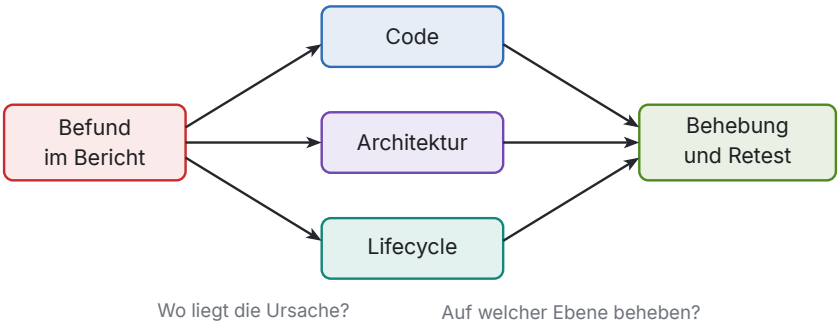
PERSÖNLICHES HANDBUCH

# Pentesting verstehen und beheben

Methodik, Kataloge und Schwachstellenklassen, Berichte lesen, Schweregrade rechnen und Befunde in Code, Architektur und Lifecycle beheben




Für Philipp Haindl
Begleitbuch zur Spezialisierung Software Security (SPE-SS), Master Cyber Security and Resilience, USTP

Stand: November 2026

# Inhalt


**Bevor es losgeht** 6

Was dieses Buch ist und was nicht . . . 6

Aufbau . . . 6

Konventionen . . . 6

TEIL I · DAS FELD VERSTEHEN

**1 Was ein Pentest ist und was nicht** 8

1.1 Eine Arbeitsdefinition . . . 8

1.2 Die Verwandten . . . 9

1.3 Wie viel der Tester weiß: Black, Grey und White Box . . . 10

1.4 Was ein Pentest aussagt . . . 11

1.5 Wer beteiligt ist . . . 11

1.6 Der Ablauf im Überblick . . . 12

1.7 Wann ein Pentest sinnvoll ist . . . 13

Selbsttest . . . 14

**2 Auftrag, Scope und Recht** 15

2.1 Dieselbe Handlung, zwei Bedeutungen . . . 15

2.2 Der Rechtsrahmen in Österreich . . . 15

2.3 Wer einwilligen kann . . . 16

2.4 Der Scope . . . 17

2.5 Rules of Engagement . . . 18

2.6 Was dabei schiefgehen kann . . . 19

2.7 KI-Werkzeuge im Test . . . 19

2.8 Vertraulichkeit des Ergebnisses . . . 20

Selbsttest . . . 20

**3 Methodiken: wie ein Test abläuft** 22

3.1 Wozu eine Methodik . . . 22

3.2 PTES: sieben Phasen . . . 22

3.3 NIST SP 800-115: vier Phasen . . . 23

3.4 OWASP WSTG: der Prüfkatalog für Web-Anwendungen . . . 24

3.5 WSTG und ASVS: Test und Anforderung . . . 25

3.6 Weitere Methodiken in Kurzform . . . 26

3.7 Was alle gemeinsam haben . . . 26

3.8 Zeit ist die knappe Größe . . . 27

3.9 Automatisierung und KI im Ablauf . . . 27

Selbsttest . . . 28

**4 Die Landkarte: CVE, CWE, CVSS und ATT&CK** 30

4.1 Warum es Kataloge braucht . . . 30

4.2 CVE: eine Nummer für jede bekannte Lücke . . . 31

4.3 CWE: die Art des Fehlers . . . 32

4.4 Ranglisten: OWASP Top 10 und CWE Top 25 . . . 34

4.5 CVSS: die Schwere in einer Zahl . . . 35

4.6 EPSS und KEV: wird die Lücke ausgenutzt? . . . 36

4.7 CAPEC: wie ein Fehler angegriffen wird . . . 37

4.8 MITRE ATT&CK: wie Angreifer vorgehen . . . 37

4.9 Wie alles zusammenhängt . . . 38

4.10 Was davon im Pentest-Bericht steht . . . 39

4.11 Grenzen der Kataloge . . . 40

Selbsttest . . . 41

**5 Sicherheit im Lebenszyklus** 42

5.1 Vom SDLC zum SSDLC . . . 42

5.2 Das NIST SSDF . . . 43

5.3 Reifegradmodelle: SAMM und BSIMM . . . 44

5.4 Die automatisierten Prüfungen . . . 45

5.5 Wo der Pentest sitzt . . . 47

5.6 Nach der Auslieferung . . . 47

5.7 Der Befund als Rückmeldung . . . 48

Selbsttest . . . 49

TEIL II · WIE EIN TESTER DENKT

**6 Angriffsfläche, Vertrauensgrenzen und STRIDE** 50

6.1 Der andere Blick . . . 50

6.2 Die Angriffsfläche . . . 50

6.3 Vertrauensgrenzen . . . 51

6.4 Bedrohungsmodellierung in vier Fragen . . . 53

6.5 Das Datenflussdiagramm . . . 53

6.6 STRIDE . . . 54

6.7 Von der Bedrohung zur Maßnahme . . . 55

6.8 Angriffsbäume . . . 56

6.9 Bedrohungen gewichten . . . 57

6.10 Grenzen der Verfahren . . . 57

Selbsttest . . . 58

**7 Aufklärung und Bestandsaufnahme** 59

7.1 Der halbe Test . . . 59

7.2 Passive Aufklärung: was ohne Anfassen zu erfahren ist . . . 59

7.3 Aktive Aufklärung: was antwortet . . . 60

7.4 Die Anwendung kartieren . . . 61

7.5 Was ein Schwachstellenscanner tut . . . 62

7.6 Was Systeme über sich verraten . . . 63

7.7 Die Bestandsaufnahme im Bericht . . . 63

7.8 Die Sicht der Verteidigung . . . 64
7.9 Aufklärung mit KI . . . 64
Selbsttest . . . 65

**8 Schwachstellenklassen I: Eingaben 66**
8.1 Der gemeinsame Mechanismus . . . 66
8.2 SQL-Injection . . . 67
8.3 Befehle, Code und Vorlagen . . . 68
8.4 Cross-Site Scripting . . . 69
8.5 Dateien: Pfade und Uploads . . . 70
8.6 Server-Side Request Forgery . . . 70
8.7 Deserialisierung und XML . . . 71
8.8 Drei verschiedene Gegenmaßnahmen . . . 72
8.9 Die drei Ebenen im Überblick . . . 72
Selbsttest . . . 74

**9 Schwachstellenklassen II: Identität und Zugriff 75**
9.1 Drei Fragen . . . 75
9.2 Authentifizierung . . . 76
9.3 Sitzungen und Tokens . . . 77
9.4 Cross-Site Request Forgery . . . 78
9.5 Autorisierung . . . 78
9.6 Zugriff zwischen Services . . . 81
9.7 Die drei Ebenen im Überblick . . . 82
Selbsttest . . . 83

**10 Schwachstellenklassen III: Konfiguration, Kryptografie, Lieferkette, Logik 84**
10.1 Fehlkonfiguration . . . 84
10.2 Kryptografische Fehler . . . 86
10.3 Lieferkette . . . 87
10.4 Fehler in der Fachlogik . . . 88
10.5 Ressourcen und das Verhalten im Fehlerfall . . . 90
10.6 Die drei Ebenen im Überblick . . . 91
Selbsttest . . . 92

**11 Nach dem ersten Zugriff 93**
11.1 Der Befund ist nicht das Ende . . . 93
11.2 Was ein Angreifer nach dem Zugriff tut . . . 94
11.3 Rechteausweitung . . . 94
11.4 Zugangsdaten . . . 95
11.5 Seitwärtsbewegung . . . 96
11.6 Unentdeckt bleiben . . . 97
11.7 Der Wirkungsradius als Architektureigenschaft . . . 97
11.8 Was davon im Bericht steht . . . 98
Selbsttest . . . 99

**12 KI-Systeme testen 101**
12.1 Zwei Rollen der KI . . . 101
12.2 Aufbau einer Anwendung mit Sprachmodell . . . 101
12.3 Die OWASP-Liste für LLM-Anwendungen . . . 102
12.4 Prompt Injection . . . 103
12.5 Die Ausgabe ist eine nicht vertrauenswürdige Eingabe . . . 103
12.6 Übermäßige Handlungsmacht . . . 104
12.7 Preisgabe, Vergiftung, Verbrauch . . . 104
12.8 Was am Testen anders ist . . . 105
Selbsttest . . . 107

**TEIL III · BEFUNDE LESEN UND BEWERTEN**

**13 Der Pentest-Bericht 108**
13.1 Drei Leser, ein Dokument . . . 108
13.2 Der Aufbau . . . 108
13.3 Die Zusammenfassung für die Leitung . . . 109
13.4 Auftrag, Scope und Einschränkungen . . . 110
13.5 Die Abdeckung . . . 110
13.6 Die Befundübersicht . . . 111
13.7 Anatomie eines Befunds . . . 111
13.8 Angriffspfade . . . 113
13.9 Was standgehalten hat . . . 114
13.10 Einen Bericht beurteilen . . . 114
13.11 Einen Bericht lesen . . . 115
13.12 Die Abschlussbesprechung . . . 115
13.13 Der Bericht als vertrauliches Dokument . . . 116
13.14 Berichte aus der Maschine . . . 116
Selbsttest . . . 117

**14 Schweregrade: CVSS rechnen und einordnen 118**
14.1 Drei Dinge, die oft verwechselt werden . . . 118
14.2 Der Aufbau von CVSS 3.1 . . . 118
14.3 Die acht Basismetriken . . . 119
14.4 Einen Vektor lesen . . . 120
14.5 Wie aus dem Vektor die Zahl wird . . . 120
14.6 Was die Zahl verschiebt . . . 121
14.7 Warum 6,5 „hoch“ sein kann . . . 121
14.8 Die Umgebungsmetriken . . . 121
14.9 Die zeitlichen Metriken . . . 122
14.10 CVSS 4.0 . . . 122
14.11 Wo CVSS nicht passt . . . 123
14.12 Von der Schwere zur Dringlichkeit . . . 124
14.13 Eine Bewertung prüfen . . . 124
Selbsttest . . . 126

**15 Triage: prüfen, bündeln, verketten 127**
15.1 Was Triage ist . . . 127
15.2 Schritt 1: Trifft der Befund zu? . . . 128
15.3 Schritt 2: Gilt der Befund in der Produktion? . . . 129

15.4 Schritt 3: Bündeln . . . 129
15.5 Schritt 4: Verketten . . . 130
15.6 Schritt 5: Ordnen . . . 132
15.7 Die Suche nach weiteren Stellen . . . 133
Selbsttest . . . 134

TEIL IV · VON DER SCHWÄCHE ZUR URSACHE

**16 Ursachenanalyse auf drei Ebenen 136**
16.1 Symptom, Fehler, Ursache . . . 136
16.2 Die drei Ebenen . . . 137
16.3 Die Warum-Kette . . . 137
16.4 Die CWE als Leiter . . . 138
16.5 Wie die Architektur Fehler wahrscheinlich macht 138
16.6 Wie der Lifecycle Fehler durchlässt . . . 139
16.7 Die Ursachen bei Kurbad . . . 140
16.8 Nur im Code, nur in der Architektur . . . 141
16.9 Die Analyse aufschreiben . . . 142
16.10 Grenzen der Ursachenanalyse . . . 143
Selbsttest . . . 144

**17 Beheben, abmildern, nachtesten 145**
17.1 Vier Antworten auf einen Befund . . . 145
17.2 Behebung oder Abmilderung . . . 146
17.3 Drei Zeithorizonte . . . 147
17.4 Die Behebung im Code prüfbar machen . 147
17.5 Der Dreispaltentest . . . 148
17.6 Der Maßnahmenplan für Kurbad . . . 149
17.7 Nebenwirkungen . . . 150
17.8 Was sonst noch zu tun ist . . . 150
17.9 Der Nachtest . . . 151
17.10 Behebungen von der Maschine . . . 152
Selbsttest . . . 153

**18 Entscheiden unter Konflikt 154**
18.1 Wer entscheidet . . . 154
18.2 Vier Positionen . . . 155
18.3 Die Positionen lesen . . . 155
18.4 Kriterien für die Reihenfolge . . . 156
18.5 Ein Vorschlag in Stufen . . . 157
18.6 Die Risikoakzeptanz . . . 158
18.7 Die Vorlage für die Entscheidung . . . 159
18.8 Die Entscheidung festhalten . . . 159
Selbsttest . . . 161

TEIL V · IN DER LEHRE

**19 Die Spezialisierung unterrichten 162**
19.1 Was die Spezialisierung leisten soll . . . 162
19.2 Die Case Study vom Ende her bauen . . . 163
19.3 Was KI in der Case Study leistet . . . 165
19.4 Die Abgabe . . . 166
19.5 Bewerten . . . 167
19.6 Die mündliche Verteidigung . . . 167
19.7 Der Ablauf des Blocks . . . 168
19.8 Abstimmung mit der Pentesting-Lehrveranstaltung 169
19.9 Was schiefgehen kann . . . 169
Selbsttest . . . 170

**20 Ein Fall von Anfang bis Ende 171**
20.1 Das System und seine Befunde . . . 171
20.2 Woche 0: Der Auftrag . . . 171
20.3 Wochen 1 und 2: Der Test . . . 172
20.4 Woche 3: Der Bericht . . . 172
20.5 Woche 3: Sofortmaßnahmen . . . 172
20.6 Woche 4: Triage . . . 172
20.7 Wochen 4 und 5: Ursachen . . . 173
20.8 Woche 5: Plan und Entscheidung . . . 173
20.9 Woche 6: Der erste Nachtest . . . 173
20.10 Das Quartal: Stufe 1 . . . 173
20.11 Ein Jahr später . . . 174
20.12 Was der Fall zeigt . . . 174
20.13 Der Fall als Übungsfall . . . 175
Selbsttest . . . 175

ANHANG

**A Spickzettel 177**
A.1 Kürzel auf einen Blick . . . 177
A.2 Einen CVSS-Vektor lesen (Version 3.1) . . . 178
A.3 STRIDE . . . 179
A.4 OWASP Top 10:2025 . . . 179
A.5 Schwächen und ihre Behebung im Überblick 180
A.6 Die drei Ebenen . . . 180
A.7 Vom Bericht zum Plan in sechs Schritten . 180
A.8 Zehn Fragen für die mündliche Verteidigung 181

**B Lösungen zu den Selbsttests 182**
B.1 Kapitel 1: Was ist ein Pentest? . . . 182
B.2 Kapitel 2: Auftrag, Recht und Rules of Engagement 182
B.3 Kapitel 3: Methodik . . . 182
B.4 Kapitel 4: Die Landkarte . . . 183
B.5 Kapitel 5: Der Lebenszyklus . . . 183
B.6 Kapitel 6: Angriffsfläche und STRIDE . . . 184
B.7 Kapitel 7: Aufklärung . . . 184
B.8 Kapitel 8: Eingaben . . . 184
B.9 Kapitel 9: Identität und Zugriff . . . 185

B.10 Kapitel 10: Konfiguration, Kryptografie, Lieferkette, Fachlogik . . . . . 185
B.11 Kapitel 11: Nach dem ersten Zugriff . . . . . 186
B.12 Kapitel 12: KI-Systeme testen . . . . . 186
B.13 Kapitel 13: Der Pentest-Bericht . . . . . 186
B.14 Kapitel 14: Schweregrade . . . . . 187
B.15 Kapitel 15: Triage . . . . . 187
B.16 Kapitel 16: Ursachenanalyse . . . . . 188
B.17 Kapitel 17: Beheben, abmildern, nachtesten . . . . . 188
B.18 Kapitel 18: Entscheiden unter Konflikt . . . . 189
B.19 Kapitel 19: Die Spezialisierung unterrichten . . . . . 189
B.20 Kapitel 20: Ein Fall von Anfang bis Ende . 190

**C Glossar 191**

**Literaturverzeichnis 193**

# Bevor es losgeht

Dieses Handbuch ist für jemanden geschrieben, der Software-Architektur und sichere Entwicklung unterrichtet, aber selbst nie Penetrationstests gemacht hat. Du kennst Architekturmuster, Qualitätsszenarien, Sagas und sichere Programmierpraktiken. Was dir fehlt, ist die andere Seite: wie ein Tester ein System ansieht, in welcher Sprache er seine Ergebnisse aufschreibt und wie man aus einem Befund im Bericht wieder bei einer Entscheidung in Architektur, Code oder Entwicklungsprozess landet.

Genau diese Lücke ist der Grund, warum es den neuen Studienplan in dieser Form gibt. In der Praxis bauen die einen Software und die anderen testen sie, und beide Gruppen verstehen die Arbeit der jeweils anderen nur ungefähr. Der Entwickler liest im Pentest-Bericht „CWE-639, CVSS 7.1, AV:N/AC:L/PR:L“ und weiß nicht, was er damit anfangen soll. Der Tester schreibt „Eingaben validieren“ als Empfehlung und weiß nicht, dass das Problem in Wahrheit eine fehlende Vertrauensgrenze in der Architektur ist. Die Spezialisierung Software Security soll Leute ausbilden, die beides können. Dieses Buch soll dich in die Lage versetzen, sie zu unterrichten.

## Was dieses Buch ist und was nicht

Das Buch erklärt Pentesting aus der Sicht dessen, der die Ergebnisse verstehen, bewerten und beheben muss. Es ist kein Angriffshandbuch. Du findest hier keine Werkzeugbefehle zum Nachtippen und keine fertigen Angriffe, sondern Mechanismen: warum eine Schwachstellenklasse entsteht, woran man sie erkennt, wie sie im Bericht aussieht, auf welcher Ebene ihre Ursache liegt und wie man sie behebt. Das ist auch die Tiefe, die du für die Lehre brauchst. Die Studierenden hören die praktische Seite in der Lehrveranstaltung Application Security and Pentesting bei einer anderen Lehrperson, bevor sie zu dir in die Spezialisierung kommen. Du musst ihre Sprache sprechen, ihre Berichte lesen und ihre Behebungen beurteilen können.

Alle Begriffe, die in diesem Feld als bekannt vorausgesetzt werden, sind hier von Grund auf erklärt: CVE, CWE, CVSS, CAPEC, MITRE ATT&CK, KEV, EPSS, STRIDE, PTES, WSTG, SSDF, SAMM und einige mehr. Jeder Begriff bekommt beim ersten Auftreten eine Definition, ein Beispiel und eine Einordnung, und er steht im Glossar in Anhang C oder, wenn es ein Kürzel ist, im Spickzettel in Anhang A.

## Aufbau

- **Teil I, Das Feld verstehen:** was ein Pentest ist, wie Auftrag, Scope und Rechtsrahmen aussehen, nach welchen Methodiken getestet wird, welche Kataloge und Bewertungssysteme es gibt und wo das Testen im Software-Lebenszyklus sitzt.
- **Teil II, Wie ein Tester denkt:** Angriffsfläche, Vertrauensgrenzen und Threat Modeling mit STRIDE, Aufklärung, die Schwachstellenklassen in drei Kapiteln, das Vorgehen nach dem ersten Zugriff und das Testen von KI-Systemen.
- **Teil III, Befunde lesen und bewerten:** der Aufbau eines Pentest-Berichts, das Rechnen mit CVSS und die Triage, also das Prüfen, Bündeln und Verketten von Befunden.
- **Teil IV, Von der Schwäche zur Ursache:** die Ursachenanalyse auf den drei Ebenen Code, Architektur und Lifecycle, das Beheben und Nachtesten und das Entscheiden, wenn Stakeholder Widersprüchliches wollen.
- **Teil V, In der Lehre:** wie du die Case Study der Spezialisierung baust und beurteilst, und ein Fall, der von Anfang bis Ende durchgespielt wird.
- **Anhang:** Spickzettel, Lösungen zu den Selbsttests, Glossar.

## Konventionen

Drei Ebenen ziehen sich durch das ganze Buch. Jede Schwäche hat ihre Ursache auf mindestens einer davon, und jede Behebung setzt auf mindestens einer an. Sie sind farblich markiert: `Code` steht für Fehler

in der Implementierung, Architektur für Fehler im Entwurf und Lifecycle für Fehler im Entwicklungs- und Betriebsprozess.

Befunde, wie sie in einem Pentest-Bericht stehen, sind als Karten gesetzt:

**F-00 Beispielbefund** MITTEL

| | |
|---|---|
| **Ort** | wo die Schwäche gefunden wurde |
| **Beschreibung** | was der Tester beobachtet hat |
| **Einordnung** | CWE-Nummer und CVSS-Vektor, erklärt in Kapitel 4 und 14 |

Kästen mit dem Titel „Brücke zu Software Security“ zeigen, wo ein Thema an deine Pflicht-Lehrveranstaltung und ihre Foliensätze anschließt. Kästen „Für die Lehre“ enthalten Hinweise für den Unterricht in der Spezialisierung. „Frage aus dem Hörsaal“ sind Fragen, mit denen du rechnen solltest, samt einer Antwort. „In der Reihe“ verweist auf die anderen Handbücher, damit sich nichts doppelt: Die Schutzseite von APIs steht in *Secure API Design*, die von Messaging-Systemen in *Sichere Messaging-Architekturen*, Container-Sicherheit in *Docker von Grund auf* und Pipeline-Sicherheit in *Git von Grund auf*.

Code-Ausschnitte sind kurz und dienen dem Lesen, nicht dem Ausführen. Wo ein Ausschnitt eine Schwäche zeigt, steht die Erklärung im Text darunter und nicht als Kommentar im Code, so wie es auch in den Übungsunterlagen für die Studierenden gehalten wird.

Zeitkritische Angaben, also Versionen von Katalogen, Rechtsstände und Fristen, tragen den Stand 12. November 2026. Kataloge wie die OWASP Top 10 oder MITRE ATT&CK ändern sich regelmäßig. Das Buch erklärt deshalb vor allem die Mechanismen dahinter, die bleiben.

Das durchgängige Beispiel ist ein erfundenes System namens *Kurbad*, die Buchungs- und Abrechnungsplattform eines Thermenbetreibers mit Web-Oberfläche, mobiler App, einigen Services, einer Nachrichtenwarteschlange und einer Anbindung an einen Zahlungsdienstleister. Es ist klein genug, um es im Kopf zu behalten, und verteilt genug, um alle Arten von Schwächen zu zeigen. Kapitel 20 spielt einen vollständigen Fall an diesem System durch.

# 1 Was ein Pentest ist und was nicht

Ein Penetrationstest ist ein beauftragter, zeitlich begrenzter Angriff auf ein System, den jemand durchführt, um Schwächen zu finden, bevor es ein echter Angreifer tut. So weit die Kurzfassung, die in jedem Angebot steht. Sie verdeckt, was für deine Arbeit wichtig ist: Ein Pentest ist eine Stichprobe und kein Beweis, sein Ergebnis ist ein Dokument und kein sichereres System, und sein Wert entsteht erst dort, wo jemand die Befunde versteht und behebt. Dieses Kapitel klärt, was ein Pentest leistet, wie er sich von seinen Verwandten unterscheidet und warum die Übergabe vom Tester zum Entwickler die Stelle ist, an der in der Praxis am meisten verloren geht.

**IN DIESEM KAPITEL**

- einen Penetrationstest definieren und von Schwachstellenscan, Audit, Code Review, Red Teaming und Bug Bounty abgrenzen
- Black-, Grey- und White-Box-Tests unterscheiden und sagen, wann welche Form sinnvoll ist
- erklären, was ein Pentest-Ergebnis aussagt und was nicht
- die Beteiligten eines Tests und ihre unterschiedlichen Sichtweisen benennen

## 1.1 Eine Arbeitsdefinition

Die Literatur ist sich im Kern einig. NIST beschreibt Penetrationstests als Sicherheitstests, bei denen Prüfer reale Angriffe nachahmen, um Wege zu finden, die Sicherheitsfunktionen einer Anwendung, eines Systems oder eines Netzes zu umgehen [1]. Für dieses Buch brauchen wir eine Definition, die die Merkmale einzeln greifbar macht.

**DEFINITION 1.1 · PENETRATIONSTEST**
Ein Penetrationstest ist eine Sicherheitsprüfung mit vier Merkmalen: Sie ist **beauftragt** (es gibt eine schriftliche Erlaubnis des Berechtigten), **begrenzt** (in Zeit, Zielsystemen und erlaubten Mitteln), **zielgerichtet** (der Tester versucht aktiv, Schwächen auszunutzen und nicht nur aufzulisten) und sie endet mit einem **Nachweis** (einem Bericht, der jede gefundene Schwäche nachvollziehbar belegt).

Jedes der vier Merkmale trennt den Pentest von etwas anderem. Ohne Auftrag ist dieselbe Tätigkeit eine Straftat, dazu mehr in Kapitel 2. Ohne Begrenzung wäre es eine dauerhafte Sicherheitsüberwachung. Ohne das aktive Ausnutzen wäre es ein Schwachstellenscan. Und ohne Nachweis wäre es eine Behauptung, mit der niemand arbeiten kann.

Das dritte Merkmal verdient einen zweiten Blick, weil es den Namen erklärt. „Penetration“ meint das Eindringen: Der Tester begnügt sich nicht mit dem Verdacht, dass eine Schwäche vorliegt, sondern zeigt, dass sie sich ausnutzen lässt und wie weit man damit kommt. Ein Scanner meldet, dass eine Bibliothek veraltet ist. Ein Tester zeigt, dass er über diese Bibliothek Kundendaten lesen konnte. Der Unterschied ist für den Empfänger des Berichts erheblich, denn der Nachweis beendet die Diskussion, ob die Schwäche „wirklich ein Problem“ ist.

**MERKE**
Ein Pentest beantwortet die Frage „Was kann ein Angreifer mit diesem System in dieser Zeit erreichen?“ Er beantwortet nicht die Frage „Ist dieses System sicher?“

## 1.2 Die Verwandten

Rund um den Pentest gibt es eine Reihe von Prüfformen, die im Alltag durcheinandergeworfen werden. Wer einen Bericht liest, muss wissen, welche Form ihn hervorgebracht hat, denn davon hängt ab, was die Befunde wert sind.

### Schwachstellenscan

Ein Schwachstellenscan (*vulnerability scan*) ist ein automatisierter Abgleich: Ein Werkzeug prüft Systeme gegen eine Datenbank bekannter Schwachstellen und Fehlkonfigurationen und gibt eine Liste aus. Das geht schnell, lässt sich wöchentlich wiederholen und deckt viel Fläche ab. Der Scan weiß aber nichts über den Zweck der Anwendung. Er findet die veraltete Serverversion, aber nicht, dass Kundin A die Rechnungen von Kunde B abrufen kann, weil das keine bekannte Schwachstelle eines Produkts ist, sondern ein Fehler in genau dieser einen Anwendung. Außerdem meldet ein Scan Verdachtsfälle. Ob eine gemeldete Schwäche in dieser Umgebung tatsächlich ausnutzbar ist, hat niemand geprüft. Das Ergebnis enthält deshalb regelmäßig falsche Treffer, die Kapitel 15 behandelt.

Viele Pentests beginnen mit einem Scan, um die naheliegenden Funde abzuräumen. Ein Bericht, der nur aus Scanner-Ausgaben besteht, ist aber kein Pentest-Bericht, auch wenn er so verkauft wird. Du erkennst ihn daran, dass die Befunde generische Texte tragen, nichts über die Fachlogik der Anwendung sagen und kein Nachweis der Ausnutzung dabei ist.

### Audit und Konformitätsprüfung

Ein Audit prüft gegen einen Maßstab: eine Norm, eine Richtlinie, eine Checkliste. Die Frage lautet, ob eine Organisation oder ein System die Anforderungen erfüllt, etwa ob es eine Passwortrichtlinie gibt und ob sie eingehalten wird. Das Ergebnis ist eine Liste von Abweichungen. Ein Audit kann vollständig bestanden werden, während die Anwendung trotzdem angreifbar ist, weil die Checkliste die konkrete Schwäche nicht kennt. Umgekehrt kann ein Pentest ohne kritischen Befund enden, während ein Audit grobe organisatorische Mängel findet. Die beiden Formen messen Verschiedenes.

### Code Review und statische Analyse

Beim Code Review liest ein Mensch oder ein Werkzeug den Quelltext. Das ist die Sicht von innen: Man sieht jede Verzweigung, auch die, die ein Tester von außen nie erreicht. Dafür sieht man nicht, wie sich das fertige, konfigurierte und verteilte System verhält. Ein Fehler in der Konfiguration des Reverse Proxy steht in keiner Quelldatei der Anwendung. Statische Analyse (*SAST*), Analyse der Abhängigkeiten (*SCA*) und dynamische Analyse (*DAST*) sind die automatisierten Geschwister. Kapitel 5 ordnet sie in den Lebenszyklus ein.

### Red Teaming

Ein Red-Team-Einsatz ist dem Pentest am ähnlichsten und wird am häufigsten mit ihm verwechselt. Der Unterschied liegt im Ziel. Ein Pentest will in begrenzter Zeit möglichst viele Schwächen in einem definierten System finden. Ein Red Team will ein bestimmtes Ziel erreichen, etwa den Zugriff auf die Kundendatenbank, und es will dabei prüfen, ob die Verteidigung den Angriff bemerkt und richtig reagiert. Das Red Team arbeitet deshalb verdeckt, über Wochen oder Monate, und nimmt den Weg des geringsten Widerstands, auch über Menschen und Gebäude. Es findet nicht viele Schwächen, sondern eine Kette, die zum Ziel führt. Getestet wird nicht das System, sondern die Organisation mitsamt ihrer Erkennung und Reaktion.

Für den Finanzsektor ist diese Form inzwischen vorgeschrieben: Die EU-Verordnung DORA verlangt von bestimmten Finanzunternehmen regelmäßige bedrohungsgeleitete Penetrationstests (*threat-led penetration testing*, TLPT) [2], deren Durchführung sich am Rahmenwerk TIBER-EU der Europäischen Zentralbank orientiert [3]. Trotz des Namens ist das methodisch ein Red-Team-Einsatz.

### Bug Bounty

Bei einem Bug-Bounty-Programm lädt ein Unternehmen Außenstehende ein, Schwächen zu suchen, und bezahlt pro gültigem Fund. Es gibt keinen einzelnen Auftragnehmer, kein festes Zeitfenster und keine Garantie, dass überhaupt jemand sucht oder dass alle Bereiche angesehen werden. Dafür prüfen viele verschiedene Köpfe mit verschiedenen Spezialgebieten. Ein Bug-Bounty-Programm ergänzt Pentests und ersetzt sie nicht. Es setzt außerdem voraus, dass die Organisation eingehende Meldungen geordnet bearbeiten kann. Dafür gibt es eigene Normen, ISO/IEC 29147 für die Offenlegung und ISO/IEC 30111 für

die interne Bearbeitung [4], [5], und eine einfache technische Konvention, die Datei `security.txt`, in der ein Betreiber angibt, wohin Meldungen gehen sollen [6].

**Tab. 1:** Prüfformen im Vergleich. Jede beantwortet eine andere Frage.

| Form | Frage | Ergebnis | Blinder Fleck |
|---|---|---|---|
| Schwachstellen-scan | Welche bekannten Schwächen könnten vorliegen? | Liste von Verdachtsfällen | Fachlogik, tatsächliche Ausnutzbarkeit |
| Audit | Werden die Anforderungen einer Norm erfüllt? | Liste von Abweichungen | alles, was die Norm nicht nennt |
| Code Review | Welche Fehler stehen im Quelltext? | Fundstellen im Code | Konfiguration, Zusammenspiel, Betrieb |
| Pentest | Was erreicht ein Angreifer in begrenzter Zeit? | belegte Befunde mit Bewertung | alles außerhalb von Scope und Zeitfenster |
| Red Teaming | Erreicht ein Angreifer ein bestimmtes Ziel unbemerkt? | Angriffspfad, Bewertung von Erkennung und Reaktion | Breite: andere Schwächen bleiben ungefunden |
| Bug Bounty | Was finden viele Außenstehende über lange Zeit? | einzelne Meldungen | Abdeckung ist dem Zufall überlassen |

FRAGE AUS DEM HÖRSAAL

***Wenn ein Red Team realistischer ist, warum macht man dann überhaupt noch Pentests?***

Weil die beiden Formen Verschiedenes liefern. Ein Red Team sucht einen Weg zum Ziel und hört auf, wenn es ihn hat. Die zwanzig anderen Schwächen links und rechts dieses Weges bleiben unentdeckt. Ein Pentest sucht in die Breite und liefert die Liste, die ein Entwicklungsteam abarbeiten kann. Red Teaming setzt außerdem eine Organisation voraus, die schon eine funktionierende Erkennung hat. Wer die noch nicht hat, erfährt aus einem Red-Team-Einsatz nur, dass er durchgefallen ist.

## 1.3 Wie viel der Tester weiß: Black, Grey und White Box

Pentests unterscheiden sich danach, wie viel Wissen der Tester vorab bekommt. Die drei üblichen Stufen heißen nach der Sichtbarkeit des Innenlebens, so wie du es von Black-Box- und White-Box-Tests aus der Softwaretechnik kennst.

Beim **Black-Box-Test** bekommt der Tester nur das Ziel genannt, zum Beispiel eine Adresse. Er hat keine Zugangsdaten, keine Dokumentation und keinen Quelltext. Das entspricht der Lage eines Außenstehenden und wirkt deshalb besonders realistisch. Es ist aber die teuerste Art, wenig herauszufinden: Ein großer Teil der bezahlten Zeit fließt in Aufklärung, die ein echter Angreifer ohne Zeitdruck erledigen würde, und alles hinter der Anmeldung bleibt ungetestet, wenn der Tester in der gegebenen Zeit nicht hineinkommt.

Beim **White-Box-Test** bekommt der Tester alles: Architekturdokumentation, Quelltext, Konfiguration, Zugangsdaten für alle Rollen. Er kann gezielt dorthin gehen, wo die Dokumentation Schwächen vermuten lässt, und er kann einen Verdacht im Code bestätigen, statt ihn von außen zu erraten. Diese Form findet in derselben Zeit am meisten.

Der **Grey-Box-Test** liegt dazwischen und ist in der Praxis der Normalfall für Anwendungen: Der Tester bekommt Benutzerkonten für die verschiedenen Rollen und eine grobe Beschreibung, aber keinen Quelltext. Damit kann er prüfen, was ein angemeldeter Benutzer anstellen kann, und das ist bei den meisten Anwendungen die ergiebigste Frage.




**Abb. 1:** Die drei Wissensstufen eines Pentests. Mehr Vorwissen bringt mehr Befunde pro Testtag, sagt aber weniger über den Aufwand eines Außenstehenden.

**BRÜCKE ZU SOFTWARE SECURITY**
Für die Verzahnung mit deiner Lehrveranstaltung ist der White-Box-Test die interessante Form. Dort bekommt der Tester genau die Unterlagen, die deine Studierenden in Software Security lesen lernen: C4-Diagramme, Architecture Decision Records, Qualitätsszenarien. Wer aus einem Container-Diagramm ablesen kann, zwischen welchen Services die Authentifizierung fehlt, testet gezielter als jemand, der von außen stochert. Das ist die Richtung „von der Architektur zum Test", die am Ende deiner Pflicht-Lehrveranstaltung steht.

## 1.4 Was ein Pentest aussagt

In deinem Foliensatz zu Software Testing steht als erstes Testprinzip, dass Testen die Anwesenheit von Fehlern zeigt und nicht ihre Abwesenheit [7]. Für Pentests gilt das in verschärfter Form, aus drei Gründen.

**Ein Pentest ist eine Stichprobe.** Der Tester hat einige Tage. Die Anwendung hat hunderte Endpunkte, Rollen und Zustände. Er wählt aus, wo er sucht, und diese Auswahl ist Erfahrung und Zufall. Zwei gleich gute Tester finden in demselben System nicht dieselben Schwächen.

**Ein Pentest ist eine Momentaufnahme.** Getestet wurde der Stand eines bestimmten Tages. Mit dem nächsten Release, der nächsten Konfigurationsänderung und der nächsten veröffentlichten Schwachstelle in einer Abhängigkeit ändert sich die Lage. Ein Bericht vom Frühjahr sagt über das System im Herbst wenig.

**Ein Pentest ist begrenzt durch seinen Auftrag.** Was nicht im Scope war, wurde nicht geprüft. Was der Auftrag verboten hat, zum Beispiel Lasttests oder Angriffe auf Mitarbeiter, wurde nicht versucht. Ein echter Angreifer hält sich an keine dieser Grenzen.

Daraus folgt, wie ein leerer Bericht zu lesen ist. „Keine kritischen Befunde" heißt: In dieser Zeit, in diesem Scope, mit diesem Vorwissen hat dieser Tester nichts Kritisches gefunden. Es heißt nicht, dass nichts da ist. Ein guter Bericht sagt das selbst und nennt, was aus Zeitgründen nicht geprüft wurde. Fehlt diese Angabe, ist das ein Qualitätsmangel des Berichts, auf den Kapitel 13 zurückkommt.

**FALLSTRICK**
„Wir hatten einen Pentest, also sind wir sicher" ist derselbe Fehlschluss wie „die Tests sind grün, also ist die Software korrekt". Der Pentest ist eine Prüfmaßnahme unter mehreren. Er ersetzt weder sichere Entwicklung noch Überwachung im Betrieb.

Umgekehrt sollte man den Pentest auch nicht kleinreden. Er ist die einzige der genannten Prüfformen, die das fertige System so ansieht, wie es ein Gegner tun würde, und die zeigt, wie sich einzelne unscheinbare Schwächen zu einem ernsten Angriff verbinden. Diese Verkettung findet kein Scanner und kein Audit. Kapitel 15 zeigt, wie sie im Bericht aussieht.

## 1.5 Wer beteiligt ist

An einem Pentest sind mehr Leute beteiligt als Tester und Auftraggeber, und sie wollen Verschiedenes. Wer das nicht vor Augen hat, versteht nicht, warum Berichte so aussehen, wie sie aussehen, und warum Befunde so oft liegen bleiben.

Der **Auftraggeber**, oft die Sicherheitsverantwortliche oder ein Produktverantwortlicher, will eine belastbare Aussage über das Risiko und häufig auch einen Nachweis für Kunden, Prüfer oder Aufsicht. Der **Tester** will in der knappen Zeit möglichst viele relevante Schwächen finden und sauber belegen. Das **Entwicklungsteam** bekommt am Ende eine Liste mit Arbeit, die es nicht eingeplant hat, formuliert in einer Sprache, die es nicht spricht. Der **Betrieb** will vor allem, dass während des Tests nichts ausfällt, und muss wissen, welche Alarme vom Test stammen. Dazu kommen je nach Lage **Compliance** und **Management**, die den Bericht nach außen vertreten.

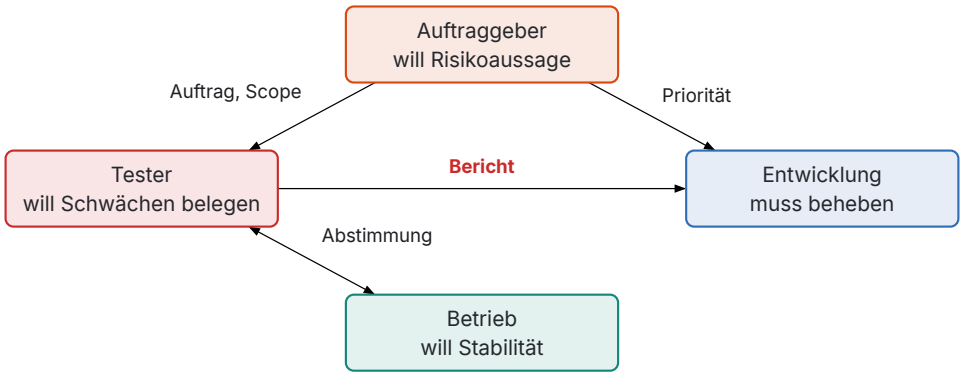



**Abb. 2:** Die Beteiligten. Der Bericht ist die einzige Verbindung zwischen Tester und Entwicklung.

Die kritische Verbindung in Abb. 2 ist die zwischen Tester und Entwicklung. Sie besteht in den meisten Projekten aus genau einem Dokument, das nach Testende übergeben wird, wenn der Tester schon im nächsten Auftrag steckt. Der Tester hat das System von außen gesehen und beschreibt Symptome: an welcher Stelle sich das System falsch verhält. Die Entwicklerin kennt das System von innen und braucht Ursachen: welche Entscheidung in Code oder Entwurf zu diesem Verhalten führt. Dazwischen liegt eine Übersetzung, die niemand ausdrücklich beauftragt hat.

Wenn diese Übersetzung misslingt, passiert eines von drei Dingen. Der Befund wird als „nicht nachvollziehbar" geschlossen. Oder das Symptom wird behoben und die Ursache bleibt, sodass derselbe Fehler an der nächsten Stelle wieder auftaucht. Oder es wird ein Fix eingebaut, der den Befund schließt und dabei etwas anderes kaputt macht. Alle drei Verläufe kommen in den folgenden Kapiteln als Beispiele vor, und alle drei zu vermeiden ist das, was die Spezialisierung den Studierenden beibringen soll.

> **AUS DER PRAXIS**
> Ein typischer Befund lautet „Die Anwendung prüft nicht, ob die angefragte Rechnung dem angemeldeten Benutzer gehört". Die Empfehlung des Testers lautet „Autorisierungsprüfung ergänzen". Das Team ergänzt die Prüfung an genau diesem Endpunkt und schließt das Ticket. Im nächsten Jahr steht derselbe Befund für die Gutschriften im Bericht, im Jahr darauf für die Mahnungen. Die Ursache war nie der einzelne Endpunkt, sondern dass die Architektur die Prüfung der Besitzverhältnisse jedem einzelnen Endpunkt überlässt, statt sie an einer Stelle zu erzwingen. Das stand in keinem der drei Berichte, weil der Tester die Architektur nicht kannte.

## 1.6 Der Ablauf im Überblick

Bevor Kapitel 3 die Methodiken im Einzelnen vergleicht, hier der Ablauf in der gröbsten Form. Jeder Pentest, egal nach welchem Standard, durchläuft fünf Abschnitte.

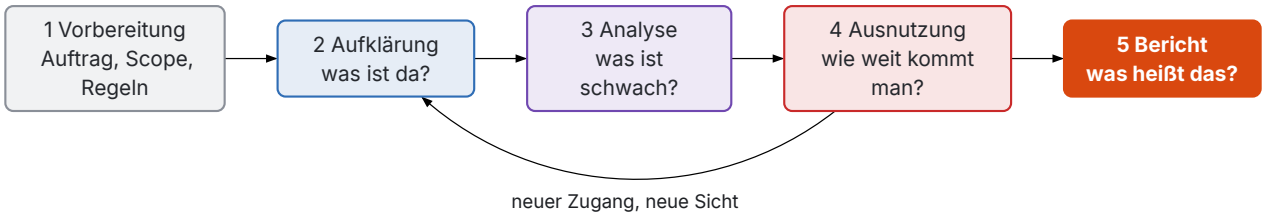



**Abb. 3:** Der Ablauf eines Pentests in fünf Abschnitten. Der Rückpfeil zeigt, dass jeder gewonnene Zugang die Aufklärung neu startet.

In der **Vorbereitung** wird geklärt, was getestet werden darf, wann, womit und wer im Notfall erreichbar ist. In der **Aufklärung** verschafft sich der Tester ein Bild: welche Systeme, Dienste, Endpunkte und Rollen es gibt. In der **Analyse** sucht er nach Stellen, an denen sich das System anders verhält, als es sollte. In der **Ausnutzung** prüft er, ob sich ein Verdacht bestätigt und was er damit erreicht. Hat er einen neuen Zugang gewonnen, etwa ein Benutzerkonto mit höheren Rechten, beginnt die Aufklärung von dort aus neu, denn jetzt sieht er Teile des Systems, die vorher unsichtbar waren. Am Ende steht der **Bericht**.

Für dich ist wichtig, dass die Abschnitte zwei bis vier das Handwerk des Testers sind, während die Abschnitte eins und fünf die Schnittstellen zum Rest der Organisation bilden. Dieses Buch behandelt das Handwerk so weit, dass du verstehst, was der Tester getan hat und warum er etwas gefunden hat (Teil II). Den Schwerpunkt legt es auf die Schnittstellen: auf Auftrag und Scope (Kapitel 2), weil dort festgelegt wird, was ein Befund überhaupt bedeuten kann, und auf Bericht, Bewertung und Behebung (Teile III und IV), weil dort deine Studierenden arbeiten werden.

## 1.7 Wann ein Pentest sinnvoll ist

Ein Pentest kostet Geld und bindet Leute. Er lohnt sich dann, wenn sein Ergebnis eine Entscheidung beeinflusst. Vier Anlässe sind üblich.

Der erste ist ein **größeres Release** oder eine neue Anwendung vor dem Gang in den Betrieb. Hier ist der Test eine letzte unabhängige Prüfung. Er kommt spät: Was er an Entwurfsfehlern findet, ist zu diesem Zeitpunkt teuer zu beheben. In deinem Foliensatz zu den Grundlagen steht die Kostenkurve, nach der ein Fehler umso teurer wird, je später er gefunden wird. Ein Pentest am Ende findet Entwurfsfehler an der teuersten Stelle dieser Kurve. Das ist kein Argument gegen den Pentest, sondern eines dafür, die früheren Prüfungen nicht durch ihn zu ersetzen.

Der zweite Anlass ist eine **wesentliche Änderung**: eine neue Schnittstelle nach außen, der Wechsel des Identitätsanbieters, die Zerlegung eines Monolithen. Der dritte ist der **regelmäßige Turnus**, meist jährlich, weil sich System und Bedrohungslage laufend ändern. Der vierte ist eine **Vorgabe von außen**: ein Kunde, der im Vertrag einen aktuellen Pentest-Bericht verlangt, oder eine Regulierung. DORA wurde schon genannt. Die NIS-2-Richtlinie, in Österreich umgesetzt durch das NISG 2026, das am 1. Oktober 2026 in Kraft getreten ist, verpflichtet wesentliche und wichtige Einrichtungen zu Maßnahmen des Risikomanagements, zu denen auch die Bewertung der Wirksamkeit dieser Maßnahmen gehört [8], [9]. Der Cyber Resilience Act verlangt von Herstellern, dass sie die Sicherheit ihrer Produkte mit digitalen Elementen wirksam und regelmäßig testen und überprüfen [10]. Keine dieser Vorschriften schreibt für alle Fälle einen Pentest im engen Sinn vor. In der Praxis ist er aber das Mittel, mit dem Organisationen die Wirksamkeit nachweisen.

**FRAGE AUS DEM HÖRSAAL**

***Warum testet man nicht einfach laufend, statt einmal im Jahr?***

Das tut man zunehmend, nur nicht mit Menschen allein. Automatisierte Prüfungen laufen in der Pipeline bei jedem Build mit (Kapitel 5). Für die Dinge, die nur ein Mensch findet, also Fehler in der Fachlogik und Verkettungen, gibt es Modelle, bei denen Tester über das Jahr verteilt Kontingente abarbeiten und jeweils die Änderungen des letzten Quartals ansehen. Der jährliche Großtest ist vor allem eine Folge davon, wie Budgets und Nachweispflichten organisiert sind, nicht eine fachliche Notwendigkeit.

**FRAGE AUS DEM HÖRSAAL**

***Kann eine KI den Pentest nicht inzwischen allein machen?***

Sie kann Teile davon. KI-gestützte Werkzeuge beschleunigen Aufklärung, das Ausprobieren von Varianten und das Schreiben des Berichts erheblich, und sie finden bekannte Muster zuverlässig. Zwei Dinge bleiben beim Menschen. Erstens die Verantwortung für den Scope: Ein Werkzeug, das selbstständig Systeme angreift, muss jemand so führen, dass es die vereinbarten Grenzen nicht verlässt. Zweitens das Urteil, ob ein Fund in dieser Anwendung, mit dieser Fachlogik, tatsächlich ein Problem ist. KI-Werkzeuge melden Schwächen, die es nicht gibt, mit derselben Überzeugung wie echte. Genau dieses Prüfen von KI-Ergebnissen ist ein Lernergebnis der Lehrveranstaltungen im neuen Studienplan.

**FÜR DIE LEHRE**

Dieses Kapitel eignet sich als Einstieg in die Spezialisierung, weil es die Rollen klärt. Lass die Studierenden zu Beginn einen echten oder nachgebauten Befund lesen und in zwei Sätzen aufschreiben, was das Entwicklungsteam jetzt tun soll. Die Antworten werden weit auseinanderliegen, von „Eingabe filtern“ bis „Architektur umbauen“. Das zeigt ihnen besser als jede Folie, dass zwischen Befund und Behebung eine Übersetzung liegt, und es gibt dir einen Vergleichspunkt für das Ende der Lehrveranstaltung.

> Eine typische Fehlvorstellung ist, dass ein Pentest mit vielen Befunden ein schlechtes und einer mit wenigen ein gutes System anzeigt. Die Zahl der Befunde hängt stark von Testzeit, Vorwissen und Scope ab. Ein White-Box-Test über zehn Tage findet in einem guten System mehr als ein Black-Box-Test über zwei Tage in einem schlechten.

## Selbsttest

1. Nenne die vier Merkmale, die einen Penetrationstest definieren, und sage zu jedem, wovon es den Pentest abgrenzt.
2. Ein Dienstleister liefert einen „Pentest-Bericht“ mit 140 Befunden, alle mit generischem Text und ohne Nachweis. Was hast du vermutlich vor dir, und woran erkennst du es?
3. Warum findet ein Grey-Box-Test bei einer Web-Anwendung in der Regel mehr als ein Black-Box-Test gleicher Dauer?
4. Ein Bericht endet mit „keine kritischen Befunde“. Welche drei Einschränkungen musst du mitlesen?
5. Worin unterscheiden sich die Ziele eines Pentests und eines Red-Team-Einsatzes?
6. Beschreibe an einem Beispiel, wie ein Befund behoben werden kann, ohne dass seine Ursache verschwindet.

# 2 Auftrag, Scope und Recht

Technisch gesehen tut ein Penetrationstester dasselbe wie ein Angreifer. Der einzige Unterschied liegt auf Papier: in einem Auftrag, der festhält, wer was wann und womit testen darf. Dieses Papier entscheidet, ob eine Handlung eine Dienstleistung oder eine Straftat ist, und es entscheidet später, was die Befunde im Bericht bedeuten. Wer einen Bericht bewerten will, muss deshalb zuerst den Auftrag kennen. Dieses Kapitel erklärt den rechtlichen Rahmen in Österreich in Grundzügen, den Scope, die Rules of Engagement und die Fragen, die der Einsatz von KI-Werkzeugen im Test aufwirft.

**IN DIESEM KAPITEL**

- erklären, warum die Einwilligung des Berechtigten die Grundlage jedes Tests ist und wer sie geben kann
- die einschlägigen Tatbestände des österreichischen Strafrechts in Grundzügen benennen
- einen Scope lesen und sagen, was er für die Aussagekraft des Berichts bedeutet
- die Bestandteile von Rules of Engagement aufzählen und begründen
- die Grenzen für den Einsatz von KI-Werkzeugen im Test benennen

**ACHTUNG**

Dieses Kapitel ist keine Rechtsberatung. Es gibt dir das Vokabular und die Grundgedanken, damit du im Hörsaal die richtigen Warnungen aussprechen kannst. Für einen konkreten Auftrag gilt der Vertrag, den Juristen geprüft haben, und der aktuelle Gesetzestext im Rechtsinformationssystem des Bundes.

## 2.1 Dieselbe Handlung, zwei Bedeutungen

Stell dir zwei Personen vor, die an einem Dienstagabend dieselbe Anfrage an die Buchungsplattform *Kurbad* schicken: eine Anfrage, in der die Kundennummer in der Adresse um eins erhöht ist, um zu sehen, ob das System fremde Buchungen herausgibt. Die eine Person ist eine beauftragte Testerin, die andere ein neugieriger Kunde. Im Protokoll des Servers sehen die beiden Anfragen gleich aus. Rechtlich liegen Welten dazwischen.

Der Unterschied ist die **Einwilligung des Berechtigten**. Ein Computersystem gehört jemandem, und dieser Jemand darf bestimmen, wer darauf zugreift. Erlaubt er den Test, ist der Zugriff befugt. Erlaubt er ihn nicht, hilft es dem neugierigen Kunden nicht, dass er „nur schauen wollte“, nichts verändert hat und den Fehler anschließend sogar meldet. Gute Absicht ersetzt die Einwilligung nicht.

Das klingt selbstverständlich und ist trotzdem die Stelle, an der Studierende am häufigsten falsch liegen. Wer in einer Lehrveranstaltung Angriffstechniken lernt, will sie ausprobieren, und das nächstliegende Ziel ist ein echtes System: die Website des Sportvereins, das Portal der Hochschule, das Smart-Home-Gerät des Mitbewohners. Nichts davon ist erlaubt, solange der Berechtigte nicht zugestimmt hat.

**MERKE**

Getestet wird nur, wofür eine schriftliche Erlaubnis des Verfügungsberechtigten vorliegt. In der Lehre heißt das: ausschließlich in der bereitgestellten Laborumgebung.

## 2.2 Der Rechtsrahmen in Österreich

Das österreichische Strafgesetzbuch enthält eine Gruppe von Tatbeständen, die auf Handlungen an Computersystemen und Daten zugeschnitten sind [11]. Du musst sie nicht auswendig kennen, aber du solltest wissen, dass es sie gibt und was sie im Kern erfassen.

**Tab. 2:** Einschlägige Tatbestände des österreichischen Strafgesetzbuchs in Grundzügen (Stand: 12. November 2026). Maßgeblich ist der Gesetzestext.

| StGB | Bezeichnung | Was im Kern erfasst ist |
|---|---|---|
| § 118a | Widerrechtlicher Zugriff auf ein Computersystem | sich unbefugt Zugang zu einem fremden System verschaffen, indem man eine Sicherheitsvorkehrung überwindet |
| § 119 | Verletzung des Telekommunikationsgeheimnisses | Inhalte fremder Kommunikation mit technischen Mitteln ausspähen |
| § 119a | Missbräuchliches Abfangen von Daten | Daten abfangen, die über ein Computersystem übertragen werden und nicht für einen bestimmt sind |
| § 126a | Datenbeschädigung | fremde Daten verändern, löschen oder unbrauchbar machen |
| § 126b | Störung der Funktionsfähigkeit eines Computersystems | den Betrieb eines fremden Systems schwer stören |
| § 126c | Missbrauch von Computerprogrammen oder Zugangsdaten | Programme oder Zugangsdaten herstellen, beschaffen oder weitergeben, um damit die genannten Taten zu begehen |

Drei Beobachtungen zu Tab. 2 sind für die Lehre nützlich.

Erstens bilden die Tatbestände die Phasen eines Angriffs ab. Der Zugriff selbst, das Mitlesen von Datenverkehr, das Verändern von Daten und das Lahmlegen eines Systems sind je eigens erfasst. Ein vollständiger Pentest berührt fast alle davon. Ohne Einwilligung wäre er also nicht ein Delikt, sondern mehrere.

Zweitens zeigt § 126c, dass schon die Vorbereitung strafbar sein kann, wenn sie auf eine solche Tat gerichtet ist. Das ist der Hintergrund der häufigen Frage, ob der Besitz von Sicherheitswerkzeugen verboten sei. Die Werkzeuge eines Testers sind dieselben wie die eines Angreifers. Es kommt auf den Zweck an, zu dem jemand sie herstellt, beschafft oder weitergibt. Wer sie für beauftragte Tests und für die Lehre in einer Laborumgebung verwendet, verfolgt keinen solchen Zweck. Für dich als Lehrenden folgt daraus eine praktische Vorsicht: Übungsmaterial gehört in die Laborumgebung und wird so ausgegeben, dass sein Ausbildungszweck außer Frage steht.

Drittens steht in keinem dieser Tatbestände, dass ein Schaden eingetreten sein muss, damit der Zugriff als solcher verboten ist. Wie die einzelnen Merkmale auszulegen sind, etwa was als Sicherheitsvorkehrung gilt und welche Absichten das Gesetz verlangt, ist Sache der Juristen. Für die Praxis des Testens genügt die einfache Regel aus dem vorigen Abschnitt.

Neben dem Strafrecht stehen zwei weitere Rechtsgebiete. Das **Datenschutzrecht** ist berührt, sobald der Tester im Zuge des Tests personenbezogene Daten sieht, was bei einer Anwendung mit Kundendaten praktisch immer der Fall ist [12]. Der Auftrag muss deshalb regeln, wie der Tester mit solchen Daten umgeht: nur so weit ansehen, wie für den Nachweis nötig, nichts aus der Umgebung herauskopieren, im Bericht schwärzen. Das **Zivilrecht** regelt die Haftung, wenn beim Test etwas kaputtgeht. Auch das gehört in den Vertrag.

> FRAGE AUS DEM HÖRSAAL
>
> ***Wenn ich eine Lücke auf einer fremden Website finde und sie sofort melde, bin ich dann nicht auf der sicheren Seite?***
>
> Nein. Die Meldung ändert nichts daran, wie die Lücke gefunden wurde. Wer ohne Erlaubnis aktiv getestet hat, hat das getan, bevor er gemeldet hat. Ob jemand deswegen verfolgt wird, steht auf einem anderen Blatt, aber darauf sollte sich niemand verlassen. Anders liegt der Fall, wenn der Betreiber ein Programm zur Meldung von Schwachstellen veröffentlicht hat, das Tests in einem bestimmten Rahmen ausdrücklich erlaubt. Dann liegt eine Einwilligung vor, allerdings nur für das, was dort steht. Wer über etwas stolpert, ohne es gesucht zu haben, meldet es und probiert nicht weiter.

## 2.3 Wer einwilligen kann

Die Einwilligung muss von dem kommen, der über das System verfügen darf. Bei einem Server im eigenen Keller ist das einfach. Bei heutigen Anwendungen ist es das nicht, weil kaum ein System einem allein gehört.

Die Anwendung *Kurbad* läuft bei einem Cloud-Anbieter. Die Anmeldung übernimmt ein Identitätsdienst, den ein anderes Unternehmen betreibt. Zahlungen wickelt ein Zahlungsdienstleister ab. Die App wird über die Stores zweier Konzerne verteilt. Der Thermenbetreiber kann in Tests seiner eigenen Anwendung einwilligen, aber nicht in Tests der Infrastruktur des Cloud-Anbieters, der Systeme des Zahlungsdienstleisters oder des Identitätsdienstes. Für jedes dieser fremden Systeme gelten die Bedingungen des jeweiligen Betreibers. Große Cloud-Anbieter veröffentlichen Regeln, welche Tests Kunden an ihren eigenen Ressourcen ohne Rückfrage durchführen dürfen und welche nicht. Zahlungsdienstleister stellen in der Regel eigene Testumgebungen bereit und verbieten Tests gegen die Produktion.

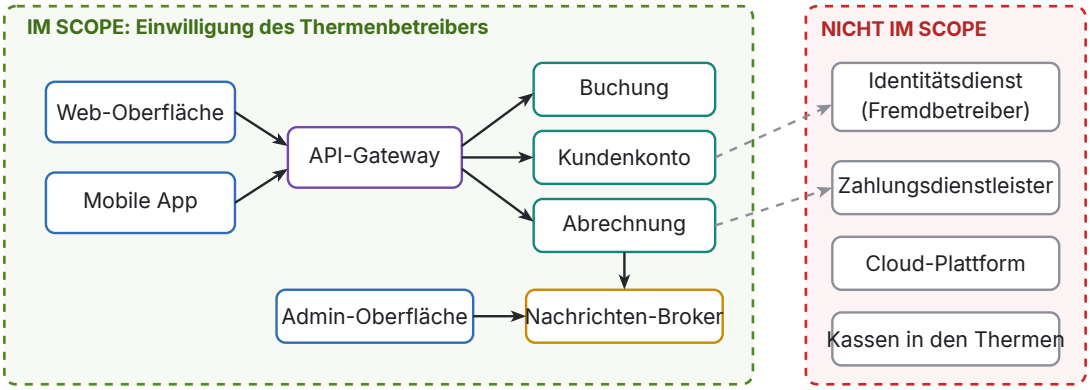



**Abb. 4:** Scope des Beispielsystems *Kurbad*. Die gestrichelten Pfeile sind Schnittstellen zu Systemen, in deren Test der Auftraggeber nicht einwilligen kann.

Abb. 4 zeigt das Ergebnis für das Beispielsystem. Die Grenze ist nicht nur eine juristische Formalität. Sie hat eine fachliche Folge, die beim Lesen des Berichts wichtig wird: Die Schnittstellen zu den ausgeschlossenen Systemen werden nur von der eigenen Seite aus geprüft. Ob *Kurbad* die Antworten des Zahlungsdienstleisters richtig prüft, ist im Scope. Ob der Zahlungsdienstleister selbst sicher ist, nicht.

## 2.4 Der Scope

Der Scope legt fest, was getestet wird. Er ist das Ergebnis einer Verhandlung zwischen dem, was der Auftraggeber wissen will, dem, was er bezahlen kann, und dem, worin er einwilligen darf. Ein brauchbarer Scope beantwortet fünf Fragen.

**Welche Ziele?** Genannt werden Anwendungen, Adressen, Schnittstellen, mobile Apps mit Version. „Die Website“ ist kein Scope. „Die Web-Oberfläche unter der Testadresse, die dahinterliegende API in Version 3 und die Admin-Oberfläche“ ist einer.

**Welche Umgebung?** Getestet wird entweder die Produktion oder eine Testumgebung. Die Testumgebung ist risikoärmer, weil ein Ausfall niemanden trifft und keine echten Kundendaten betroffen sind. Dafür stimmt sie selten genau mit der Produktion überein: andere Konfiguration, andere Datenmengen, manchmal abgeschaltete Schutzmechanismen. Ein Befund in der Testumgebung muss deshalb auf die Produktion übertragen werden, und ein Fehler, der nur in der Konfiguration der Produktion steckt, wird nicht gefunden.

**Welche Rollen und welches Vorwissen?** Hier steht, ob Black, Grey oder White Box getestet wird und welche Benutzerkonten der Tester bekommt. Für eine Anwendung mit Kunden, Mitarbeitern und Administratoren braucht er mindestens zwei Konten je Rolle, sonst kann er nicht prüfen, ob ein Kunde auf die Daten eines anderen Kunden zugreifen kann.

**Welche Tiefe?** Wie viele Testtage stehen zur Verfügung, und soll der Tester nach dem ersten Erfolg weitergehen? Darf er von einem übernommenen Service aus das interne Netz erkunden, oder endet der Test an der Anwendung?

**Was ist ausdrücklich ausgeschlossen?** Systeme Dritter, bestimmte Funktionen wie der echte Zahlungsvorgang, bestimmte Techniken.

**Tab. 3:** Ein Scope in fünf Punkten.

| Punkt | Festlegung für *Kurbad* |
|---|---|
| Ziele | Web-Oberfläche, mobile App (iOS und Android, aktuelle Version), API-Gateway, Services Buchung, Kundenkonto und Abrechnung, Admin-Oberfläche |
| Umgebung | Testumgebung, die der Produktion nachgebaut ist, mit erzeugten Testdaten |
| Vorwissen | Grey Box: je zwei Konten für Kunde, Kassenkraft und Administrator, dazu die Beschreibung der API |
| Tiefe | acht Testtage, nach Übernahme eines Service kein Vordringen in die Cloud-Plattform |
| Ausgeschlossen | Identitätsdienst, Zahlungsdienstleister, Cloud-Plattform, Kassen in den Thermen, Lasttests, Täuschung von Mitarbeitern |

Der Scope ist das Erste, was du in einem Bericht suchen solltest, denn er begrenzt jede Aussage darin. Drei Fehlschlüsse entstehen, wenn man ihn überliest. Aus „in der Admin-Oberfläche wurde nichts gefunden“ wird „die Admin-Oberfläche ist sicher“, obwohl sie in acht Testtagen nur einen halben Tag bekommen hat. Aus „kein Befund zu Überlastung“ wird „das System hält Last aus“, obwohl Lasttests ausgeschlossen waren. Und aus einem Befund in der Testumgebung wird ohne Prüfung ein Befund in der Produktion, obwohl dort eine vorgeschaltete Schutzschicht läuft, die in der Testumgebung fehlt.

**BRÜCKE ZU SOFTWARE SECURITY**
Der Scope ist eine Architekturaussage. Wer ihn festlegt, zieht Grenzen durch das System, und er zieht sie gut oder schlecht, je nachdem, ob er die Architektur kennt. In einem C4-Container-Diagramm, wie es deine Studierenden in Software Security lesen, lässt sich der Scope direkt einzeichnen: welche Container dazugehören und welche Beziehungen die Grenze überqueren. Genau an diesen Beziehungen sitzen später häufig die interessanten Befunde.

## 2.5 Rules of Engagement

Der Scope sagt, *was* getestet wird. Die Rules of Engagement, kurz RoE, sagen, *wie*. Der Begriff stammt aus dem Militärischen und bezeichnet dort die Einsatzregeln. In einem Pentest sind es die Spielregeln, auf die sich Auftraggeber und Tester vor Beginn einigen. Der Standard PTES widmet dieser Vorbereitung eine eigene Phase, die *Pre-engagement Interactions* [13], und auch NIST verlangt in der Planungsphase klare Regeln und eine dokumentierte Genehmigung [1].

**Tab. 4:** Übliche Bestandteile von Rules of Engagement.

| Regel | Was festgelegt wird | Warum |
|---|---|---|
| Zeitfenster | Tage und Uhrzeiten, in denen getestet wird | damit der Betrieb Alarme zuordnen kann und im Störfall jemand da ist |
| Herkunft des Tests | Adressen, von denen der Tester arbeitet | damit Testverkehr von echten Angriffen unterscheidbar ist |
| Erlaubte Techniken | zum Beispiel keine Überlastung, keine Täuschung von Mitarbeitern, kein physischer Zutritt | damit der Test keinen Schaden anrichtet, den niemand wollte |
| Umgang mit Daten | ansehen nur so weit nötig, nichts mitnehmen, Belege schwärzen | Datenschutz und Vertraulichkeit |
| Meldewege | Ansprechpartner auf beiden Seiten, Erreichbarkeit im Notfall | damit ein Problem in Minuten und nicht in Tagen ankommt |
| Sofortmeldung | kritische Funde werden gleich gemeldet, nicht erst im Bericht | eine offene kritische Lücke soll nicht zwei Wochen auf den Bericht warten |
| Abbruch | Bedingungen, unter denen der Test angehalten wird | zum Beispiel wenn ein System instabil wird oder Spuren eines echten Angriffs auftauchen |
| Aufräumen | Testkonten, abgelegte Dateien und geänderte Einstellungen werden entfernt und dokumentiert | damit der Test selbst keine Schwäche hinterlässt |

Zwei Punkte aus Tab. 4 werden oft unterschätzt.

Die **Sofortmeldung** ändert, wie du einen Bericht liest. Steht bei einem kritischen Befund, dass er am dritten Testtag gemeldet und am vierten behoben wurde, dann beschreibt der Bericht an dieser Stelle einen Zustand, den es nicht mehr gibt. Ein guter Bericht vermerkt das und hält fest, ob die Behebung nachgetestet wurde.

Das **Aufräumen** ist die Regel, deren Verletzung am peinlichsten ist. Ein Testkonto mit Administratorrechten und einfachem Passwort, das nach dem Test vergessen wird, ist eine echte Schwäche, die der Pentest selbst erzeugt hat. Dasselbe gilt für Dateien, die der Tester zum Nachweis abgelegt hat. Seriöse Berichte enthalten deshalb eine Liste aller Veränderungen am System.

Ein dritter Punkt steht nicht in jeder Tabelle, gehört aber in jede Vereinbarung: Der Auftraggeber bestätigt schriftlich, dass er berechtigt ist, den Test zu beauftragen. Dieses Schreiben trägt in der Branche den etwas flapsigen Namen *Get out of jail free letter*. Es schützt den Tester, falls sich später herausstellt, dass ein System doch einem anderen gehört hat.

## 2.6 Was dabei schiefgehen kann

Die Regeln wirken bürokratisch, bis man die Fälle kennt, die sie verhindern sollen. Vier Muster kommen in der Praxis immer wieder vor.

**Das falsche Ziel.** Eine Adresse im Scope enthält einen Tippfehler oder gehört inzwischen jemand anderem, weil die Cloud-Ressource freigegeben und neu vergeben wurde. Der Tester greift ein fremdes System an. Deshalb prüfen sorgfältige Tester vor Beginn, ob die genannten Ziele tatsächlich dem Auftraggeber zuzuordnen sind.

**Die Nebenwirkung.** Ein Test füllt ein Formular tausendfach aus, und jede Einsendung löst eine E-Mail an den Kundendienst aus. Oder ein Test in der Testumgebung löst über eine vergessene Verbindung echte Buchungen im Produktivsystem eines Partners aus. Solche Verbindungen kennt nur, wer die Architektur kennt. Das ist ein weiteres Argument dafür, den Scope mit einem Architekturdiagramm in der Hand festzulegen.

**Der Alarm.** Die Überwachung im Betrieb bemerkt den Test und behandelt ihn als echten Angriff: Konten werden gesperrt, Adressen blockiert, im schlimmsten Fall wird ein Notfallplan ausgelöst. Deshalb werden Zeitfenster und Herkunftsadressen vorab mitgeteilt. Ausnahme ist der Red-Team-Einsatz aus Kapitel 1, bei dem die Reaktion der Verteidigung gerade der Prüfgegenstand ist und nur ein kleiner Kreis eingeweiht wird.

**Der echte Angreifer.** Der Tester stößt auf Spuren, die nicht von ihm stammen: eine fremde Hintertür, auffällige Konten, abgelegte Dateien. Dann endet der Test und beginnt die Behandlung eines Sicherheitsvorfalls. Die RoE müssen sagen, wen der Tester in diesem Fall sofort anruft.

## 2.7 KI-Werkzeuge im Test

Seit Tester KI-Assistenten einsetzen, hat die Vereinbarung einen neuen Abschnitt. Die Werkzeuge helfen an vielen Stellen: Sie werten die Ergebnisse der Aufklärung aus, schlagen Varianten einer Eingabe vor, erklären unbekannte Antworten eines Systems und formulieren Befunde. Dabei entstehen drei Fragen, die der Auftrag beantworten muss.

**Wohin gehen die Daten?** Wer eine Serverantwort mit Kundendaten oder einen Ausschnitt der internen API-Beschreibung in einen KI-Dienst im Internet kopiert, hat vertrauliche Daten des Auftraggebers an einen Dritten weitergegeben. Das kann gegen die Vertraulichkeitsvereinbarung und gegen das Datenschutzrecht verstoßen. Der Auftrag muss festlegen, ob und welche KI-Dienste verwendet werden dürfen und ob nur solche in Frage kommen, die der Tester selbst betreibt oder die vertraglich zusichern, die Eingaben nicht weiterzuverwenden.

**Wer hält den Scope ein?** Ein KI-Agent, der selbstständig Werkzeuge aufruft, kennt die Grenzen aus Abb. 4 nur, wenn man sie ihm gibt, und hält sie nur ein, wenn man es technisch erzwingt. Folgt der Agent einem Verweis auf das System des Zahlungsdienstleisters, ist das Verlassen des Scope nicht dem Werkzeug anzulasten, sondern dem Tester, der es geführt hat. Autonome Werkzeuge brauchen deshalb harte Begrenzungen, etwa eine Liste erlaubter Ziele auf Netzwerkebene, und einen Menschen, der eingreift.

**Wer steht für das Ergebnis ein?** Ein KI-Werkzeug meldet auch Schwächen, die es nicht gibt, und beschreibt sie überzeugend. Ein Befund im Bericht muss vom Tester selbst nachvollzogen sein. Der Auftraggeber kann das verlangen, und er sollte es tun.

BRÜCKE ZU SOFTWARE SECURITY

Diese drei Fragen sind die Praxisform der Lernergebnisse, die im neuen Studienplan unter „Human-AI-Agency" stehen. „Steward" heißt, den Einsatz von KI innerhalb rechtlicher, vertraglicher und ethischer Grenzen zu halten. „Validate" heißt, KI-Ergebnisse gegen überprüfbare Kriterien zu prüfen. In der Case Study der Spezialisierung arbeiten die Studierenden mit KI an einem Pentest-Bericht. Sie haften für jede falsche Aussage, die sie daraus übernehmen, so wie ein Tester für jeden Befund haftet, den er in den Bericht schreibt.

## 2.8 Vertraulichkeit des Ergebnisses

Ein Pentest-Bericht ist eine Anleitung, wie man in das System eines Unternehmens eindringt, mit Belegen, dass es funktioniert. Entsprechend wird er behandelt. Er geht an einen benannten Kreis von Empfängern, wird verschlüsselt übertragen und nicht in allgemein zugänglichen Ablagen gespeichert. Der Tester bewahrt seine Arbeitsdaten nur so lange auf, wie vereinbart, und löscht sie danach.

Für die Lehre hat das eine unmittelbare Folge: Echte Berichte aus der Praxis kannst du nicht einfach als Unterrichtsmaterial verwenden, auch nicht geschwärzt, solange der Auftraggeber nicht zugestimmt hat. Es gibt öffentlich verfügbare Berichte, die Unternehmen und Open-Source-Projekte freiwillig veröffentlicht haben, und es gibt die Möglichkeit, Berichte für ein erfundenes System selbst zu erzeugen. Für die Case Study ist der zweite Weg der bessere, weil du dann den Lösungsschlüssel kennst. Kapitel 19 beschreibt, wie das geht.

FRAGE AUS DEM HÖRSAAL

***Was passiert, wenn der Tester im Scope etwas findet, das auf eine Straftat des Auftraggebers hindeutet?***

Das ist selten, kommt aber vor, und es ist heikel, weil der Tester zur Vertraulichkeit verpflichtet ist. Seriöse Verträge enthalten dafür eine Klausel. Im Hörsaal reicht die Antwort, dass der Tester den Test an dieser Stelle anhält, den Fund dokumentiert, nichts weiter ansieht und sich rechtlich beraten lässt, bevor er irgendetwas unternimmt. Die Frage zeigt gut, dass Pentesting ein Vertrauensberuf ist.

FÜR DIE LEHRE

Lass die Studierenden zu Beginn der Lehrveranstaltung eine Verpflichtung unterschreiben, dass sie die gelernten Techniken ausschließlich in der Laborumgebung anwenden. Das ist mehr als eine Formalität. Es gibt dir Gelegenheit, den Unterschied zwischen befugtem und unbefugtem Zugriff einmal ausdrücklich zu machen, und es dokumentiert den Ausbildungszweck.

Eine gute Übung zu diesem Kapitel: Gib den Studierenden das Container-Diagramm eines Systems und die Liste der Betreiber seiner Teile. Sie sollen den Scope einzeichnen, die Schnittstellen markieren, die die Grenze überqueren, und drei Regeln für die RoE formulieren, die sich aus genau dieser Architektur ergeben. Wer hier die Verbindung zum Partnersystem übersieht, hat verstanden, warum Architekturwissen beim Testen zählt.

Die häufigste Fehlvorstellung ist, dass ein Test „nur lesend" und deshalb harmlos sei. Schon das Lesen fremder Daten ohne Befugnis ist das Problem, und viele scheinbar lesende Anfragen lösen im System Wirkungen aus.

## Selbsttest

1. Was unterscheidet die Anfrage einer beauftragten Testerin von der identischen Anfrage eines neugierigen Kunden?
2. Warum kann der Betreiber von *Kurbad* nicht in einen Test des Zahlungsdienstleisters einwilligen, und was bedeutet das für die Prüfung der Schnittstelle?
3. Nenne die fünf Fragen, die ein brauchbarer Scope beantwortet.

4. Ein Bericht enthält keinen Befund zur Verfügbarkeit. Was musst du prüfen, bevor du daraus etwas schließt?
5. Warum gehört das Aufräumen in die Rules of Engagement?
6. Ein Tester kopiert eine Serverantwort mit echten Kundendaten in einen öffentlichen KI-Dienst, um sie erklären zu lassen. Welche Probleme entstehen?
7. Welche zwei Nachteile hat ein Test in der Testumgebung gegenüber einem Test in der Produktion?

# 3 Methodiken: wie ein Test abläuft

Zwei Tester, dasselbe System, dieselbe Woche: Ohne gemeinsame Methodik prüft die eine vor allem die Anmeldung und der andere vor allem die Eingabefelder, und beide liefern einen Bericht, der vollständig aussieht. Methodiken gibt es, damit ein Test nicht vom Geschmack des Testers abhängt. Sie legen fest, in welchen Schritten vorgegangen wird und welche Bereiche zu prüfen sind. Für dich sind sie aus einem zweiten Grund wichtig: Berichte verweisen auf sie. Wer in einem Befund „WSTG-ATHZ-04“ liest oder im Angebot „Vorgehen nach PTES“, sollte wissen, was dahintersteht und was er daraus ableiten darf.

**IN DIESEM KAPITEL**

- begründen, wozu ein Pentest eine Methodik braucht
- die Phasen von PTES und NIST SP 800-115 nennen und einander zuordnen
- den Aufbau des OWASP Web Security Testing Guide beschreiben und eine Test-ID lesen
- den Unterschied zwischen einem Testleitfaden (WSTG) und einem Anforderungskatalog (ASVS) erklären
- aus der Methodik ableiten, welche Angaben zur Abdeckung ein Bericht enthalten muss

## 3.1 Wozu eine Methodik

Ein Softwaretest ohne Testplan ist Herumprobieren. Dein Foliensatz zu Software Testing beschreibt den generischen Testprozess mit Planung, Analyse, Entwurf, Realisierung, Durchführung und Abschluss [7]. Eine Pentest-Methodik ist dasselbe für Sicherheitstests, und sie leistet vier Dinge.

**Abdeckung.** Die Methodik zählt auf, was zu prüfen ist. Ohne sie prüft jeder, was er am besten kann. Mit ihr lässt sich am Ende sagen, welche Bereiche angesehen wurden und welche nicht.

**Wiederholbarkeit.** Ein Befund, den niemand nachvollziehen kann, ist für die Behebung wertlos und lässt sich nach der Behebung nicht nachtesten. Die Methodik verlangt, dass jeder Schritt dokumentiert ist.

**Vergleichbarkeit.** Wird ein System jedes Jahr von einem anderen Dienstleister getestet, lassen sich die Berichte nur vergleichen, wenn alle nach einem ähnlichen Raster gearbeitet haben.

**Nachweis.** Gegenüber Kunden, Prüfern und Aufsicht zählt nicht nur, dass getestet wurde, sondern wie. „Nach anerkanntem Standard“ ist eine Aussage, die sich überprüfen lässt.

Keine Methodik ersetzt das Urteil des Testers. Sie gibt den Rahmen vor, in dem er seine Zeit einteilt. Die folgenden Abschnitte stellen die drei Methodiken vor, denen du in Berichten und Angeboten am häufigsten begegnest, und zwei weitere in Kurzform.

## 3.2 PTES: sieben Phasen

Der *Penetration Testing Execution Standard*, kurz PTES, wurde ab 2009 von Praktikern geschrieben, die sich über wertlose Tests ärgerten [13]. Er ist kein Standard im Sinn einer Normungsorganisation, sondern eine öffentlich verfügbare Sammlung von Leitlinien. Seine Wirkung liegt in der Gliederung: Die sieben Phasen sind zum allgemeinen Sprachgebrauch der Branche geworden.

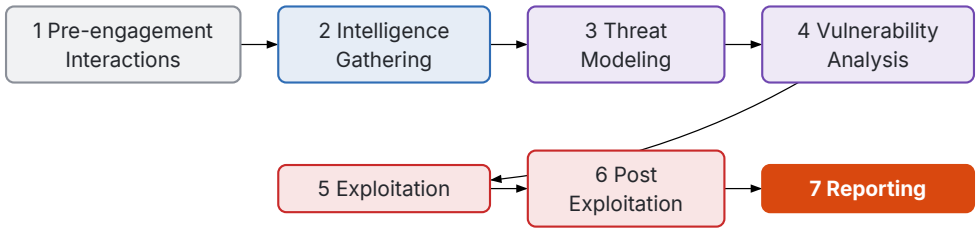



**Abb. 5:** Die sieben Phasen des PTES.

**Phase 1, Pre-engagement Interactions.** Alles, was Kapitel 2 beschrieben hat: Ziele, Scope, Rules of Engagement, Zeitplan, Ansprechpartner, Vertrag. PTES verwendet darauf viel Platz, weil ein schlecht vereinbarter Test auch bei bester Technik ein schlechter Test wird.

**Phase 2, Intelligence Gathering.** Die Aufklärung. Der Tester sammelt, was über das Ziel herauszufinden ist: welche Systeme und Adressen dazugehören, welche Technologien im Einsatz sind, welche Funktionen und Rollen es gibt, und was sich aus öffentlichen Quellen über die Organisation erfahren lässt. Kapitel 7 behandelt das näher.

**Phase 3, Threat Modeling.** Die Bedrohungsmodellierung. Der Tester überlegt, was in diesem System schützenswert ist, wer es angreifen würde und über welche Wege. Daraus leitet er ab, wo er seine begrenzte Zeit einsetzt. Diese Phase wird in der Praxis am häufigsten übersprungen, und sie ist zugleich die, in der Architekturwissen den größten Unterschied macht. Kapitel 6 erklärt die Technik dahinter.

**Phase 4, Vulnerability Analysis.** Die Suche nach Schwächen. Der Tester prüft die gefundenen Systeme und Funktionen, teils mit Werkzeugen, teils von Hand, und sammelt Verdachtsfälle. Am Ende dieser Phase steht eine Liste von Stellen, die schwach sein könnten.

**Phase 5, Exploitation.** Das Ausnutzen. Der Tester prüft, welche Verdachtsfälle sich bestätigen, indem er die Schwäche tatsächlich ausnutzt. Erst hier wird aus dem Verdacht ein Befund. PTES betont, dass dies gezielt und mit Rücksicht auf die Stabilität des Systems geschehen soll.

**Phase 6, Post Exploitation.** Was nach dem ersten Erfolg kommt. Der Tester bestimmt den Wert des übernommenen Systems: An welche Daten kommt er von hier, kann er seine Rechte ausweiten, kann er von hier aus weitere Systeme erreichen? Diese Phase beantwortet die Frage nach der Auswirkung, die für die Bewertung eines Befunds entscheidend ist. Kapitel 11 geht darauf ein.

**Phase 7, Reporting.** Der Bericht, getrennt in eine Zusammenfassung für die Leitung und einen technischen Teil. Kapitel 13 behandelt ihn ausführlich.

Die Reihenfolge ist auf dem Papier linear und in der Wirklichkeit ein Kreislauf. Jeder Erfolg in Phase 5 öffnet eine neue Sicht auf das System, und mit ihr beginnen Aufklärung und Analyse von vorne. Abb. 3 in Kapitel 1 hat diesen Rückpfeil schon gezeigt.

> **MERKE**
> Wenn ein Angebot „Vorgehen nach PTES“ verspricht, darfst du im Bericht zu jeder der sieben Phasen ein Ergebnis erwarten: einen dokumentierten Scope, die Ergebnisse der Aufklärung, die angenommenen Bedrohungen, die geprüften Bereiche, die bestätigten Schwächen, eine Aussage zur Auswirkung und den Bericht selbst.

## 3.3 NIST SP 800-115: vier Phasen

Das US-amerikanische Normungsinstitut NIST hat 2008 mit der Special Publication 800-115 einen technischen Leitfaden für Sicherheitstests veröffentlicht [1]. Er ist älter und knapper als PTES, wird aber bis heute zitiert, vor allem dort, wo Behörden und regulierte Unternehmen einen Nachweis nach einem amtlichen Dokument brauchen.

NIST ordnet zunächst die Prüftechniken in drei Gruppen. **Review-Techniken** sind passiv: Dokumente, Konfigurationen, Protokolle und Regelwerke werden durchgesehen, ohne das System anzugreifen. **Techniken zur Identifikation und Analyse von Zielen** finden heraus, welche Systeme, offenen Ports und Dienste es gibt und wo Schwächen liegen könnten. **Techniken zur Validierung von Schwachstellen**

bestätigen den Verdacht, und hierher gehört der Penetrationstest. Diese Dreiteilung ist nützlich, weil sie den Pentest als eine von mehreren Techniken einordnet und nicht als die einzige.

Für den Pentest selbst beschreibt NIST vier Phasen.

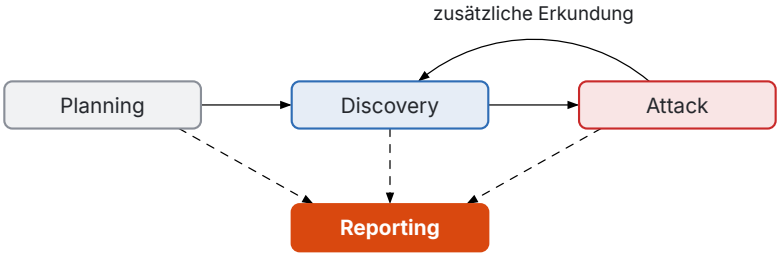



**Abb. 6:** Die vier Phasen nach NIST SP 800-115. Berichtet wird aus jeder Phase, und die Angriffsphase führt zurück in die Erkundung.

**Planning** entspricht der Vorbereitung. **Discovery** fasst Aufklärung und Schwachstellenanalyse zusammen. **Attack** ist das Ausnutzen, und NIST zerlegt diese Phase in vier Schritte, die du als Begriffe kennen solltest, weil sie in Berichten auftauchen: Zugang erlangen (*gaining access*), Rechte ausweiten (*escalating privileges*), das System erkunden (*system browsing*) und weitere Werkzeuge nachladen (*installing additional tools*). **Reporting** läuft bei NIST parallel zu den anderen Phasen mit.

Der bemerkenswerteste Unterschied zu PTES ist dieser letzte Punkt. Nach NIST entsteht der Bericht nicht am Ende, sondern laufend. Das ist die methodische Grundlage für die Sofortmeldung kritischer Funde aus Kapitel 2.

## 3.4 OWASP WSTG: der Prüfkatalog für Web-Anwendungen

PTES und NIST beschreiben den Ablauf. Sie sagen wenig darüber, was an einer Web-Anwendung im Einzelnen zu prüfen ist. Diese Lücke füllt der *Web Security Testing Guide* der OWASP, kurz WSTG.

> **DEFINITION 3.1 · OWASP**
> Das *Open Worldwide Application Security Project* ist eine gemeinnützige Stiftung, unter deren Dach Freiwillige frei verfügbare Kataloge, Leitfäden und Werkzeuge zur Anwendungssicherheit erarbeiten. Aus ihr stammen unter anderem die OWASP Top 10, der WSTG, der ASVS und das Reifegradmodell SAMM. OWASP-Dokumente sind keine Normen. Ihre Verbindlichkeit entsteht dadurch, dass Verträge, Ausschreibungen und Regelwerke auf sie verweisen.

Der WSTG liegt in der stabilen Version 4.2 vor, an Version 5 wird gearbeitet (Stand: 12. November 2026) [14]. Sein Kern ist Kapitel 4, ein Katalog von rund hundert einzelnen Tests in zwölf Kategorien. Jeder Test hat eine Kennung, eine Beschreibung, Testziele, eine Anleitung zur Durchführung und Hinweise zur Behebung.

**Tab. 5:** Die zwölf Testkategorien des WSTG 4.2 [14].

| Kennung | Kategorie | Leitfrage |
|---|---|---|
| WSTG-INFO | Information Gathering | Was verrät die Anwendung über sich? |
| WSTG-CONF | Configuration and Deployment Management | Sind Server und Plattform sicher eingerichtet? |
| WSTG-IDNT | Identity Management | Wie entstehen Konten und Rollen? |
| WSTG-ATHN | Authentication | Lässt sich die Anmeldung umgehen oder schwächen? |
| WSTG-ATHZ | Authorization | Kommt ein Benutzer an Daten und Funktionen, die ihm nicht zustehen? |
| WSTG-SESS | Session Management | Lässt sich eine Sitzung stehlen oder fälschen? |
| WSTG-INPV | Input Validation | Was passiert mit Eingaben, die das System nicht erwartet? |
| WSTG-ERRH | Error Handling | Was verraten Fehlermeldungen? |
| WSTG-CRYP | Weak Cryptography | Sind Transport und gespeicherte Daten wirksam geschützt? |
| WSTG-BUSL | Business Logic | Lässt sich die Fachlogik gegen ihren Zweck verwenden? |
| WSTG-CLNT | Client-side | Was lässt sich im Browser des Opfers anstellen? |
| WSTG-APIT | API Testing | Sind die Programmierschnittstellen eigens geprüft? |

Eine Test-ID setzt sich aus der Kategorie und einer laufenden Nummer zusammen. `WSTG-ATHZ-04` ist der vierte Test der Kategorie Authorization, er prüft auf unsichere direkte Objektreferenzen, also darauf, ob jemand durch Ändern einer Kennung in der Anfrage an fremde Datensätze kommt. Steht diese ID in einem Befund, weißt du, nach welchem Prüfschritt die Schwäche gefunden wurde, und kannst im WSTG nachlesen, wie der Test aussieht und welche Behebung dort empfohlen wird.

Der größere Nutzen des Katalogs liegt aber nicht beim einzelnen Befund, sondern bei dem, was *nicht* im Bericht steht. Ein Bericht, der seine Tests den WSTG-Kategorien zuordnet, kann eine Abdeckungsmatrix enthalten: für jede Kategorie die Angabe, ob sie geprüft wurde, mit welchem Ergebnis, und wenn nicht, warum nicht. Erst damit wird aus „kein Befund“ eine Aussage. Kapitel 13 kommt darauf zurück.

Neben dem WSTG gibt es für mobile Apps ein Gegenstück, den *Mobile Application Security Testing Guide*, und für Programmierschnittstellen die eigene Liste der häufigsten Risiken, die Kapitel 9 vorstellt.

> **IN DER REIHE**
> Wie man die Autorisierung einer API als Tabelle testet und wie ein dynamischer Scan in der Pipeline läuft, steht in *Secure API Design*, Kapitel 11. Dort geht es um Tests, die das Entwicklungsteam selbst automatisiert. Hier geht es um den Katalog, nach dem ein externer Tester vorgeht.

## 3.5 WSTG und ASVS: Test und Anforderung

Mit dem WSTG wird oft der *Application Security Verification Standard* verwechselt, kurz ASVS, ebenfalls von OWASP. Der Unterschied ist für die Verzahnung von Entwicklung und Test grundlegend.

Der ASVS ist ein Katalog von **Anforderungen**. Er sagt, was eine sichere Anwendung erfüllen muss, zum Beispiel dass die Anwendung bei jedem Zugriff auf einen Datensatz prüft, ob der Benutzer dazu berechtigt ist. Die Anforderungen sind in drei Stufen gestaffelt, von einem Grundschutz bis zu Anwendungen mit sehr hohem Schutzbedarf. Version 5.0 ist 2025 erschienen [15]. Der WSTG ist ein Katalog von **Tests**. Er sagt, wie man prüft, ob eine Anwendung eine Schwäche hat.

Die beiden verhalten sich zueinander wie Spezifikation und Testfall. Der ASVS gehört an den Anfang eines Projekts: Aus ihm wählt das Team die Sicherheitsanforderungen aus, die für dieses System gelten sollen. Der WSTG gehört ans Ende: Mit ihm wird geprüft. Wo beides zusammenkommt, entsteht Rückverfolgbarkeit: Ein Befund verletzt eine benannte Anforderung, die zu Projektbeginn vereinbart war. Dann streitet niemand mehr, ob der Befund „wirklich“ ein Fehler ist.

**BRÜCKE ZU SOFTWARE SECURITY**
In deinem Foliensatz zu den Prozessmodellen steht das V-Modell: Jeder Spezifikationsebene links steht eine Testebene rechts gegenüber. ASVS und WSTG sind genau dieses Paar für die Sicherheit. Die Anforderung aus dem ASVS ist die linke Seite, der Test aus dem WSTG die rechte. Ein Pentest ohne vorher festgelegte Sicherheitsanforderungen ist wie ein Abnahmetest ohne Pflichtenheft: Er findet etwas, aber niemand kann sagen, wogegen geprüft wurde.

## 3.6 Weitere Methodiken in Kurzform

Drei weitere Namen solltest du einordnen können.

Das **OSSTMM**, das *Open Source Security Testing Methodology Manual* des Instituts ISECOM, ist der Versuch, Sicherheitstests messbar zu machen [16]. Es teilt die Prüfung in Kanäle ein, nämlich Mensch, physischer Zugang, Funk, Telekommunikation und Datennetze, und definiert eine eigene Kennzahl für die Angriffsfläche. In Berichten zu Web-Anwendungen begegnet es dir selten, in Ausschreibungen gelegentlich.

Die **BSI-Studie** „Durchführungskonzept für Penetrationstests" des deutschen Bundesamts für Sicherheit in der Informationstechnik ist im deutschsprachigen Raum verbreitet [17]. Sie ist alt, aber ihre Klassifikation ist bis heute brauchbar: Ein Test wird nach sechs Kriterien eingeordnet, nämlich Informationsbasis (Black oder White Box), Aggressivität, Umfang, Vorgehensweise (verdeckt oder offensichtlich), Technik und Ausgangspunkt (von außen oder von innen). Wenn in einem deutschsprachigen Angebot „Aggressivität: abwägend" steht, stammt das aus dieser Studie.

**TIBER-EU** ist das Rahmenwerk der Europäischen Zentralbank für bedrohungsgeleitete Tests im Finanzsektor [3]. Es schreibt vor, dass ein eigener Dienstleister zuerst ein Bedrohungsbild für das konkrete Institut erstellt und das Red Team danach genau diese Bedrohungen nachstellt. Für die Anwendungssicherheit im engeren Sinn ist es weniger wichtig als für das Verständnis, wohin sich regulierte Tests entwickeln.

**Tab. 6:** Methodiken im Vergleich.

| Methodik | Herausgeber | Stärke | Wo du ihr begegnest |
|---|---|---|---|
| PTES | Praktiker-Initiative | vollständiger Ablauf in sieben Phasen | Angebote, Gliederung von Berichten |
| NIST SP 800-115 | NIST | amtliches Dokument, ordnet den Pentest unter andere Techniken ein | regulierte Branchen, Behörden |
| OWASP WSTG | OWASP | konkreter Prüfkatalog für Web-Anwendungen mit Test-IDs | Befunde und Abdeckungsmatrix |
| OSSTMM | ISECOM | Messbarkeit, deckt auch Mensch und Gebäude ab | Ausschreibungen |
| BSI-Studie | BSI | Klassifikation von Tests nach sechs Kriterien | deutschsprachige Angebote |
| TIBER-EU | EZB | bedrohungsgeleitetes Red Teaming | Finanzsektor, DORA |

## 3.7 Was alle gemeinsam haben

Legt man die Methodiken nebeneinander, bleibt ein gemeinsamer Kern von fünf Abschnitten, derselbe wie in Kapitel 1. Tab. 7 ordnet die Phasen zu.

**Tab. 7:** Zuordnung der Phasen zum gemeinsamen Kern.

| Abschnitt | PTES | NIST SP 800-115 | Ergebnis für den Empfänger |
|---|---|---|---|
| Vorbereitung | Pre-engagement Interactions | Planning | Scope, Rules of Engagement |
| Aufklärung | Intelligence Gathering | Discovery | Bestandsaufnahme: was es gibt |
| Analyse | Threat Modeling, Vulnerability Analysis | Discovery | geprüfte Bereiche, Verdachtsfälle |
| Ausnutzung | Exploitation, Post Exploitation | Attack | bestätigte Befunde mit Auswirkung |
| Bericht | Reporting | Reporting | Bericht mit Bewertung und Empfehlungen |

Die rechte Spalte ist die für dich wichtige. Jeder Abschnitt hinterlässt ein Ergebnis, das im Bericht stehen sollte, und nicht nur der vierte. Die Bestandsaufnahme aus der Aufklärung ist für den Auftraggeber oft schon für sich wertvoll, weil sie zeigt, was von außen sichtbar ist, darunter regelmäßig Systeme, von denen niemand mehr wusste. Die Liste der geprüften Bereiche ist die Grundlage für jede Aussage über die Abdeckung.

## 3.8 Zeit ist die knappe Größe

Keine Methodik sagt, wie viel Zeit jede Phase bekommt, und doch entscheidet genau das über das Ergebnis. Ein Tester mit acht Tagen für das System *Kurbad* kann den WSTG nicht Zeile für Zeile abarbeiten. Er muss gewichten, und dafür ist die Bedrohungsmodellierung da.

Ein Beispiel. Die Plattform hat eine Funktion zum Hochladen eines Profilbilds und eine Funktion zum Einlösen von Gutscheinen. Aus Sicht eines Prüfkatalogs sind beides Eingaben, die zu testen sind. Aus Sicht der Bedrohung sind sie sehr verschieden: Beim Gutschein geht es unmittelbar um Geld, und die Logik dahinter läuft über mehrere Services. Ein Tester, der das System verstanden hat, wird dort einen ganzen Tag verbringen und beim Profilbild eine Stunde. Ein Tester, der es nicht verstanden hat, teilt die Zeit gleichmäßig auf und findet die teure Schwäche nicht.

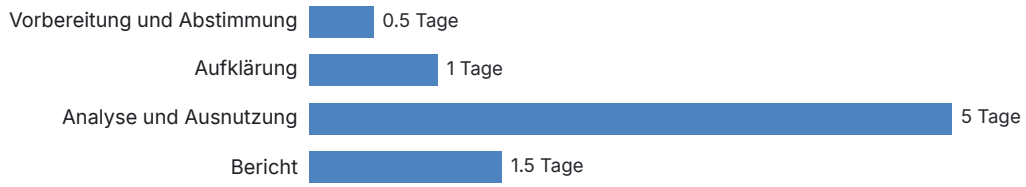



**Abb. 7:** Eine typische Aufteilung von acht Testtagen als Beispiel. Die Zahlen sind eine Annahme zur Veranschaulichung, keine Erhebung.

Zwei Dinge fallen an Abb. 7 auf. Der Bericht braucht einen spürbaren Teil der Zeit, und das ist richtig so, denn er ist das einzige Ergebnis, das beim Auftraggeber bleibt. Wo am Bericht gespart wird, entstehen die unverständlichen Befunde aus Kapitel 1. Und für die eigentliche Prüfung bleiben fünf Tage. Wer weiß, dass eine Anwendung dieser Größe dutzende Endpunkte hat, versteht, warum ein Pentest eine Stichprobe bleibt.

> **BRÜCKE ZU SOFTWARE SECURITY**
> Hier setzt die Lehre aus deiner Pflicht-Lehrveranstaltung an: Wer dem Tester ein gutes Architekturdokument gibt, schenkt ihm Zeit. Jede Stunde, die er nicht mit dem Erraten der Struktur verbringt, verbringt er mit dem Prüfen. Die Aufgabe „Testschwerpunkte aus der Architektur ableiten“ am Ende von Software Security ist genau die Vorarbeit für Phase 3 des PTES.

## 3.9 Automatisierung und KI im Ablauf

In jeder Phase gibt es Werkzeuge, und in den letzten Jahren sind KI-gestützte dazugekommen. Es hilft, sich klarzumachen, was sich automatisieren lässt und was nicht.

Die **Aufklärung** ist weitgehend automatisierbar. Systeme, Dienste, Versionen, Endpunkte und Parameter finden Werkzeuge schneller und vollständiger als Menschen. Die **Analyse** ist dort automatisierbar, wo es um bekannte Muster geht: eine veraltete Komponente, ein fehlender Sicherheits-Header, eine Eingabe, die unverändert in der Antwort erscheint. Sie ist dort nicht automatisierbar, wo die Schwäche in der Bedeutung liegt. Dass eine Kassenkraft keine Gutscheine für sich selbst ausstellen sollte, weiß kein Werkzeug, solange es ihm niemand sagt. KI-Assistenten verschieben diese Grenze, weil sie Beschreibungen einer Anwendung lesen und daraus Vermutungen über die Fachlogik ableiten können. Sie heben sie nicht auf, weil jede Vermutung geprüft werden muss.

Die **Ausnutzung** bleibt eine Entscheidung mit Verantwortung: Ob ein Schritt das System gefährdet und ob er noch im Scope liegt, entscheidet ein Mensch. Beim **Bericht** helfen KI-Werkzeuge beim Formulieren, mit dem Risiko, dass ein Befund überzeugend klingt und trotzdem nicht stimmt.

**FALLSTRICK**

Ein Bericht, in dem jeder Befund denselben glatten Aufbau und dieselben allgemeinen Empfehlungen hat, ist oft maschinell erzeugt und nicht nachgeprüft. Das Erkennungsmerkmal ist dasselbe wie beim reinen Scanner-Bericht aus Kapitel 1: Es fehlt der Bezug zu genau dieser Anwendung und der Nachweis der Ausnutzung.

**FRAGE AUS DEM HÖRSAAL**

***Warum arbeitet nicht jeder Tester einfach den WSTG von vorne bis hinten ab? Dann wäre die Abdeckung vollständig.***

Aus zwei Gründen. Erstens reicht die Zeit nicht: Rund hundert Tests, jeder für jede Funktion der Anwendung, sind in einer Woche nicht zu schaffen. Zweitens ist der Katalog nach Technik gegliedert und nicht nach Risiko. Wer ihn gleichmäßig abarbeitet, verbringt genauso viel Zeit mit einer harmlosen Fehlermeldung wie mit der Zahlungslogik. Der Katalog ist eine Checkliste gegen das Vergessen, die Gewichtung kommt aus der Bedrohungsmodellierung.

**FRAGE AUS DEM HÖRSAAL**

***Ist ein Test „nach OWASP“ besser als einer „nach PTES“?***

Die Frage vergleicht zwei verschiedene Dinge. PTES beschreibt den Ablauf eines Tests von der Vereinbarung bis zum Bericht. Der WSTG beschreibt, was an einer Web-Anwendung zu prüfen ist. Ein guter Test einer Web-Anwendung verwendet beides: den Ablauf nach PTES oder NIST und den Prüfkatalog des WSTG. „Nach OWASP“ allein ist außerdem ungenau, weil OWASP viele Dokumente herausgibt. Gemeint ist meist der WSTG, manchmal nur die Top 10, und das ist ein großer Unterschied.

**FÜR DIE LEHRE**

Für die Spezialisierung reicht es, wenn die Studierenden die Phasen kennen und einen Bericht einer Methodik zuordnen können. Die Durchführung lernen sie in der Pentesting-Lehrveranstaltung.

Eine kurze Übung: Gib ihnen das Inhaltsverzeichnis eines Berichts und lass sie prüfen, zu welchen der fünf Abschnitte aus Tab. 7 ein Ergebnis dokumentiert ist. Fehlt die Bestandsaufnahme oder die Liste der geprüften Bereiche, sollen sie formulieren, welche Aussagen des Berichts dadurch nicht mehr belastbar sind.

Die häufigste Fehlvorstellung ist, dass die OWASP Top 10 eine Testmethodik seien. Sie sind eine Rangliste von Risiken zur Bewusstseinsbildung. Ein Test, der „die OWASP Top 10 abdeckt“, hat zehn Risikokategorien angesehen und sagt nichts darüber, wie gründlich.

## Selbsttest

1. Nenne vier Dinge, die eine Methodik für einen Pentest leistet.
2. Ordne die sieben Phasen des PTES den vier Phasen nach NIST SP 800-115 zu.
3. Was bedeutet die Kennung `WSTG-ATHZ-04`, und was kannst du mit ihr anfangen?
4. Erkläre den Unterschied zwischen ASVS und WSTG mit dem Bild von Spezifikation und Testfall.

5. Warum ist die Bedrohungsmodellierung die Phase, in der Architekturwissen am meisten bringt?
6. Ein Bericht enthält dreißig Befunde, aber keine Angabe, welche Bereiche geprüft wurden. Was fehlt dir, um ihn zu bewerten?
7. Welche Teile eines Pentests lassen sich gut automatisieren und welche nicht? Begründe an je einem Beispiel.

# 4 Die Landkarte: CVE, CWE, CVSS und ATT&CK

In einem Pentest-Bericht steht unter einem Befund eine Zeile wie diese: „CWE-639, CVSS 3.1: 6.5 (AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N)“. Unter einem anderen: „CVE-2021-44228, in KEV gelistet, ATT&CK T1190“. Wer diese Kürzel nicht lesen kann, kann den Bericht nicht lesen. Sie stammen aus einer Handvoll öffentlicher Kataloge und Bewertungssysteme, die in den letzten fünfundzwanzig Jahren entstanden sind und heute die gemeinsame Sprache des Feldes bilden. Dieses Kapitel erklärt jeden davon einzeln und zeigt dann, wie sie zusammenhängen. Es ist das Kapitel, zu dem du am häufigsten zurückblättern wirst.

**IN DIESEM KAPITEL**

- CVE, CWE, CVSS, EPSS, KEV, CAPEC und ATT&CK definieren und voneinander abgrenzen
- eine CVE-Kennung und einen CWE-Eintrag lesen und die Hierarchie der CWE nutzen
- den Unterschied zwischen Schwachstelle und Schwäche, zwischen Schweregrad und Risiko erklären
- die OWASP Top 10 und die CWE Top 25 als Ranglisten einordnen
- sagen, welche dieser Angaben bei welcher Art von Befund zu erwarten sind

## 4.1 Warum es Kataloge braucht

Bis Ende der 1990er Jahre hatte jede Organisation, die Sicherheitslücken sammelte, ihre eigenen Namen dafür. Ein Hersteller von Scannern nannte eine Lücke anders als der Hersteller der betroffenen Software, und der wieder anders als die Sicherheitsforscher, die sie gefunden hatten. Zwei Werkzeuge konnten dieselbe Lücke melden, ohne dass jemand merkte, dass es dieselbe war. Wer wissen wollte, ob sein System von „der neuen Lücke im Webserver“ betroffen war, musste erst herausfinden, welche gemeint ist.

Die Lösung war dieselbe wie in jeder Wissenschaft, die erwachsen wird: eine gemeinsame Benennung. Zuerst für einzelne Lücken, dann für Arten von Fehlern, dann für die Schwere, dann für Angriffsmuster und für das Vorgehen von Angreifern. Daraus sind sieben Systeme geworden, die du kennen musst. Sie beantworten sieben verschiedene Fragen, und die meisten Missverständnisse entstehen, wenn man sie verwechselt.

**Tab. 8:** Sieben Systeme, sieben Fragen.

| Kürzel | Frage, die es beantwortet | Herausgeber |
|---|---|---|
| CVE | Welche konkrete Lücke in welchem Produkt ist gemeint? | CVE-Programm (MITRE) |
| CWE | Welche Art von Fehler steckt dahinter? | MITRE |
| CVSS | Wie schwer ist die Lücke technisch? | FIRST |
| EPSS | Wie wahrscheinlich wird sie demnächst ausgenutzt? | FIRST |
| KEV | Wird sie nachweislich schon ausgenutzt? | CISA |
| CAPEC | Nach welchem Muster wird so ein Fehler angegriffen? | MITRE |
| ATT&CK | Wie gehen Angreifer insgesamt vor? | MITRE |

Zwei Namen in der rechten Spalte tauchen im ganzen Buch auf. **MITRE** ist eine gemeinnützige US-amerikanische Forschungsorganisation, die im Auftrag von Behörden arbeitet und mehrere dieser Kataloge pflegt. **FIRST**, das *Forum of Incident Response and Security Teams*, ist ein internationaler Verband von Notfallteams, der die beiden Bewertungssysteme herausgibt. **CISA** ist die US-Behörde für Cybersicherheit und Infrastruktursicherheit.

## 4.2 CVE: eine Nummer für jede bekannte Lücke

**DEFINITION 4.1 · CVE**
*Common Vulnerabilities and Exposures* ist ein öffentliches Verzeichnis, das jeder bekannt gewordenen Schwachstelle in einem konkreten Produkt eine eindeutige Kennung gibt. Eine Kennung hat die Form `CVE-Jahr-Nummer`, zum Beispiel `CVE-2021-44228`. Das Jahr ist das der Vergabe, die Nummer ist fortlaufend und hat mindestens vier Stellen [18].

Eine CVE-Kennung ist ein Name und nicht mehr. Der zugehörige Eintrag enthält eine kurze Beschreibung, die betroffenen Produkte und Versionen und Verweise auf weitere Quellen. Er enthält von sich aus keine Bewertung der Schwere und keine Anleitung zur Behebung. Sein Zweck ist, dass alle vom selben Loch reden: der Hersteller in seiner Sicherheitsmeldung, der Scanner in seinem Ergebnis, die Zeitung in ihrem Artikel und der Tester in seinem Bericht.

Die Nummern vergibt nicht eine zentrale Stelle allein. Das Programm wird von MITRE betrieben und von der US-Regierung finanziert. Vergeben werden die Nummern von mehreren hundert berechtigten Organisationen, den *CVE Numbering Authorities* oder CNAs. Das sind vor allem Hersteller, die für ihre eigenen Produkte Nummern vergeben, dazu Notfallteams und Forschungseinrichtungen. Ein Hersteller, der eine Lücke in seinem Produkt behebt, reserviert eine Nummer, veröffentlicht die Korrektur und danach den Eintrag.

Drei bekannte Beispiele zeigen, wie verschieden die Lücken hinter einer solchen Nummer sein können.

`CVE-2014-0160`, bekannt als *Heartbleed*, war ein Fehler in der Verschlüsselungsbibliothek OpenSSL. Eine Funktion las mehr Speicher aus, als sie durfte, und schickte ihn an den Anfragenden zurück, darunter Schlüssel und Passwörter anderer Benutzer [19]. Die Art des Fehlers ist ein Lesezugriff außerhalb der Grenzen eines Speicherbereichs.

`CVE-2021-44228`, bekannt als *Log4Shell*, war ein Fehler in der Java-Protokollbibliothek Log4j. Die Bibliothek wertete bestimmte Zeichenfolgen in Protokolltexten aus und lud daraufhin Code von fremden Servern nach. Wer einen Text in das Protokoll einer Anwendung bringen konnte, etwa über ein Eingabefeld, konnte auf dem Server Code ausführen [20]. Die Bibliothek steckte in einer unüberschaubaren Zahl von Produkten, die meisten Betreiber wussten nicht, dass sie sie einsetzen.

`CVE-2024-3094` bezeichnet keine versehentliche Lücke, sondern eine absichtlich eingebaute Hintertür in der Kompressionsbibliothek xz, die über Jahre durch das Erschleichen von Vertrauen im Projekt vorbereitet worden war [21]. Dein Foliensatz zu den Grundlagen verwendet den Fall als Beispiel für Angriffe auf die Lieferkette.

Für das Lesen von Berichten sind drei Eigenschaften des Verzeichnisses wichtig.

**CVE-Nummern gibt es nur für Produkte.** Eine Lücke in einer Bibliothek, einem Betriebssystem, einem Gerät oder einer verbreiteten Anwendung bekommt eine Nummer. Ein Fehler in der selbst entwickelten Anwendung *Kurbad* bekommt keine, weil er nur diese eine Installation betrifft und niemand außer dem Betreiber davon wissen muss. In einem Pentest-Bericht zu einer Eigenentwicklung tragen deshalb die meisten Befunde keine CVE-Nummer. Eine Nummer steht nur dort, wo eine eingesetzte Fremdkomponente eine bekannte Lücke hat.

**Nicht jede Lücke hat eine Nummer.** Das Verzeichnis enthält, was gemeldet und aufgenommen wurde. Lücken, die ein Hersteller stillschweigend behebt, und Lücken, die nur Angreifer kennen, fehlen. Letztere heißen *Zero-Day-Lücken*: Der Hersteller hatte null Tage Zeit, sie zu beheben, bevor sie ausgenutzt wurden.

**Die Zahl der Einträge wächst stark.** Dein Foliensatz zeigt die Kurve: von rund 16.500 neuen Einträgen im Jahr 2018 auf rund 40.000 im Jahr 2024. Niemand kann das von Hand verfolgen. Daraus folgt der Bedarf an den Bewertungs- und Priorisierungssystemen weiter unten.

**FRAGE AUS DEM HÖRSAAL**
***Wenn eine Lücke keine CVE-Nummer hat, ist sie dann weniger schlimm?***

Nein. Die Nummer sagt nur, dass die Lücke ein verbreitetes Produkt betrifft und öffentlich verzeichnet ist. Die schwersten Befunde in einem Pentest einer Eigenentwicklung haben fast nie eine Nummer, weil sie Fehler in der eigenen Logik sind. Umgekehrt haben viele harmlose Lücken eine Nummer. Die Schwere steht woanders, nämlich in der CVSS-Bewertung.

**Wo die Einträge angereichert werden**

Der CVE-Eintrag selbst ist knapp. Die Angaben, mit denen Werkzeuge arbeiten, kommen aus Datenbanken, die auf dem Verzeichnis aufbauen.

Die bekannteste ist die *National Vulnerability Database*, kurz **NVD**, des NIST [22]. Sie ergänzt jeden CVE-Eintrag um eine CVSS-Bewertung, um die Art der Schwäche als CWE-Nummer und um eine maschinenlesbare Liste der betroffenen Produkte in einem Format namens CPE, *Common Platform Enumeration*. Scanner und Werkzeuge zur Prüfung von Abhängigkeiten beziehen ihre Daten meist von hier. Seit 2024 kommt die NVD mit der Anreicherung nicht mehr nach, viele Einträge warten lange auf ihre Bewertung. Das hat die Abhängigkeit von einer einzelnen Quelle sichtbar gemacht. Im Frühjahr 2025 stand zudem kurzzeitig die Finanzierung des CVE-Programms selbst in Frage.

In Europa betreibt die Agentur ENISA seit 2025 eine eigene Datenbank, die *European Vulnerability Database* oder **EUVD** [23]. Daneben gibt es die Meldungen der Hersteller selbst und Datenbanken, die auf Open-Source-Pakete zugeschnitten sind und von den Werkzeugen der Entwicklungsplattformen verwendet werden.

Für dich reicht der Grundsatz: Die Nummer kommt vom CVE-Programm, die Anreicherung von einer Datenbank, und zwei Datenbanken können dieselbe Lücke unterschiedlich bewerten.

## 4.3 CWE: die Art des Fehlers

Eine CVE-Nummer bezeichnet einen einzelnen Fall. Wer aus Fällen lernen will, braucht Kategorien. Dafür gibt es die CWE.

**DEFINITION 4.2 · CWE**

Die *Common Weakness Enumeration* ist ein Katalog von Arten von Fehlern in Software und Hardware, die zu Schwachstellen führen können. Jeder Eintrag hat eine Nummer, zum Beispiel `CWE-89` für SQL-Injection, eine Beschreibung, Beispiele, typische Folgen und mögliche Gegenmaßnahmen [24].

Der Katalog unterscheidet zwei Begriffe, die im Deutschen leicht verschwimmen und die dieses Buch ab hier streng trennt.

**DEFINITION 4.3 · SCHWÄCHE UND SCHWACHSTELLE**

Eine **Schwäche** (*weakness*) ist eine Art von Fehler, die unter bestimmten Umständen ausnutzbar wird, zum Beispiel „Benutzereingaben werden ungeprüft in eine Datenbankabfrage eingesetzt“. Eine **Schwachstelle** (*vulnerability*) ist das konkrete Auftreten einer Schwäche in einem bestimmten Produkt, das sich ausnutzen lässt, zum Beispiel „das Suchfeld der Anwendung X in Version 2.3“. Die CWE katalogisiert Schwächen, das CVE-Verzeichnis Schwachstellen.

Der Unterschied ist derselbe wie zwischen einer Krankheit und einem Krankheitsfall. Wer nur Fälle behandelt, behandelt immer wieder denselben. Wer die Krankheit kennt, kann vorbeugen. Genau deshalb ist die CWE-Nummer für die Ursachenanalyse wichtiger als jede andere Angabe im Befund: Sie benennt, *was für ein* Fehler gemacht wurde, und damit, wo er sonst noch stecken könnte.

**Die Hierarchie**

Der Katalog enthält über 900 Einträge, und sie stehen nicht nebeneinander, sondern in einer Hierarchie von allgemein zu speziell. Vier Ebenen der Abstraktion sind festgelegt.

**Tab. 9:** Die vier Abstraktionsebenen der CWE [24].

| Ebene | Bedeutung | Beispiel |
|---|---|---|
| Pillar | oberste Kategorie, sehr allgemein, beschreibt ein Grundproblem | CWE-284 Improper Access Control |
| Class | allgemeine Schwäche, unabhängig von Sprache und Technologie | CWE-863 Incorrect Authorization |
| Base | konkrete Schwäche, an der sich Erkennung und Behebung festmachen lassen | CWE-639 Authorization Bypass Through User-Controlled Key |
| Variant | Schwäche in einer bestimmten Sprache oder Technologie | CWE-564 SQL Injection: Hibernate |

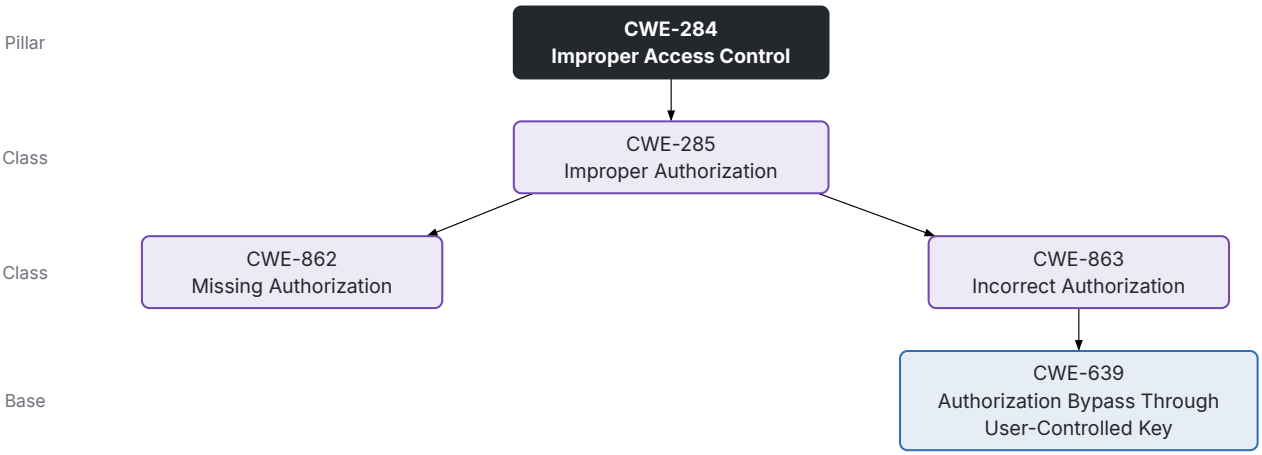



**Abb. 8:** Ein Ast der CWE-Hierarchie: von der Zugriffskontrolle allgemein bis zur konkreten Schwäche, dass ein vom Benutzer gelieferter Schlüssel über den Zugriff entscheidet.

Die Hierarchie ist ein Werkzeug für die Ursachenanalyse, und Kapitel 16 verwendet sie so. Liest man Abb. 8 von unten nach oben, bekommt man auf jede Stufe eine allgemeinere Antwort auf die Frage, was falsch ist. Unten: An diesem Endpunkt entscheidet eine Kennung aus der Anfrage, welcher Datensatz geliefert wird. Eine Stufe höher: Die Autorisierung ist falsch umgesetzt. Ganz oben: Die Zugriffskontrolle des Systems ist mangelhaft. Die untere Antwort führt zu einer Korrektur im Code, die obere zu einer Frage an die Architektur.

Daraus ergibt sich eine Regel für die Zuordnung, die MITRE selbst ausgibt: Ein Befund soll so genau wie möglich eingeordnet werden, also auf der Ebene Base oder Variant. Eine Zuordnung zu einem Pillar oder einer sehr allgemeinen Class ist zu unscharf, um daraus eine Behebung abzuleiten. Der bekannteste Fall ist `CWE-20`, *Improper Input Validation*. Fast jede Schwäche, bei der Eingaben eine Rolle spielen, lässt sich dort einsortieren, und gerade deshalb sagt die Zuordnung nichts. Steht in einem Bericht bei der Hälfte der Befunde CWE-20, hat sich der Tester die Einordnung gespart.

Manche Schwächen treten nur zusammen auf. Die CWE kennt dafür zwei Begriffe. Eine **Kette** (*chain*) ist eine Folge, in der eine Schwäche die nächste ermöglicht: Ein Rechenfehler bei einer Längenangabe führt zu einem zu kleinen Speicherbereich, der zu einem Schreibzugriff außerhalb der Grenzen führt. Ein **Kompositum** (*composite*) ist eine Schwäche, die nur entsteht, wenn mehrere andere gleichzeitig vorliegen. Für die Ursachenanalyse ist das wichtig, weil bei einer Kette die Behebung am ersten Glied ansetzen sollte.

### Sichten auf den Katalog

Weil 900 Einträge unhandlich sind, bietet MITRE vorgefertigte Sichten an. Eine ordnet die Einträge so, wie Entwickler sie im Alltag antreffen, eine andere so, wie Forscher sie systematisch gliedern. Eine dritte ist eine stark verkürzte Auswahl von rund 130 Einträgen, auf die die NVD ihre Zuordnungen beschränkt. Das erklärt, warum die NVD einer Lücke manchmal eine allgemeinere CWE-Nummer gibt als der Hersteller: Der genauere Eintrag kommt in ihrer Auswahl nicht vor.

## 4.4 Ranglisten: OWASP Top 10 und CWE Top 25

Zwei Ranglisten sind bekannter als die Kataloge, auf denen sie beruhen.

Die **CWE Top 25** erscheinen jährlich. MITRE wertet aus, welchen Schwächen die CVE-Einträge der letzten Zeit zugeordnet wurden, und gewichtet nach Häufigkeit und Schwere [25]. Tab. 10 zeigt die ersten zehn Plätze der Liste von 2025.

**Tab. 10:** Die ersten zehn der CWE Top 25 von 2025 [25].

| Rang | CWE | Name | Worum es geht |
|---|---|---|---|
| 1 | CWE-79 | Cross-site Scripting | fremde Eingaben landen als Skript im Browser anderer Benutzer |
| 2 | CWE-89 | SQL Injection | fremde Eingaben werden Teil einer Datenbankabfrage |
| 3 | CWE-352 | Cross-Site Request Forgery | der Browser eines angemeldeten Benutzers wird zu einer ungewollten Aktion gebracht |
| 4 | CWE-862 | Missing Authorization | eine Berechtigungsprüfung fehlt ganz |
| 5 | CWE-787 | Out-of-bounds Write | es wird außerhalb eines Speicherbereichs geschrieben |
| 6 | CWE-22 | Path Traversal | über einen manipulierten Pfad werden Dateien außerhalb des erlaubten Verzeichnisses erreicht |
| 7 | CWE-416 | Use After Free | Speicher wird verwendet, nachdem er freigegeben wurde |
| 8 | CWE-125 | Out-of-bounds Read | es wird außerhalb eines Speicherbereichs gelesen |
| 9 | CWE-78 | OS Command Injection | fremde Eingaben werden Teil eines Betriebssystembefehls |
| 10 | CWE-94 | Code Injection | fremde Eingaben werden als Programmcode ausgeführt |

Die Liste zeigt zwei Familien, die das Feld seit Jahrzehnten beherrschen. Die eine sind Fehler im Umgang mit dem Speicher (Plätze 5, 7 und 8), die in Sprachen wie C und C++ entstehen und in Web-Anwendungen, die in speichersicheren Sprachen geschrieben sind, kaum vorkommen. Die andere sind Fehler, bei denen Daten als Anweisungen behandelt werden (Plätze 1, 2, 9 und 10). Kapitel 8 führt sie auf einen gemeinsamen Mechanismus zurück. Dazu kommt mit Platz 4 die Zugriffskontrolle, das Thema von Kapitel 9.

Die **OWASP Top 10** sind die bekannteste Liste der Anwendungssicherheit überhaupt. Sie erscheinen alle paar Jahre und nennen die zehn wichtigsten Risikokategorien für Web-Anwendungen, ermittelt aus Testdaten vieler Organisationen und einer Umfrage unter Fachleuten. Die aktuelle Fassung trägt die Jahreszahl 2025 und wurde im Januar 2026 fertiggestellt [26].

**Tab. 11:** Die OWASP Top 10:2025 [26].

| Nr. | Kategorie 2025 | Veränderung gegenüber 2021 |
|---|---|---|
| A01 | Broken Access Control | bleibt auf Platz 1, umfasst jetzt auch SSRF |
| A02 | Security Misconfiguration | von Platz 5 aufgestiegen |
| A03 | Software Supply Chain Failures | neu, erweitert die frühere Kategorie zu veralteten Komponenten |
| A04 | Cryptographic Failures | von Platz 2 abgestiegen |
| A05 | Injection | von Platz 3 abgestiegen |
| A06 | Insecure Design | von Platz 4 abgestiegen |
| A07 | Authentication Failures | unverändert |
| A08 | Software and Data Integrity Failures | unverändert |
| A09 | Security Logging and Alerting Failures | umbenannt, „Alerting“ statt „Monitoring“ |
| A10 | Mishandling of Exceptional Conditions | neu, ersetzt SSRF als eigene Kategorie |

Jede Kategorie der Top 10 fasst mehrere CWE-Einträge zusammen. Die Top 10 sind also gröber als die CWE: „Broken Access Control“ ist ein Sammelbegriff für Dutzende Schwächen. Das macht die Liste gut für Schulungen und schlecht für Befunde. Eine Angabe wie „A01“ im Befund sagt dem Entwickler so wenig wie „CWE-284“.

Drei Verschiebungen in der Liste von 2025 sind für deine Lehre bemerkenswert. Die Lieferkette steht jetzt auf Platz 3, das stützt das Gewicht, das dein Foliensatz dem Thema gibt. Fehlkonfiguration ist auf Platz 2 gestiegen, ein Hinweis, dass immer mehr Sicherheit in Einstellungen statt im Code liegt. Und die neue Kategorie A10 zur Behandlung von Ausnahmesituationen umfasst unter anderem nicht abgefangene Fehler und Wettlaufsituationen, also genau das, was du unter TOCTOU und Sagas unterrichtest.

**FALLSTRICK**
Die Top 10 sind keine Prüfliste und kein Standard. Eine Anwendung, die „die OWASP Top 10 erfüllt“, gibt es nicht, weil die Liste keine prüfbaren Anforderungen enthält. Wer Anforderungen will, nimmt den ASVS aus Kapitel 3.

## 4.5 CVSS: die Schwere in einer Zahl

Zigtausende neue Lücken im Jahr kann niemand einzeln durchdenken. Es braucht ein Maß, das auf einen Blick sagt, wie ernst eine Lücke ist.

**DEFINITION 4.4 · CVSS**
Das *Common Vulnerability Scoring System* ist ein offenes Verfahren, das die technische Schwere einer Schwachstelle aus einer festen Reihe von Merkmalen berechnet. Das Ergebnis ist eine Zahl von 0,0 bis 10,0 und eine Zeichenkette, der *Vektor*, der die Ausprägung jedes Merkmals festhält [27].

Die Zahl wird in fünf Stufen übersetzt, die du in jedem Bericht findest.

**Tab. 12:** Die Schweregradstufen des CVSS [27].

| Stufe | Wert | Übliche deutsche Bezeichnung |
|---|---|---|
| None | 0,0 | keine |
| Low | 0,1 bis 3,9 | niedrig |
| Medium | 4,0 bis 6,9 | mittel |
| High | 7,0 bis 8,9 | hoch |
| Critical | 9,0 bis 10,0 | kritisch |

Der Vektor ist wichtiger als die Zahl, weil er die Begründung enthält. In `AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N` steht, in dieser Reihenfolge: Die Lücke ist über das Netz ausnutzbar, der Angriff ist nicht aufwendig, der Angreifer braucht ein gewöhnliches Benutzerkonto, kein anderer Benutzer muss mitwirken, die Auswirkung bleibt auf die verwundbare Komponente beschränkt, die Vertraulichkeit ist stark betroffen, Integrität

und Verfügbarkeit sind es nicht. Kapitel 14 erklärt jedes Merkmal, rechnet Beispiele durch und zeigt, wo Bewertungen in der Praxis danebenliegen.

Es gibt zwei Fassungen, die nebeneinander im Gebrauch sind. Version 3.1 stammt von 2019 und ist in bestehenden Datenbanken und Berichten noch die häufigere [27]. Version 4.0 wurde im November 2023 veröffentlicht, ändert einige Merkmale und trennt deutlicher zwischen der Bewertung der Lücke an sich und der Bewertung in einer bestimmten Umgebung [28]. Ein Bericht muss angeben, welche Fassung er verwendet, weil dieselbe Lücke in beiden unterschiedliche Zahlen bekommen kann.

Schon hier ist die wichtigste Einschränkung festzuhalten.

> **MERKE**
> CVSS misst die **Schwere** einer Schwachstelle, nicht das **Risiko**. Die Schwere sagt, wie schlimm es wäre, wenn die Lücke ausgenutzt wird, und wie leicht das technisch ist. Das Risiko hängt zusätzlich davon ab, ob jemand es tatsächlich tut und was in dieser Organisation dabei auf dem Spiel steht.

Eine Lücke mit dem Wert 9,8 in einem Testsystem ohne Daten und ohne Verbindung zu anderen Systemen ist ein geringes Risiko. Eine Lücke mit 5,3 in dem Service, über den alle Zahlungen laufen, kann ein hohes sein. Die folgenden zwei Systeme versuchen, den fehlenden Teil zu liefern.

## 4.6 EPSS und KEV: wird die Lücke ausgenutzt?

Von den vielen tausend Lücken, die jedes Jahr als hoch oder kritisch eingestuft werden, wird nur ein kleiner Teil jemals tatsächlich angegriffen. Wer alle gleich behandelt, verschwendet den größten Teil seiner Arbeit. Zwei Quellen helfen bei der Auswahl.

> **DEFINITION 4.5 · KEV**
> Der Katalog *Known Exploited Vulnerabilities* der US-Behörde CISA führt Schwachstellen, für die es verlässliche Belege gibt, dass sie in echten Angriffen ausgenutzt wurden oder werden [29].

Der Katalog entstand im November 2021 zusammen mit einer Anordnung, die US-Bundesbehörden verpflichtet, gelistete Lücken innerhalb einer Frist zu beheben [30]. Aufgenommen wird eine Lücke nur, wenn sie eine CVE-Nummer hat, wenn die Ausnutzung belegt ist und wenn es eine klare Abhilfe gibt. Obwohl die Anordnung nur für US-Behörden gilt, verwenden Organisationen weltweit den Katalog als Liste dessen, was sofort zu erledigen ist. Die Logik ist einfach: Hier muss niemand mehr schätzen, ob ein Angriff wahrscheinlich ist. Er findet statt.

KEV beantwortet die Frage nach der Vergangenheit und Gegenwart. Für die nahe Zukunft gibt es eine Schätzung.

> **DEFINITION 4.6 · EPSS**
> Das *Exploit Prediction Scoring System* ist ein statistisches Modell, das für jede veröffentlichte CVE die Wahrscheinlichkeit schätzt, dass sie in den nächsten 30 Tagen in freier Wildbahn ausgenutzt wird. Das Ergebnis ist ein Wert zwischen 0 und 1 und wird täglich neu berechnet [31], [32].

Das Modell wird aus beobachteten Angriffen gelernt und verwendet Merkmale wie das Alter der Lücke, den betroffenen Hersteller, die Art der Schwäche und ob Angriffscode öffentlich verfügbar ist. Es wird regelmäßig erneuert, im Juni 2026 wurde die fünfte Fassung vorgestellt [33]. EPSS ist kein Ersatz für CVSS, sondern die zweite Achse: CVSS sagt, wie schwer ein Treffer wäre, EPSS, wie wahrscheinlich er ist.

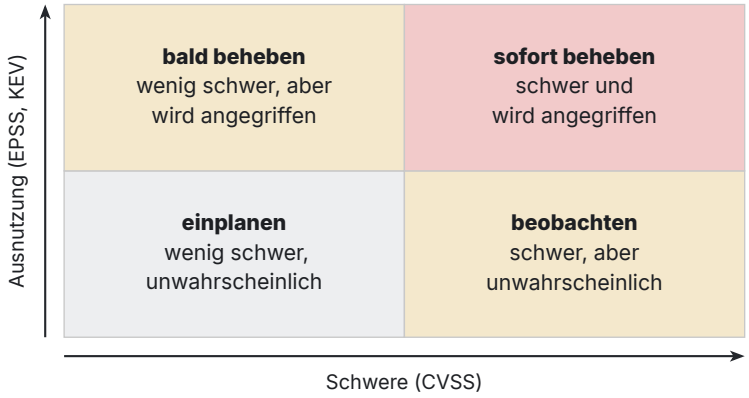



**Abb. 9:** Schwere und Wahrscheinlichkeit der Ausnutzung als zwei Achsen der Priorisierung.

Beide Quellen haben dieselbe Grenze wie das CVE-Verzeichnis: Sie kennen nur Lücken mit CVE-Nummer. Für Befunde in einer Eigenentwicklung gibt es weder einen EPSS-Wert noch einen Eintrag in KEV. Dort muss die Wahrscheinlichkeit von Menschen eingeschätzt werden, und Kapitel 14 zeigt ein Verfahren dafür.

## 4.7 CAPEC: wie ein Fehler angegriffen wird

Die CWE beschreibt Fehler aus der Sicht dessen, der Software baut. Der Blick von der anderen Seite steht in einem Schwesterkatalog.

> **DEFINITION 4.7 · CAPEC**
> Die *Common Attack Pattern Enumeration and Classification* ist ein Katalog von Angriffsmustern. Ein Muster beschreibt, wie ein Angreifer eine bestimmte Art von Schwäche ausnutzt: welche Voraussetzungen er braucht, in welchen Schritten er vorgeht und was er damit erreicht. Jedes Muster verweist auf die CWE-Einträge, gegen die es sich richtet [34].

Zu `CWE-89`, der SQL-Injection als Schwäche, gehört zum Beispiel `CAPEC-66`, die SQL-Injection als Angriff. Der Unterschied wirkt klein, ist aber derselbe wie zwischen einer offenen Tür und einem Einbruch: Die CWE beschreibt den Zustand, die CAPEC die Handlung.

In Pentest-Berichten kommt CAPEC seltener vor als CWE. Nützlich ist der Katalog an zwei Stellen. Bei der Bedrohungsmodellierung hilft er, von einer vermuteten Schwäche auf mögliche Angriffe zu schließen. Und er ist das Bindeglied zum letzten und größten System in dieser Reihe.

## 4.8 MITRE ATT&CK: wie Angreifer vorgehen

Alles bisher Genannte betrachtet einzelne Lücken. Ein echter Angriff besteht aber aus vielen Schritten, von denen die wenigsten eine Lücke im engen Sinn brauchen. Der Angreifer verschafft sich Zugang, richtet sich ein, sieht sich um, weitet seine Rechte aus, bewegt sich zu anderen Systemen und holt am Ende Daten heraus oder richtet Schaden an. Dieses Vorgehen beschreibt ATT&CK.

> **DEFINITION 4.8 · MITRE ATT&CK**
> ATT&CK, kurz für *Adversarial Tactics, Techniques, and Common Knowledge*, ist eine Wissensbasis über das Verhalten von Angreifern, zusammengetragen aus der Beobachtung echter Angriffe. Sie ist als Matrix aufgebaut: Die Spalten sind **Taktiken**, also die Ziele, die ein Angreifer in einem Schritt verfolgt. In den Spalten stehen **Techniken**, also die Wege, auf denen er ein solches Ziel erreicht [35], [36].

Die Unterscheidung ist die zwischen Warum und Wie. „Zugang erlangen“ ist eine Taktik. „Eine öffentlich erreichbare Anwendung ausnutzen“ ist eine Technik dazu, „gültige Zugangsdaten verwenden“ eine andere, „eine gefälschte E-Mail schicken“ eine dritte. Techniken können in **Untertechniken** zerfallen, und zu jeder sind **Prozeduren** dokumentiert, also wie bestimmte Angreifergruppen sie tatsächlich eingesetzt haben.

Die Wissensbasis wird zweimal im Jahr aktualisiert. Version 20 vom Oktober 2026 zählt für Unternehmensumgebungen 15 Taktiken, 227 Techniken und 479 Untertechniken [37]. Daneben gibt es eigene Matrizen für mobile Geräte und für industrielle Steuerungssysteme.

**Tab. 13:** Die Taktiken der ATT&CK-Matrix für Unternehmensumgebungen (Version 20, Oktober 2026). Die frühere Taktik *Defense Evasion* wurde mit Version 19 in *Stealth* und *Defense Impairment* geteilt [37].

| | Taktik | Ziel des Angreifers in diesem Schritt |
|---|---|---|
| 1 | Reconnaissance | Informationen über das Ziel sammeln |
| 2 | Resource Development | Infrastruktur und Werkzeuge für den Angriff bereitstellen |
| 3 | Initial Access | in das Netz oder System hineinkommen |
| 4 | Execution | eigenen Code zur Ausführung bringen |
| 5 | Persistence | den Zugang über Neustarts und Passwortwechsel hinweg behalten |
| 6 | Privilege Escalation | höhere Rechte erlangen |
| 7 | Stealth | unentdeckt bleiben |
| 8 | Defense Impairment | Schutz- und Überwachungsfunktionen schwächen oder abschalten |
| 9 | Credential Access | Zugangsdaten stehlen |
| 10 | Discovery | die Umgebung von innen erkunden |
| 11 | Lateral Movement | sich zu weiteren Systemen bewegen |
| 12 | Collection | die gesuchten Daten zusammentragen |
| 13 | Command and Control | mit den übernommenen Systemen kommunizieren |
| 14 | Exfiltration | Daten aus dem Netz herausbringen |
| 15 | Impact | Systeme und Daten manipulieren, stören oder zerstören |

Jede Taktik und Technik hat eine Kennung. Taktiken beginnen mit `TA`, Techniken mit `T`: `TA0001` ist *Initial Access*, `T1190` ist die Technik *Exploit Public-Facing Application*, `T1078` ist *Valid Accounts*. Steht in einem Bericht „T1190“, heißt das: Der erste Zugang gelang über eine Schwäche in einer von außen erreichbaren Anwendung.

Für Pentests von Anwendungen ist ATT&CK weniger zentral als für Red-Team-Einsätze, weil der Test einer Anwendung meist bei den ersten Taktiken bleibt. Drei Gründe sprechen trotzdem dafür, dass du die Matrix kennst.

Sie zeigt, **was nach dem Befund kommt**. Ein Befund „Codeausführung auf dem Buchungsservice“ ist für den Tester das Ende. Für einen Angreifer ist es Schritt 4 von 15. Die Matrix macht greifbar, was mit dem gewonnenen Zugang alles möglich wäre, und das ist die Grundlage für die Einschätzung der Auswirkung. Kapitel 11 geht diesen Weg nach.

Sie ist die **Sprache der Verteidigung**. Die Leute im Betrieb, die Angriffe erkennen sollen, ordnen ihre Erkennungsregeln den Techniken zu. Wenn ein Pentest-Bericht die verwendeten Techniken nennt, kann der Betrieb prüfen, ob er sie bemerkt hätte.

Sie **wächst mit dem Feld**. Version 20 hat Techniken aufgenommen, die KI-Systeme betreffen, darunter direkte und indirekte Prompt Injection [37]. Für KI-Systeme gibt es mit **ATLAS** zusätzlich eine eigene Wissensbasis nach demselben Muster [38], die Kapitel 12 aufgreift. Und als Gegenstück zu ATT&CK führt MITRE mit **D3FEND** einen Katalog von Verteidigungsmaßnahmen [39].

Ein älteres und einfacheres Modell für denselben Gedanken ist die *Cyber Kill Chain*, die ein Angriff in sieben aufeinanderfolgende Stufen teilt [40]. Der Begriff fällt in Berichten und Vorträgen noch oft. ATT&CK ist feiner und nicht streng linear, die Grundidee ist dieselbe: Ein Angriff ist eine Kette, und jede Stufe ist eine Gelegenheit, ihn zu unterbrechen.

## 4.9 Wie alles zusammenhängt

Die sieben Systeme bilden ein zusammenhängendes Bild, wenn man sie um eine einzelne Schwachstelle herum anordnet.

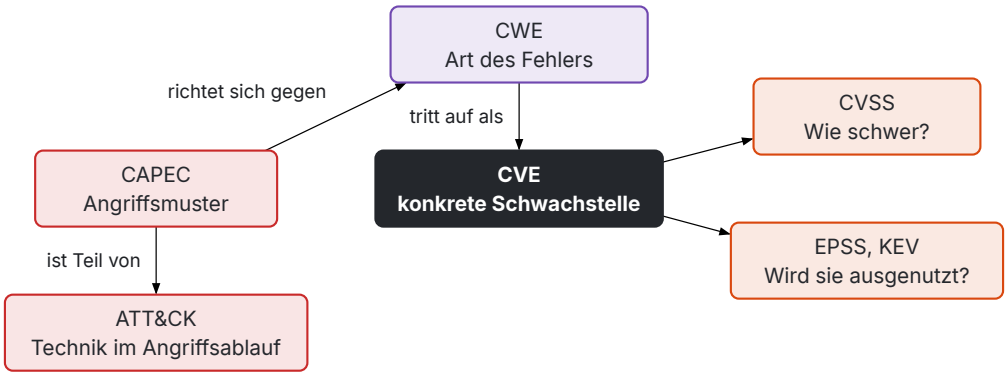



**Abb. 10:** Die Kataloge und Bewertungssysteme rund um eine Schwachstelle.

Am Beispiel Log4Shell liest sich Abb. 10 so. Die konkrete Schwachstelle heißt `CVE-2021-44228`. Die Art des Fehlers ist, dass eine Bibliothek von außen gelieferte Zeichenfolgen als Anweisungen auswertet und daraufhin fremde Objekte lädt. Die NVD ordnet ihr mehrere CWE-Einträge zu, darunter `CWE-502`, die Deserialisierung nicht vertrauenswürdiger Daten, und `CWE-917`, das Einschleusen von Ausdrücken einer Ausdruckssprache. Die Schwere nach CVSS 3.1 ist 10,0, der höchste mögliche Wert: über das Netz, ohne Aufwand, ohne Konto, ohne Mitwirkung eines Benutzers, mit voller Auswirkung auch über die verwundbare Komponente hinaus [20]. Die Lücke steht im KEV-Katalog, sie wurde innerhalb von Tagen massenhaft angegriffen. Im Ablauf eines Angriffs ist sie ein Weg zum ersten Zugang, also die Technik `T1190`.

Damit sind zu einer einzigen Lücke fünf Fragen beantwortet: welche, was für eine, wie schwer, wie dringend und wozu ein Angreifer sie braucht. Für die Behebung im eigenen Haus ist davon die zweite die lehrreichste. Die Lücke selbst wurde durch eine neue Version der Bibliothek behoben. Die Frage „Wissen wir überhaupt, in welchen unserer Anwendungen diese Bibliothek steckt?“ konnten viele Organisationen nicht beantworten. Das war kein Fehler im Code, sondern einer im Lifecycle, und er ist der Grund, warum heute Stücklisten für Software verlangt werden. Kapitel 10 kommt darauf zurück.

## 4.10 Was davon im Pentest-Bericht steht

Nicht jeder Befund trägt alle Angaben, und ob eine fehlt, hängt davon ab, was für ein Befund es ist.

**Tab. 14:** Welche Angaben bei welcher Art von Befund zu erwarten sind.

| Art des Befunds | CVE | CWE | CVSS | EPSS, KEV | ATT&CK |
|---|---|---|---|---|---|
| bekannte Lücke in einer Fremdkomponente, etwa einer Bibliothek | ja | ja | ja | ja | selten |
| Fehler im eigenen Code, etwa eine fehlende Berechtigungsprüfung | nein | ja | ja | nein | selten |
| Fehlkonfiguration, etwa eine öffentlich lesbare Ablage | nein | ja | ja | nein | selten |
| Fehler in der Fachlogik, etwa ein mehrfach einlösbarer Gutschein | nein | unge-nau | ja | nein | nein |
| Schritt in einem Red-Team-Einsatz | teils | teils | selten | teils | ja |

Zwei Zeilen in Tab. 14 verdienen eine Anmerkung. Bei Fehlern in der Fachlogik passt die CWE oft schlecht, weil der Katalog technische Fehlerarten beschreibt und nicht die Regeln eines Geschäfts. Tester greifen dann zu allgemeinen Einträgen, und die Einordnung hilft bei der Behebung wenig. Und die Zeile zu Fremdkomponenten ist die einzige, bei der sich die Dringlichkeit aus öffentlichen Quellen ablesen lässt. Bei allen anderen muss sie jemand begründen.

> **BRÜCKE ZU SOFTWARE SECURITY**
> Für die Schwachstellenanalyse in der Case Study deiner Pflicht-Lehrveranstaltung brauchen die Studierenden aus diesem Kapitel genau eines: die CWE als Vokabular für Fehlerarten. Wer eine gefundene Schwäche einem Base-Eintrag zuordnen kann, hat sie verstanden. In der Spezialisierung kommen CVSS und die Frage der Dringlichkeit dazu, weil dort ein Bericht bewertet und priorisiert werden muss.

## 4.11 Grenzen der Kataloge

Die Systeme sind unverzichtbar, und sie sind nicht neutral. Vier Grenzen solltest du kennen, auch um kritische Fragen im Hörsaal beantworten zu können.

**Die Abdeckung ist lückenhaft.** Verzeichnet ist, was jemand gemeldet hat. Verbreitete Produkte großer Hersteller sind gut erfasst, Nischenprodukte und vieles aus der Welt der eingebetteten Geräte schlecht.

**Die Zuordnung ist Ermessenssache.** Zwei Fachleute ordnen dieselbe Lücke unterschiedlichen CWE-Einträgen zu und bewerten sie mit unterschiedlichen CVSS-Werten. Die Zahl sieht objektiver aus, als sie ist. Kapitel 14 zeigt an Beispielen, wie weit Bewertungen auseinanderliegen können.

**Die Ranglisten spiegeln ihre Datenquellen.** Die CWE Top 25 zählen, was in CVE-Einträgen vorkommt, also Fehler in Produkten. Die OWASP Top 10 beruhen auf Daten aus Tests von Web-Anwendungen. Keine der beiden sagt, was in *deiner* Anwendung am wahrscheinlichsten falsch ist.

**Die Infrastruktur ist verletzlich.** Der Rückstau bei der NVD und die Unsicherheit um die Finanzierung des CVE-Programms haben gezeigt, dass das Fundament des ganzen Feldes an wenigen Organisationen und einem Geldgeber hängt. Die europäische Datenbank ist auch eine Antwort darauf.

FRAGE AUS DEM HÖRSAAL

***Warum gibt es so viele verschiedene Kataloge? Könnte man das nicht in einem zusammenfassen?***

Die Kataloge beantworten verschiedene Fragen für verschiedene Leute. Ein Administrator will wissen, welche Lücke er heute schließen muss (CVE, KEV). Eine Entwicklerin will wissen, welche Art von Fehler sie nicht mehr machen soll (CWE). Ein Analyst im Betrieb will wissen, woran er einen laufenden Angriff erkennt (ATT&CK). Ein einziger Katalog für alles wäre für jeden zu groß. Dass die Systeme aufeinander verweisen, ist der bessere Weg.

FRAGE AUS DEM HÖRSAAL

***Wenn EPSS sagt, eine Lücke wird zu zwei Prozent ausgenutzt, kann ich sie dann ignorieren?***

Der Wert ist eine Schätzung über alle Organisationen hinweg, nicht für deine. Wenn dein System für einen bestimmten Angreifer besonders lohnend ist, gilt der Durchschnitt nicht. Der Wert ändert sich außerdem täglich, eine Lücke kann von zwei auf neunzig Prozent springen, sobald Angriffscode veröffentlicht wird. EPSS hilft, tausend Lücken zu sortieren. Es erlaubt nicht, eine einzelne zu vergessen.

FRAGE AUS DEM HÖRSAAL

***Ist eine CWE-Nummer im Befund nicht nur Bürokratie?***

Sie ist das Gegenteil, wenn sie genau genug gewählt ist. Die Nummer macht aus einem Einzelfall eine Klasse. Mit „CWE-639“ kann das Team gezielt nach allen Stellen suchen, an denen eine Kennung aus der Anfrage über den Zugriff entscheidet, und es findet im Katalog die bewährten Gegenmaßnahmen. Wertlos wird die Nummer, wenn der Tester einen Sammeleintrag wie CWE-20 oder CWE-284 hinschreibt.

FÜR DIE LEHRE

Lass die Studierenden die Kataloge einmal selbst benutzen, statt sie nur erklärt zu bekommen. Eine geeignete Aufgabe: Jede Person bekommt eine bekannte CVE-Nummer und trägt in zehn Minuten zusammen, welche CWE zugeordnet ist, welchen CVSS-Wert und Vektor sie hat, ob sie in KEV steht und in welche ATT&CK-Taktik ihre Ausnutzung fällt. Danach erklärt sie den anderen in drei Sätzen, was für ein Fehler es war. Das dauert eine Einheit und nimmt den Kürzeln ihren Schrecken.

Für die CWE-Hierarchie eignet sich eine zweite Aufgabe: Drei Befunde aus einem Bericht, jeweils nur mit einem Sammeleintrag versehen. Die Studierenden sollen im Katalog den passenden Base-Eintrag finden und begründen, warum der genauere Eintrag bei der Behebung hilft.

Typische Fehlvorstellungen: dass jede Schwachstelle eine CVE-Nummer habe, dass ein hoher CVSS-Wert ein hohes Risiko bedeute, und dass die OWASP Top 10 eine vollständige Liste seien.

## Selbsttest

1. Welche Frage beantwortet jedes der sieben Systeme aus Tab. 8? Antworte in je einem Satz.
2. Erkläre den Unterschied zwischen Schwäche und Schwachstelle an einem eigenen Beispiel.
3. Warum hat ein Fehler in einer selbst entwickelten Anwendung in der Regel keine CVE-Nummer, und welche Folgen hat das für EPSS und KEV?
4. Ein Bericht ordnet zwölf von zwanzig Befunden CWE-20 zu. Was sagt dir das, und was würdest du verlangen?
5. Was ist der Unterschied zwischen einer Taktik und einer Technik in ATT&CK?
6. Eine Lücke hat CVSS 9,8, steht nicht in KEV und hat einen EPSS-Wert nahe null. Eine andere hat CVSS 6,5 und steht in KEV. Welche behebst du zuerst, und was müsstest du noch wissen?
7. Warum sind die OWASP Top 10 als Angabe in einem Befund ungeeignet?
8. Nenne zwei Grenzen der Kataloge und je eine praktische Folge.

# 5 Sicherheit im Lebenszyklus

Ein Pentest findet Fehler, die irgendwann vorher gemacht wurden: beim Festlegen der Anforderungen, beim Entwurf, beim Programmieren, beim Einbinden einer Bibliothek, beim Konfigurieren der Umgebung. Jeder Befund ist deshalb auch eine Aussage über den Entwicklungsprozess, der ihn zugelassen hat. Um diese Aussage lesen zu können, brauchst du ein Bild davon, wo im Lebenszyklus einer Software welche Sicherheitsarbeit stattfinden sollte, und die Namen der Rahmenwerke, die das beschreiben. Dieses Kapitel erklärt SDLC und SSDLC, das NIST SSDF, die Reifegradmodelle SAMM und BSIMM, die automatisierten Prüfverfahren mit ihren Kürzeln und die Pflichten nach der Auslieferung. Am Ende steht die Frage, die Teil IV wieder aufnimmt: Welche Tätigkeit im Lebenszyklus hätte diesen Befund verhindern müssen?

**IN DIESEM KAPITEL**

- SDLC und SSDLC erklären und Sicherheitsaktivitäten den Phasen zuordnen
- den Aufbau des NIST SSDF mit seinen vier Gruppen beschreiben
- SAMM und BSIMM unterscheiden und sagen, wozu ein Reifegradmodell dient
- SAST, SCA, DAST und IAST abgrenzen und sagen, was jedes Verfahren sieht und übersieht
- den Pentest in den Lebenszyklus einordnen und einen Befund als Rückmeldung an den Prozess lesen

## 5.1 Vom SDLC zum SSDLC

**DEFINITION 5.1 · SDLC UND SSDLC**
Der *Software Development Life Cycle* (SDLC) ist die Abfolge der Tätigkeiten, mit denen Software entsteht und betrieben wird: Anforderungen, Entwurf, Implementierung, Test, Auslieferung, Betrieb und Wartung. Ein *Secure Software Development Life Cycle* (SSDLC) ist ein SDLC, in dem jede dieser Phasen ausdrücklich festgelegte Sicherheitsaktivitäten enthält.

Die Phasen kennst du aus deinem Foliensatz zu den Prozessmodellen. Ob sie nacheinander ablaufen wie im Wasserfall oder in kurzen Schleifen wie in agilen Vorgehen, ändert nichts an den Tätigkeiten selbst. Der Gedanke hinter dem SSDLC ist ebenso einfach wie alt: Sicherheit ist keine Phase am Ende, sondern eine Eigenschaft, an der in jeder Phase gearbeitet wird. Saltzer und Schroeder haben die Entwurfsprinzipien dafür schon 1975 formuliert [41]. Zum Programm wurde der Gedanke Anfang der 2000er Jahre, als Microsoft nach einer Serie schwerer Sicherheitsvorfälle die Entwicklung anhielt und einen verbindlichen Prozess einführte, den *Security Development Lifecycle* [42]. Etwa zur selben Zeit beschrieb Gary McGraw sieben *Touchpoints*, also Berührungspunkte, an denen Sicherheitsarbeit in den Entwicklungsprozess eingreift [43]. Der Penetrationstest ist einer davon, und McGraw stellte ihn bewusst nicht an die erste Stelle.

**Tab. 15:** Sicherheitsaktivitäten im Lebenszyklus und was ihr Fehlen kostet.

| Phase | Sicherheitsaktivität | Typische Schwäche, wenn sie fehlt |
|---|---|---|
| Anforderungen | Sicherheitsanforderungen festlegen, Missbrauchsfälle beschreiben, Schutzbedarf bestimmen | niemand hat festgelegt, wer was darf |
| Entwurf | Bedrohungsmodellierung, Architekturanalyse, Vertrauensgrenzen festlegen | Services vertrauen einander ungeprüft |
| Implementierung | sichere Programmierpraktiken, Code Review, statische Analyse | Eingaben werden als Anweisungen behandelt |
| Abhängigkeiten | Komponenten auswählen, Stückliste führen, bekannte Lücken prüfen | eine Bibliothek mit bekannter Lücke ist im Einsatz |
| Test | Sicherheitstests aus den Anforderungen, dynamische Analyse, Fuzzing, Pentest | Fehler erreichen die Produktion |
| Auslieferung | sichere Voreinstellungen, signierte Artefakte, gehärtete Umgebung | Debug-Zugänge und Standardpasswörter in der Produktion |
| Betrieb | Überwachung, Umgang mit gemeldeten Schwachstellen, Aktualisierungen | ein Angriff wird nicht bemerkt, eine bekannte Lücke bleibt offen |

Die rechte Spalte von Tab. 15 ist ein Vorgriff auf Teil II. Jede der genannten Schwächen findet sich später als Befund in einem Bericht wieder. Wer die Tabelle von rechts nach links liest, hat die Grundform der Ursachenanalyse auf der Ebene Lifecycle: Vom Befund zur Phase, in der er hätte vermieden werden können.

### Shift Left

Zu den häufigsten Schlagworten des Feldes gehört *Shift Left*. Gemeint ist: Zeichnet man den Lebenszyklus von links nach rechts, sollen Sicherheitsprüfungen möglichst weit links stattfinden, also früh. Die Begründung ist die Kostenkurve aus deinem Foliensatz zu den Grundlagen. Ein Fehler in den Anforderungen kostet am wenigsten, wenn er in den Anforderungen gefunden wird, und am meisten, wenn ihn ein Angreifer in der Produktion findet.

Der Gedanke ist richtig und wird leicht überzogen. Manche Fehler lassen sich früh gar nicht finden, weil sie erst im Zusammenspiel der fertigen Teile entstehen: die Konfiguration des Gateways, das Verhalten unter gleichzeitigen Anfragen, die Wirkung einer Einstellung der Cloud-Plattform. Deshalb gehört zum Verschieben nach links immer auch das Prüfen ganz rechts, am laufenden System. Der Pentest ist die Prüfung am rechten Rand. Er ersetzt die frühen Prüfungen nicht, und sie ersetzen ihn nicht.

> **MERKE**
> Frühe Prüfungen finden Fehler billig, aber nur die, die auf dem Papier oder im Quelltext sichtbar sind. Späte Prüfungen finden auch Fehler im Zusammenspiel, aber teuer. Ein reifer Prozess hat beides.

## 5.2 Das NIST SSDF

Der Gedanke des SSDLC ist allgemein. Wer wissen will, was eine Organisation im Einzelnen tun soll, braucht einen Katalog von Praktiken. Der heute am häufigsten zitierte stammt vom NIST.

> **DEFINITION 5.2 · SSDF**
> Das *Secure Software Development Framework*, NIST Special Publication 800-218, ist ein Katalog von Praktiken für sichere Softwareentwicklung. Es schreibt kein Vorgehensmodell vor, sondern beschreibt Ergebnisse, die eine Organisation erreichen soll, und verweist zu jeder Praktik auf bestehende Standards und Leitfäden [44].

In Kraft ist Version 1.1 von 2022. Eine Version 1.2 wurde Ende 2025 als Entwurf zur Kommentierung veröffentlicht (Stand: 12. November 2026). Für die Entwicklung von Systemen mit generativer KI gibt es mit SP 800-218A eine Ergänzung [45]. Das SSDF ist deshalb so verbreitet, weil die US-Regierung von ihren Softwarelieferanten eine Bestätigung verlangt, dass sie danach arbeiten. Wer an US-Behörden liefert, kommt an ihm nicht vorbei, und viele andere Kunden haben die Anforderung übernommen.

Das SSDF gliedert seine Praktiken in vier Gruppen.

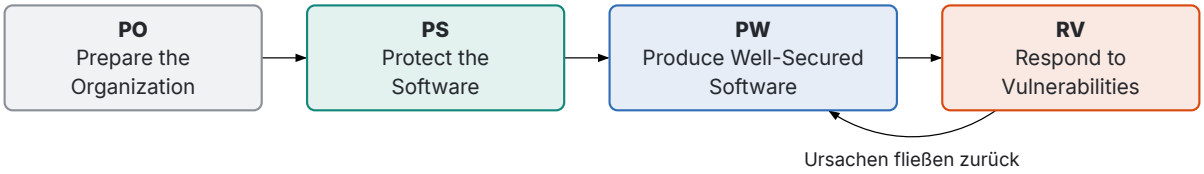



**Abb. 11:** Die vier Praktikengruppen des SSDF [44].

**Prepare the Organization (PO)** betrifft die Voraussetzungen: Sicherheitsanforderungen für die Entwicklung sind festgelegt, Rollen und Verantwortungen sind vergeben, die Werkzeugkette ist eingerichtet, und es gibt Kriterien, wann Software als geprüft gilt.

**Protect the Software (PS)** betrifft den Schutz der Software selbst vor Manipulation: Der Quelltext ist vor unbefugtem Zugriff geschützt, die Echtheit einer Auslieferung lässt sich prüfen, und jede Auslieferung wird archiviert. Der Fall der Hintertür in xz aus Kapitel 4 ist ein Versagen in dieser Gruppe, nur eben in einem Projekt, von dem andere abhängen.

**Produce Well-Secured Software (PW)** ist die größte Gruppe und betrifft die Entwicklung im engeren Sinn: sicher entwerfen, den Entwurf prüfen, vorhandene sichere Bausteine wiederverwenden, sicher programmieren, den Code prüfen, das ausführbare Programm testen und sichere Voreinstellungen ausliefern.

**Respond to Vulnerabilities (RV)** betrifft den Umgang mit dem, was trotzdem durchrutscht: Schwachstellen erkennen und bestätigen, sie bewerten, priorisieren und beheben, und ihre Ursachen untersuchen.

Jede Praktik hat eine Kennung aus dem Gruppenkürzel und einer Nummer. Vier davon sind für dieses Buch besonders wichtig.

**Tab. 16:** Vier Praktiken des SSDF, die den Weg vom Befund zur Ursache beschreiben [44].

| Kennung | Praktik | Bezug zu diesem Buch |
|---|---|---|
| PW.8 | Test Executable Code: das lauffähige Programm auf Schwachstellen testen | hier ist der Pentest zu Hause |
| RV.1 | Identify and Confirm Vulnerabilities on an Ongoing Basis | Befunde entgegennehmen und bestätigen (Kapitel 15) |
| RV.2 | Assess, Prioritize, and Remediate Vulnerabilities | bewerten, priorisieren, beheben (Kapitel 14 und 17) |
| RV.3 | Analyze Vulnerabilities to Identify Their Root Causes | Ursachenanalyse (Kapitel 16) |

RV.3 ist die Praktik, um die sich die Spezialisierung dreht. Das SSDF verlangt dort nicht nur, die gefundene Schwachstelle zu beheben. Es verlangt, ihre Ursache zu ermitteln, nach weiteren Vorkommen derselben Art zu suchen und den Entwicklungsprozess so zu ändern, dass sie künftig nicht mehr entsteht. Der Rückpfeil in Abb. 11 steht dafür. Eine Organisation, die Befunde nur einzeln schließt, erfüllt RV.2 und verfehlt RV.3.

> **BRÜCKE ZU SOFTWARE SECURITY**
> Wenn dich jemand fragt, wo der Inhalt der Spezialisierung in einem anerkannten Rahmenwerk steht, ist die Antwort: SSDF, Praktiken RV.1 bis RV.3. Die Pflicht-Lehrveranstaltung Software Security deckt vor allem die Gruppe PW ab, die Pentesting-Lehrveranstaltung die Praktik PW.8. Die Spezialisierung schließt den Kreis über RV zurück zu PW. Das ist eine brauchbare Kurzfassung der „Story“ des Studienplans in der Sprache eines Standards.

## 5.3 Reifegradmodelle: SAMM und BSIMM

Das SSDF sagt, was getan werden soll. Es sagt nicht, wie gut eine Organisation darin schon ist und in welcher Reihenfolge sie besser werden soll. Dafür gibt es Reifegradmodelle. Zwei sind verbreitet, und sie gehen gegensätzlich vor.

**DEFINITION 5.3 · SAMM**
Das *Software Assurance Maturity Model* der OWASP beschreibt, was eine Organisation für sichere Software tun sollte, in fünf Geschäftsfunktionen mit je drei Sicherheitspraktiken. Für jede Praktik sind drei Reifestufen mit prüfbaren Kriterien festgelegt [46].

Die fünf Geschäftsfunktionen von SAMM folgen dem Lebenszyklus: *Governance*, *Design*, *Implementation*, *Verification* und *Operations*. Zu *Design* gehören zum Beispiel die Praktiken Bedrohungsbewertung, Sicherheitsanforderungen und sichere Architektur. Zu *Verification* gehören die Bewertung der Architektur, das anforderungsgetriebene Testen und das Sicherheitstesten. Der Pentest fällt unter die letzte. Eine Organisation bewertet sich in jeder der fünfzehn Praktiken auf einer Stufe von null bis drei und legt fest, wohin sie will.

SAMM ist ein **vorschreibendes** Modell: Es sagt, was gut wäre. Das zweite Modell fragt etwas anderes.

**DEFINITION 5.4 · BSIMM**
Das *Building Security In Maturity Model* ist eine jährlich aktualisierte Erhebung dessen, was Unternehmen in ihren Programmen für Softwaresicherheit tatsächlich tun. Es ordnet die beobachteten Aktivitäten in zwölf Praktiken in vier Domänen und zeigt, wie verbreitet jede Aktivität ist [47].

BSIMM ist ein **beschreibendes** Modell. Es behauptet nicht, dass eine Aktivität richtig ist, sondern stellt fest, wie viele der untersuchten Unternehmen sie ausüben. Eine Organisation kann sich damit vergleichen: Tun wir, was die meisten in unserer Branche tun? Der Penetrationstest ist in BSIMM eine eigene Praktik, und das Modell unterscheidet, ob eine Organisation nur externe Tester beauftragt, ob sie die Ergebnisse in ihr Fehlerverfolgungssystem einspeist und ob sie daraus Rückschlüsse für die Entwicklung zieht.

**Tab. 17:** Die beiden verbreiteten Reifegradmodelle im Vergleich.

| | SAMM | BSIMM |
|---|---|---|
| Herausgeber | OWASP, frei verfügbar | kommerzieller Anbieter, Bericht frei verfügbar |
| Art | vorschreibend: was man tun sollte | beschreibend: was andere tun |
| Aufbau | 5 Geschäftsfunktionen, 15 Praktiken, 3 Reifestufen | 4 Domänen, 12 Praktiken, Aktivitäten nach Verbreitung |
| Frage | Wo stehen wir, und was ist der nächste Schritt? | Wo stehen wir im Vergleich zu anderen? |
| Einsatz | Selbstbewertung und Fahrplan | Vergleich und Argumentation gegenüber der Leitung |

Wozu braucht jemand, der Pentest-Berichte liest, ein Reifegradmodell? Weil es die Frage beantwortet, was nach dem Bericht passieren soll. Ein Bericht mit dreißig Befunden lässt sich auf zwei Arten lesen. Als Liste von dreißig Reparaturen. Oder als Messung: In welchen Praktiken ist diese Organisation offenbar schwach? Häufen sich Befunde zur Zugriffskontrolle, fehlt es an Sicherheitsanforderungen und Architekturprüfung. Häufen sich veraltete Komponenten, fehlt es am Umgang mit Abhängigkeiten. Das Reifegradmodell gibt den Befunden einen Ort und der Verbesserung eine Reihenfolge.

## 5.4 Die automatisierten Prüfungen

Zwischen den frühen Tätigkeiten auf dem Papier und dem Pentest am Ende liegen die automatisierten Prüfungen, die in der Build-Pipeline bei jeder Änderung mitlaufen. Sie tragen Kürzel, die in jedem Gespräch über sichere Entwicklung fallen.

**DEFINITION 5.5 · SAST, SCA, DAST, IAST**
**SAST** (*Static Application Security Testing*) untersucht den Quelltext, ohne ihn auszuführen. **SCA** (*Software Composition Analysis*) ermittelt, welche Fremdkomponenten eine Anwendung enthält, und gleicht sie mit Verzeichnissen bekannter Schwachstellen ab. **DAST** (*Dynamic Application Security Testing*) schickt Anfragen an die laufende Anwendung und wertet die Antworten aus. **IAST** (*Interactive Application Security Testing*) beobachtet die laufende Anwendung von innen, während sie getestet wird.

Die vier Verfahren unterscheiden sich darin, was sie sehen können, und daraus folgt, was sie finden und übersehen.

**SAST** sieht den Code, aber nicht die Laufzeit. Es verfolgt, wie Daten von einer Eingabe durch das Programm zu einer gefährlichen Stelle fließen, etwa zu einer Datenbankabfrage. Das findet viele Fehler aus Kapitel 8 zuverlässig. Es findet nicht, was nicht im Code steht: Konfiguration, das Verhalten der Plattform, das Zusammenspiel mehrerer Services. Und es meldet viel Falsches, weil es nicht weiß, ob ein verdächtiger Pfad im Betrieb überhaupt erreichbar ist.

**SCA** ist eine Buchhaltung. Das Werkzeug liest die Liste der Abhängigkeiten und fragt für jede die Schwachstellen-Datenbanken ab. Sein Ergebnis sind CVE-Nummern. Es findet verlässlich, dass eine Bibliothek mit bekannter Lücke enthalten ist. Es weiß nicht, ob die Anwendung den verwundbaren Teil der Bibliothek überhaupt verwendet. Dein Foliensatz zu den Grundlagen nennt den Anteil von Fremdcode in heutigen Anwendungen mit 70 bis 90 Prozent. Entsprechend lang sind die Ergebnislisten, und entsprechend wichtig ist die Priorisierung mit KEV und EPSS aus Kapitel 4.

**DAST** sieht, was ein Angreifer von außen sieht. Es kennt weder Code noch Architektur, es probiert Eingaben aus und beobachtet. Im Kern ist es ein automatisierter Ausschnitt dessen, was ein Tester tut. Es findet Fehlkonfigurationen und einfache Fälle von Eingabefehlern am laufenden System. Es scheitert an allem, wofür man die Anwendung verstehen muss: mehrstufige Abläufe, Rollen, Fachlogik.

**IAST** verbindet beide Sichten, indem ein Sensor in der laufenden Anwendung mitverfolgt, welche Codepfade ein Test auslöst. Das verringert falsche Meldungen, setzt aber voraus, dass die Anwendung mit dem Sensor betrieben und überhaupt getestet wird.

Dazu kommen Prüfungen für alles, was nicht Anwendungscode ist: die Suche nach versehentlich eingecheckten Zugangsdaten (*Secrets Scanning*), die Prüfung von Container-Images auf bekannte Lücken und die Prüfung von Infrastrukturbeschreibungen auf unsichere Einstellungen. Und es gibt das Fuzzing, das du in deinem Foliensatz zu Software Testing behandelst.

**Tab. 18:** Automatisierte Prüfverfahren und der Pentest im Vergleich.

| Verfahren | Sieht | Findet gut | Übersieht |
|---|---|---|---|
| SAST | Quelltext | Datenflüsse von Eingabe zu gefährlicher Stelle, unsichere Funktionen | Konfiguration, Zusammenspiel, Fachlogik |
| SCA | Liste der Abhängigkeiten | Komponenten mit bekannter Lücke (CVE) | ob der verwundbare Teil genutzt wird, unbekannte Lücken |
| DAST | laufende Anwendung von außen | Fehlkonfiguration, einfache Eingabefehler | alles hinter mehrstufigen Abläufen und Rollen, Fachlogik |
| IAST | laufende Anwendung von innen | bestätigte Datenflüsse mit Fundstelle im Code | nicht ausgelöste Pfade |
| Pentest | laufende Anwendung, je nach Vorwissen auch Code und Architektur | Fachlogik, Zugriffskontrolle, Verkettung mehrerer Schwächen | alles außerhalb von Scope und Zeit |

Die letzte Zeile von Tab. 18 zeigt, wofür man Menschen bezahlt. Die drei Dinge, die der Pentest gut findet, sind genau die, die kein Werkzeug findet: Fehler in der Fachlogik, Fehler in der Zugriffskontrolle, die vom Verständnis der Rollen abhängen, und Verkettungen. Daraus folgt eine Faustregel für die Qualität eines Tests: Ein Pentest, dessen Befunde ein Scanner auch gefunden hätte, war sein Geld nicht wert. Und umgekehrt: Wenn ein Pentest viele Befunde liefert, die SAST oder SCA hätten finden müssen, fehlen diese Prüfungen im Prozess, und das ist der eigentliche Befund.

> **IN DER REIHE**
> Wie die automatisierten Prüfungen in eine Pipeline eingebaut werden, mit Stufen, Gates und dem Umgang mit Geheimnissen, steht in *Git von Grund auf*, Kapitel 17. Das Prüfen von Images und ihre Herkunft behandelt *Docker von Grund auf*, Kapitel 12. Statische Analyse, Fuzzing aus der API-Beschreibung und ein dynamischer Scan für APIs stehen in *Secure API Design*, Kapitel 11.

## 5.5 Wo der Pentest sitzt

Setzt man alles zusammen, ergibt sich ein Bild des Lebenszyklus, in dem jede Prüfung ihren Ort hat.

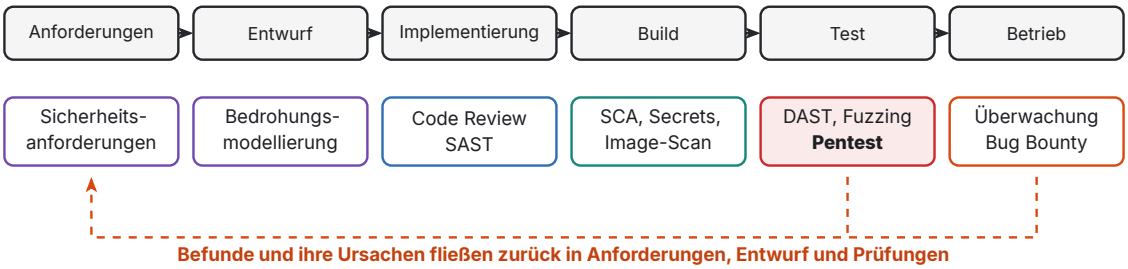



**Abb. 12:** Prüfungen im Lebenszyklus. Der Pentest sitzt am rechten Rand, sein Ergebnis wirkt bis ganz nach links.

Der Pentest steht in Abb. 12 in der Testphase, also spät. Sein Wert hängt davon ab, ob der gestrichelte Pfeil existiert. In einer Organisation ohne diesen Pfeil ist der Pentest eine jährliche Reparaturliste. In einer mit Pfeil ist er eine Messung, die den Prozess verbessert: Ein Befund zur Zugriffskontrolle führt zu einer neuen Sicherheitsanforderung, zu einer Regel für die Architekturprüfung und zu einem automatisierten Test, der denselben Fehler künftig in der Pipeline abfängt.

## 5.6 Nach der Auslieferung

Der Lebenszyklus endet nicht mit dem Release. Für den Umgang mit Schwachstellen in ausgelieferter Software haben sich eigene Begriffe und in Europa inzwischen gesetzliche Pflichten entwickelt.

> **DEFINITION 5.6 · SBOM**
> Eine *Software Bill of Materials* ist eine maschinenlesbare Stückliste aller Komponenten, aus denen eine Software besteht, mit Namen, Versionen und Herkunft. Verbreitete Formate sind CycloneDX und SPDX [48].

Die Stückliste beantwortet die Frage, an der bei Log4Shell so viele gescheitert sind: Wo überall steckt diese Bibliothek? Wer für jede seiner Anwendungen eine aktuelle SBOM hat, kann nach Bekanntwerden einer neuen Lücke in Minuten sagen, was betroffen ist. Als Ergänzung gibt es *VEX*, kurz für *Vulnerability Exploitability eXchange*: eine maschinenlesbare Aussage des Herstellers, ob sein Produkt von einer bestimmten Lücke in einer enthaltenen Komponente tatsächlich betroffen ist. Damit lässt sich der häufige Fall ausdrücken, dass eine verwundbare Bibliothek zwar enthalten ist, der verwundbare Teil aber nie aufgerufen wird.

Zur Lieferkette gehört auch die Frage, ob ein ausgeliefertes Artefakt wirklich aus dem geprüften Quelltext gebaut wurde. Dafür gibt es das Rahmenwerk *SLSA*, das Stufen der Nachweisbarkeit von Herkunft und Build beschreibt [49]. Dein Foliensatz zu den Grundlagen nennt beide Begriffe unter Supply-Chain-Integrität.

Eine Organisation, die Software herstellt, braucht außerdem einen geordneten Weg, auf dem Meldungen von außen ankommen und bearbeitet werden. In größeren Unternehmen gibt es dafür ein eigenes Team, oft *PSIRT* genannt, *Product Security Incident Response Team*. Der abgestimmte Ablauf zwischen Finder und Hersteller, bei dem eine Lücke erst veröffentlicht wird, wenn eine Korrektur bereitsteht, heißt *Coordinated Vulnerability Disclosure*. Die Normen dazu wurden in Kapitel 1 genannt.

### Der Cyber Resilience Act

Mit dem Cyber Resilience Act hat die EU vieles davon zur Pflicht gemacht [10]. Die Verordnung gilt für Produkte mit digitalen Elementen, die in der EU auf den Markt kommen, also für Geräte ebenso wie für eigenständige Software. Hersteller müssen ihre Produkte so entwerfen und entwickeln, dass sie ein angemessenes Sicherheitsniveau haben, sie müssen eine Stückliste führen, Schwachstellen über die vorgesehene Nutzungsdauer hinweg behandeln, Sicherheitsaktualisierungen bereitstellen und die Sicherheit ihrer Produkte wirksam und regelmäßig testen.

Zwei Daten sind festzuhalten. Seit dem 11. September 2026 gelten die Meldepflichten: Ein Hersteller, der erfährt, dass eine Schwachstelle in seinem Produkt aktiv ausgenutzt wird, muss das binnen 24 Stunden als Frühwarnung melden, mit weiteren Meldungen in den Tagen danach. Ab dem 11. Dezember 2027 gilt

die Verordnung in vollem Umfang. Dein Foliensatz nennt den Cyber Resilience Act unter den Treibern sicherer Softwareentwicklung, das hier ist der Teil davon, der Pentest-Befunde unmittelbar betrifft.

Für die Lehre ist daran vor allem wichtig, was sich dadurch an der Bedeutung eines Befunds ändert. Früher war ein offener Befund ein technisches Risiko, das eine Organisation nach eigenem Ermessen tragen konnte. Für Hersteller im Geltungsbereich der Verordnung ist ein bekannter, unbehandelter Befund auch ein Verstoß gegen eine gesetzliche Pflicht. In der Case Study der Spezialisierung ist das die Position der Compliance, mit der die Studierenden umgehen müssen (Kapitel 18).

## 5.7 Der Befund als Rückmeldung

Damit lässt sich die Leitfrage dieses Kapitels beantworten. Jeder Befund hat eine Stelle im Lebenszyklus, an der er hätte verhindert oder früher gefunden werden können. Tab. 19 ordnet typische Befunde zu.

**Tab. 19:** Befunde als Rückmeldung an den Lebenszyklus.

| Befund im Bericht | Hätte verhindert werden können durch | Hätte früher gefunden werden können durch |
|---|---|---|
| Kunde kann fremde Rechnungen abrufen | Sicherheitsanforderung zur Besitzprüfung, zentrale Durchsetzung in der Architektur | Architekturprüfung, automatisierter Test der Berechtigungen |
| SQL-Injection im Suchfeld | sichere Programmierpraktik (parametrisierte Abfragen) | SAST, Code Review |
| Bibliothek mit bekannter kritischer Lücke | Regel zur Auswahl und Aktualisierung von Komponenten | SCA in der Pipeline |
| Debug-Endpunkt in der Produktion erreichbar | sichere Voreinstellungen, getrennte Konfiguration je Umgebung | Prüfung der Konfiguration, DAST |
| Gutschein lässt sich durch gleichzeitige Anfragen mehrfach einlösen | Entwurf des Ablaufs mit atomarer Prüfung | Bedrohungsmodellierung, gezielter Test |
| Services nehmen Nachrichten ohne Prüfung des Absenders an | Vertrauensgrenzen im Entwurf, Authentifizierung zwischen Services | Architekturprüfung |

Die Tabelle zeigt ein Muster, das Teil IV ausbaut. Bei manchen Befunden liegt die Ursache dort, wo auch die Korrektur ansetzt, nämlich im Code. Bei anderen liegt sie eine oder zwei Phasen früher, im Entwurf oder in den Anforderungen, und eine Korrektur im Code behebt nur das Symptom. Und bei allen stellt sich zusätzlich die Frage aus der rechten Spalte: Warum hat keine der Prüfungen im Prozess den Fehler gefunden? Die Antwort darauf ist die Ursache auf der Ebene Lifecycle, und sie fehlt in fast jedem Pentest-Bericht, weil der Tester den Prozess nicht kennt.

**FRAGE AUS DEM HÖRSAAL**

***Wenn wir SAST, SCA und DAST in der Pipeline haben, brauchen wir dann noch einen Pentest?***

Ja, für das, was diese Werkzeuge nicht sehen: Fachlogik, Zugriffskontrolle über Rollen hinweg und Verkettungen. Gute Automatisierung verändert aber, was der Pentest findet. Die einfachen Funde verschwinden, und der Tester kann seine Zeit auf die schwierigen verwenden. Ein Pentest in einer Organisation mit guter Pipeline hat weniger und interessantere Befunde.

**FRAGE AUS DEM HÖRSAAL**

***Ist das SSDF verpflichtend?***

In Europa nicht unmittelbar. Verpflichtend ist es für Lieferanten der US-Bundesverwaltung, die eine entsprechende Bestätigung abgeben müssen. Viele Unternehmen übernehmen die Anforderung in ihre eigenen Lieferverträge. In Europa entsteht eine vergleichbare Wirkung über den Cyber Resilience Act, der Anforderungen an den Entwicklungsprozess stellt, ohne das SSDF zu nennen. Wer nach dem SSDF arbeitet, hat einen großen Teil dieser Anforderungen inhaltlich abgedeckt.

**FRAGE AUS DEM HÖRSAAL**

***Was ist der Unterschied zwischen DevOps und DevSecOps?***

DevOps bezeichnet die Zusammenarbeit von Entwicklung und Betrieb mit weitgehend automatisierter Auslieferung. DevSecOps ist der Zusatz, dass die Sicherheitsprüfungen in dieselbe Automatisierung eingebaut sind, statt als eigener Schritt am Ende von einer eigenen Abteilung durchgeführt zu werden. Inhaltlich sind es die Prüfungen aus Tab. 18, die bei jeder Änderung mitlaufen. Das Wort beschreibt also weniger eine neue Technik als eine Zuständigkeit: Das Team, das baut, prüft auch.

**FÜR DIE LEHRE**

Dieses Kapitel ist das Scharnier zwischen deiner Pflicht-Lehrveranstaltung und der Spezialisierung. In Software Security lernen die Studierenden den Lebenszyklus von links kennen. Hier sehen sie ihn von rechts: vom Befund zurück zur Tätigkeit, die versagt hat.

Eine wirksame Übung ist Tab. 19 ohne die beiden rechten Spalten. Die Studierenden bekommen sechs Befunde und füllen aus, was jeden hätte verhindern und was ihn früher hätte finden können. In der Besprechung zeigt sich regelmäßig, dass sie für fast alles „SAST“ eintragen. Das ist der Anlass, über die Grenzen der Werkzeuge zu sprechen.

Typische Fehlvorstellungen: dass Shift Left bedeute, späte Prüfungen seien überflüssig, dass eine SBOM selbst etwas sicherer mache (sie macht nur sichtbar), und dass ein Reifegradmodell eine Zertifizierung sei.

## Selbsttest

1. Was unterscheidet einen SSDLC von einem SDLC?
2. Nenne die vier Praktikengruppen des SSDF und je ein Beispiel für eine Praktik.
3. Welche Praktik des SSDF beschreibt die Ursachenanalyse, und was verlangt sie über das Beheben hinaus?
4. Worin unterscheiden sich SAMM und BSIMM in ihrem Ansatz?
5. Ein SCA-Werkzeug meldet eine kritische Lücke in einer Bibliothek. Welche Frage kann es nicht beantworten, und mit welcher Angabe ließe sie sich ausdrücken?
6. Warum findet DAST keine Fehler in der Fachlogik?
7. Ein Pentest-Bericht enthält acht Befunde zu Komponenten mit bekannten Lücken. Was ist der Befund hinter den Befunden?
8. Was ändert der Cyber Resilience Act an der Bedeutung eines offenen Befunds für einen Hersteller?

# 6 Angriffsfläche, Vertrauensgrenzen und STRIDE

Eine Entwicklerin fragt bei einer Funktion: Was soll hier passieren? Ein Tester fragt: Was kann man hier passieren lassen? Die erste Frage hat eine Antwort, die in der Spezifikation steht. Die zweite hat viele, und die meisten hat niemand aufgeschrieben. Dieses Kapitel erklärt die Denkwerkzeuge, mit denen ein Tester die zweite Frage systematisch stellt: die Angriffsfläche, die Vertrauensgrenze, das Datenflussdiagramm, das Merkschema STRIDE und den Angriffsbaum. Es sind dieselben Werkzeuge, mit denen ein Architekt sein System vorab prüfen kann, und genau deshalb sind sie die natürliche Brücke zwischen Bauen und Testen.

**IN DIESEM KAPITEL**

- Angriffsfläche und Vertrauensgrenze definieren und in einem Architekturdiagramm einzeichnen
- die vier Fragen der Bedrohungsmodellierung nennen
- ein Datenflussdiagramm lesen und seine fünf Elemente unterscheiden
- die sechs Kategorien von STRIDE erklären und auf die Elemente eines Diagramms anwenden
- aus einer Bedrohung eine Testfrage und eine Gegenmaßnahme ableiten
- einen Angriffsbaum lesen und die Grenzen der Verfahren benennen

## 6.1 Der andere Blick

Stell dir die Funktion „Gutschein einlösen“ der Plattform *Kurbad* vor. Die Spezifikation sagt: Der Kunde gibt einen Code ein, das System prüft, ob der Code gültig und noch nicht eingelöst ist, zieht den Betrag vom Preis ab und markiert den Gutschein als verwendet. Die Entwicklerin baut das, schreibt Tests für gültige, ungültige und schon eingelöste Codes, und alles ist grün.

Der Tester sieht dieselbe Funktion und stellt andere Fragen. Was passiert, wenn ich denselben Code in zwei Anfragen gleichzeitig schicke? Kann ich einen Gutschein auf eine Buchung anwenden, die nicht mir gehört? Lässt sich der Betrag in der Anfrage ändern? Wie viele Codes kann ich pro Minute durchprobieren? Was steht in der Fehlermeldung bei einem falschen Code, verrät sie, ob der Code existiert? Wer kann Gutscheine ausstellen, und wird das irgendwo festgehalten?

Keine dieser Fragen steht in der Spezifikation, und keine wird von den Tests der Entwicklerin beantwortet. Das liegt nicht an mangelnder Sorgfalt. Dein Foliensatz zu Software Testing nennt den Grund: Wer sein eigenes Werk prüft, prüft, was er erwartet. Der Tester erwartet nichts. Er hat aber auch kein Zaubermittel. Seine Fragen folgen einem Raster, und dieses Raster lässt sich lernen.

Das Raster hat drei Teile. Zuerst wird festgestellt, wo das System überhaupt berührt werden kann (Angriffsfläche). Dann, wo es Annahmen darüber trifft, wem es vertrauen darf (Vertrauensgrenzen). Und schließlich wird für jede dieser Stellen gefragt, was dort schiefgehen kann (STRIDE).

## 6.2 Die Angriffsfläche

**DEFINITION 6.1 · ANGRIFFSFLÄCHE**
Die Angriffsfläche (*attack surface*) eines Systems ist die Gesamtheit der Stellen, an denen jemand von außerhalb einer Vertrauensgrenze Daten hineingeben, Daten herausbekommen oder eine Funktion auslösen kann.

Die naheliegenden Stellen sind die Eingabefelder und Endpunkte, die für Benutzer gedacht sind. Die Angriffsfläche ist aber fast immer größer als das, woran das Team denkt. Für das Beispielsystem gehören dazu:

- jeder Endpunkt der API, auch die, die nur die mobile App verwendet, und die, die aus einer früheren Version übrig sind
- die Admin-Oberfläche, auch wenn sie „nur intern" gedacht ist
- jede Stelle, an der Dateien angenommen werden, etwa der Upload eines Profilbilds
- die Rückmeldungen des Zahlungsdienstleisters, die als Anfragen von außen bei *Kurbad* ankommen
- die Nachrichten auf dem Broker, aus Sicht jedes Service, der sie liest
- Verweise in E-Mails, etwa zum Zurücksetzen des Passworts
- Schnittstellen für Überwachung, Diagnose und Verwaltung
- der Weg, auf dem neue Versionen der Software und ihrer Abhängigkeiten ins System kommen
- die Menschen mit Zugang: Kassenkräfte, Administratoren, der Kundendienst

Die letzten beiden Punkte wirken wie eine andere Kategorie, gehören aber dazu. Der Fall xz aus Kapitel 4 war ein Angriff über den Weg der Software ins System, und ein Kundendienst, der am Telefon Passwörter zurücksetzt, ist eine Schnittstelle mit eigener Berechtigungsprüfung.

Zwei Eigenschaften machen den Begriff praktisch nützlich. Die Angriffsfläche lässt sich **aufzählen**: Sie ist eine Liste, die man gegen ein Architekturdiagramm und gegen die Beschreibung der API abgleichen kann. Und sie lässt sich **verkleinern**: Jeder Endpunkt, den es nicht gibt, kann nicht angegriffen werden. Das Abschalten ungenutzter Funktionen, das Schließen nicht benötigter Ports und das Entfernen alter API-Versionen sind die billigsten Sicherheitsmaßnahmen überhaupt. In deinem Foliensatz zu den Architekturtaktiken steht das unter den Taktiken zum Widerstehen gegen Angriffe.

Für den Tester ist das Feststellen der Angriffsfläche der erste Arbeitsschritt, Kapitel 7 beschreibt ihn. Für die Leserin eines Berichts ist sie ein Prüfstein: Zählt der Bericht auf, was er als Angriffsfläche gefunden hat, und stimmt das mit dem überein, was die Architektur erwarten lässt? Tauchen Endpunkte auf, die niemand kennt, ist das schon ein Befund. Fehlen welche, die es geben müsste, wurde nicht alles gesehen.

> **BRÜCKE ZU SOFTWARE SECURITY**
> Die Angriffsfläche lässt sich aus einem C4-Container-Diagramm ablesen: Jede Beziehung, die von einer Person oder einem Fremdsystem zu einem eigenen Container führt, ist ein Teil davon, und jede Beziehung zwischen zwei eigenen Containern ist ein Kandidat für die innere Angriffsfläche. Die Aufgabe „Testschwerpunkte ableiten" am Ende deiner Pflicht-Lehrveranstaltung beginnt genau mit dieser Aufzählung.

## 6.3 Vertrauensgrenzen

Der zweite Begriff ist der wichtigste des ganzen Kapitels und vielleicht des ganzen Buchs. Die meisten schweren Befunde, die auf die Architektur zurückgehen, sind Fehler an einer Vertrauensgrenze.

> **DEFINITION 6.2 · VERTRAUENSGRENZE**
> Eine Vertrauensgrenze (*trust boundary*) ist eine Linie im System, an der sich ändert, wie weit den Daten und dem Absender vertraut werden darf. Daten, die eine Vertrauensgrenze überqueren, müssen auf der empfangenden Seite geprüft werden, weil die sendende Seite nicht unter derselben Kontrolle steht.

Die offensichtliche Vertrauensgrenze liegt zwischen dem Internet und dem eigenen System. Dass Eingaben eines anonymen Besuchers geprüft werden müssen, bezweifelt niemand. Die Fehler entstehen an den weniger offensichtlichen Grenzen.

**Zwischen Browser und Server.** Alles, was im Browser oder in der App läuft, steht unter der Kontrolle des Benutzers. Eine Prüfung im Browser, etwa dass ein Feld nur Zahlen annimmt oder dass ein Knopf für Kunden ausgeblendet ist, ist eine Bedienhilfe und keine Sicherheitsmaßnahme. Der Server muss jede Prüfung selbst wiederholen. Ein erheblicher Teil der Befunde aus Kapitel 9 entsteht, weil der Server dem vertraut, was der eigene Client schickt.

**Zwischen angemeldeten Benutzern.** Angemeldet zu sein heißt nicht, alles zu dürfen. Zwischen Kundin A und Kunde B verläuft eine Vertrauensgrenze, ebenso zwischen Kunde und Kassenkraft und zwischen Kassenkraft und Administrator.

**Zwischen eigenen Services.** Hier liegt der häufigste Architekturfehler in verteilten Systemen. Das Gateway prüft die Anmeldung, und die Services dahinter nehmen an, dass jede Anfrage, die bei ihnen ankommt, schon geprüft wurde. Diese Annahme heißt *implizites Vertrauen*. Sie hält genau so lange, bis ein einziger Service übernommen wird oder ein Weg am Gateway vorbei gefunden ist. Dann steht dem Angreifer das gesamte Innere offen. Deine eigene Übersichtsarbeit zu Bedrohungen zwischen Services beschreibt diese Klasse ausführlich [50].

**Zwischen dem System und seinen Zulieferern.** Antworten des Zahlungsdienstleisters, Nachrichten eines Partners, Daten aus einer eingebundenen Bibliothek kommen von außerhalb der eigenen Kontrolle. Dein Foliensatz zur asynchronen Kommunikation bringt das auf den Satz, dass auch eine Nachricht aus dem Inneren des Systems eine nicht vertrauenswürdige Eingabe ist.

**Zwischen Daten und Anweisungen.** Das ist die abstrakteste Grenze und die Wurzel der Injection-Schwächen aus Kapitel 8: Ein Teil dessen, was eine Komponente verarbeitet, ist als Anweisung gemeint, ein anderer als Daten, und die Grenze dazwischen darf von den Daten nicht verschoben werden können.

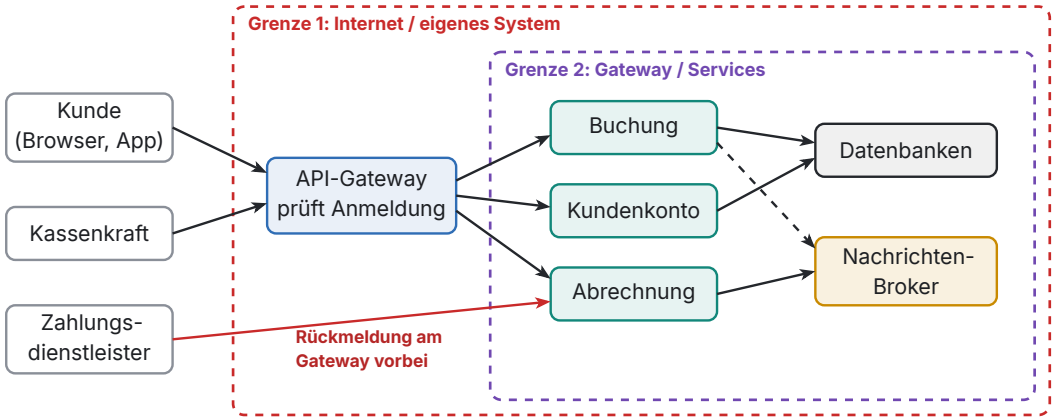



**Abb. 13:** Vertrauensgrenzen im Beispielsystem. Die Rückmeldung des Zahlungsdienstleisters überquert beide Grenzen auf einem Weg, der nicht über das Gateway führt.

Abb. 13 zeigt zwei der Grenzen im Beispielsystem und eine Stelle, an der ein Tester sofort stehen bleibt. Der Zahlungsdienstleister meldet eine erfolgte Zahlung direkt an den Abrechnungsservice. Dieser Weg führt nicht über das Gateway, das die Anmeldung prüft. Die Frage ist: Woran erkennt der Abrechnungsservice, dass die Meldung wirklich vom Zahlungsdienstleister stammt? Wenn die Antwort lautet „an der Absenderadresse" oder „gar nicht", kann jeder, der die Adresse des Endpunkts kennt, eine Buchung als bezahlt markieren. Das steht in keiner Zeile Code als Fehler. Es ist eine Lücke im Entwurf.

> **MERKE**
> An jeder Vertrauensgrenze stellt ein Tester drei Fragen: Wer steht auf der anderen Seite? Woran erkennt das System das? Was passiert, wenn es sich irrt?

Der Gegenentwurf zum impliziten Vertrauen heißt *Zero Trust*: Kein Teil des Systems vertraut einem anderen allein wegen seiner Position im Netz. Jede Anfrage wird authentifiziert und autorisiert, auch zwischen eigenen Services. Die Mittel dafür kennst du aus deinen Foliensätzen: gegenseitig authentifizierte Verbindungen (mTLS), ein Service Mesh, signierte Nachrichten.

> **IN DER REIHE**
> Wie man Vertrauensgrenzen einer API zieht und die Absender zwischen Services prüft, steht in *Secure API Design*, Kapitel 1 bis 3. Das Signieren von Nachrichten und minimale Rechte pro Dienst am Broker behandelt *Sichere Messaging-Architekturen*, Kapitel 9 und 10.

## 6.4 Bedrohungsmodellierung in vier Fragen

Das systematische Durchdenken von Angriffsfläche und Vertrauensgrenzen hat einen Namen.

**DEFINITION 6.3 · BEDROHUNGSMODELLIERUNG**
Bedrohungsmodellierung (*threat modeling*) ist die strukturierte Analyse eines Systems mit dem Ziel, mögliche Bedrohungen zu finden, bevor sie eintreten, und über Gegenmaßnahmen zu entscheiden. Sie arbeitet auf einem Modell des Systems, nicht auf dem laufenden System.

Adam Shostack, dessen Buch das Standardwerk zum Thema ist, fasst das Vorgehen in vier Fragen zusammen [51], die auch das *Threat Modeling Manifesto* übernommen hat [52].

1. **Woran arbeiten wir?** Ein Modell des Systems erstellen: Bestandteile, Datenflüsse, Vertrauensgrenzen.
2. **Was kann schiefgehen?** Bedrohungen finden, mit einem Schema wie STRIDE.
3. **Was tun wir dagegen?** Für jede Bedrohung entscheiden: verhindern, abmildern, übertragen oder bewusst tragen.
4. **Haben wir gute Arbeit geleistet?** Prüfen, ob das Modell stimmt und die Maßnahmen wirken.

Die Bedrohungsmodellierung kommt in diesem Buch an zwei Stellen vor, und es ist wichtig, sie auseinanderzuhalten. Im Entwurf macht sie das Entwicklungsteam, um Fehler zu vermeiden. Das ist die Tätigkeit aus Tab. 15. Im Pentest macht sie der Tester in Phase 3 des PTES, um seine Zeit einzuteilen. Beide stellen dieselben Fragen an dasselbe System. Der Unterschied liegt in Frage 3: Das Team antwortet mit einer Maßnahme, der Tester mit einem Test.

Daraus folgt die schönste Form der Verzahnung. Die vierte Frage, ob das Team gute Arbeit geleistet hat, beantwortet der Pentest. Wenn ein Team ein Bedrohungsmodell hat und es dem Tester gibt, kann der Tester gezielt prüfen, ob die vorgesehenen Maßnahmen halten, und er kann nach Bedrohungen suchen, die im Modell fehlen. Ein Befund ist dann entweder eine Maßnahme, die nicht wirkt, oder eine Bedrohung, an die niemand gedacht hat. Beides ist eine genauere Auskunft als „hier ist eine Lücke“.

## 6.5 Das Datenflussdiagramm

Das übliche Modell für die erste Frage ist das Datenflussdiagramm, kurz DFD. Es ist älter als die Bedrohungsmodellierung und einfacher als jede Architekturnotation: Es zeigt nur, wo Daten herkommen, wo sie verarbeitet werden, wo sie liegen und wie sie fließen.

**Tab. 20:** Die fünf Elemente eines Datenflussdiagramms [51].

| Element | Darstellung | Bedeutung |
|---|---|---|
| Externe Entität | Rechteck | Person oder System außerhalb der eigenen Kontrolle: Kunde, Zahlungsdienstleister |
| Prozess | Kreis oder abgerundetes Rechteck | etwas, das Daten verarbeitet: ein Service, eine Funktion |
| Datenspeicher | zwei parallele Linien | etwas, das Daten aufbewahrt: Datenbank, Datei, Warteschlange |
| Datenfluss | Pfeil | Daten bewegen sich von einem Element zu einem anderen |
| Vertrauensgrenze | gestrichelte Linie | hier ändert sich das Vertrauen |

Abb. 13 ist bereits ein solches Diagramm in vereinfachter Zeichenweise. Wer aus deiner Lehrveranstaltung C4 kennt, findet sich sofort zurecht: Die Container eines C4-Diagramms sind die Prozesse und Datenspeicher, die Personen und Fremdsysteme sind die externen Entitäten, die Beziehungen sind die Datenflüsse. Was C4 nicht von sich aus zeigt, sind die Vertrauensgrenzen. Sie in ein vorhandenes Container-Diagramm einzuzeichnen ist der schnellste Weg zu einem brauchbaren Bedrohungsmodell.

Ein Datenflussdiagramm muss nicht vollständig sein, um nützlich zu sein. Es muss an den richtigen Stellen genau sein: dort, wo Daten eine Grenze überqueren. Ein Diagramm, in dem zwischen zwei Services nur ein Pfeil ohne Beschriftung steht, lässt die entscheidende Frage offen, was dort fließt und wer es prüft.

## 6.6 STRIDE

Für die zweite Frage, was schiefgehen kann, braucht es ein Schema, das verhindert, dass man nur an das denkt, woran man immer denkt. Das bekannteste stammt aus dem Jahr 1999 von Loren Kohnfelder und Praerit Garg bei Microsoft [53].

**DEFINITION 6.4 · STRIDE**
STRIDE ist ein Merkwort für sechs Kategorien von Bedrohungen. Jede Kategorie ist die Verletzung einer Sicherheitseigenschaft: **S**poofing verletzt die Authentizität, **T**ampering die Integrität, **R**epudiation die Nachweisbarkeit, **I**nformation Disclosure die Vertraulichkeit, **D**enial of Service die Verfügbarkeit und **E**levation of Privilege die Autorisierung.

**Tab. 21:** Die sechs Kategorien von STRIDE.

| Kategorie | Verletzt | Die Frage dazu | Beispiel im System *Kurbad* |
|---|---|---|---|
| **S**poofing (Vortäuschen) | Authentizität | Kann sich jemand als ein anderer ausgeben? | jemand schickt eine gefälschte Zahlungsbestätigung im Namen des Zahlungsdienstleisters |
| **T**ampering (Verfälschen) | Integrität | Kann jemand Daten unbefugt verändern? | der Preis einer Buchung wird in der Anfrage geändert |
| **R**epudiation (Abstreiten) | Nachweisbarkeit | Kann jemand eine Handlung bestreiten, ohne dass sich das Gegenteil belegen lässt? | eine Kassenkraft stellt Gutscheine aus, und es wird nicht protokolliert, wer es war |
| **I**nformation Disclosure (Preisgabe) | Vertraulichkeit | Kann jemand Daten sehen, die nicht für ihn bestimmt sind? | ein Kunde ruft die Buchungen eines anderen ab |
| **D**enial of Service (Verweigerung) | Verfügbarkeit | Kann jemand das System für andere unbenutzbar machen? | massenhafte Anfragen blockieren alle freien Termine mit unbezahlten Reservierungen |
| **E**levation of Privilege (Rechteausweitung) | Autorisierung | Kann jemand mehr tun, als ihm zusteht? | ein Kunde ruft eine Funktion auf, die nur Kassenkräften zugedacht ist |

Wer die klassischen drei Schutzziele kennt, also Vertraulichkeit, Integrität und Verfügbarkeit, erkennt sie in der Mitte von Tab. 21 wieder. STRIDE ergänzt drei Eigenschaften, die im Qualitätsmodell ISO/IEC 25010 aus deinem Foliensatz ebenfalls unter Sicherheit stehen: Authentizität, Nachweisbarkeit und, als eigene Kategorie, die Autorisierung.

Die Kategorie, die am häufigsten vergessen wird, ist *Repudiation*. Sie hat keinen Angriff im üblichen Sinn zum Gegenstand, sondern eine Lücke im Nachweis. Eine Anwendung, in der Administratoren Buchungen stornieren können, ohne dass festgehalten wird, wer es war, ist an dieser Stelle verwundbar, auch wenn technisch alles funktioniert. In Pentest-Berichten erscheinen solche Schwächen als „fehlende Protokollierung“ und werden gern als niedrig eingestuft. Ihr Gewicht zeigt sich erst, wenn etwas passiert ist und niemand rekonstruieren kann, was.

### STRIDE auf die Elemente anwenden

STRIDE wird nicht auf das System als Ganzes angewendet, sondern auf jedes Element des Datenflussdiagramms einzeln. Dabei kommen nicht bei jedem Element alle sechs Kategorien in Frage.

**Tab. 22:** Welche Kategorien bei welchem Element in Frage kommen. Bei Datenspeichern ist *Repudiation* relevant, wenn der Speicher Protokolle enthält [51].

| Element | S | T | R | I | D | E |
|---|---|---|---|---|---|---|
| Externe Entität | x | | x | | | |
| Prozess | x | x | x | x | x | x |
| Datenspeicher | | x | (x) | x | x | |
| Datenfluss | | x | | x | x | |

Die Tabelle erspart Arbeit und lenkt den Blick. Ein Datenfluss kann abgehört, verfälscht oder unterbrochen werden. Eine externe Entität kann vorgetäuscht werden oder etwas abstreiten. Nur ein Prozess ist allen sechs Kategorien ausgesetzt, und nur bei einem Prozess gibt es eine Rechteausweitung.

Wie das in der Anwendung aussieht, zeigt die Funktion „Gutschein einlösen“. Beteiligt sind der Kunde als externe Entität, der Buchungsservice als Prozess, die Gutscheintabelle als Datenspeicher und die Datenflüsse dazwischen.

**Tab. 23:** STRIDE angewendet auf die Funktion „Gutschein einlösen“.

| Element | Kat. | Bedrohung | Testfrage |
|---|---|---|---|
| Kunde | S | jemand löst einen Gutschein im Namen eines anderen Kunden ein | Lässt sich die Buchungskennung in der Anfrage gegen eine fremde tauschen? |
| Buchungsservice | T | der Rabattbetrag kommt aus der Anfrage statt aus der Gutscheintabelle | Übernimmt der Server einen Betrag, den der Client mitschickt? |
| Buchungsservice | E | ein Kunde ruft die Funktion zum Ausstellen von Gutscheinen auf | Ist der Endpunkt für Kassenkräfte auch mit einem Kundenkonto erreichbar? |
| Buchungsservice | D | massenhaftes Durchprobieren von Codes | Gibt es eine Begrenzung der Versuche pro Konto und Zeit? |
| Gutscheintabelle | T | zwei gleichzeitige Anfragen lösen denselben Gutschein zweimal ein | Wird der Zustand in einem Schritt geprüft und geändert? |
| Gutscheintabelle | I | die Antwort verrät, ob ein Code existiert | Unterscheiden sich die Meldungen für „unbekannt“ und „schon eingelöst“? |
| Gutscheintabelle | R | Einlösungen und Ausstellungen sind nicht nachvollziehbar | Wird festgehalten, wer wann welchen Gutschein ausgestellt und eingelöst hat? |
| Datenfluss | I | der Code wird unverschlüsselt übertragen oder landet im Protokoll | Erscheint der Code in Adressen oder Protokolldateien? |

Die Spalte „Testfrage“ in Tab. 23 ist der Arbeitsplan des Testers für diese Funktion. Vergleiche sie mit den Fragen am Anfang des Kapitels: Es sind dieselben, nur diesmal vollständig und geordnet. Das ist der ganze Nutzen des Schemas. Es bringt keine Einfälle hervor, auf die ein erfahrener Tester nicht auch so käme. Es sorgt dafür, dass ein weniger erfahrener keine Kategorie vergisst.

Die Zeile zur gleichzeitigen Einlösung kennst du aus deinem Foliensatz zu den Transaktionen: Es ist die Überschreitung eines Limits durch eine Wettlaufsituation, und die Behebung dort ist eine einzige bedingte Aktualisierung, die Prüfen und Ändern in einem Schritt erledigt [54]. Hier siehst du, an welcher Stelle der Bedrohungsmodellierung diese Schwäche gefunden wird: als Verfälschung des Datenspeichers.

## 6.7 Von der Bedrohung zur Maßnahme

Die dritte Frage der Bedrohungsmodellierung lautet, was man gegen eine Bedrohung tut. Für jede STRIDE-Kategorie gibt es eine Familie bewährter Antworten. Sie decken sich mit den Sicherheitstaktiken aus deinem Foliensatz zu den Architekturtaktiken [55].

**Tab. 24:** Gegenmaßnahmen je Kategorie und die zugehörige Prüfung.

| Kategorie | Übliche Gegenmaßnahmen | Woran ein Tester prüft, ob sie halten |
|---|---|---|
| Spoofing | Authentifizierung von Benutzern und Services, signierte Nachrichten, gegenseitig authentifizierte Verbindungen | Wird eine Anfrage ohne oder mit gefälschtem Nachweis abgelehnt? |
| Tampering | Prüfung aller Eingaben am Server, Integritätsschutz durch Signatur oder Prüfsumme, minimale Schreibrechte | Werden vom Client gelieferte Werte übernommen, die der Server selbst bestimmen müsste? |
| Repudiation | manipulationssichere Protokollierung sicherheitsrelevanter Handlungen mit Zeit und Urheber | Hinterlässt eine kritische Handlung eine Spur, und lässt sich die Spur verändern? |
| Information Disclosure | Zugriffskontrolle je Datensatz, Verschlüsselung bei Transport und Speicherung, sparsame Fehlermeldungen | Liefert das System Daten an jemanden, dem sie nicht gehören? |
| Denial of Service | Begrenzung von Anfragen und Ressourcen, Zeitlimits, Isolation durch Bulkheads | Lässt sich mit wenig Aufwand viel Last erzeugen? |
| Elevation of Privilege | Autorisierung bei jeder Funktion, minimale Rechte, Trennung der Rollen | Ist eine Funktion mit zu geringen Rechten aufrufbar? |

Für eine Bedrohung gibt es vier mögliche Entscheidungen. Man kann sie **beseitigen**, indem man die Funktion entfernt, an der sie hängt. Man kann sie **abmildern**, indem man eine Gegenmaßnahme einbaut. Man kann sie **übertragen**, etwa an einen Dienstleister oder eine Versicherung. Oder man kann sie **akzeptieren**, also bewusst und dokumentiert tragen. Die vierte Möglichkeit ist legitim. Nicht legitim ist die häufige fünfte: die Bedrohung zu übersehen und sie dadurch unbewusst zu tragen. Kapitel 18 kommt auf die bewusste Risikoakzeptanz zurück, weil sie in der Case Study eine Rolle spielt.

## 6.8 Angriffsbäume

STRIDE geht vom System aus und fragt, was jedem Teil zustoßen kann. Die umgekehrte Richtung geht vom Ziel des Angreifers aus. Das Werkzeug dafür hat Bruce Schneier 1999 beschrieben [56].

**DEFINITION 6.5 · ANGRIFFSBAUM**
Ein Angriffsbaum (*attack tree*) stellt die Wege dar, auf denen ein Angreifer ein bestimmtes Ziel erreichen kann. Die Wurzel ist das Ziel. Die Knoten darunter sind Teilziele, die entweder alle erreicht werden müssen (UND) oder von denen eines genügt (ODER). Die Blätter sind einzelne Angriffsschritte.

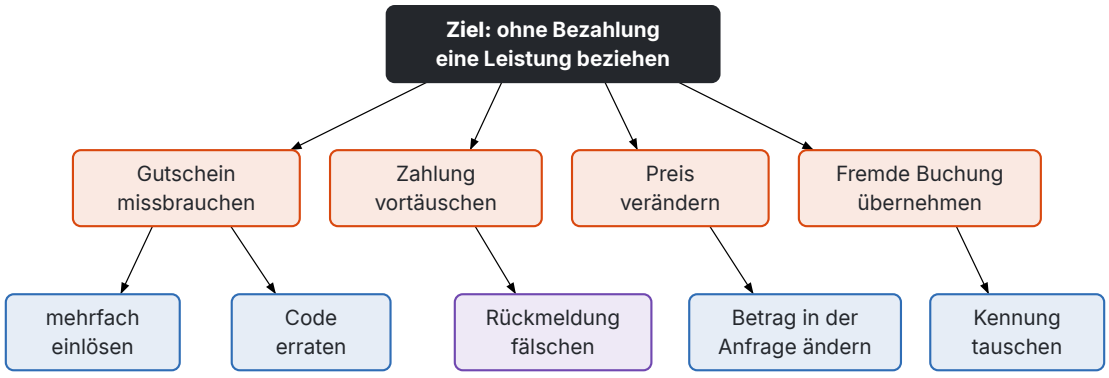



**Abb. 14:** Ein Angriffsbaum für das Beispielsystem. Alle Verzweigungen sind ODER-Verknüpfungen. Blau sind Blätter, deren Ursache im Code liegen kann, violett ist ein Blatt, dessen Ursache im Entwurf liegt.

Ein Angriffsbaum hat gegenüber STRIDE zwei Vorzüge. Er ist vom Geschäft her gedacht: Die Wurzel in Abb. 14 ist ein Schaden, den auch die Geschäftsführung versteht. Und er zeigt Alternativen: Wer einen Ast durch eine Maßnahme abschneidet, sieht, welche anderen Äste zum selben Ziel führen. Das schützt vor dem Irrtum, mit einer einzelnen Korrektur sei das Ziel des Angreifers vereitelt.

Für die Arbeit mit Befunden ist der Baum aus einem dritten Grund nützlich. Die Blätter eines Angriffsbaums sind genau die Dinge, die in einem Pentest-Bericht als einzelne Befunde stehen. Wer die Befunde eines

Berichts in einen Baum einordnet, sieht, welche zum selben Schaden führen, und kann sie gemeinsam bewerten. Kapitel 15 verwendet das beim Bündeln.

## 6.9 Bedrohungen gewichten

Die Bedrohungsmodellierung liefert schnell mehr Bedrohungen, als sich behandeln lassen. Für die Auswahl gilt die Grundformel des Risikos: Wie wahrscheinlich ist es, und wie groß wäre der Schaden? Für die Einschätzung gibt es Verfahren verschiedener Strenge, von einer einfachen Matrix aus drei mal drei Feldern bis zur Methode der OWASP, die Wahrscheinlichkeit und Auswirkung aus je acht Faktoren zusammensetzt [57].

Aus der Frühzeit der Bedrohungsmodellierung stammt ein Bewertungsschema namens DREAD, das dir in älterer Literatur begegnet. Es wird heute kaum noch empfohlen, weil seine Faktoren zu sehr vom Bauchgefühl des Bewertenden abhängen. Für bestätigte Schwachstellen hat sich CVSS durchgesetzt, das Kapitel 14 behandelt. Für Bedrohungen im Entwurf, die es noch gar nicht als Schwachstelle gibt, bleibt die einfache Matrix das ehrlichste Werkzeug.

Eine hilfreiche Ergänzung sind *Missbrauchsfälle* (*abuse cases*), das Gegenstück zu Anwendungsfällen und User Stories. Neben „Als Kunde möchte ich einen Gutschein einlösen“ steht dann „Als Betrüger möchte ich einen Gutschein mehrfach einlösen“. Der Missbrauchsfall macht die Bedrohung zu einer Anforderung, die geplant, umgesetzt und getestet werden kann. Er ist die handlichste Form, in der Ergebnisse der Bedrohungsmodellierung in einem agilen Team ankommen.

## 6.10 Grenzen der Verfahren

Die Werkzeuge dieses Kapitels sind einfach, und ihre Grenzen sind es auch.

**STRIDE findet Kategorien, keine Schwachstellen.** Das Schema sagt, dass am Buchungsservice eine Rechteausweitung denkbar ist. Ob es sie gibt, weiß man erst nach dem Blick in den Code oder nach dem Test. Die Bedrohungsmodellierung ersetzt weder das eine noch das andere.

**Das Ergebnis ist so gut wie das Modell.** Fehlt im Diagramm der Weg vom Zahlungsdienstleister zum Abrechnungsservice, wird die zugehörige Bedrohung nicht gefunden. Veraltete oder geschönte Architekturdiagramme sind die häufigste Ursache wertloser Bedrohungsmodelle. Dein Foliensatz zur Architekturdokumentation nennt das Problem beim Namen: die Abweichung zwischen der beabsichtigten und der tatsächlich gebauten Architektur.

**Fachlogik bleibt schwierig.** Dass ein Storno nach dem Besuch der Therme nicht mehr möglich sein sollte, ergibt sich aus keinem Schema. Es ergibt sich aus dem Verständnis des Geschäfts. Hier helfen Missbrauchsfälle mehr als STRIDE.

**Der Aufwand skaliert schlecht.** Ein vollständiges STRIDE über jedes Element eines großen Systems erzeugt hunderte Einträge, von denen die meisten uninteressant sind. In der Praxis wendet man es auf die Stellen an, an denen Vertrauensgrenzen überquert werden, und auf die Funktionen, an denen Geld oder besonders schützenswerte Daten hängen.

Es gibt weitere Verfahren, deren Namen du einordnen können solltest. *PASTA* ist ein siebenstufiges, stärker am Geschäftsrisiko ausgerichtetes Vorgehen. *LINDDUN* ist ein Schema nach dem Vorbild von STRIDE für Bedrohungen der Privatsphäre. Für den Zweck dieses Buchs genügen STRIDE und der Angriffsbaum.

> **BRÜCKE ZU SOFTWARE SECURITY**
> STRIDE steht nicht im Akkreditierungstext deiner Pflicht-Lehrveranstaltung und wird dort nicht geprüft. Die Denkweise dahinter brauchen die Studierenden trotzdem, sobald sie am Ende von Software Security Testschwerpunkte aus einer Architektur ableiten sollen. Dort genügt die abgespeckte Form: Vertrauensgrenzen einzeichnen und an jeder die drei Fragen aus dem Merkkasten stellen. In der Spezialisierung, in der die Studierenden nach einem Pentest die Architektur umbauen, ist das volle Schema nützlich, um zu prüfen, ob der Umbau neue Bedrohungen erzeugt.

**FRAGE AUS DEM HÖRSAAL**

***Wenn das Entwicklungsteam ein Bedrohungsmodell hat, warum findet der Pentest dann überhaupt noch etwas?***

Aus drei Gründen. Das Modell beschreibt, was gebaut werden sollte, und der Pentest prüft, was gebaut wurde. Dazwischen liegen Fehler in der Umsetzung. Zweitens ist kein Modell vollständig, und die Bedrohungen, an die niemand gedacht hat, findet nur, wer von außen kommt. Drittens ändert sich das System nach dem Modell weiter, und das Modell wird selten nachgezogen. Ein Team mit gutem Bedrohungsmodell bekommt vom Pentest aber andere Befunde: weniger grundsätzliche und mehr solche, die sich in einer Zeile beheben lassen.

**FRAGE AUS DEM HÖRSAAL**

***Ist STRIDE nicht veraltet? Es stammt aus den Neunzigern.***

Die Kategorien sind Verletzungen von Sicherheitseigenschaften, und die haben sich nicht geändert. Was sich geändert hat, sind die Systeme, auf die man das Schema anwendet. Für verteilte Systeme mit vielen Services muss man es auf die Verbindungen zwischen ihnen anwenden, nicht nur auf den Rand. Für Systeme mit KI-Komponenten kommen Bedrohungen hinzu, die sich nur mit Mühe in die sechs Kategorien pressen lassen, etwa das Einschleusen von Anweisungen über Daten. Kapitel 12 zeigt das. Als Einstieg und als Schutz gegen das Vergessen ist STRIDE unverändert brauchbar.

**FRAGE AUS DEM HÖRSAAL**

***Warum zeichnet man die Vertrauensgrenzen nicht einfach um jeden einzelnen Service?***

Genau das ist der Gedanke von Zero Trust, und für besonders kritische Systeme ist er richtig. Er hat einen Preis: Jede Grenze bedeutet Prüfungen, Schlüssel, Zertifikate, Latenz und Betriebsaufwand. Ein kleines Team ohne Erfahrung mit einem Service Mesh kann daran scheitern. Die Entscheidung, wo Grenzen liegen, ist deshalb eine Architekturentscheidung mit einer Abwägung zwischen Sicherheit, Leistung und Betreibbarkeit. Sie gehört in ein Architecture Decision Record, mit Begründung.

**FÜR DIE LEHRE**

In der Spezialisierung ist dieses Kapitel die Grundlage für alles, was mit Ursachen in der Architektur zu tun hat. Lass die Studierenden früh ein Container-Diagramm des Systems aus der Case Study nehmen und die Vertrauensgrenzen einzeichnen. Wenn sie später im Pentest-Bericht einen Befund lesen, sollen sie sagen können, an welcher Grenze er liegt. Befunde, die an keiner Grenze liegen, sind meist reine Code-Fehler. Befunde an einer Grenze, die im Diagramm fehlt, sind Entwurfsfehler.

Eine gute Übung in 45 Minuten: Kleine Gruppen bekommen je eine Funktion des Systems und füllen eine Tabelle wie Tab. 23 aus. Danach tauschen die Gruppen und ergänzen, was fehlt. Fast immer fehlt *Repudiation*, und fast immer fehlen die Bedrohungen an den Verbindungen zwischen den Services.

Typische Fehlvorstellungen: dass eine Prüfung im Client eine Vertrauensgrenze sichere, dass das interne Netz vertrauenswürdig sei, und dass ein Bedrohungsmodell ein Dokument sei, das man einmal schreibt.

## Selbsttest

1. Zähle fünf Teile der Angriffsfläche des Beispielsystems auf, die keine Eingabefelder für Kunden sind.
2. Was ist eine Vertrauensgrenze, und welche drei Fragen stellt ein Tester an jeder?
3. Erkläre „implizites Vertrauen“ zwischen Services und warum es ein Architekturfehler ist.
4. Nenne die vier Fragen der Bedrohungsmodellierung. Welche davon beantwortet ein Pentest?
5. Ordne zu jeder der sechs STRIDE-Kategorien die verletzte Sicherheitseigenschaft zu.
6. Warum ist bei einer externen Entität keine Rechteausweitung zu prüfen?
7. Wende STRIDE auf den Datenfluss „Zahlungsdienstleister meldet Zahlung an Abrechnung“ an. Welche Bedrohung ist die gewichtigste, und welche Gegenmaßnahme gehört dazu?
8. Was zeigt ein Angriffsbaum, das STRIDE nicht zeigt?
9. Nenne zwei Grenzen von STRIDE.

# 7 Aufklärung und Bestandsaufnahme

Bevor ein Tester irgendetwas angreift, will er wissen, was es gibt. Welche Systeme gehören zum Ziel, welche Dienste laufen darauf, welche Endpunkte hat die Anwendung, welche Rollen kennt sie, und was verrät sie über ihr Innenleben? Diese Phase heißt Aufklärung, und sie entscheidet über die Qualität des ganzen Tests, weil niemand prüfen kann, was er nicht gefunden hat. Für dich hat sie einen zweiten Wert: Ihr Ergebnis ist ein Bild des Systems, wie es tatsächlich ist, und dieses Bild weicht fast immer von dem ab, was in der Architekturdokumentation steht.

**IN DIESEM KAPITEL**

- passive und aktive Aufklärung unterscheiden und sagen, was öffentliche Quellen über ein System verraten
- erklären, was Ports, Dienste und Fingerprinting sind und was ein Portscan feststellt
- beschreiben, wie eine Anwendung kartiert wird und wozu eine Rollenmatrix dient
- erklären, was ein Schwachstellenscanner tut und warum seine Ergebnisse geprüft werden müssen
- die Bestandsaufnahme im Bericht als Messung der tatsächlichen Architektur lesen

## 7.1 Der halbe Test

In der Aufteilung der Testzeit aus Kapitel 3 bekam die Aufklärung einen von acht Tagen. Das unterschätzt ihr Gewicht, denn sie hört nie ganz auf. Jeder neue Zugang, jede neue Rolle, jeder übernommene Service eröffnet einen neuen Ausschnitt des Systems, der wieder erkundet wird. Erfahrene Tester sagen, dass die meisten guten Befunde nicht auf einer raffinierten Technik beruhen, sondern darauf, dass jemand eine Stelle gefunden hat, die alle anderen übersehen haben: den vergessenen Endpunkt, die alte Version der Schnittstelle, die Verwaltungsoberfläche auf einem ungewöhnlichen Port.

Das erklärt auch, warum Black-Box-Tests so viel Zeit kosten. Was ein Außenstehender mühsam zusammensuchen muss, steht in einem White-Box-Test in den Unterlagen. Die Aufklärung fällt dort nicht weg, sie ändert ihren Zweck: Der Tester prüft, ob die Unterlagen stimmen.

## 7.2 Passive Aufklärung: was ohne Anfassen zu erfahren ist

Die Aufklärung beginnt mit Quellen, die das Zielsystem gar nicht berühren.

**DEFINITION 7.1 · OSINT**
*Open Source Intelligence* bezeichnet das Gewinnen von Erkenntnissen aus öffentlich zugänglichen Quellen. Im Pentest ist damit die Aufklärung gemeint, die ohne direkte Anfragen an das Zielsystem auskommt.

Was sich auf diesem Weg über ein System wie *Kurbad* herausfinden lässt, überrascht viele Betreiber.

**Namen und Adressen.** Das Namenssystem des Internets ist öffentlich. Aus ihm und aus den öffentlichen Protokollen, in denen jede Ausstellung eines TLS-Zertifikats verzeichnet wird, lassen sich die Rechnernamen einer Organisation weitgehend rekonstruieren. Dabei tauchen regelmäßig Namen auf, die niemand öffentlich machen wollte: eine Testumgebung, eine alte Version, ein Verwaltungszugang.

**Technik.** Stellenanzeigen nennen die eingesetzten Programmiersprachen, Frameworks und Cloud-Dienste. Vorträge und Blogbeiträge des Entwicklungsteams beschreiben die Architektur. Die Antworten des Servers verraten Produkte und oft Versionen.

**Code und Geheimnisse.** Öffentliche Code-Ablagen enthalten immer wieder Dinge, die dort nicht hingehören: Konfigurationsdateien mit Zugangsdaten, interne Adressen, Schlüssel für Schnittstellen. Auch

wenn sie später gelöscht werden, bleiben sie in der Versionsgeschichte. Die mobile App ist öffentlich herunterladbar und lässt sich zerlegen. In ihr stehen die Adressen aller Endpunkte, die sie verwendet, und manchmal Schlüssel, die jemand für geheim hielt.

**Vergangenheit.** Archive des Webs bewahren frühere Fassungen einer Website auf, mitsamt Verweisen auf Funktionen, die es angeblich nicht mehr gibt.

**Menschen.** Namen, Rollen und das Schema der E-Mail-Adressen einer Organisation sind über berufliche Netzwerke leicht zu ermitteln. Für den Test einer Anwendung spielt das nur am Rand eine Rolle, für Angriffe über Täuschung ist es die Grundlage.

Die passive Aufklärung berührt das Ziel nicht und hinterlässt dort keine Spuren. Sie ist deshalb auch der Teil, den ein echter Angreifer ohne Eile und ohne Risiko erledigt, lange bevor irgendjemand etwas bemerken könnte.

AUS DER PRAXIS

Der häufigste Fund der passiven Aufklärung ist keine Schwachstelle im technischen Sinn, sondern ein Geheimnis am falschen Ort: ein Zugangsschlüssel in einer öffentlichen Code-Ablage oder in der App. Der Befund dazu trägt eine niedrige technische Komplexität und kann trotzdem der schwerste des Berichts sein, weil der Schlüssel den Weg an allen anderen Schutzmaßnahmen vorbei öffnet. Seine Ursache liegt nie im Code allein. Sie liegt im Umgang mit Geheimnissen, also auf der Ebene Lifecycle.

IN DER REIHE

Wie Geheimnisse richtig verwaltet werden, steht in *Docker von Grund auf*, Kapitel 12, in *Git von Grund auf*, Kapitel 17, und für APIs in *Secure API Design*, Kapitel 8.

## 7.3 Aktive Aufklärung: was antwortet

Aktiv wird die Aufklärung, sobald der Tester Anfragen an die Zielsysteme schickt. Ab hier braucht er den Auftrag aus Kapitel 2.

### Rechner, Ports und Dienste

Die erste Frage ist, welche Rechner erreichbar sind und was auf ihnen läuft. Dafür braucht es drei Begriffe.

DEFINITION 7.2 · PORT, DIENST, FINGERPRINTING

Ein **Port** ist eine Nummer zwischen 0 und 65535, über die ein Rechner mehrere Netzverbindungen auseinanderhält. Ein **Dienst** ist ein Programm, das an einem Port auf Verbindungen wartet, zum Beispiel ein Webserver an Port 443. **Fingerprinting** ist das Erkennen von Art und Version eines Dienstes oder Betriebssystems aus der Art, wie es auf Anfragen antwortet.

Ein **Portscan** stellt fest, an welchen Ports eines Rechners ein Dienst antwortet. Das Verfahren ist einfach: Das Werkzeug versucht, zu jedem Port eine Verbindung aufzubauen, und wertet aus, was zurückkommt. Ein Port gilt als *offen*, wenn ein Dienst die Verbindung annimmt, als *geschlossen*, wenn der Rechner sie ablehnt, und als *gefiltert*, wenn gar keine Antwort kommt, weil eine Firewall die Anfrage verworfen hat. Das Ergebnis ist eine Liste: An diesem Rechner sind diese Ports offen.

Für jeden offenen Port versucht das Werkzeug anschließend, den Dienst zu erkennen. Viele Dienste stellen sich beim Verbindungsaufbau selbst vor, mit Produktname und Version. Andere erkennt man an Eigenheiten ihres Verhaltens. Am Ende steht eine Tabelle aus Rechner, Port, Dienst und Version.

Diese Tabelle ist für sich schon aufschlussreich. Ein System, das nach außen genau den Port für verschlüsselte Web-Verbindungen anbietet, hat eine kleine Angriffsfläche. Ein System, bei dem zusätzlich eine Datenbank, ein Verwaltungszugang und ein Diagnosedienst von außen erreichbar sind, hat eine große, und zwar unabhängig davon, ob einer dieser Dienste eine bekannte Lücke hat. In deinem Foliensatz zu den Architekturtaktiken ist das die Taktik, die Angriffsfläche durch Schließen ungenutzter Ports zu verkleinern.

Die erkannte Version führt außerdem direkt in die Kataloge aus Kapitel 4. Ist ein Dienst in einer Version im Einsatz, für die CVE-Einträge vorliegen, hat der Tester einen ersten Verdacht. Mehr als ein Verdacht ist es

noch nicht: Die Version kann falsch erkannt sein, die Lücke kann in dieser Konfiguration nicht ausnutzbar sein, oder der Betreiber hat eine Korrektur eingespielt, ohne dass sich die angezeigte Versionsnummer geändert hat.

**Vergessene Systeme**
Der ergiebigste Fund dieser Phase sind Systeme, von denen der Auftraggeber nichts mehr wusste. Dafür gibt es den Begriff *Schatten-IT* und, in den OWASP-Listen für Schnittstellen, die Kategorie der mangelhaften Inventarisierung [58]. Drei Formen kommen ständig vor.

Die **vergessene Umgebung**: eine Test- oder Abnahmeumgebung, die aus dem Internet erreichbar ist, oft mit schwächeren Einstellungen und manchmal mit einer Kopie echter Daten. Die **alte Version**: Die Schnittstelle in Version 1 wurde durch Version 2 ersetzt, läuft aber weiter, ohne die Korrekturen, die Version 2 bekommen hat. Der **Zweitzugang**: eine Verwaltungsoberfläche, eine Überwachungsansicht oder eine Dokumentationsseite, die für den internen Gebrauch gedacht war und nie abgesichert wurde.

> **BRÜCKE ZU SOFTWARE SECURITY**
> Jeder dieser Funde ist ein Unterschied zwischen dokumentierter und tatsächlicher Architektur. Dein Foliensatz zur Architekturdokumentation nennt das Erosion und Drift: Die gebaute Architektur entfernt sich schleichend von der beabsichtigten. Die Bestandsaufnahme eines Pentests ist eine unabhängige Messung dieser Abweichung. Legt man sie neben das Deployment-Diagramm, sieht man, was das Diagramm verschweigt.

## 7.4 Die Anwendung kartieren

Für den Test einer Anwendung ist die Liste der Rechner und Ports nur der Anfang. Die eigentliche Angriffsfläche sind die Funktionen der Anwendung. Der Tester erstellt dafür eine Karte mit vier Schichten.




**Abb. 15:** Die Aufklärung wird von links nach rechts feiner. Werkzeuge leisten die linke Hälfte, Verständnis die rechte.

**Endpunkte.** Jede Adresse, unter der die Anwendung etwas tut, wird erfasst. Der Tester bedient die Anwendung dafür in allen Rollen von Hand, während ein Werkzeug den Datenverkehr mitschneidet, und lässt zusätzlich ein Programm allen Verweisen folgen. Bei Schnittstellen hilft die maschinenlesbare Beschreibung, falls sie erreichbar ist. Ist sie in der Produktion öffentlich abrufbar, ist das selbst ein Fund: Sie ist die vollständige Liste der Angriffsfläche, frei Haus.

Dazu kommen die Endpunkte, auf die kein Verweis zeigt. Sie stehen im Programmcode, den der Browser lädt, in der mobilen App, in älteren Fassungen der Beschreibung. Und manche findet man durch Raten naheliegender Namen. Eine Verwaltungsfunktion, die nur dadurch geschützt ist, dass kein Menüpunkt auf sie zeigt, ist nicht geschützt.

**Parameter.** Für jeden Endpunkt wird festgehalten, welche Eingaben er annimmt: in der Adresse, im Inhalt der Anfrage, in Kopfzeilen und Cookies. Auch hier gibt es Verborgenes. Ein Endpunkt zum Ändern des Profils, der laut Oberfläche Name und Adresse annimmt, akzeptiert vielleicht auch ein Feld für die Rolle, das die Oberfläche nur nicht anzeigt. Diese Schwäche heißt *Mass Assignment*, Kapitel 9 erklärt sie.

**Rollen.** Welche Arten von Benutzern gibt es, und was soll jede dürfen? Diese Frage lässt sich von außen nicht vollständig beantworten. Der Tester braucht dafür eine Auskunft des Auftraggebers oder leitet sie aus der Oberfläche ab.

**Abläufe.** Welche Schritte gehören zusammen und in welcher Reihenfolge? Eine Buchung wird angelegt, bezahlt, bestätigt, eventuell storniert. Die Fehler der Fachlogik sitzen an den Übergängen: Lässt sich ein Schritt überspringen, wiederholen oder nach einem späteren Schritt noch einmal ausführen?

Das wichtigste Arbeitsmittel, das aus dieser Karte entsteht, ist die Rollenmatrix.

**Tab. 25:** Eine Rollenmatrix für das Beispielsystem: was jede Rolle dürfen soll.

| Funktion | anonym | Kunde | Kassenkraft | Administrator |
|---|---|---|---|---|
| Termine ansehen | ja | ja | ja | ja |
| eigene Buchung anlegen und ansehen | nein | ja | ja | ja |
| fremde Buchung ansehen | nein | nein | ja | ja |
| Gutschein einlösen | nein | ja | ja | ja |
| Gutschein ausstellen | nein | nein | ja | ja |
| Buchung ohne Zahlung bestätigen | nein | nein | ja | ja |
| Preise ändern | nein | nein | nein | ja |
| Benutzer und Rollen verwalten | nein | nein | nein | ja |

Die Matrix in Tab. 25 beschreibt den Sollzustand. Der Test der Zugriffskontrolle besteht im Kern darin, jede Zelle mit „nein“ zu überprüfen: Die Funktion wird mit einem Konto der betreffenden Rolle aufgerufen, und das System muss ablehnen. Bei acht Funktionen und vier Rollen sind das überschaubar viele Fälle. Bei einer echten Anwendung mit hundert Funktionen und sechs Rollen sind es mehrere hundert, und dazu kommt die zweite Dimension, die die Matrix gar nicht zeigt: Darf ein Kunde die Funktion nur auf seine eigenen Daten anwenden?

Hier liegt ein Punkt, an dem Entwicklung und Test einander viel Arbeit abnehmen können. Die Rollenmatrix ist eine Sicherheitsanforderung. Gibt es sie schon, weil das Team sie beim Entwurf festgelegt hat, bekommt der Tester sie und prüft dagegen. Gibt es sie nicht, muss er sie erraten, und jede Abweichung, die er meldet, wird zur Diskussion, ob das Verhalten nicht doch so gewollt war.

> IN DER REIHE
> Wie man die Rollenmatrix als automatisierten Test in die Pipeline bringt, zeigt *Secure API Design*, Kapitel 11 („Rechte als Tabelle testen“).

## 7.5 Was ein Schwachstellenscanner tut

Auf die Bestandsaufnahme folgt in den meisten Tests ein automatisierter Scan. Kapitel 1 hat ihn vom Pentest abgegrenzt. Hier geht es darum, was er technisch tut, damit du seine Ergebnisse einordnen kannst.

Ein Scanner arbeitet mit zwei Arten von Prüfungen. Die erste ist ein **Abgleich**: Er nimmt die erkannte Version eines Dienstes oder einer Komponente und schlägt in einer Datenbank nach, welche Lücken für diese Version bekannt sind. Das ist schnell und ungefährlich, beruht aber allein auf der Versionsangabe. Die zweite ist eine **aktive Probe**: Der Scanner schickt eine Anfrage, die bei einem verwundbaren System eine erkennbare Reaktion auslöst, und wertet die Antwort aus. Das ist verlässlicher, kann aber Nebenwirkungen haben.

Scanner für Web-Anwendungen gehen ähnlich vor. Sie rufen jeden gefundenen Endpunkt mit einer Reihe von Testeingaben auf und suchen in den Antworten nach Anzeichen bekannter Fehlerbilder. Das ist das Verfahren, das Kapitel 5 als DAST eingeführt hat.

Zwei Eigenschaften bestimmen, was ein Scan wert ist.

**Mit oder ohne Anmeldung.** Ein Scan ohne Zugangsdaten sieht nur, was ein anonymer Besucher sieht. Bei einer Anwendung, deren Funktionen fast alle hinter der Anmeldung liegen, ist das ein Bruchteil. Ein Scan mit Zugangsdaten sieht mehr, scheitert aber oft an mehrstufigen Anmeldungen und an Abläufen, die eine bestimmte Reihenfolge verlangen.

**Fehler in beide Richtungen.** Ein Scanner meldet Schwächen, die nicht vorliegen, und übersieht Schwächen, die vorliegen. Die ersten heißen falsch-positive, die zweiten falsch-negative Ergebnisse. Falsch-positive entstehen vor allem beim Versionsabgleich und bei mehrdeutigen Antworten. Falsch-negative entstehen überall dort, wo das Erkennen ein Verständnis der Anwendung voraussetzt.

**Tab. 26:** Die vier möglichen Ausgänge einer automatisierten Prüfung.

| | Scanner meldet eine Schwäche | Scanner meldet nichts |
|---|---|---|
| Schwäche liegt vor | richtig-positiv: echter Fund | falsch-negativ: übersehen, vor allem Fachlogik und Zugriffskontrolle |
| Schwäche liegt nicht vor | falsch-positiv: Fehlalarm, kostet Prüfaufwand und Vertrauen | richtig-negativ |

Die Tabelle erklärt, warum die Ergebnisse eines Scans in einem Pentest nur Rohmaterial sind. Der Tester prüft jede Meldung nach und nimmt in den Bericht nur auf, was er bestätigt hat. Kapitel 15 zeigt, woran du erkennst, ob das geschehen ist.

## 7.6 Was Systeme über sich verraten

Ein eigener Teil der Aufklärung gilt den Auskünften, die ein System ungefragt gibt. Für sich genommen sind sie harmlos. Zusammen ersparen sie dem Angreifer das Raten.

**Fehlermeldungen.** Eine ausführliche Fehlermeldung mit dem Verlauf der aufgerufenen Funktionen zeigt Namen von Klassen und Dateien, das verwendete Framework, manchmal die fehlgeschlagene Datenbankabfrage. Sie verwandelt einen Black-Box-Test für einen Moment in einen White-Box-Test.

**Kopfzeilen und Standardseiten.** Viele Server nennen in jeder Antwort Produkt und Version. Standardseiten, Beispielanwendungen und Dokumentation, die mit einem Produkt ausgeliefert werden, verraten, was im Einsatz ist und dass es niemand aufgeräumt hat.

**Unterschiedliche Antworten.** Antwortet die Anmeldung bei einem unbekannten Benutzernamen anders als bei einem falschen Passwort, lässt sich herausfinden, welche Konten es gibt. Derselbe Effekt entsteht durch messbar unterschiedliche Antwortzeiten.

**Kommentare und Reste.** Im ausgelieferten Code für den Browser stehen Kommentare der Entwickler, auskommentierte Funktionen, Verweise auf interne Systeme.

**Metadaten.** Dokumente, die eine Organisation veröffentlicht, enthalten Namen von Verfassern, Pfade und Programmversionen.

In Berichten stehen solche Funde als „Preisgabe von Informationen“ mit niedrigem oder rein informativem Schweregrad. Das ist als Einzelbewertung richtig und als Gesamtbild irreführend. Ihr Gewicht liegt darin, dass sie andere Befunde erleichtern. Die genaue Versionsangabe macht aus einem unsicheren Verdacht einen gezielten Versuch. Die Liste gültiger Benutzernamen macht aus blindem Raten ein gezieltes Durchprobieren. Kapitel 15 zeigt, wie solche Stufen in einer Verkettung zu bewerten sind.

In deinem Foliensatz zu den sicheren Programmierpraktiken steht die Gegenmaßnahme unter Fehlerbehandlung. Die OWASP Top 10:2025 haben dem Thema mit A10, dem falschen Umgang mit Ausnahmesituationen, eine eigene Kategorie gegeben [26].

> IN DER REIHE
> Einheitliche Fehlerantworten, Request-IDs und die Frage, was ins Protokoll gehört, behandelt *Secure API Design*, Kapitel 7.

## 7.7 Die Bestandsaufnahme im Bericht

Was aus der Aufklärung in den Bericht gehört, ist mehr als ein Anhang mit Werkzeugausgaben. Ein guter Bericht enthält drei Dinge.

Erstens eine **Liste der gefundenen Systeme und Dienste** mit dem Vermerk, was davon im Scope war. Systeme, die zum Auftraggeber gehören, aber nicht im Scope standen, werden genannt und nicht getestet.

Zweitens eine **Übersicht der Angriffsfläche der Anwendung**: Anzahl und Art der Endpunkte, die Rollen, die geprüften Abläufe. Das ist die Grundlage für die Abdeckungsmatrix aus Kapitel 3.

Drittens die **Abweichungen von der Erwartung**: alles, was es laut Auftraggeber nicht geben sollte, und alles, was es geben sollte und nicht gefunden wurde.

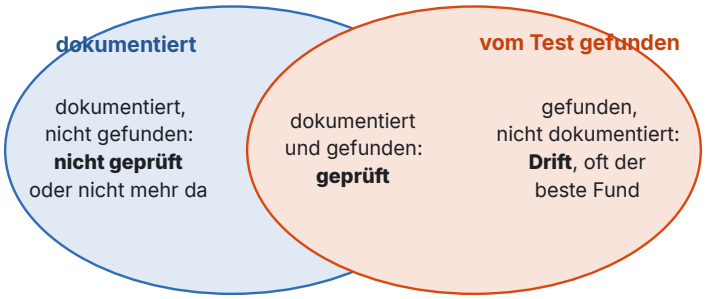



**Abb. 16:** Dokumentierte und gefundene Angriffsfläche. Die beiden äußeren Bereiche sind die interessanten.

Abb. 16 zeigt, wie die Bestandsaufnahme zu lesen ist. Der linke Bereich sagt, wo die Aussagen des Berichts enden: Was dokumentiert ist, aber nicht gefunden wurde, ist ungeprüft. Der rechte Bereich ist ein Befund über die Organisation. Ein System, das niemand kennt, wird von niemandem aktualisiert, überwacht oder abgeschaltet.

## 7.8 Die Sicht der Verteidigung

Aufklärung lässt sich nicht verhindern, aber erschweren und bemerken. In der ATT&CK-Matrix aus Kapitel 4 steht sie als erste Taktik, *Reconnaissance*, und die Erkundung von innen nach einem ersten Zugang als eigene Taktik, *Discovery* [35]. Aus Sicht der Verteidigung folgen daraus drei Aufgaben.

**Wissen, was man hat.** Die eigene Angriffsfläche regelmäßig mit denselben Mitteln zu erfassen, die ein Angreifer verwendet, ist der wirksamste Schutz gegen vergessene Systeme. Dafür hat sich der Begriff *Attack Surface Management* eingebürgert.

**Weniger verraten.** Versionsangaben abschalten, Fehlermeldungen vereinheitlichen, Beispiel- und Standardinhalte entfernen, Schnittstellenbeschreibungen in der Produktion nicht ausliefern. Keine dieser Maßnahmen macht ein verwundbares System sicher. Sie nehmen dem Angreifer aber die Abkürzungen.

**Bemerken.** Ein Portscan und das systematische Abrufen hunderter Adressen in kurzer Zeit sind auffällig. Ob die Überwachung im Betrieb so etwas meldet, ist ein Nebenergebnis jedes Pentests, das im Bericht stehen sollte. Hat der Betrieb vom angekündigten Test nichts bemerkt, hätte er auch von einem unangekündigten nichts bemerkt. In den OWASP Top 10:2025 ist das die Kategorie A09, mangelhafte Protokollierung und Alarmierung [26].

## 7.9 Aufklärung mit KI

Keine Phase des Tests hat sich durch KI-Werkzeuge so sehr verändert wie diese. Das Zusammentragen und Auswerten öffentlicher Quellen, das Lesen von Programmcode aus dem Browser auf der Suche nach Endpunkten, das Verstehen einer fremden Schnittstellenbeschreibung, das Ableiten einer Rollenmatrix aus der Oberfläche: All das sind Aufgaben, bei denen viel Text gelesen und verdichtet wird, und darin sind Sprachmodelle gut.

Die Folge ist zwiespältig. Für Tester wird die linke Hälfte von Abb. 15 billiger, und es bleibt mehr Zeit für die rechte. Für Angreifer gilt dasselbe. Die Schonfrist, die ein unübersichtliches System früher dadurch hatte, dass seine Erkundung mühsam war, schrumpft.

Für die Einschränkungen gilt, was Kapitel 2 gesagt hat: Was der Tester einem KI-Dienst zeigt, verlässt unter Umständen die vereinbarte Umgebung, und was der Dienst behauptet, muss geprüft werden. Ein Sprachmodell, das aus einer Schnittstellenbeschreibung Endpunkte „ableitet", erfindet gelegentlich welche dazu.

> **FRAGE AUS DEM HÖRSAAL**
> ***Ist ein Portscan schon ein Angriff?***
>
> Technisch ist er ein Verbindungsversuch zu vielen Ports, also nichts anderes als das, was ein Browser mit einem Port tut. Rechtlich und praktisch ist die Lage weniger eindeutig. Ein Scan ist die typische Vorbereitung eines Angriffs, er kann empfindliche Systeme stören, und ohne Auftrag gibt es keinen

legitimen Grund, fremde Systeme systematisch abzutasten. Für die Lehre gilt deshalb dieselbe Regel wie für alles andere: nur in der Laborumgebung oder mit Auftrag.

**FRAGE AUS DEM HÖRSAAL**

***Wenn wir unsere Versionsnummern verstecken, sind wir dann sicherer?***

Etwas, aber nicht so, wie man hofft. Das Verstecken erschwert die gezielte Suche und hält automatisierte Massenangriffe ab, die nach bestimmten Versionen suchen. Es ändert nichts an der Lücke selbst, und viele Angriffe werden einfach auf Verdacht ausprobiert. Das Prinzip hat einen Namen, *Security through Obscurity*, und es ist als einzige Maßnahme wertlos und als zusätzliche Schicht in Ordnung.

**FRAGE AUS DEM HÖRSAAL**

***Warum bekommt der Tester nicht einfach immer alle Unterlagen? Dann spart man sich die Aufklärung.***

Für den Test einer Anwendung ist das fast immer die bessere Wahl, wie Kapitel 1 argumentiert hat. Zwei Dinge sprechen dafür, die Aufklärung trotzdem nicht ganz zu streichen. Die Unterlagen können falsch oder veraltet sein, und die Abweichung ist selbst ein Ergebnis. Und der Auftraggeber will manchmal ausdrücklich wissen, was ein Außenstehender ohne Hilfe herausfindet. Ein sinnvoller Mittelweg ist, einen kleinen Teil der Zeit für die Sicht von außen zu verwenden und danach mit vollen Unterlagen weiterzuarbeiten.

**FÜR DIE LEHRE**

In der Spezialisierung bekommen die Studierenden die Bestandsaufnahme als Teil des Pentest-Berichts. Eine lohnende Aufgabe ist der Abgleich mit der Architekturdokumentation des Systems: Was hat der Test gefunden, das in keinem Diagramm steht? Wenn du die Case Study baust, lege bewusst ein oder zwei solche Abweichungen an, etwa eine alte Version der Schnittstelle und eine erreichbare Verwaltungsansicht. Wer sie nur als zwei weitere Befunde abhakt, hat die Ursache nicht gesehen: Es gibt keinen Prozess, der abgelöste Teile stilllegt.

Die Rollenmatrix eignet sich als durchgehendes Arbeitsmittel. Lass die Studierenden sie für das System der Case Study aufstellen, bevor sie die Befunde zur Zugriffskontrolle lesen. Jeder dieser Befunde ist dann eine Zelle, in der Soll und Ist auseinanderfallen.

Typische Fehlvorstellungen: dass Aufklärung „noch kein richtiges Testen“ sei, dass ein Scan-Ergebnis ein Befund sei, und dass Funde mit dem Schweregrad „Info“ ignoriert werden könnten.

## Selbsttest

1. Nenne vier Arten öffentlicher Quellen und je ein Beispiel, was sie über ein Zielsystem verraten.
2. Was bedeuten die Zustände offen, geschlossen und gefiltert bei einem Portscan?
3. Warum ist eine erkannte Version mit bekannter Lücke noch kein Befund?
4. Beschreibe drei Formen vergessener Systeme und sage, warum sie gefährlich sind.
5. Wozu dient eine Rollenmatrix, und welche Dimension der Zugriffskontrolle zeigt sie nicht?
6. Erkläre falsch-positive und falsch-negative Ergebnisse eines Scanners an je einem Beispiel.
7. Ein Bericht enthält sechs Funde mit dem Schweregrad „Info“, alle zur Preisgabe von Versionsangaben und Fehlermeldungen. Warum solltest du sie nicht überblättern?
8. Was sagt es über den Betrieb aus, wenn ein angekündigter Pentest in der Überwachung keinen einzigen Alarm ausgelöst hat?

# 8 Schwachstellenklassen I: Eingaben

Die älteste und bis heute größte Familie von Schwachstellen hat einen einzigen Mechanismus: Ein Programm bekommt Daten von außen und reicht sie an etwas weiter, das sie als Anweisungen versteht. SQL-Injection, Cross-Site Scripting, Command Injection, Path Traversal, SSRF und unsichere Deserialisierung sehen in Berichten wie sechs verschiedene Themen aus. Sie sind sechs Erscheinungsformen desselben Fehlers. Wer den Mechanismus einmal verstanden hat, kann jeden Befund dieser Familie lesen, auch die, deren Namen er noch nie gehört hat. Dieses Kapitel erklärt den Mechanismus, geht die wichtigsten Formen durch und zeigt für jede, wie sie im Bericht aussieht, wo ihre Ursache liegt und wie sie behoben wird.

**IN DIESEM KAPITEL**

- den gemeinsamen Mechanismus der Injection-Schwächen erklären: Daten überschreiten die Grenze zu Anweisungen
- SQL-Injection, Command Injection, Cross-Site Scripting, Path Traversal, SSRF, unsichere Deserialisierung und XXE in ihrer Wirkungsweise beschreiben
- Validierung, Kodierung und Parametrisierung als drei verschiedene Gegenmaßnahmen unterscheiden
- zu jeder Klasse die CWE-Nummer und die Behebung auf den Ebenen Code, Architektur und Lifecycle nennen
- begründen, warum „Eingaben validieren“ als Empfehlung nicht ausreicht

## 8.1 Der gemeinsame Mechanismus

Fast jedes Programm setzt Texte zusammen und übergibt sie an eine andere Komponente, die sie auswertet. Eine Datenbankabfrage ist ein Text, den die Datenbank auswertet. Ein Aufruf eines Systemprogramms ist ein Text, den die Kommandozeile auswertet. Eine Webseite ist ein Text, den der Browser auswertet. Jede dieser Komponenten ist ein *Interpreter*: Sie liest einen Text und entscheidet anhand bestimmter Zeichen, was davon Anweisung ist und was Daten.

Die Schwäche entsteht, wenn in so einen Text Daten eingesetzt werden, die von außen kommen, und wenn diese Daten Zeichen enthalten können, die für den Interpreter eine Bedeutung haben. Dann bestimmt nicht mehr das Programm, wo die Anweisung endet und die Daten beginnen. Das bestimmt, wer die Daten liefert.

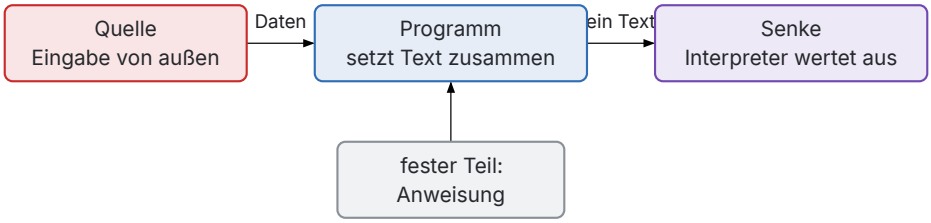



**Abb. 17:** Der Mechanismus aller Injection-Schwächen. Der Interpreter bekommt einen einzigen Text und kann nicht mehr erkennen, welcher Teil vom Programm stammt und welcher von außen.

In Abb. 17 stehen die zwei Begriffe, mit denen Werkzeuge und Tester über diese Schwächen reden. Die **Quelle** (*source*) ist die Stelle, an der nicht vertrauenswürdige Daten ins Programm kommen. Die **Senke** (*sink*) ist die Stelle, an der sie einem Interpreter übergeben werden. Eine Injection-Schwäche liegt vor, wenn es einen Weg von einer Quelle zu einer Senke gibt, auf dem die Daten nicht unschädlich gemacht werden. Statische Analysewerkzeuge suchen genau solche Wege.

In der Sprache von Kapitel 6 ist das ein Fehler an einer Vertrauensgrenze, und zwar an der abstraktesten: der Grenze zwischen Daten und Anweisungen. In der Sprache der CWE ist es der Ast unter `CWE-707`, der mangelhaften Neutralisierung, mit `CWE-74` als allgemeinem Eintrag für Injection.

**Tab. 27:** Dieselbe Schwäche bei verschiedenen Interpretern. Die letzte Zeile ist Thema von Kapitel 12.

| Interpreter | Was er als Anweisung versteht | Name der Schwäche | CWE |
|---|---|---|---|
| Datenbank | SQL-Befehle | SQL-Injection | 89 |
| Kommandozeile | Systembefehle | OS Command Injection | 78 |
| Laufzeitumgebung | Programmcode | Code Injection | 94 |
| Template-Engine | Ausdrücke in Vorlagen | Server-Side Template Injection | 1336 |
| Browser | HTML und JavaScript | Cross-Site Scripting | 79 |
| Dateisystem | Pfadangaben | Path Traversal | 22 |
| HTTP-Client des Servers | Adressen | Server-Side Request Forgery | 918 |
| Deserialisierer | Typangaben und Objektstruktur | unsichere Deserialisierung | 502 |
| XML-Parser | Verweise auf externe Entitäten | XML External Entity | 611 |
| Sprachmodell | Anweisungen in natürlicher Sprache | Prompt Injection | - |

Die Tabelle ist die Landkarte dieses Kapitels. Sie zeigt auch, warum die Familie nicht ausstirbt: Jede neue Technik, die Text auswertet, bringt ihre eigene Form mit. Die jüngste ist das Sprachmodell.

## 8.2 SQL-Injection

Die SQL-Injection ist das Lehrbuchbeispiel der Familie und steht seit über zwanzig Jahren in jeder Rangliste, 2025 auf Platz 2 der CWE Top 25 [25].

Angenommen, der Buchungsservice sucht die Buchungen eines Kunden mit einem bestimmten Status. Die Kundennummer kommt aus der Sitzung, der Status aus der Anfrage.

`BuchungRepository.cs`

```
public List<Buchung> Suche(string kunde, string status)
{
    var sql = "SELECT id, datum, preis FROM buchung " +
              "WHERE kunde = '" + kunde + "' AND status = '" + status + "'";
    var cmd = new SqlCommand(sql, verbindung);
    return Lies(cmd.ExecuteReader());
}
```

Der Code setzt die Abfrage als Text zusammen. Solange `status` Werte wie `bezahlt` enthält, entsteht die beabsichtigte Abfrage. Enthält `status` aber ein einfaches Anführungszeichen, endet für die Datenbank an dieser Stelle der Textwert, und alles, was danach kommt, liest sie als Teil des Befehls. Wer den Wert liefert, kann die Bedingung so ergänzen, dass sie für jede Zeile gilt, und bekommt die Buchungen aller Kunden. Er kann mit weiteren SQL-Mitteln andere Tabellen in das Ergebnis holen, und je nach Datenbank und Berechtigung Daten ändern oder löschen.

Das Wesentliche daran: Die Datenbank tut nichts Falsches. Sie führt einen gültigen Befehl aus, den das Programm ihr geschickt hat. Der Fehler liegt darin, dass das Programm die Struktur des Befehls aus Teilen zusammensetzt, von denen einer nicht ihm gehört.

### Erscheinungsformen

Im Bericht begegnen dir drei Bezeichnungen, die beschreiben, wie der Tester die Schwäche nachgewiesen hat.

Bei der **direkten** Form erscheint das Ergebnis der veränderten Abfrage in der Antwort der Anwendung. Bei der **blinden** Form zeigt die Anwendung keine Ergebnisse an, verhält sich aber je nach Ausgang der Abfrage unterschiedlich, etwa mit einer anderen Seite oder einer messbar längeren Antwortzeit. Daraus lassen sich Daten Zeichen für Zeichen erschließen, was langsam, aber vollständig automatisierbar ist. Bei der Form **zweiter Ordnung** wird die Eingabe zunächst unauffällig gespeichert und erst später, an einer

ganz anderen Stelle, ungeprüft in eine Abfrage eingesetzt. Diese Form wird oft übersehen, weil Entwickler Daten aus der eigenen Datenbank für vertrauenswürdig halten. Das ist derselbe Irrtum wie beim impliziten Vertrauen zwischen Services.

Für die Bewertung ist die Unterscheidung fast gleichgültig. Eine blinde SQL-Injection ist nicht weniger schwer als eine direkte, sie ist nur mühsamer auszunutzen.

**Behebung**

Die Behebung im Code ist die **parametrisierte Abfrage**. Der Befehl wird mit Platzhaltern an die Datenbank geschickt, die Werte getrennt davon.

BuchungRepository.cs

```
public List<Buchung> Suche(string kunde, string status)
{
    var cmd = new SqlCommand(
        "SELECT id, datum, preis FROM buchung " +
        "WHERE kunde = @kunde AND status = @status", verbindung);
    cmd.Parameters.AddWithValue("@kunde", kunde);
    cmd.Parameters.AddWithValue("@status", status);
    return Lies(cmd.ExecuteReader());
}
```

Der Unterschied ist grundsätzlich und kein Trick mit Sonderzeichen. Die Datenbank erfährt die Struktur des Befehls, bevor sie die Werte sieht. Ein Wert kann danach nichts mehr an der Struktur ändern, egal welche Zeichen er enthält. Die Grenze zwischen Anweisung und Daten wird nicht bewacht, sie wird durch die Bauweise unverrückbar. Dein Foliensatz zu den sicheren Programmierpraktiken führt parametrisierte Abfragen aus diesem Grund als eigene Praktik.

Zwei verbreitete Annahmen sind falsch. Ein objektrelationaler Abbilder schützt nur, solange man seine vorgesehenen Wege benutzt. Die meisten erlauben auch, Abfragen als Text zu übergeben, und dann ist die Schwäche wieder da. Die CWE hat dafür eigene Einträge auf der Ebene Variant. Und das Herausfiltern einzelner Zeichen aus der Eingabe ist keine Behebung. Es bricht legitime Eingaben, etwa Namen mit Apostroph, und es wird regelmäßig durch andere Schreibweisen umgangen.

Auf der Ebene Architektur begrenzt das Prinzip der minimalen Rechte den Schaden: Das Datenbankkonto des Buchungsservice braucht keinen Zugriff auf die Tabellen der Abrechnung und kein Recht, Tabellen zu löschen. Eine Datenbank pro Service, wie in deinem Foliensatz zu Microservices, ist auch eine Sicherheitsmaßnahme. Auf der Ebene Lifecycle gehört die SQL-Injection zu den Schwächen, die statische Analyse zuverlässig findet. Taucht sie in einem Pentest auf, stellt sich die Frage, warum sie nicht schon in der Pipeline aufgefallen ist.

**F-03 SQL-Injection im Parameter `status` der Buchungssuche** — KRITISCH

| | |
|---|---|
| **Ort** | `GET /api/v3/buchungen?status=...`, Buchungsservice |
| **Beobachtung** | Der Parameter wird ungeprüft in eine Datenbankabfrage eingesetzt. Mit einem gewöhnlichen Kundenkonto ließen sich die Buchungen aller Kunden abrufen. |
| **Einordnung** | CWE-89, CVSS 3.1: 9.9 (AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H) |
| **Empfehlung** | Parametrisierte Abfragen an allen Stellen, an denen SQL als Text zusammengesetzt wird. Rechte des Datenbankkontos auf die Tabellen des Service beschränken. |

So sieht ein guter Befund aus. Er nennt den Ort genau, sagt, was der Tester erreicht hat, ordnet auf der Ebene Base ein und empfiehlt nicht nur die Korrektur an der einen Stelle, sondern an allen Stellen derselben Art.

## 8.3 Befehle, Code und Vorlagen

Drei Verwandte der SQL-Injection unterscheiden sich nur im Interpreter.

Bei der **Command Injection** (`CWE-78`) setzt die Anwendung einen Befehl für die Kommandozeile des Betriebssystems aus Text zusammen, etwa um ein Bild umzuwandeln oder eine Datei zu packen, und ein Teil davon stammt aus einer Eingabe. Zeichen, mit denen die Kommandozeile Befehle aneinanderreiht,

erlauben es, einen zweiten Befehl anzuhängen. Die Folge ist die Ausführung beliebiger Befehle mit den Rechten der Anwendung und damit in der Regel die Übernahme des Servers. Die Behebung folgt demselben Gedanken wie die parametrisierte Abfrage: Das Programm ruft das Hilfsprogramm direkt auf und übergibt die Argumente als getrennte Liste, statt eine Befehlszeile zu bauen. Noch besser ist es, statt des Hilfsprogramms eine Bibliothek zu verwenden.

Bei der **Code Injection** (`CWE-94`) wird eine Eingabe als Programmcode der Anwendung selbst ausgewertet, zum Beispiel weil eine Funktion zum Auswerten von Ausdrücken auf Benutzereingaben angewendet wird. Die Behebung ist, solche Funktionen nicht auf fremde Daten anzuwenden.

Die **Server-Side Template Injection** (`CWE-1336`) betrifft Vorlagen, aus denen Anwendungen Seiten, E-Mails oder Dokumente erzeugen. Template-Engines kennen Platzhalter und oft eine kleine Ausdruckssprache. Der richtige Gebrauch ist, eine feste Vorlage mit Daten zu füllen. Der Fehler ist, Benutzereingaben in die Vorlage selbst einzubauen, etwa wenn Kunden den Text einer Benachrichtigung anpassen dürfen. Dann wertet die Engine aus, was der Benutzer geschrieben hat.

In diese Gruppe gehört Log4Shell aus Kapitel 4. Die Protokollbibliothek wertete in den Texten, die sie aufzeichnen sollte, eine Ausdruckssprache aus. Jeder Text, der im Protokoll landete, konnte damit Anweisungen enthalten: der Name eines Benutzers, eine Kopfzeile der Anfrage, eine Suchanfrage [20]. Bemerkenswert ist der Ort des Fehlers. Die betroffenen Anwendungen hatten ihre Eingaben oft sauber behandelt. Die Senke lag in einer Bibliothek, von der niemand erwartet hatte, dass sie überhaupt etwas interpretiert.

## 8.4 Cross-Site Scripting

Beim Cross-Site Scripting, kurz XSS, ist der Interpreter der Browser eines anderen Benutzers. Um zu verstehen, warum das schwer wiegt, braucht es einen Begriff aus der Sicherheitsarchitektur des Browsers.

> **DEFINITION 8.1 · SAME-ORIGIN POLICY**
> Die *Same-Origin Policy* ist die grundlegende Vertrauensgrenze des Browsers. Ein Skript darf nur auf Inhalte und Daten zugreifen, die vom selben Ursprung stammen wie es selbst, also von derselben Kombination aus Protokoll, Rechnername und Port. Ein Skript von einer fremden Website kann deshalb nicht lesen, was der Benutzer in der Bankanwendung im Nachbarfenster sieht [59].

Der Browser vertraut also jedem Skript, das eine Website selbst ausliefert, und gibt ihm alle Rechte dieser Website: Es kann die Seite verändern, im Namen des angemeldeten Benutzers Anfragen stellen und lesen, was die Seite anzeigt. Cross-Site Scripting heißt, dass ein Angreifer sein eigenes Skript von der angegriffenen Website ausliefern lässt. Aus Sicht des Browsers stammt es dann vom richtigen Ursprung.

Das gelingt auf dem bekannten Weg. Die Anwendung setzt eine Seite als Text zusammen, baut dabei Daten von außen ein, und der Browser liest spitze Klammern in diesen Daten als Beginn eines HTML-Elements. Steht in einem Kommentarfeld, einem Namen oder einem Suchbegriff ein Element wie `<script>` mit Inhalt, führt der Browser diesen Inhalt aus.

Drei Formen werden unterschieden.

Beim **gespeicherten** XSS legt der Angreifer seine Eingabe in der Anwendung ab, etwa als Bewertung einer Therme. Jeder, der die Seite mit der Bewertung später öffnet, führt das Skript aus. Das ist die schwerste Form, weil sie ohne Zutun des Opfers wirkt und viele trifft, darunter Mitarbeiter mit hohen Rechten, die die Bewertung in ihrer Verwaltungsansicht sehen.

Beim **reflektierten** XSS wird die Eingabe nicht gespeichert, sondern unmittelbar in der Antwort zurückgegeben, zum Beispiel ein Suchbegriff in der Zeile „Ergebnisse für …“. Der Angreifer muss das Opfer dazu bringen, einen vorbereiteten Verweis zu öffnen.

Beim **DOM-basierten** XSS liegt der Fehler nicht im Server, sondern im Code, der im Browser läuft. Er nimmt einen Wert, etwa aus der Adresszeile, und schreibt ihn auf eine Weise in die Seite, bei der der Browser ihn als HTML auswertet. Der Server sieht die Eingabe unter Umständen nie.

Was ein Angreifer mit XSS erreicht, wird in Berichten gern verharmlost, weil der übliche Nachweis ein harmloses Meldungsfenster ist. Das Fenster belegt nur, dass fremder Code läuft. Dieser Code kann alles,

was der angemeldete Benutzer kann: Daten lesen und abschicken, Einstellungen ändern, Buchungen auslösen, die Seite so umbauen, dass sie nach dem Passwort fragt. Trifft es einen Administrator, ist die Anwendung übernommen.

**Behebung**

Die Behebung im Code ist die **kontextgerechte Kodierung bei der Ausgabe**. Bevor Daten in eine Seite eingesetzt werden, werden die Zeichen, die im jeweiligen Kontext eine Bedeutung haben, durch eine harmlose Schreibweise ersetzt. Auf das Wort „Kontext" kommt es an. In einem HTML-Text sind es andere Zeichen als in einem HTML-Attribut, in einem JavaScript-Block oder in einer Adresse. Eine Kodierung für den falschen Kontext schützt nicht.

In der Praxis nimmt ein modernes Framework für Oberflächen diese Arbeit ab. Es kodiert alle Ausgaben standardmäßig richtig, und XSS entsteht dort, wo Entwickler diesen Schutz ausdrücklich umgehen, etwa um Benutzern formatierten Text zu erlauben. Ein Tester sucht deshalb gezielt nach diesen Ausnahmen, und eine Suche im Quelltext nach den Funktionen, die den Schutz abschalten, findet die Kandidaten in Minuten.

Auf der Ebene Architektur gibt es eine zweite Schicht, die *Content Security Policy*. Der Server teilt dem Browser in einer Kopfzeile mit, aus welchen Quellen Skripte geladen werden dürfen und dass in die Seite eingebettete Skripte nicht auszuführen sind. Ein eingeschleustes Skript bleibt dann wirkungslos, selbst wenn die Kodierung an einer Stelle fehlt. Das ist ein Beispiel für gestaffelte Verteidigung: Die zweite Schicht behebt die Schwäche nicht, sie verhindert ihre Ausnutzung. Dazu kommt, Sitzungs-Cookies so zu kennzeichnen, dass Skripte sie nicht lesen können.

> IN DER REIHE
> Sicherheits-Header, CORS und Cookie-Einstellungen für APIs stehen in *Secure API Design*, Kapitel 8.

## 8.5 Dateien: Pfade und Uploads

Auch das Dateisystem ist ein Interpreter, und seine Sprache sind Pfade.

Beim **Path Traversal** (`CWE-22`) baut die Anwendung einen Dateipfad aus einem festen Verzeichnis und einem Namen aus der Anfrage, etwa um eine Rechnung als Dokument auszuliefern. Enthält der Name die Zeichenfolge für „ein Verzeichnis nach oben", verlässt der Pfad das vorgesehene Verzeichnis. Der Angreifer liest dann Dateien, die nie zum Ausliefern gedacht waren: Konfigurationsdateien mit Zugangsdaten, Schlüssel, Quelltext. Die Behebung ist, den fertigen Pfad in seine eindeutige Form aufzulösen und danach zu prüfen, ob er noch im erlaubten Verzeichnis liegt. Besser ist ein Entwurf, in dem der Benutzer gar keinen Dateinamen liefert, sondern eine Kennung, zu der die Anwendung den Pfad selbst nachschlägt.

Der **uneingeschränkte Datei-Upload** (`CWE-434`) ist das Gegenstück beim Schreiben. Nimmt die Anwendung Dateien entgegen, ohne Art und Inhalt zu prüfen, und legt sie an einem Ort ab, an dem der Server sie ausführt oder als Seite ausliefert, kann ein Angreifer eigenen Code hochladen. Die Gegenmaßnahmen liegen vor allem in der Architektur: Hochgeladene Dateien gehören in einen eigenen Speicher, der nichts ausführt, werden unter einem vom System vergebenen Namen abgelegt und von einem anderen Ursprung ausgeliefert als die Anwendung.

> IN DER REIHE
> Datei-Uploads in APIs behandelt *Secure API Design*, Kapitel 5.

## 8.6 Server-Side Request Forgery

Bei der Server-Side Request Forgery, kurz SSRF (`CWE-918`), ist der Interpreter der HTTP-Client des Servers, und die Anweisung ist eine Adresse. Die Anwendung ruft im Auftrag des Benutzers eine Adresse ab: um ein Profilbild von einer angegebenen Adresse zu laden, eine Vorschau für einen Verweis zu erzeugen oder einen Webhook auszulösen. Bestimmt der Benutzer die Adresse frei, kann er den Server Anfragen an Ziele stellen lassen, die er selbst nicht erreicht. Dein Foliensatz zu den sicheren Programmierpraktiken zeigt das Bild dazu: Der Angreifer leiht sich die Netzposition des Servers.

Interessant ist diese Schwäche, weil ihre Schwere fast vollständig von der Architektur abhängt. Der fehlerhafte Code ist immer derselbe, eine ungeprüfte Adresse. Was der Angreifer damit erreicht, entscheidet die Umgebung: welche internen Dienste vom Server aus erreichbar sind und ob sie Anfragen aus dem internen Netz ohne weitere Prüfung annehmen. In Cloud-Umgebungen gibt es einen besonders lohnenden internen Dienst, den Metadatendienst, über den eine Maschine ihre eigenen Zugangsdaten zur Cloud-Plattform abrufen kann.

**STUDIE 8.2 · CAPITAL ONE, 2019**
Der bekannteste dokumentierte Fall ist der Einbruch bei der US-Bank Capital One im Jahr 2019. Eine fehlerhaft konfigurierte Schutzkomponente vor einer Anwendung ließ sich dazu bringen, Anfragen an den Metadatendienst der Cloud-Plattform zu stellen. Die Angreiferin erhielt darüber temporäre Zugangsdaten einer Rolle, die Zugriff auf Speicherbereiche mit Kundendaten hatte, und kopierte Daten von rund hundert Millionen Menschen. Die Analyse des Falls nennt als Ursachen neben der Fehlkonfiguration zu weit gefasste Berechtigungen der Rolle und fehlende Erkennung des ungewöhnlichen Zugriffs [60]. Der Fall ist ein Lehrstück für die drei Ebenen: eine Fehlkonfiguration als Auslöser, eine Architektur ohne Begrenzung des Schadens, ein Betrieb, der den Abfluss nicht bemerkte.

Die OWASP Top 10 führten SSRF 2021 als eigene Kategorie. In der Fassung 2025 ist sie in der Zugriffskontrolle aufgegangen [26]. Das ist folgerichtig: Im Kern geht es darum, dass der Server im Namen des Benutzers auf etwas zugreift, worauf der Benutzer keinen Zugriff haben sollte.

Die Behebung im Code ist eine Liste erlaubter Ziele, mit den Fallstricken, die dein Foliensatz zeigt: Der Vergleich muss den ganzen Rechnernamen treffen, und geprüft werden muss die Adresse, zu der tatsächlich verbunden wird. In der Architektur helfen ein eigener, abgeschotteter Dienst für ausgehende Abrufe, die Beschränkung dessen, was der Server im internen Netz erreicht, und eine abgesicherte Form des Metadatendienstes.

**IN DER REIHE**
SSRF in APIs, das Konsumieren fremder Schnittstellen und das Absichern von Webhooks behandelt *Secure API Design*, Kapitel 9, ausführlich.

## 8.7 Deserialisierung und XML

Zwei weitere Formen kennst du aus deinem Foliensatz, hier nur ihre Einordnung in das Bild.

Bei der **unsicheren Deserialisierung** (`CWE-502`) baut die Anwendung aus einem Datenstrom wieder Objekte auf. Manche Formate und Bibliotheken erlauben dem Datenstrom zu bestimmen, *welche* Klassen dabei erzeugt werden. Dann ist der Datenstrom nicht mehr nur Daten. Er ist eine Bauanleitung, und unter den verfügbaren Klassen finden sich fast immer solche, deren Erzeugung Nebenwirkungen hat, bis hin zur Ausführung von Code. Die Behebung ist, Formate zu verwenden, die nur Daten transportieren, und die Zieltypen im Programm festzulegen statt im Datenstrom.

Bei **XML External Entity**, kurz XXE (`CWE-611`), nutzt der Angreifer eine Eigenschaft des XML-Formats: Ein Dokument kann Verweise auf externe Quellen enthalten, die der Parser beim Lesen auflöst. Ein Parser in seiner Standardeinstellung liest dann auf Anweisung des Dokuments lokale Dateien oder ruft Adressen ab. Die Behebung ist eine Einstellung: die Verarbeitung solcher Verweise abschalten. Das macht XXE zu einem Grenzfall zwischen Code-Fehler und Fehlkonfiguration, und in vielen aktuellen Laufzeitumgebungen ist die sichere Einstellung inzwischen die Voreinstellung.

Beide Fälle zeigen eine Wendung des Grundmusters. Die Anwendung hat hier keinen Text zusammengesetzt. Sie hat ein Format gewählt, das mächtiger ist, als sie braucht, und dessen zusätzliche Fähigkeiten dem Absender der Daten zur Verfügung stehen.

**MERKE**
Frage bei jeder Stelle, an der fremde Daten verarbeitet werden: Kann der Absender damit nur Werte liefern, oder kann er auch bestimmen, was mit ihnen geschieht? Je mächtiger das Format oder die Sprache an dieser Stelle, desto größer die Angriffsfläche.

## 8.8 Drei verschiedene Gegenmaßnahmen

In Berichten steht als Empfehlung zu Befunden dieser Familie sehr oft derselbe Satz: „Eingaben validieren.“ Er ist nicht falsch, aber er vermischt drei Maßnahmen, die Verschiedenes leisten.

**Tab. 28:** Validierung, Kodierung und Parametrisierung.

| Maßnahme | Was sie tut | Wo sie ansetzt | Was sie leistet |
|---|---|---|---|
| Validierung | prüft, ob eine Eingabe dem entspricht, was fachlich erlaubt ist, und weist alles andere zurück | an der Quelle, beim Eintritt ins System | hält offensichtlich Falsches fern, verkleinert die Angriffsfläche, behebt aber keine Injection |
| Kodierung | ersetzt Zeichen mit besonderer Bedeutung durch eine harmlose Darstellung für einen bestimmten Kontext | an der Senke, beim Einsetzen in den Text | verhindert, dass Daten als Anweisung gelesen werden, wenn der Kontext stimmt |
| Parametrisierung | übergibt Anweisung und Daten getrennt an den Interpreter | an der Senke, durch die Schnittstelle selbst | macht die Verwechslung strukturell unmöglich |

Die **Validierung** gehört an den Eingang. Sie prüft gegen die fachliche Erwartung: Ein Status ist einer von fünf festen Werten, eine Kundennummer besteht aus acht Ziffern, ein Datum ist ein Datum. Das ist eine Erlaubnisliste, und Erlaubnislisten sind Verbotslisten immer vorzuziehen, weil niemand alle gefährlichen Eingaben aufzählen kann. Validierung ist gute Praxis und fängt vieles ab. Sie löst das Problem aber nicht grundsätzlich, weil es legitime Eingaben gibt, die gefährliche Zeichen enthalten. Der Name O’Brien ist ein gültiger Name und enthält ein Anführungszeichen. Ein Kommentar darf spitze Klammern enthalten. Die Validierung kann nicht wissen, in welchen Interpreter die Daten später gelangen.

Die **Kodierung** gehört an die Senke, weil erst dort bekannt ist, welche Zeichen gefährlich sind. Dieselben Daten brauchen in einer HTML-Seite eine andere Behandlung als in einer Protokollzeile.

Die **Parametrisierung** ist die stärkste Maßnahme und sollte überall verwendet werden, wo der Interpreter sie anbietet: bei Datenbanken, beim Aufruf von Programmen, bei Template-Engines.

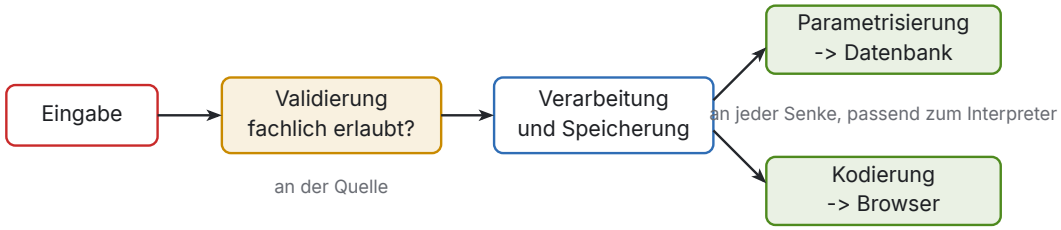



**Abb. 18:** Wo die drei Maßnahmen im Datenfluss sitzen. Validierung am Eingang ersetzt die Maßnahme an der Senke nicht.

Für die Bewertung von Behebungen in der Spezialisierung folgt daraus eine klare Regel. Wurde ein Injection-Befund durch eine zusätzliche Prüfung der Eingabe „behoben“, ist die Schwäche in aller Regel noch da, nur schwerer zu erreichen. Die Behebung ist erst vollständig, wenn die Senke selbst sicher ist.

Dasselbe gilt für vorgeschaltete Schutzsysteme. Eine *Web Application Firewall*, kurz WAF, sitzt vor der Anwendung und blockiert Anfragen, die bekannten Angriffsmustern ähneln. Sie ist ein nützliches Netz für die Zeit, bis eine Korrektur ausgeliefert ist, und sie hält viel automatisierten Lärm ab. Als Behebung taugt sie nicht, denn sie rät anhand von Mustern und kennt weder die Anwendung noch ihre Interpreter. Kapitel 17 kommt bei der Unterscheidung von Beheben und Abmildern darauf zurück.

## 8.9 Die drei Ebenen im Überblick

Zum Abschluss die Klassen dieses Kapitels mit ihren Behebungen auf allen drei Ebenen. Die Tabelle ist als Nachschlagehilfe für die Arbeit an Befunden gedacht.

**Tab. 29:** Behebungen der Eingabeschwächen auf drei Ebenen.

| Klasse | Code | Architektur | Lifecycle |
|---|---|---|---|
| SQL-Injection | parametrisierte Abfragen | minimale Rechte des Datenbankkontos, Datenbank pro Service | statische Analyse in der Pipeline, Regel im Code Review |
| Command Injection | Programme mit Argumentliste aufrufen, Bibliothek statt Hilfsprogramm | Dienst ohne Kommandozeile im Image, minimale Rechte des Prozesses | statische Analyse, minimale Basis-Images |
| Cross-Site Scripting | kontextgerechte Kodierung, Schutz des Frameworks nicht umgehen | Content Security Policy, Cookies vor Skripten schützen | statische Analyse, Prüfregel für Ausnahmen vom Schutz |
| Path Traversal | Pfad auflösen und gegen erlaubtes Verzeichnis prüfen | Kennungen statt Dateinamen, getrennter Dateispeicher | Missbrauchsfall in den Anforderungen |
| Datei-Upload | Art und Größe prüfen, Namen selbst vergeben | Speicher ohne Ausführung, eigener Ursprung für die Auslieferung | Vorgabe in den Architekturrichtlinien |
| SSRF | Erlaubnisliste für Ziele, tatsächlich verbundene Adresse prüfen | abgeschotteter Dienst für ausgehende Abrufe, Netzsegmentierung, abgesicherter Metadatendienst | Bedrohungsmodell für jede Funktion, die Adressen abruft |
| Deserialisierung | reine Datenformate, Zieltypen im Programm festlegen | keine serialisierten Objekte über Vertrauensgrenzen | gefährliche Schnittstellen per Regel verbieten |
| XXE | externe Entitäten im Parser abschalten | XML an Grenzen vermeiden, wo es nicht nötig ist | sichere Voreinstellungen als Vorgabe, aktuelle Laufzeitumgebung |
| Bibliothek mit Injection, etwa Log4Shell | Bibliothek aktualisieren | ausgehende Verbindungen des Servers beschränken | Stückliste, Prüfung der Abhängigkeiten, schneller Aktualisierungsweg |

Lies die Tabelle spaltenweise. Die Spalte Code enthält die eigentliche Korrektur. Die Spalte Architektur enthält, was den Schaden begrenzt, wenn die Korrektur irgendwo fehlt, und das wird sie, weil kein Team jede Stelle erwischt. Die Spalte Lifecycle enthält, was dafür sorgt, dass der Fehler nicht wiederkommt. Ein Befund ist erst dann wirklich erledigt, wenn in jeder Spalte etwas geschehen ist. Das ist der Maßstab, an dem Kapitel 17 Behebungen misst.

**BRÜCKE ZU SOFTWARE SECURITY**

Die Code-Spalte dieser Tabelle deckt sich weitgehend mit deinen Foliensätzen zu den sicheren Programmierpraktiken, die Architektur-Spalte mit den Sicherheitstaktiken und den Mustern für Microservices. Neu ist für die Studierenden der Zusammenhang: dass ein Befund aus einem Bericht an allen drei Stellen eine Antwort braucht. In der Pflicht-Lehrveranstaltung erkennen sie die Muster im Code. In der Spezialisierung entscheiden sie, auf welcher Ebene sie ansetzen.

**FRAGE AUS DEM HÖRSAAL**

***Wir verwenden ein modernes Framework und einen ORM. Gibt es bei uns überhaupt noch Injection?***

Deutlich weniger, aber ja. Die Frameworks haben die naheliegenden Fälle sicher gemacht. Was bleibt, sind die Stellen, an denen jemand den vorgesehenen Weg verlässt: eine handgeschriebene Abfrage für einen Bericht, weil der ORM zu langsam war, formatierter Text von Benutzern, für den der Schutz der Oberfläche abgeschaltet wurde, ein Hilfsprogramm, das über die Kommandozeile aufgerufen wird. Dazu kommen die Senken, an die niemand denkt: Protokolle, Vorlagen für E-Mails, exportierte Tabellen. Ein Tester sucht genau diese Ausnahmen.

**FRAGE AUS DEM HÖRSAAL**

***Warum reicht es nicht, gefährliche Zeichen einfach herauszufiltern?***

Weil niemand vollständig aufzählen kann, was gefährlich ist. Es hängt vom Interpreter ab, von seiner Version, von der Zeichenkodierung und davon, wie die Daten auf dem Weg umgeformt werden. Die Geschichte der Filter ist eine Geschichte ihrer Umgehungen. Außerdem zerstört ein Filter legitime Daten. Die Parametrisierung braucht keine Liste gefährlicher Zeichen, weil sie die Frage gar nicht stellt.

**FRAGE AUS DEM HÖRSAAL**

***Ist XSS wirklich so schlimm? Da geht ja nur ein Fenster auf.***

Das Fenster ist der Nachweis, nicht der Schaden. Es zeigt, dass der Tester eigenen Code im Browser eines anderen Benutzers ausführen kann, im Namen der Anwendung. Derselbe Code kann stattdessen die Daten des Benutzers auslesen, in seinem Namen handeln oder die Anmeldeseite fälschen. Trifft ein gespeichertes XSS einen Administrator in seiner Verwaltungsansicht, hat der Angreifer dessen Rechte. Die niedrige Einstufung vieler XSS-Befunde ist einer der häufigeren Bewertungsfehler.

**FÜR DIE LEHRE**

Für die Spezialisierung ist nicht das Erkennen der Klassen das Lernziel, das können die Studierenden aus den Pflicht-Lehrveranstaltungen. Das Lernziel ist die vollständige Behebung. Gib ihnen einen Injection-Befund samt dem zugehörigen Code und lass sie drei Dinge liefern: die Korrektur an der gemeldeten Stelle, eine Suche nach allen weiteren Stellen derselben Art und je eine Maßnahme auf den Ebenen Architektur und Lifecycle. Beurteile die Suche am strengsten. Wer nur die gemeldete Stelle korrigiert, hat den Befund geschlossen und die Schwäche behalten.

Ein gutes Mittel gegen blindes Übernehmen von KI-Vorschlägen: Lass ein Sprachmodell eine Behebung für einen XSS-Befund vorschlagen. Die Vorschläge filtern oft Zeichen heraus oder kodieren für den falschen Kontext. Die Studierenden sollen begründen, ob der Vorschlag die Senke sicher macht.

Typische Fehlvorstellungen: dass Validierung am Eingang Injection behebe, dass Daten aus der eigenen Datenbank vertrauenswürdig seien, und dass eine WAF eine Behebung sei.

## Selbsttest

1. Beschreibe den gemeinsamen Mechanismus der Injection-Schwächen mit den Begriffen Quelle, Senke und Interpreter.
2. Warum behebt eine parametrisierte Abfrage die SQL-Injection grundsätzlich, während ein Zeichenfilter das nicht tut?
3. Was ist eine SQL-Injection zweiter Ordnung, und welcher Denkfehler führt zu ihr?
4. Erkläre mit der Same-Origin Policy, warum Cross-Site Scripting dem Angreifer die Rechte des angemeldeten Benutzers gibt.
5. Worin unterscheiden sich gespeichertes, reflektiertes und DOM-basiertes XSS?
6. Warum hängt die Schwere einer SSRF-Schwäche stärker von der Architektur ab als vom Code?
7. Ordne zu: Validierung, Kodierung, Parametrisierung. Welche setzt an der Quelle an, welche an der Senke?
8. Ein Team behebt einen Befund zu Command Injection, indem es Sonderzeichen aus der Eingabe entfernt. Beurteile die Behebung.
9. Nenne für Path Traversal je eine Maßnahme auf den Ebenen Code, Architektur und Lifecycle.

# 9 Schwachstellenklassen II: Identität und Zugriff

Die Schwächen aus Kapitel 8 haben eine angenehme Eigenschaft: Es gibt für jede eine technische Regel, deren Einhaltung sie verhindert, und Werkzeuge, die Verstöße finden. Für die Schwächen dieses Kapitels gilt das nicht. Ob ein Kunde eine bestimmte Rechnung sehen darf, steht in keiner Programmiersprache und in keinem Framework. Es steht in den Regeln des Geschäfts, und die Anwendung muss sie an jeder einzelnen Stelle durchsetzen. Fehler in der Zugriffskontrolle führen seit Jahren die OWASP Top 10 an [26], und sie sind die Befunde, die am häufigsten auf den Entwurf zurückgehen und am häufigsten falsch behoben werden. Dieses Kapitel behandelt die drei Fragen, die jede Anwendung beantworten muss: Wer bist du, bist du es noch, und darfst du das?

**IN DIESEM KAPITEL**

- Authentifizierung, Sitzungsverwaltung und Autorisierung unterscheiden und typische Schwächen jeder Stufe benennen
- die Schwächen von Passwörtern, Zurücksetzverfahren und Mehrfaktor-Anmeldung erklären
- den Aufbau eines Tokens und die typischen Fehler bei seiner Prüfung beschreiben
- Cross-Site Request Forgery erklären
- Autorisierungsfehler auf Funktions-, Objekt- und Eigenschaftsebene unterscheiden und die Kürzel BOLA, BFLA und IDOR zuordnen
- begründen, warum Zugriffskontrolle eine Architekturfrage ist

## 9.1 Drei Fragen

Jede Anfrage an eine Anwendung durchläuft, ausgesprochen oder nicht, drei Prüfungen.




**Abb. 19:** Die drei Prüfungen vor jeder Funktion.

**DEFINITION 9.1 · AUTHENTIFIZIERUNG, SITZUNG, AUTORISIERUNG**

**Authentifizierung** ist die Feststellung, wer der Absender ist, anhand eines Nachweises wie Passwort, Schlüssel oder Zertifikat. Die **Sitzung** ist der Mechanismus, der diese Feststellung über viele Anfragen hinweg aufrechterhält, ohne dass der Nachweis jedes Mal neu erbracht wird. **Autorisierung** ist die Entscheidung, ob der festgestellte Absender die verlangte Handlung an dem verlangten Objekt ausführen darf.

Im Englischen werden Authentifizierung und Autorisierung oft zu *AuthN* und *AuthZ* abgekürzt, weil die Wörter so ähnlich sind. In Berichten und im Hörsaal werden sie ständig verwechselt. Die Unterscheidung ist einfach zu merken: Die Authentifizierung prüft einen Ausweis. Die Autorisierung prüft eine Berechtigung. Ein gültiger Ausweis berechtigt zu nichts Bestimmtem.

Die drei Prüfungen sind verschieden schwer richtig zu bauen. Für Authentifizierung und Sitzung gibt es fertige, ausgereifte Bausteine, und der häufigste Fehler ist, sie nicht zu verwenden oder falsch einzustellen. Für die Autorisierung gibt es Hilfsmittel, aber keinen Baustein, der einem die Regeln abnimmt. Entsprechend verschiebt sich das Gewicht dieses Kapitels: zwei kürzere Abschnitte zu den ersten beiden Fragen, ein langer zur dritten.

## 9.2 Authentifizierung

Die Schwächen der Anmeldung lassen sich danach ordnen, an welchem Teil des Verfahrens sie ansetzen. Die OWASP Top 10 fassen sie unter A07 zusammen [26].

### Passwörter

Das Passwort ist ein Geheimnis, das der Benutzer wählt, und die meisten Schwächen folgen daraus.

**Erraten und Durchprobieren.** Begrenzt die Anwendung die Zahl der Anmeldeversuche nicht, kann ein Angreifer Passwörter in großer Zahl ausprobieren (`CWE-307`). Die wirksamste Form davon braucht keine Rechenleistung: Beim *Credential Stuffing* verwendet der Angreifer Paare aus Benutzername und Passwort, die bei Einbrüchen in andere Dienste erbeutet wurden. Weil viele Menschen dasselbe Passwort mehrfach verwenden, passt ein Teil davon. Beim *Password Spraying* probiert er wenige sehr häufige Passwörter gegen sehr viele Konten, um Sperren pro Konto zu unterlaufen.

**Auskunft über Konten.** Unterscheidet sich die Antwort bei einem unbekannten Benutzernamen von der bei einem falschen Passwort, lässt sich feststellen, welche Konten es gibt. Kapitel 7 hat das als Preisgabe von Informationen beschrieben. Hier zeigt sich ihr Nutzen für den Angreifer.

**Schwache Regeln.** Lässt die Anwendung sehr kurze oder allgemein bekannte Passwörter zu, ist das Durchprobieren aussichtsreich (`CWE-521`). Die aktuellen Empfehlungen des NIST haben mit einigen alten Gewohnheiten aufgeräumt: Länge zählt mehr als Sonderzeichenregeln, neue Passwörter sollen gegen Listen bekannt gewordener Passwörter geprüft werden, und ein erzwungener regelmäßiger Wechsel ohne Anlass wird nicht mehr empfohlen, weil er zu vorhersehbaren Varianten führt [61].

**Feste und voreingestellte Zugangsdaten.** Ein Passwort, das im Quelltext oder in einer Konfigurationsdatei steht (`CWE-798`), kennt jeder, der die Datei kennt. Ein vom Hersteller voreingestelltes Passwort, das nie geändert wurde, kennt jeder, der das Handbuch kennt. Beides sind Funde, die ein Tester in Minuten macht und die zu den folgenschwersten gehören.

**Speicherung.** Wie Passwörter auf dem Server abzulegen sind, behandelt dein Foliensatz zu den sicheren Programmierpraktiken: als langsamer, gesalzener Hash, nie im Klartext und nie umkehrbar verschlüsselt. Von außen sieht ein Tester die Speicherung nicht. Er findet sie, wenn er über eine andere Schwäche an die Datenbank kommt, und dann entscheidet sie darüber, ob aus einem Datenabfluss auch ein Verlust aller Passwörter wird.

### Zurücksetzen und Wiederherstellen

Jede Anwendung mit Passwörtern braucht einen Weg für den Fall, dass jemand seines vergessen hat. Dieser Weg ist eine zweite Anmeldung, und er ist oft schwächer als die erste (`CWE-640`). Typische Fehler: Der Verweis zum Zurücksetzen enthält einen vorhersehbaren oder zu kurzen Wert. Er bleibt nach Gebrauch oder nach langer Zeit gültig. Die Anwendung baut den Verweis aus einer Angabe in der Anfrage zusammen, sodass ein Angreifer ihn auf seinen eigenen Server zeigen lassen kann. Oder die Wiederherstellung fragt nach Dingen, die sich herausfinden lassen, etwa dem Geburtsdatum.

Für den Tester gilt eine einfache Überlegung: Die Sicherheit eines Kontos ist die Sicherheit seines schwächsten Zugangswegs. Dazu gehören neben der Anmeldemaske das Zurücksetzen, die Anmeldung über einen Drittanbieter, der Kundendienst am Telefon und jede ältere Schnittstelle, die noch die alte Anmeldung akzeptiert.

### Mehrere Faktoren

Die Mehrfaktor-Authentifizierung, kurz MFA, verlangt neben dem Passwort einen zweiten Nachweis, etwa einen Code aus einer App oder die Bestätigung an einem Gerät. Sie ist die wirksamste einzelne Maßnahme gegen gestohlene und erratene Passwörter. In Berichten erscheinen dazu zwei Arten von Befunden.

Die erste ist das **Fehlen**: Für Konten mit hohen Rechten, etwa Administratoren und Kassenkräfte, gibt es keinen zweiten Faktor. Die zweite ist das **Umgehen**: Der zweite Faktor wird nur an der Anmeldemaske geprüft und nicht an der Schnittstelle dahinter. Der Code lässt sich unbegrenzt oft raten. Es gibt einen Ersatzweg ohne zweiten Faktor. Oder der Benutzer wird so lange mit Bestätigungsanfragen überhäuft, bis er eine annimmt. Version 20 von ATT&CK führt Techniken zum Umgehen der Mehrfaktor-Anmeldung ausdrücklich auf [37].

> **IN DER REIHE**
> Die Anmeldung über OAuth 2.0 und OpenID Connect, also wie eine Anwendung die Authentifizierung an einen Identitätsdienst abgibt, erklärt *Secure API Design* in den Kapiteln 2 und 3 mit allen Abläufen. Dieses Buch setzt den Aufbau als gegeben voraus und behandelt die Fehler, die ein Tester an ihm findet.

## 9.3 Sitzungen und Tokens

Nach der Anmeldung muss die Anwendung den Benutzer bei jeder weiteren Anfrage wiedererkennen. Dafür gibt es zwei Bauweisen. Bei der klassischen merkt sich der Server die Sitzung und gibt dem Browser eine zufällige Kennung, meist in einem Cookie. Bei der zweiten stellt ein Dienst dem Client ein signiertes *Token* aus, das die Angaben zum Benutzer selbst enthält, und jeder Service prüft die Signatur. In beiden Fällen gilt: Wer die Kennung oder das Token besitzt, *ist* für die Anwendung der Benutzer. Der Diebstahl einer Sitzung ersetzt Passwort und zweiten Faktor.

### Sitzungskennungen

Die Schwächen klassischer Sitzungen sind seit langem bekannt. Die Kennung ist vorhersehbar. Sie wird nach der Anmeldung nicht erneuert, sodass ein Angreifer dem Opfer vorab eine Kennung unterschieben kann, die nach dessen Anmeldung gültig wird (*Session Fixation*, `CWE-384`). Sie bleibt nach dem Abmelden oder über sehr lange Zeit gültig (`CWE-613`). Oder sie ist schlecht geschützt: Das Cookie wird auch über unverschlüsselte Verbindungen gesendet, oder Skripte können es lesen, was jedes XSS aus Kapitel 8 zum Sitzungsdiebstahl macht.

Gegen die letzten beiden Punkte helfen Kennzeichen am Cookie, die du in Befunden lesen wirst. `Secure` bedeutet, dass der Browser das Cookie nur über verschlüsselte Verbindungen sendet. `HttpOnly` bedeutet, dass Skripte es nicht lesen können. `SameSite` regelt, ob es bei Anfragen mitgeschickt wird, die von einer fremden Website ausgelöst wurden. Fehlende Kennzeichen sind einer der häufigsten Befunde überhaupt, meist mit niedrigem Schweregrad. Ihr Gewicht hängt davon ab, was sonst noch im Argen liegt.

### Tokens

In verteilten Systemen haben sich signierte Tokens durchgesetzt, am häufigsten im Format *JSON Web Token*, kurz JWT. Ein solches Token besteht aus drei Teilen: einem Kopf, der unter anderem das Signaturverfahren nennt, einem Inhalt mit den Angaben zum Benutzer und zur Gültigkeit, den sogenannten *Claims*, und einer Signatur über beides. Kopf und Inhalt sind nur kodiert, nicht verschlüsselt. Jeder, der das Token in die Hand bekommt, kann sie lesen.

Die Sicherheit des Verfahrens hängt allein an der Prüfung auf der empfangenden Seite, und dort liegen die Fehler.

**Tab. 30:** Typische Fehler bei der Prüfung signierter Tokens. Die Einordnung ist meist CWE-347, die mangelhafte Prüfung einer kryptografischen Signatur.

| Fehler bei der Prüfung | Was passiert | Folge |
|---|---|---|
| Signatur wird nicht geprüft | der Service liest den Inhalt, ohne die Signatur zu kontrollieren | jeder kann sich beliebige Angaben ausstellen |
| Verfahren aus dem Token übernommen | der Service lässt sich vom Kopf des Tokens vorschreiben, wie zu prüfen ist, im Extremfall „gar nicht“ | die Signatur lässt sich aushebeln |
| Schwacher Schlüssel | der Signaturschlüssel ist kurz oder ein bekannter Beispielwert | der Schlüssel lässt sich erraten, Tokens lassen sich fälschen |
| Gültigkeit nicht geprüft | Ablaufzeit, Aussteller oder Empfänger werden ignoriert | alte Tokens und Tokens für andere Dienste werden angenommen |
| Kein Widerruf | ein ausgestelltes Token bleibt bis zum Ablauf gültig | Abmelden und Sperren wirken erst verzögert |
| Vertrauliches im Inhalt | das Token enthält Daten, die der Benutzer nicht sehen soll | Preisgabe von Informationen |

Die zweite Zeile von Tab. 30 ist ein Sonderfall des Grundfehlers aus Kapitel 8: Der Empfänger lässt sich von den Daten vorschreiben, wie er mit ihnen umgehen soll. Die Behebung ist dieselbe wie dort. Der Service legt fest, welches Verfahren und welcher Schlüssel gelten, und lehnt alles andere ab.

Für die Architektur stellt sich bei Tokens die Frage, wo sie geprüft werden. Prüft nur das Gateway und reicht die Angaben in einer Kopfzeile an die Services weiter, vertrauen die Services dieser Kopfzeile. Wer einen Service direkt erreicht, kann sie selbst setzen. Das ist das implizite Vertrauen aus Kapitel 6 in seiner häufigsten Gestalt.

## 9.4 Cross-Site Request Forgery

Eine eigene Schwäche entsteht aus der Art, wie Browser mit Cookies umgehen. Ein Browser schickt die Cookies einer Website bei *jeder* Anfrage an diese Website mit, auch wenn die Anfrage von einer ganz anderen Seite ausgelöst wurde.

> **DEFINITION 9.2 · CROSS-SITE REQUEST FORGERY**
> Bei der *Cross-Site Request Forgery*, kurz CSRF (`CWE-352`), bringt eine fremde Website den Browser eines angemeldeten Benutzers dazu, eine Anfrage an die Anwendung zu senden. Weil der Browser die Sitzungs-Cookies mitschickt, führt die Anwendung die Anfrage im Namen des Benutzers aus, obwohl er sie nie beabsichtigt hat.

Der Unterschied zum Cross-Site Scripting wird oft übersehen. Bei XSS läuft fremder Code *in* der Anwendung und kann lesen und schreiben. Bei CSRF kommt die Anfrage von außen. Der Angreifer kann eine Handlung auslösen, sieht aber die Antwort nicht. Betroffen sind deshalb Anfragen, die etwas ändern: eine E-Mail-Adresse, ein Passwort, eine Überweisung, eine Stornierung.

Die Schwäche stand 2025 auf Platz 3 der CWE Top 25 [25], obwohl es gute Gegenmittel gibt. Das wirksamste ist inzwischen das Kennzeichen `SameSite` am Cookie, mit dem der Browser die Cookies bei fremd ausgelösten Anfragen gar nicht mehr mitschickt. Dazu kommt ein geheimer, pro Sitzung vergebener Wert, den jede ändernde Anfrage mitführen muss und den eine fremde Seite nicht kennen kann. Anwendungen, die sich nicht über Cookies, sondern über ein Token in einer Kopfzeile ausweisen, sind von der Bauweise her nicht betroffen, weil der Browser diese Kopfzeile nicht von selbst setzt.

Verwandt, aber nicht dasselbe, ist die Fehlkonfiguration von CORS (`CWE-942`). CORS ist ein Mechanismus, mit dem ein Server die Same-Origin Policy gezielt lockert und bestimmten fremden Ursprüngen erlaubt, seine Antworten zu lesen. Eine zu großzügige Einstellung, die jeden Ursprung zulässt und dabei auch Cookies akzeptiert, hebt den Schutz des Browsers auf.

> **IN DER REIHE**
> CORS, Cookie-Sitzungen und CSRF aus der Sicht dessen, der eine API baut, stehen in *Secure API Design*, Kapitel 8.

## 9.5 Autorisierung

Damit zur dritten Frage und zum Kern des Kapitels. Die Anwendung weiß, wer anfragt. Darf er, was er verlangt?

Die Antwort hat drei Ebenen, und auf jeder gibt es eine eigene Art von Fehler. Die Liste der häufigsten Risiken für Programmierschnittstellen der OWASP führt alle drei unter den ersten fünf [58].

**Tab. 31:** Die drei Ebenen der Autorisierung.

| Ebene | Frage | Kürzel | Beispiel im System *Kurbad* |
|---|---|---|---|
| Funktion | Darf diese Rolle diese Funktion aufrufen? | BFLA | ein Kunde ruft den Endpunkt zum Ausstellen von Gutscheinen auf |
| Objekt | Darf dieser Benutzer auf diesen einen Datensatz zugreifen? | BOLA, IDOR | Kundin A ruft die Rechnung von Kunde B ab |
| Eigenschaft | Darf dieser Benutzer dieses Feld des Datensatzes sehen oder ändern? | BOPLA | ein Kunde setzt beim Ändern seines Profils das Feld für die Rolle |

Die Kürzel stammen aus der OWASP-Liste und stehen so in Berichten. *BFLA* ist *Broken Function Level Authorization*, *BOLA* ist *Broken Object Level Authorization*, *BOPLA* ist *Broken Object Property Level Authorization*. Der ältere Name für BOLA ist *IDOR*, *Insecure Direct Object Reference*, und er beschreibt den Mechanismus anschaulicher: Die Anfrage verweist direkt auf ein Objekt, und nichts prüft, ob der Verweis erlaubt ist.

Zwei weitere Begriffe ordnen dieselben Fehler nach der Richtung. Eine **vertikale** Rechteausweitung führt nach oben, zu den Rechten einer höheren Rolle. Das ist der Fehler auf Funktionsebene. Eine **horizontale** führt zur Seite, zu den Daten eines anderen Benutzers derselben Rolle. Das ist der Fehler auf Objektebene.

### Funktionsebene

Der einfachste Fall ist die fehlende Prüfung an einer Funktion. Die Oberfläche zeigt einem Kunden den Menüpunkt „Gutschein ausstellen“ nicht an, aber der Endpunkt dahinter prüft die Rolle nicht. Wer die Adresse kennt, kann ihn aufrufen. Der Fehler heißt auch *Forced Browsing* (`CWE-425`), und die CWE unterscheidet, ob die Prüfung ganz fehlt (`CWE-862`) oder vorhanden, aber falsch ist (`CWE-863`).

Die Ursache ist fast immer dieselbe: Die Zugriffskontrolle wurde in der Oberfläche umgesetzt statt am Server. Das ist der Fehler an der Vertrauensgrenze zwischen Browser und Server aus Kapitel 6. Besonders oft trifft es Verwaltungsfunktionen, weil sie später dazukommen, von anderen Leuten gebaut werden und in den Tests der Kundensicht nicht vorkommen.

Gefunden werden diese Fehler mit der Rollenmatrix aus Kapitel 7: jede Funktion mit jeder Rolle aufrufen, die sie nicht haben soll.

### Objektebene

Der häufigste und folgenreichste Fehler der Zugriffskontrolle sitzt eine Ebene tiefer. Die Funktion darf der Benutzer aufrufen, nur nicht für jedes Objekt.

`RechnungController.cs`

```
[Authorize]
[HttpGet("rechnungen/{id}")]
public IActionResult Hole(int id)
{
    var rechnung = db.Rechnungen.Find(id);
    if (rechnung == null) return NotFound();
    return Ok(rechnung);
}
```

Der Code verlangt eine Anmeldung, das leistet das Attribut in der ersten Zeile. Er prüft, ob es die Rechnung gibt. Er prüft nicht, wem sie gehört. Jeder angemeldete Kunde bekommt jede Rechnung, deren Nummer er angibt, und weil die Nummern fortlaufend sind, kann er sie alle der Reihe nach abrufen.

Dieser Fehler ist aus drei Gründen so verbreitet. Erstens ist der Code vollkommen plausibel. Er tut, was die Funktion soll, und jeder Test mit den eigenen Daten gelingt. Zweitens findet ihn kein Werkzeug von selbst, denn die Anfrage ist gültig und die Antwort ist gültig. Dass sie dem Falschen gegeben wurde, weiß nur, wer die Besitzverhältnisse kennt. Drittens muss die Prüfung an jeder Stelle stehen, an der ein Objekt über eine Kennung angesprochen wird, und bei hundert Endpunkten wird eine vergessen.

`RechnungController.cs`

```
[Authorize]
[HttpGet("rechnungen/{id}")]
public IActionResult Hole(int id)
{
    var kundeId = AktuellerKunde();
    var rechnung = db.Rechnungen
        .SingleOrDefault(r => r.Id == id && r.KundeId == kundeId);
    if (rechnung == null) return NotFound();
    return Ok(rechnung);
}
```

Die Korrektur im **Code** bindet die Abfrage an den Besitzer. Die Kundennummer stammt dabei aus der geprüften Sitzung und nicht aus der Anfrage. Für eine fremde Rechnung antwortet der Server genauso wie für eine, die es nicht gibt, damit sich aus der Antwort nicht ablesen lässt, welche Nummern vergeben sind.

Diese Korrektur ist richtig, und sie ist die Stelle, an der die meisten Behebungen stehen bleiben. Kapitel 1 hat erzählt, was dann geschieht: Im nächsten Jahr steht derselbe Befund für die Gutschriften im Bericht. Die Schwäche war nie dieser eine Endpunkt. Sie liegt in einer **Architektur**, die es jedem einzelnen Endpunkt überlässt, an die Besitzprüfung zu denken. Die Behebung auf dieser Ebene macht das Vergessen unmöglich oder wenigstens auffällig:

- Die Datenzugriffsschicht liefert Objekte grundsätzlich nur im Kontext eines Besitzers. Eine Methode, die eine Rechnung allein über ihre Nummer holt, gibt es für die Kundensicht nicht.
- Die Regeln stehen an einer Stelle, in einer zentralen Richtlinie, und die Endpunkte verweisen darauf, statt sie nachzubauen.
- Es gilt *Deny by Default*: Was nicht ausdrücklich erlaubt ist, ist verboten. Ein neuer Endpunkt ohne Regel ist dann gesperrt und nicht offen.
- Bei Mandantenfähigkeit wird die Trennung der Mandanten in der Datenbank selbst erzwungen.

Eine verbreitete Scheinlösung ist, die fortlaufenden Nummern durch lange zufällige Kennungen zu ersetzen. Das erschwert das Durchzählen und ist als zusätzliche Schicht sinnvoll. Es behebt die Schwäche nicht, denn die Kennungen tauchen in Verweisen, Protokollen und weitergeleiteten Nachrichten auf, und wer eine kennt, bekommt weiterhin das Objekt.

**F-07 Abruf fremder Rechnungen über fortlaufende Kennung** — HOCH

| | |
|---|---|
| **Ort** | `GET /api/v3/rechnungen/{id}`, Abrechnungsservice |
| **Beobachtung** | Mit dem Konto eines Testkunden ließen sich die Rechnungen anderer Kunden abrufen, indem die Kennung in der Adresse verändert wurde. Die Rechnungen enthalten Name, Anschrift und gebuchte Leistungen. |
| **Einordnung** | CWE-639, WSTG-ATHZ-04, CVSS 3.1: 6.5 (AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N) |
| **Empfehlung** | Besitzprüfung im Endpunkt ergänzen. Alle Endpunkte prüfen, die Objekte über eine Kennung ansprechen. Besitzprüfung zentral in der Datenzugriffsschicht erzwingen. |

Der Vektor dieses Befunds ist der aus Kapitel 4. Beachte den Wert: 6,5, also „mittel“ nach der Tabelle der Stufen, obwohl jeder Kunde die Daten aller anderen Kunden lesen kann. Der Tester hat hier auf „hoch“ angehoben und das begründet. Warum die Zahl und das Urteil auseinanderfallen können, ist Thema von Kapitel 14.

### Eigenschaftsebene

Die feinste Ebene betrifft einzelne Felder eines Objekts, in beide Richtungen.

Beim **Schreiben** heißt der Fehler *Mass Assignment* (`CWE-915`). Viele Frameworks können die Felder einer eingehenden Anfrage automatisch auf die Felder eines Objekts übertragen. Das ist bequem und gefährlich, wenn das Objekt Felder hat, die der Benutzer nicht setzen soll. Schickt ein Kunde beim Ändern seines Profils zusätzlich ein Feld für die Rolle mit oder für den Kontostand seiner Bonuspunkte, übernimmt die Anwendung es, sofern niemand festgelegt hat, welche Felder erlaubt sind.

Beim **Lesen** liefert die Anwendung mehr, als der Empfänger sehen soll. Der Endpunkt gibt das ganze Objekt aus der Datenbank zurück, und die Oberfläche zeigt nur drei Felder davon an. Die übrigen, etwa interne Vermerke, Kennungen anderer Systeme oder der Hash des Passworts, stehen trotzdem in der Antwort. Die Annahme, die Oberfläche werde schon filtern, ist wieder der Fehler an der Grenze zwischen Browser und Server.

Die Behebung für beides ist ein eigenes Datenmodell je Richtung: eines, das festlegt, was hereinkommen darf, und eines, das festlegt, was hinausgeht. Nichts wird automatisch von einem auf das andere übertragen.

> IN DER REIHE
> Getrennte Modelle für Eingang und Ausgang, das Begrenzen und Validieren von Eingaben und die Prüfung von Scopes und Gruppen im Token behandelt *Secure API Design* in den Kapiteln 3 bis 5 mit Beispielen.

**Warum Werkzeuge hier scheitern**

Die Prüfung der Autorisierung hat zwei Dimensionen. Die Rollenmatrix deckt eine davon ab: Funktion mal Rolle. Die zweite ist Funktion mal Objekt, und sie hat so viele Fälle, wie es Daten gibt. Kein Tester kann jede Kombination prüfen. Er prüft jede Funktion mit zwei Konten derselben Rolle über Kreuz und schließt von dort auf die Bauweise.

Automatisierte Werkzeuge können das nur, wenn man ihnen die Regeln gibt: zwei Konten und die Erwartung, dass keines die Objekte des anderen sehen darf. Ohne diese Erwartung sieht jede Antwort richtig aus. Darin liegt der Grund, warum die Zugriffskontrolle die Rangliste anführt. Sie ist die Schwachstellenklasse, für die es eine Spezifikation braucht, und die meisten Anwendungen haben keine.

> MERKE
> Ein Befund zur Zugriffskontrolle ist fast nie ein Einzelfall. Die erste Frage nach dem Lesen lautet: Wo wird die Regel durchgesetzt, und was garantiert, dass sie an den anderen Stellen nicht fehlt?

## 9.6 Zugriff zwischen Services

In einem verteilten System stellen sich die drei Fragen nicht nur zwischen Benutzer und Anwendung, sondern an jeder Verbindung zwischen zwei Services. Die Fehler sind dieselben, nur schwerer zu sehen.

**Fehlende Authentifizierung zwischen Services.** Der Abrechnungsservice nimmt Anfragen von jedem an, der ihn im internen Netz erreicht. Wer einen beliebigen anderen Service übernimmt oder per SSRF Anfragen ins interne Netz schicken kann, erreicht ihn auch.

**Fehlende Autorisierung zwischen Services.** Jeder Service darf jede Funktion jedes anderen aufrufen. Der Benachrichtigungsservice, der nur E-Mails verschickt, könnte auch Rechnungen stornieren. Das Prinzip der minimalen Rechte gilt für Services genauso wie für Menschen.

**Der verwirrte Stellvertreter.** Ein Service mit hohen Rechten führt im Auftrag eines Benutzers mit niedrigen Rechten eine Handlung aus und verwendet dabei seine eigenen Rechte statt der des Benutzers. Das Muster heißt *Confused Deputy*. Der Buchungsservice ruft für einen Kunden beim Abrechnungsservice eine Rechnung ab und weist sich dabei nur als Buchungsservice aus. Der Abrechnungsservice sieht einen vertrauenswürdigen Aufrufer und liefert. Wem die Rechnung gehört, prüft niemand mehr, weil jeder annimmt, der andere habe es getan. SSRF aus Kapitel 8 ist ein Sonderfall desselben Musters.

Die Abhilfe liegt in der Architektur: Die Identität des Benutzers wird über die Kette der Aufrufe mitgeführt, jeder Service weist sich mit einer eigenen Identität aus, und jeder Service entscheidet selbst über seine eigenen Daten. Die Mittel dazu, gegenseitig authentifizierte Verbindungen, ein Service Mesh und ein Gateway, stehen in deinem Foliensatz zu Microservices. Deine Übersichtsarbeit ordnet die Bedrohungen und Gegenmaßnahmen dieser Art systematisch [50].

> IN DER REIHE
> Für Systeme, die über einen Broker sprechen, stellt sich dieselbe Frage bei jeder Nachricht: Wer darf was veröffentlichen und lesen? *Sichere Messaging-Architekturen* behandelt das in den Kapiteln 8 bis 10.

## 9.7 Die drei Ebenen im Überblick

**Tab. 32:** Behebungen der Schwächen von Identität und Zugriff auf drei Ebenen.

| Schwäche | Code | Architektur | Lifecycle |
|---|---|---|---|
| Durchprobieren von Passwörtern | Versuche zählen und begrenzen | Anmeldung an einen ausgereiften Identitätsdienst abgeben, Mehrfaktor-Anmeldung | Vorgabe, keine eigene Anmeldung zu bauen |
| Feste Zugangsdaten | aus dem Code entfernen | Geheimnisse aus einem Tresor zur Laufzeit beziehen | Suche nach Geheimnissen in der Pipeline, Schlüssel nach Fund tauschen |
| Schwaches Zurücksetzen | zufällige, kurzlebige, einmalige Werte | Zurücksetzen über den Identitätsdienst | Missbrauchsfall in den Anforderungen |
| Fehler bei der Token-Prüfung | Verfahren und Schlüssel festlegen, alle Angaben prüfen | Prüfung in jedem Service, kurze Laufzeiten, Widerruf | geprüfte Bibliothek vorschreiben, eigene Implementierungen verbieten |
| CSRF | geheimer Wert je Sitzung bei ändernden Anfragen | `SameSite`-Cookies oder Token in Kopfzeile statt Cookie | sichere Voreinstellungen im Projektgerüst |
| Fehlende Prüfung der Funktion | Rolle am Endpunkt prüfen | zentrale Richtlinie, Verbot als Voreinstellung | Rollenmatrix als Anforderung und als automatisierter Test |
| Fehlende Prüfung des Objekts | Abfrage an den Besitzer binden | Besitzprüfung in der Datenzugriffsschicht erzwingen | Test mit zwei Konten über Kreuz in der Pipeline, Regel im Code Review |
| Mass Assignment | erlaubte Felder ausdrücklich benennen | getrennte Modelle für Eingang und Ausgang | automatische Übertragung per Regel verbieten |
| Implizites Vertrauen zwischen Services | Token des Aufrufers prüfen | eigene Identität je Service, Authentifizierung und minimale Rechte an jeder Verbindung | Vertrauensgrenzen in der Architekturprüfung |

Im Vergleich mit der entsprechenden Tabelle in Kapitel 8 fällt etwas auf. Dort enthielt die Spalte Code die eigentliche Behebung und die Spalte Architektur die Begrenzung des Schadens. Hier ist es umgekehrt. Die Korrektur im Code schließt die gemeldete Stelle, die eigentliche Behebung steht in der Spalte Architektur. Bei Schwächen der Zugriffskontrolle ist eine Behebung, die nur im Code ansetzt, fast immer unvollständig.

**BRÜCKE ZU SOFTWARE SECURITY**
Für die Analyse von Architekturen in deiner Pflicht-Lehrveranstaltung liefert dieses Kapitel die wichtigste einzelne Prüffrage: Wo in diesem System wird entschieden, wer was darf, und ist das eine Stelle oder sind es viele? In einem C4-Diagramm lässt sich das zeigen. Liegt die Autorisierung nur im Gateway, ist jede Verbindung hinter dem Gateway ungeprüft. Ein Dossier, in dem die Studierenden genau das finden sollen, braucht nur ein ADR mit dem Satz „Die Services vertrauen dem Gateway“ und einen zweiten Weg ins System, der am Gateway vorbeiführt.

**FRAGE AUS DEM HÖRSAAL**
***Wir haben überall eine Anmeldung davor. Warum sollte die Zugriffskontrolle trotzdem kaputt sein?***

Weil die Anmeldung nur die erste der drei Fragen beantwortet. Sie stellt fest, wer anfragt. Die allermeisten Befunde zur Zugriffskontrolle werden mit einem gültigen, ordentlich angemeldeten Konto gefunden. Der Angreifer ist ein Kunde, der sich Dinge ansieht, die anderen Kunden gehören. Eine starke Anmeldung mit zweitem Faktor ändert daran nichts.

**FRAGE AUS DEM HÖRSAAL**
***Reicht es nicht, wenn wir zufällige Kennungen statt fortlaufender Nummern verwenden?***

Es hilft gegen das einfache Durchzählen und ist als zusätzliche Maßnahme zu empfehlen. Die Schwäche bleibt, denn die Prüfung fehlt weiterhin. Kennungen sind keine Geheimnisse. Sie stehen in Adressen, in Protokollen, in E-Mails, in Antworten anderer Endpunkte. Sobald jemand eine fremde Kennung in die Hand bekommt, bekommt er auch das Objekt. Die CWE führt dieses Missverständnis als eigenen Fall: Schutz durch einen vom Benutzer kontrollierbaren Schlüssel.

**FRAGE AUS DEM HÖRSAAL**

***Warum steht Zugriffskontrolle seit Jahren auf Platz 1, wenn das Problem so gut bekannt ist?***

Weil sie die einzige große Klasse ist, die sich nicht durch eine technische Regel erledigen lässt. Injection verschwindet, wenn das Framework parametrisiert. Für die Zugriffskontrolle muss jemand die Regeln des Geschäfts aufschreiben, und jemand muss sie an jeder Stelle durchsetzen. Das erste wird selten gemacht, das zweite wird bei wachsenden Systemen irgendwo vergessen. Dazu kommt, dass moderne Anwendungen aus vielen Schnittstellen bestehen, die direkt auf Objekte zugreifen. Jede davon ist eine Gelegenheit für den Fehler.

**FÜR DIE LEHRE**

Wenn du in der Spezialisierung nur für eine Schwachstellenklasse Zeit hast, nimm diese. An ihr lässt sich der Unterschied zwischen Symptom und Ursache am deutlichsten zeigen. Gib den Studierenden einen Befund wie F-07 und den Code von drei Endpunkten, von denen einer gemeldet ist und ein zweiter denselben Fehler hat. Wer nur den gemeldeten korrigiert, hat die Aufgabe nicht gelöst. Wer beide korrigiert, hat sorgfältig gearbeitet. Wer die Datenzugriffsschicht so umbaut, dass der Fehler nicht mehr möglich ist, und das in einem ADR begründet, hat verstanden, worum es geht.

Lass die Studierenden außerdem die Bewertung hinterfragen. Der Vektor ergibt 6,5. Ist „mittel“ angemessen, wenn alle Kundendaten lesbar sind? Die Diskussion führt direkt zu Kapitel 14.

Typische Fehlvorstellungen: dass Authentifizierung und Autorisierung dasselbe seien, dass ausgeblendete Menüpunkte Funktionen schützen, und dass zufällige Kennungen eine Berechtigungsprüfung ersetzen.

## Selbsttest

1. Erkläre den Unterschied zwischen Authentifizierung und Autorisierung an einem Beispiel aus dem Alltag.
2. Was ist Credential Stuffing, und warum hilft eine strenge Passwortrichtlinie dagegen wenig?
3. Nenne drei Fehler bei der Prüfung eines signierten Tokens und je eine Folge.
4. Worin unterscheiden sich XSS und CSRF in dem, was der Angreifer erreichen kann?
5. Ordne BFLA, BOLA und BOPLA den drei Ebenen der Autorisierung zu und gib je ein Beispiel.
6. Warum findet ein Scanner einen Fehler auf Objektebene nicht?
7. Ein Team ersetzt nach einem BOLA-Befund die fortlaufenden Rechnungsnummern durch zufällige Kennungen und schließt den Befund. Beurteile das.
8. Was ist das Muster des verwirrten Stellvertreters, und wie hängt es mit SSRF zusammen?
9. Warum steht bei Schwächen der Zugriffskontrolle die eigentliche Behebung in der Spalte Architektur?

# 10 Schwachstellenklassen III: Konfiguration, Kryptografie, Lieferkette, Logik

Die Schwächen der letzten beiden Kapitel stecken im Code der Anwendung. Ein großer Teil der Befunde in heutigen Berichten tut das nicht. Er steckt in den Einstellungen der Umgebung, in der falschen Verwendung an sich guter Kryptografie, in Bausteinen, die jemand anderer geschrieben hat, und in der Fachlogik, die technisch einwandfrei umgesetzt ist und sich trotzdem gegen ihren Zweck verwenden lässt. Diese vier Gruppen haben eines gemeinsam: Wer die Ursache im Quelltext der Anwendung sucht, findet sie nicht. Sie sind deshalb die Befunde, an denen Entwicklungsteams am häufigsten ratlos sind, und die besten Beispiele für Ursachen auf den Ebenen Architektur und Lifecycle.

**IN DIESEM KAPITEL**

- typische Fehlkonfigurationen nennen und erklären, warum ihre Ursache selten im Code liegt
- die häufigen Fehler bei der Verwendung von Kryptografie beschreiben und sagen, welche davon ein Tester von außen sieht
- Befunde zu Fremdkomponenten lesen und ihre tatsächliche Bedeutung einschätzen
- Fehler in der Fachlogik als verbotene Zustandsübergänge beschreiben und Wettlaufsituationen einordnen
- den Unterschied zwischen offenem und geschlossenem Versagen erklären

## 10.1 Fehlkonfiguration

In den OWASP Top 10 ist die Fehlkonfiguration von Platz 5 im Jahr 2021 auf Platz 2 im Jahr 2025 gestiegen [26]. Der Grund ist eine Verschiebung in der Art, wie Software gebaut wird. Ein immer größerer Teil dessen, was ein System tut und darf, steht nicht mehr im Code, sondern in Einstellungen: des Frameworks, des Webservers, des Containers, des Orchestrators, der Cloud-Plattform. Jede dieser Schichten hat hunderte Schalter, und viele davon stehen in der Voreinstellung so, dass das System bequem in Betrieb zu nehmen ist und nicht so, dass es sicher ist.

**DEFINITION 10.1 · FEHLKONFIGURATION**
Eine Fehlkonfiguration ist eine Schwäche, die nicht aus fehlerhaftem Code entsteht, sondern aus unsicheren Einstellungen eines an sich korrekt arbeitenden Systems: einer unsicheren Voreinstellung, die nie geändert wurde, einer Funktion, die eingeschaltet blieb, obwohl sie nicht gebraucht wird, oder einer Berechtigung, die weiter reicht als nötig.

Die Erscheinungsformen sind zahlreich und einzeln meist banal.

**Tab. 33:** Gruppen von Fehlkonfigurationen.

| Gruppe | Typische Befunde | Was ein Angreifer davon hat |
|---|---|---|
| Übriggebliebenes | Beispielanwendungen, Standardseiten, Testkonten, voreingestellte Passwörter | fertige Zugänge und Auskunft über das eingesetzte Produkt |
| Entwicklungshilfen in der Produktion | Debug-Modus, Diagnose-Endpunkte, ausführliche Fehlermeldungen, öffentliche Schnittstellenbeschreibung | Einblick ins Innenleben, manchmal direkter Zugriff auf Speicher und Konfiguration |
| Zu viel erreichbar | Verwaltungsoberflächen, Datenbanken, Überwachungsansichten aus dem Internet erreichbar | Angriffsfläche, die es nicht geben müsste |
| Zu weite Berechtigungen | öffentlich lesbare Ablagen in der Cloud, Dienstkonten mit Administratorrechten, Container mit Root-Rechten | aus einem kleinen Einbruch wird ein großer |
| Fehlende Härtung | fehlende Sicherheits-Header, zugelassene alte Protokollversionen, großzügige CORS-Einstellung | Schutzschichten, die andere Schwächen abfangen würden, fehlen |
| Abweichung zwischen Umgebungen | Schutz in der Produktion aktiv, in einer vergessenen Abnahmeumgebung nicht | derselbe Zugang über die Hintertür |

Die vierte Zeile verdient einen genaueren Blick, weil sie die schwersten Vorfälle hervorbringt. Die öffentlich lesbare Datenablage ist seit Jahren einer der häufigsten Wege, auf denen große Datenmengen abhandenkommen, und sie braucht keinen Angriff im eigentlichen Sinn. Jemand hat beim Anlegen eine Einstellung gewählt, die den Inhalt für jeden abrufbar macht, der die Adresse kennt oder errät. Ebenso das zu mächtige Dienstkonto: Im Fall Capital One aus Kapitel 8 war der Auslöser eine Fehlkonfiguration und die Schadenshöhe die Folge einer Rolle, die mehr durfte, als sie brauchte [60].

In deinem Foliensatz zu den sicheren Programmierpraktiken steht ein ganzer Abschnitt zur Härtung von Containern: kein Root-Benutzer, kein privilegierter Modus, alle unnötigen Fähigkeiten entzogen, kein Zugriff auf den Docker-Socket, minimale Images. Jeder Punkt dort ist die Vermeidung einer Fehlkonfiguration. Im Pentest-Bericht stehen sie, wenn der Tester nach der Übernahme eines Service prüft, wie weit er vom Container aus kommt.

### Wo die Ursache liegt

Ein Befund zu einer Fehlkonfiguration lässt sich in Minuten beheben: Schalter umlegen, Endpunkt abschalten, Berechtigung entziehen. Genau das macht ihn tückisch. Die Einstellung ist ein Symptom. Die Fragen, die zur Ursache führen, lauten: Warum war sie so, und warum hat es niemand bemerkt?

Die Antworten liegen fast immer auf der Ebene Lifecycle.

- Es gibt keinen **Härtungsstandard**, also keine verbindliche Liste, wie die eingesetzten Produkte einzustellen sind. Solche Listen muss niemand selbst schreiben. Für verbreitete Betriebssysteme, Server und Cloud-Dienste gibt es öffentlich gepflegte Vorgaben, die bekanntesten sind die *CIS Benchmarks* [62].
- Die Konfiguration wird **von Hand** gepflegt und nicht als Code versioniert. Dann weiß niemand, wer wann was geändert hat, und zwei Umgebungen, die gleich sein sollten, sind es nicht.
- Es gibt **keine Prüfung**, die Abweichungen vom Soll findet. Werkzeuge, die Infrastrukturbeschreibungen und laufende Cloud-Umgebungen gegen Regeln prüfen, gibt es. Sie müssen eingesetzt werden.
- **Abgelöstes wird nicht stillgelegt.** Die alte Umgebung läuft weiter, weil niemand zuständig ist.

Auf der Ebene Architektur liegt die Ursache dort, wo ein System nicht sicher voreingestellt ist. Das Prinzip heißt *Secure by Default* und steht in deinem Foliensatz zu den Grundlagen: Ein System soll in dem Zustand, in dem es ausgeliefert wird, sicher sein, und unsichere Einstellungen sollen eine bewusste Handlung erfordern. Der Cyber Resilience Act macht das für Hersteller zur Pflicht [10]. Für das eigene System heißt das: Wenn ein Entwickler für einen neuen Service erst zehn Schalter umlegen muss, damit er sicher ist, wird er einen vergessen. Ein Projektgerüst, in dem die sicheren Einstellungen schon gesetzt sind, ist eine Architekturentscheidung.

> **IN DER REIHE**
> Die Härtung von Containern und Images steht ausführlich in *Docker von Grund auf*, Kapitel 12. Sicherheits-Header, TLS und der Umgang mit Dokumentation und Debug-Funktionen in der Produktion stehen in *Secure API Design*, Kapitel 8.

## 10.2 Kryptografische Fehler

Die Kategorie hieß in den OWASP Top 10 früher „Preisgabe sensibler Daten" und wurde umbenannt, um die Ursache statt der Folge zu nennen [26]. Der Name führt trotzdem leicht in die Irre. Befunde dieser Gruppe haben so gut wie nie damit zu tun, dass ein Verfahren mathematisch gebrochen wurde. Sie entstehen, weil gute Verfahren falsch oder gar nicht verwendet werden.

**Tab. 34:** Häufige Fehler bei der Verwendung von Kryptografie.

| Fehler | Beschreibung | CWE | Von außen sichtbar? |
|---|---|---|---|
| Keine Verschlüsselung beim Transport | Daten gehen im Klartext über das Netz, auch intern zwischen Services | 319 | ja |
| Veraltete Protokolle und Verfahren | alte TLS-Versionen, schwache Verfahren wie MD5 oder SHA-1 für Sicherheitszwecke | 327 | teilweise |
| Zu kurze Schlüssel | Schlüssellängen, die heute nicht mehr ausreichen | 326 | teilweise |
| Zertifikate nicht geprüft | ein Client akzeptiert jedes Zertifikat | 295 | nein, nur im Code oder im internen Netz |
| Schlüssel im Code | ein Schlüssel steht im Quelltext oder in der App | 321 | bei Apps ja |
| Schlechter Zufall | Kennungen, Tokens oder Codes sind vorhersagbar | 330 | ja, durch Sammeln vieler Werte |
| Falsche Betriebsart | gleiche Eingaben ergeben gleiche Ausgaben, oder ein Einmalwert wird wiederverwendet | 323 | selten |
| Schwache Passwort-Hashes | schnelle oder ungesalzene Hashes | 916 | nein, erst nach Zugriff auf die Daten |
| Keine Verschlüsselung gespeicherter Daten | vertrauliche Daten liegen im Klartext in Datenbank oder Sicherung | 311 | nein |

Die rechte Spalte von Tab. 34 erklärt eine Eigenheit von Pentest-Berichten. Was ein Tester von außen leicht prüfen kann, sind die Einstellungen der verschlüsselten Verbindung: welche Protokollversionen und Verfahren der Server anbietet, ob das Zertifikat in Ordnung ist. Dafür gibt es Werkzeuge, die in Minuten eine lange Liste liefern. Entsprechend häufig stehen solche Befunde im Bericht, meist mit niedrigem Schweregrad. Die schwerwiegenden Fehler dieser Gruppe sieht ein Tester von außen dagegen kaum: wie Passwörter gespeichert sind, ob Schlüssel sauber verwaltet werden, ob ein Service die Zertifikate seiner Gegenstellen prüft. Sie zeigen sich im White-Box-Test, im Code Review oder nach einem Einbruch.

Daraus folgt eine Warnung für das Lesen von Berichten: Ein Bericht mit sechs Befunden zur TLS-Konfiguration und keinem zur Speicherung sagt nichts darüber, ob die Speicherung in Ordnung ist. Er sagt nur, was von außen zu sehen war.

Drei Fehler aus der Tabelle kennst du aus deinem Foliensatz zu den sicheren Programmierpraktiken, mit Code: die abgeschaltete Zertifikatsprüfung, die falsche Betriebsart einer Blockverschlüsselung und die Speicherung von Passwörtern. Auch die Vorsorge für den Wechsel von Verfahren, die Krypto-Agilität, steht dort. Hier genügen zwei Ergänzungen aus der Sicht des Testers.

**Zufall ist ein Sicherheitsmerkmal.** Überall, wo eine Anwendung einen Wert erzeugt, den niemand erraten können soll, also Sitzungskennungen, Verweise zum Zurücksetzen von Passwörtern, Gutscheincodes, Einmalcodes, entscheidet die Qualität des Zufalls über die Sicherheit. Ein gewöhnlicher Zufallsgenerator einer Programmiersprache ist für Simulationen gedacht und liefert Folgen, die sich aus wenigen beobachteten Werten fortsetzen lassen. Für Sicherheitszwecke gibt es eigene Generatoren. Ein Tester sammelt einige hundert solcher Werte und sieht nach, ob sich ein Muster zeigt. Gutscheincodes, die aus dem Zeitpunkt ihrer Erzeugung abgeleitet sind, findet man auf diesem Weg.

**Verschlüsselung ersetzt keine Zugriffskontrolle.** Ein häufiger Trugschluss lautet, die Daten seien „ja verschlüsselt". Verschlüsselung der Datenbank schützt gegen jemanden, der die Festplatte stiehlt. Sie schützt nicht gegen eine SQL-Injection oder einen Fehler der Zugriffskontrolle, denn die Anwendung entschlüsselt die Daten für jeden, dem sie sie ausliefert.

> **MERKE**
> Die Regel für Kryptografie hat zwei Teile. Erstens: keine eigenen Verfahren und keine eigenen Implementierungen, sondern geprüfte Bibliotheken in ihrer empfohlenen Verwendung. Zweitens: Die meiste Arbeit steckt nicht im Verschlüsseln, sondern in der Verwaltung der Schlüssel. Wo sie liegen, wer sie lesen kann und wie sie getauscht werden, ist eine Architekturfrage.

## 10.3 Lieferkette

Dein Foliensatz zu den Grundlagen nennt die Zahl: Eine heutige Anwendung besteht zu 70 bis 90 Prozent aus Code, den andere geschrieben haben. Die OWASP Top 10:2025 führen Fehler der Lieferkette neu auf Platz 3 [26]. In einem Pentest-Bericht erscheinen sie in zwei sehr verschiedenen Formen.

### Komponenten mit bekannter Lücke

Die häufige Form ist ein Befund wie dieser.

| F-12 Bibliothek mit bekannter Schwachstelle | HOCH |
|---|---|
| **Ort** | Buchungsservice, Bibliothek zur Verarbeitung von Bilddateien in einer veralteten Version |
| **Beobachtung** | Die eingesetzte Version ist von einer öffentlich verzeichneten Schwachstelle betroffen, die beim Verarbeiten einer präparierten Datei zur Ausführung von Code führen kann. |
| **Einordnung** | CVE-Nummer der Schwachstelle, CVSS-Wert aus der Schwachstellendatenbank, CWE-1395 für die Abhängigkeit von einer verwundbaren Komponente |
| **Empfehlung** | Aktualisierung auf eine korrigierte Version |

Solche Befunde stammen meist aus einem Werkzeug, das die Versionen der eingesetzten Komponenten mit den Schwachstellendatenbanken abgleicht. Sie tragen den Schweregrad, den die Datenbank der Lücke gibt. Bevor du so einen Befund bewertest, sind vier Fragen zu klären, und ein guter Bericht beantwortet sie selbst.

1. **Wird der betroffene Teil verwendet?** Eine Bibliothek kann hundert Funktionen haben, die Lücke steckt in einer, und die Anwendung ruft sie vielleicht nie auf. Dann ist die Komponente verwundbar und die Anwendung nicht.
2. **Kommt ein Angreifer an die Stelle heran?** Verarbeitet der Buchungsservice überhaupt Bilder, die von außen kommen?
3. **Wird die Lücke ausgenutzt?** Hier helfen KEV und EPSS aus Kapitel 4.
4. **Hat der Tester es geprüft oder nur das Werkzeug?** Ein Befund, der eine Ausnutzung belegt, wiegt anders als einer, der eine Versionsnummer zitiert.

Die zweite Art der Bewertung ist die wichtigere, und sie gilt dem Prozess. Eine einzelne veraltete Bibliothek kann jedem passieren. Zwölf Befunde dieser Art in einem Bericht sind eine Aussage über die Organisation: Es gibt keine Stückliste, keine laufende Prüfung der Abhängigkeiten und keinen geregelten Weg, Aktualisierungen einzuspielen. Das ist die Ursache auf der Ebene Lifecycle, und die Behebung sind nicht zwölf Aktualisierungen, sondern die Dinge aus Kapitel 5: SBOM, Prüfung in der Pipeline, automatisierte Vorschläge für Aktualisierungen und eine Frist, innerhalb derer kritische Lücken zu schließen sind.

Zu den Komponenten mit bekannter Lücke gehören auch die, die niemand mehr pflegt (`CWE-1104`). Sie haben keine bekannte Lücke, weil niemand mehr nachsieht, und wenn eine gefunden wird, kommt keine Korrektur.

**Angriffe auf die Lieferkette selbst**
Die seltenere und schwerere Form ist, dass der Weg der Software ins System angegriffen wird. Der Fall xz aus Kapitel 4 war eine über Jahre vorbereitete Hintertür in einer Bibliothek [21]. Daneben gibt es alltäglichere Muster.

Beim **Typosquatting** veröffentlicht jemand ein Paket, dessen Name einem beliebten Paket zum Verwechseln ähnlich ist, und hofft auf Tippfehler. Bei der **Dependency Confusion** trägt ein öffentliches Paket denselben Namen wie ein internes, und das Build-Werkzeug zieht das öffentliche. Bei der **Übernahme eines Kontos** wird das Konto eines Maintainers gekapert und eine neue Version mit Schadcode veröffentlicht. Eine jüngere Variante hängt an KI-Assistenten: Sprachmodelle erfinden gelegentlich Paketnamen, die es nicht gibt, und Angreifer registrieren solche Namen vorab. Dein Foliensatz nennt das *Package Hallucination* [63].

Dazu kommt der Angriff auf den Build selbst. Wer die Pipeline kontrolliert, in der aus Quelltext ein auslieferbares Artefakt wird, braucht keine Lücke in der Anwendung mehr. Die OWASP Top 10 führen das als A08, Fehler bei der Integrität von Software und Daten: Code oder Aktualisierungen werden übernommen, ohne dass ihre Herkunft und Unverändertheit geprüft werden (`CWE-494`).

Ein Pentest der Anwendung deckt diese Angriffe in der Regel nicht ab, weil Entwicklungsumgebung und Pipeline nicht im Scope sind. Das ist eine der wichtigeren Lücken, die beim Lesen des Scope auffallen sollten.

> **IN DER REIHE**
> Die Absicherung der Pipeline, signierte Artefakte und der Umgang mit Geheimnissen im Build stehen in *Git von Grund auf*, Kapitel 16 und 17. Die Herkunft und Integrität von Images behandelt *Docker von Grund auf*, Kapitel 12.

## 10.4 Fehler in der Fachlogik

Die bisher besprochenen Schwächen sind Verstöße gegen technische Regeln. Die folgenden sind es nicht. Jede einzelne Anfrage ist gültig, jede Antwort korrekt, und trotzdem erreicht jemand etwas, das nie vorgesehen war.

> **DEFINITION 10.2 · FEHLER IN DER FACHLOGIK**
> Ein Fehler in der Fachlogik (*business logic flaw*) ist eine Schwäche, bei der sich die vorgesehenen Funktionen einer Anwendung so verwenden oder kombinieren lassen, dass ein Ergebnis entsteht, das den Regeln des Geschäfts widerspricht. Die Anwendung tut, was programmiert wurde, aber nicht, was gemeint war.

Die OWASP Top 10 fassen die Ursache unter A06, dem unsicheren Entwurf. Die Liste für Programmierschnittstellen hat mit dem ungehinderten Zugriff auf sensible Geschäftsabläufe eine eigene Kategorie dafür [58]. Die Einordnung in die CWE ist unscharf, wie Kapitel 4 angemerkt hat: Es gibt eine Sammelkategorie und einzelne Einträge wie `CWE-841` für die mangelhafte Durchsetzung eines Ablaufs.

Die typischen Muster lassen sich aufzählen.

- **Schritte überspringen.** Die Buchung wird bestätigt, ohne dass der Schritt der Zahlung durchlaufen wurde, weil der Endpunkt für die Bestätigung nicht prüft, in welchem Zustand die Buchung ist.
- **Schritte wiederholen.** Ein Gutschein, ein Rabatt oder eine Erstattung lässt sich mehrfach auslösen.
- **Werte außerhalb der gedachten Grenzen.** Eine negative Menge führt zu einer Gutschrift. Eine sehr große Menge läuft über. Ein Preis von null wird akzeptiert.
- **Vertrauen in Angaben des Clients.** Der Preis, der Rabatt oder die Kennzeichnung „bezahlt“ kommt aus der Anfrage statt aus den eigenen Daten.
- **Funktionen kombinieren.** Buchen mit Gutschein, stornieren, Erstattung in Geld erhalten. Jede Funktion ist für sich richtig, die Kombination tauscht Gutscheine in Geld.

- **Fristen und Reihenfolgen.** Eine Stornierung mit voller Erstattung ist auch nach dem Besuch noch möglich.
- **Massenhafte Nutzung.** Eine Funktion, die für einzelne Menschen gedacht ist, wird automatisiert: alle freien Termine eines beliebten Tags reservieren und nicht bezahlen.

Ein hilfreiches Denkmodell ist der Zustandsautomat. Jedes Geschäftsobjekt durchläuft Zustände, und nur bestimmte Übergänge sind erlaubt. Ein Fehler in der Fachlogik ist ein Übergang, den es nicht geben dürfte und den die Anwendung trotzdem zulässt.

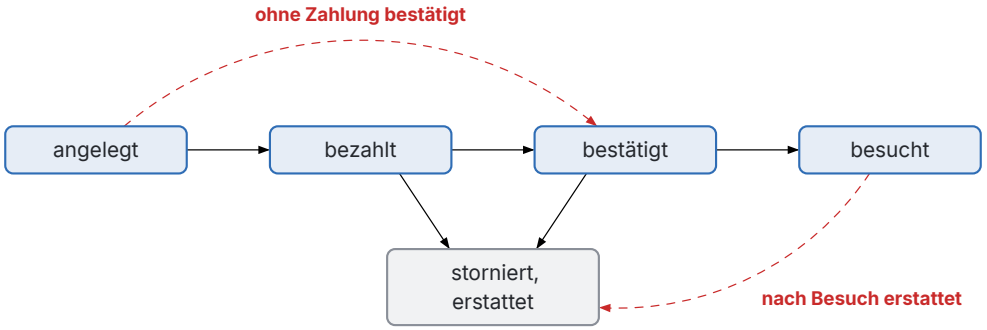



**Abb. 20:** Die Zustände einer Buchung. Durchgezogen sind die erlaubten Übergänge, rot gestrichelt zwei Übergänge, die ein Tester versucht.

Der Zustandsautomat ist auch das Mittel zur Behebung. Im Code prüft jede Funktion, ob das Objekt in einem Zustand ist, aus dem der verlangte Übergang erlaubt ist. In der Architektur liegt diese Prüfung an einer Stelle, bei dem Service, dem das Objekt gehört, und nicht verteilt über alle, die es verändern wollen. Im Lifecycle steht der Automat in der Spezifikation, zusammen mit Missbrauchsfällen aus Kapitel 6, und zu jedem verbotenen Übergang gibt es einen Test.

Fehler in der Fachlogik findet kein Werkzeug, weil kein Werkzeug die Regeln des Geschäfts kennt. Sie sind der Grund, warum Pentests von Menschen gemacht werden, und sie sind die Befunde, bei denen ein Tester mit Kenntnis der Architektur und der Fachlichkeit am meisten findet. In Berichten sind sie oft unterbewertet, weil die technische Komplexität niedrig ist und CVSS für sie schlecht passt. Ihr Schaden ist dafür unmittelbar in Geld auszudrücken.

### Wettlaufsituationen

Eine besondere Gruppe von Logikfehlern entsteht erst durch Gleichzeitigkeit. Die Anwendung prüft eine Bedingung und handelt danach, und zwischen Prüfen und Handeln vergeht Zeit. Schickt jemand in diesem Fenster eine zweite Anfrage, sieht auch sie die Bedingung als erfüllt.

Dein Foliensatz zu den sicheren Programmierpraktiken zeigt das Muster an Dateien unter dem Namen TOCTOU, *Time of Check to Time of Use* (`CWE-367`). Dein Foliensatz zu den Transaktionen zeigt es an einem Gutschein über 50 Euro und zwei gleichzeitigen Bestellungen, mit der Behebung als einer einzigen bedingten Aktualisierung. Die allgemeine Klasse heißt Wettlaufsituation, *Race Condition* (`CWE-362`). Neuere Forschung hat gezeigt, dass solche Fenster in Web-Anwendungen viel häufiger offenstehen und viel zuverlässiger zu treffen sind, als lange angenommen wurde, weil sich viele Anfragen so absenden lassen, dass sie praktisch gleichzeitig beim Server ankommen [54].

In verteilten Systemen verschärft sich das Problem. Eine Saga verzichtet auf Isolation, Zwischenzustände sind für andere sichtbar. Jede Stelle, an der zwischen dem Lesen eines Zustands in einem Service und dem Schreiben in einem anderen etwas dazwischenkommen kann, ist ein Kandidat. Die OWASP Top 10:2025 nennen Wettlaufsituationen ausdrücklich in der neuen Kategorie A10 [26].

> **BRÜCKE ZU SOFTWARE SECURITY**
> Hier schließt sich ein Kreis zu deinen Foliensätzen. Was du als Anomalien von Sagas unterrichtest, also verlorene Aktualisierungen und schmutziges Lesen, sind aus der Sicht eines Testers Angriffsflächen. Die Gegenmaßnahmen sind dieselben: semantische Sperren, bedingte Aktualisierungen, Versionsprüfungen und Idempotenz. In einem Pentest-Bericht steht dazu ein Satz wie „Gutschein durch parallele Anfragen mehrfach einlösbar“. Dass dahinter eine fehlende Isolation im Entwurf des Ablaufs steckt, sieht nur, wer beide Seiten kennt. Das ist ein Musterfall für die Spezialisierung.

## 10.5 Ressourcen und das Verhalten im Fehlerfall

Zwei letzte Gruppen betreffen nicht, was ein System tut, sondern wie es sich an seinen Grenzen verhält.

### Ungebremster Verbrauch

Jede Anfrage kostet den Server etwas: Rechenzeit, Speicher, Datenbankverbindungen, manchmal Geld, wenn ein bezahlter Dienst aufgerufen oder eine SMS verschickt wird. Setzt die Anwendung keine Grenzen, kann ein Einzelner mit wenig Aufwand viel verbrauchen (`CWE-770`). Die Liste der OWASP für Schnittstellen führt das als unbeschränkten Ressourcenverbrauch [58].

Die Formen reichen vom einfachen Fehlen einer Begrenzung der Anfragen pro Zeit über Abfragen ohne Obergrenze für die Ergebnismenge bis zu Eingaben, die gezielt teure Verarbeitung auslösen. Dein Foliensatz zeigt für das letzte die Vervielfachung von XML-Entitäten. In Pentests werden solche Schwächen oft nur festgestellt und nicht ausgereizt, weil Lasttests meist ausgeschlossen sind. Der Befund lautet dann „keine Begrenzung erkennbar", und seine Schwere muss jemand aus der Architektur ableiten: Was passiert mit dem Rest des Systems, wenn dieser eine Service überlastet ist? Die Antwort geben die Verfügbarkeitstaktiken aus deinem Foliensatz, also Begrenzung, Zeitlimits, Circuit Breaker und Bulkheads.

> **IN DER REIHE**
> Grenzen auf allen Ebenen, Rate Limiting und der Schutz von Geschäftsabläufen gegen Missbrauch stehen in *Secure API Design*, Kapitel 6.

### Offen oder geschlossen versagen

Die neue Kategorie A10 der OWASP Top 10:2025 behandelt den falschen Umgang mit Ausnahmesituationen. Ihr wichtigster Gedanke ist ein altes Entwurfsprinzip, das schon Saltzer und Schroeder nennen [41].

> **DEFINITION 10.3 · FAIL OPEN UND FAIL CLOSED**
> Ein System versagt **geschlossen** (*fail closed*, auch *fail secure*), wenn es im Fehlerfall den Zugriff verweigert. Es versagt **offen** (*fail open*), wenn es im Fehlerfall den Zugriff gewährt. Für Sicherheitsfunktionen ist das geschlossene Versagen die richtige Voreinstellung (`CWE-636`).

Das Muster ist leicht zu übersehen. Der Abrechnungsservice fragt beim Kundenkonto nach, ob ein Benutzer berechtigt ist. Das Kundenkonto antwortet nicht rechtzeitig. Was nun? Wirft der Code eine Ausnahme, die irgendwo abgefangen und ignoriert wird, läuft die Anfrage ohne Prüfung weiter. Ein Tester sucht solche Stellen, indem er Fehler provoziert: unvollständige Eingaben, unerwartete Typen, abgebrochene Abläufe, überlastete Abhängigkeiten.

Hier entsteht ein echter Zielkonflikt, der in der Spezialisierung nützlich ist. Die Verfügbarkeitstaktiken wollen, dass ein System bei Ausfall einer Abhängigkeit weiterarbeitet. Dein Foliensatz zeigt den Circuit Breaker, bei dem die Kasse weiterläuft, wenn die Empfehlungen ausfallen. Für Empfehlungen ist das richtig. Für eine Berechtigungsprüfung ist dieselbe Taktik ein Sicherheitsfehler. Welche Abhängigkeit im Fehlerfall übergangen werden darf und welche nicht, ist eine Entscheidung, die in den Entwurf gehört und in ein Architecture Decision Record.

### Protokollierung und Alarmierung

Zuletzt die Kategorie A09, mangelhafte Protokollierung und Alarmierung [26]. Sie ist die einzige der zehn, die keinen Angriff ermöglicht, sondern das Bemerken eines Angriffs verhindert. In STRIDE ist sie die Bedrohung des Abstreitens aus Kapitel 6.

Ein Tester beurteilt sie indirekt. Er hat tagelang auffällige Dinge getan: tausende fehlgeschlagene Anmeldungen, systematisches Durchzählen von Kennungen, Zugriffe auf Verwaltungsfunktionen mit Kundenkonten. Hat der Betrieb davon etwas bemerkt? Wurden sicherheitsrelevante Ereignisse überhaupt aufgezeichnet, mit Zeitpunkt, Urheber und Objekt? Kann jemand die Aufzeichnungen verändern? Die Antworten gehören in den Bericht, auch wenn sie sich nicht als einzelne Schwachstelle mit CWE-Nummer fassen lassen.

## 10.6 Die drei Ebenen im Überblick

**Tab. 35:** Behebungen der Schwächen dieses Kapitels auf drei Ebenen.

| Schwäche | Code | Architektur | Lifecycle |
|---|---|---|---|
| Fehlkonfiguration | selten betroffen | sichere Voreinstellungen im Projektgerüst, minimale Rechte für Dienstkonten | Härtungsstandard, Konfiguration als Code, automatische Prüfung auf Abweichungen, Stilllegen von Abgelöstem |
| Kryptografische Fehler | geprüfte Bibliotheken richtig verwenden, sichere Zufallsquellen | Schlüsselverwaltung in einem Tresor, Verschlüsselung an allen Verbindungen, Austauschbarkeit der Verfahren | Vorgaben für erlaubte Verfahren, Prüfregeln, Inventar von Schlüsseln und Zertifikaten |
| Komponente mit bekannter Lücke | aktualisieren | Schaden begrenzen: minimale Rechte, eingeschränkte ausgehende Verbindungen | Stückliste, Prüfung in der Pipeline, Fristen für Aktualisierungen |
| Angriff auf die Lieferkette | nicht betroffen | getrennte, abgesicherte Build-Umgebung | feste Versionen, geprüfte Herkunft, signierte Artefakte, Vier-Augen-Prinzip |
| Fehler in der Fachlogik | Zustand vor jedem Übergang prüfen, Werte am Server bestimmen | Zustandsautomat an einer Stelle, beim Eigentümer des Objekts | Missbrauchsfälle in den Anforderungen, Tests für verbotene Übergänge |
| Wettlaufsituation | Prüfen und Ändern in einem Schritt | Idempotenz, Sperren oder Versionsprüfung im Entwurf des Ablaufs | gezielte Tests auf Gleichzeitigkeit |
| Ungebremster Verbrauch | Obergrenzen für Eingaben und Ergebnismengen | Begrenzung am Gateway, Isolation durch Bulkheads, Zeitlimits | Lasttests, Überwachung des Verbrauchs |
| Offenes Versagen | Fehler in Sicherheitsprüfungen führen zur Ablehnung | festlegen, welche Abhängigkeit übergangen werden darf | Tests mit ausgefallenen Abhängigkeiten |
| Fehlende Protokollierung | sicherheitsrelevante Ereignisse aufzeichnen | zentrale, manipulationsgeschützte Sammlung | Alarmregeln, regelmäßige Auswertung, Übungen |

Im Vergleich der drei Übersichtstabellen der Kapitel 8 bis 10 zeigt sich eine Verschiebung. Bei den Eingabeschwächen lag der Schwerpunkt in der Spalte Code, bei der Zugriffskontrolle in der Spalte Architektur, hier liegt er in der Spalte Lifecycle. Für die Fehlkonfiguration und die Lieferkette steht in der Spalte Code fast nichts. Ein Team, das jeden Befund als Aufgabe für die Entwickler der Anwendung versteht, kann diese Befunde nur als Symptom beheben.

FRAGE AUS DEM HÖRSAAL

***Wenn eine Fehlkonfiguration in fünf Minuten behoben ist, warum zählt sie dann zu den größten Risiken?***

Weil die fünf Minuten nur das eine Vorkommen betreffen. Die Einstellung war falsch, weil niemand festgelegt hat, wie sie sein soll, und niemand prüft, ob sie so ist. Solange das so bleibt, entsteht bei der nächsten neuen Umgebung, dem nächsten neuen Service oder der nächsten Änderung derselbe Fehler wieder. Dazu kommt die schiere Menge an Einstellungen in heutigen Cloud-Umgebungen. Die Fehlkonfiguration ist das beste Beispiel dafür, dass ein Befund schnell geschlossen und trotzdem nicht behoben sein kann.

FRAGE AUS DEM HÖRSAAL

***Unser Scanner meldet vierzig verwundbare Bibliotheken. Müssen wir alle sofort aktualisieren?***

Alle aktualisieren solltet ihr, aber nicht alle sofort. Sortiert nach drei Fragen: Steht die Lücke im KEV-Katalog? Ist der betroffene Teil in eurer Anwendung von außen erreichbar? Was wäre die Folge? Daraus ergibt sich eine kleine Zahl von Fällen, die heute zu erledigen sind, und ein Rest, der mit der regulären

Pflege mitläuft. Wichtiger als die vierzig Einzelfälle ist die Frage, warum es vierzig geworden sind. Ein Prozess, der Abhängigkeiten laufend aktuell hält, hat nie so einen Rückstand.

**FRAGE AUS DEM HÖRSAAL**

***Wie soll ein Tester Fehler in der Fachlogik finden, wenn er unser Geschäft nicht kennt?***

Genau das ist sein Problem, und es ist euer Vorteil, wenn ihr es löst. Ein Tester, dem jemand eine Stunde lang die wichtigsten Abläufe und Regeln erklärt, findet in dieser Kategorie ein Vielfaches. Noch besser ist es, ihm die Zustandsautomaten und Missbrauchsfälle aus dem Entwurf zu geben. Diese Unterlagen muss das Team ohnehin haben, wenn es die Logik sauber bauen will. Fehlen sie, ist das ein Hinweis, dass die Logik auch nicht sauber gebaut ist.

**FÜR DIE LEHRE**

Dieses Kapitel liefert die besten Fälle für die Unterscheidung der drei Ebenen. Eine wirksame Aufgabe: Gib den Studierenden neun Befunde, je drei aus den Kapiteln 8, 9 und 10, und lass sie für jeden entscheiden, auf welcher Ebene die Hauptursache liegt. Erwartungsgemäß ordnen sie fast alles dem Code zu. Die Besprechung der Befunde zu Fehlkonfiguration und Lieferkette, bei denen im Code der Anwendung nichts zu korrigieren ist, verschiebt das Bild.

Für die Case Study sind Logikfehler mit Wettlaufsituation besonders ergiebig, weil sie an deine Folien-sätze zu Sagas anschließen und weil KI-Assistenten bei ihrer Behebung oft danebenliegen: Sie schlagen eine zusätzliche Prüfung vor dem Schreiben vor, also genau das Muster, das die Lücke erzeugt.

Typische Fehlvorstellungen: dass verschlüsselte Daten vor Fehlern der Anwendung geschützt seien, dass ein Befund zu einer Bibliothek immer bedeute, die Anwendung sei verwundbar, und dass Logik-fehler „kein Sicherheitsthema“ seien.

## Selbsttest

1. Warum ist die Fehlkonfiguration in den OWASP Top 10 aufgestiegen?
2. Ein Befund meldet einen aus dem Internet erreichbaren Diagnose-Endpunkt. Er wird abgeschaltet. Welche zwei Fragen führen zur Ursache?
3. Welche kryptografischen Fehler sieht ein Tester von außen, welche nicht? Was folgt daraus für das Lesen eines Berichts?
4. Welche vier Fragen stellst du, bevor du einen Befund zu einer Bibliothek mit bekannter Lücke bewertest?
5. Erkläre Typosquatting und Dependency Confusion in je einem Satz.
6. Beschreibe einen Fehler in der Fachlogik als verbotenen Zustandsübergang an einem eigenen Beispiel.
7. Warum ist eine zusätzliche Prüfung vor dem Schreiben keine Behebung einer Wettlaufsituation?
8. Was bedeutet geschlossenes Versagen, und warum kann ein Circuit Breaker an der falschen Stelle ein Sicherheitsfehler sein?
9. In welcher Spalte der Tabelle steht bei Fehlkonfiguration und Lieferkette die eigentliche Behebung, und was folgt daraus?

# 11 Nach dem ersten Zugriff

Ein Tester hat über eine Schwäche im Buchungsservice die Möglichkeit erlangt, dort eigene Befehle auszuführen. Für den Bericht ist das ein Befund. Für einen Angreifer ist es der Anfang. Was er von dieser Stelle aus erreicht, also welche Daten, welche weiteren Systeme, welche Rechte, entscheidet darüber, ob der Vorfall eine Fußnote bleibt oder das Unternehmen gefährdet. Diese Reichweite hängt kaum noch von der ursprünglichen Schwäche ab. Sie hängt von der Architektur ab. Dieses Kapitel erklärt, was nach dem ersten Zugriff geschieht, mit den Begriffen, die in Berichten dafür stehen, und zeigt, warum der Wirkungsradius eines Einbruchs eine Eigenschaft des Entwurfs ist.

**IN DIESEM KAPITEL**

- erklären, warum die Auswirkung eines Befunds von der Umgebung der Schwäche abhängt
- die Schritte nach dem ersten Zugriff mit den Taktiken von ATT&CK benennen
- Rechteausweitung, Zugriff auf Zugangsdaten und Seitwärtsbewegung in ihrer Wirkungsweise beschreiben
- erklären, was das Knacken von Passwort-Hashes bedeutet und was es verhindert
- den Wirkungsradius als Architektureigenschaft beschreiben und die Entscheidungen nennen, die ihn bestimmen

## 11.1 Der Befund ist nicht das Ende

Kapitel 3 hat die Phase genannt, um die es hier geht: *Post Exploitation* bei PTES, die Schritte „Rechte ausweiten“ und „System erkunden“ bei NIST. Ihr Zweck im Pentest ist, die Auswirkung eines Befunds zu bestimmen. Die Frage lautet nicht mehr, ob die Tür aufgeht, sondern was hinter der Tür liegt.

Dieselbe technische Schwäche kann sehr Verschiedenes bedeuten. Angenommen, die Bildverarbeitung im Buchungsservice hat eine Lücke, über die ein Angreifer Befehle ausführen kann. In einer ersten Umgebung läuft der Service als gewöhnlicher Benutzer in einem Container ohne besondere Rechte, erreicht im Netz nur seine eigene Datenbank und den Broker, und sein Datenbankkonto darf nur die Tabellen der Buchungen lesen und schreiben. Der Angreifer kommt an die Buchungsdaten, und dort endet es. In einer zweiten Umgebung läuft derselbe Service als Administrator im Container, im Container liegen die Zugangsdaten für die Cloud-Plattform, das Netz ist flach und jeder Service erreicht jeden anderen, und alle Services verwenden dasselbe Datenbankkonto. Der Angreifer hat nach einer Stunde das gesamte System.

Der CVSS-Basiswert der Lücke ist in beiden Umgebungen derselbe. Das Risiko ist es nicht. Ein Bericht, der nur die Lücke nennt, lässt die wichtigere Hälfte weg.

**MERKE**

Die Schwäche bestimmt, ob ein Angreifer hineinkommt. Die Architektur bestimmt, wie weit er kommt.

Aus dieser Einsicht ist eine Haltung geworden, die unter dem Namen *Assume Breach* läuft: Ein System wird so entworfen, als sei ein Einbruch in einen seiner Teile bereits geschehen. Die Frage an den Entwurf lautet dann nicht nur, wie man Einbrüche verhindert, sondern auch, was ein Angreifer von jedem einzelnen Teil aus erreichen kann. Der Name des Studiengangs, *Cyber Security and Resilience*, enthält diesen Gedanken: Resilienz ist die Fähigkeit, einen Treffer zu überstehen.

## 11.2 Was ein Angreifer nach dem Zugriff tut

Die ATT&CK-Matrix aus Kapitel 4 ordnet das Vorgehen nach dem ersten Zugriff in Taktiken [35]. An ihnen entlang lässt sich erzählen, was im Beispiel geschieht.

**Tab. 36:** Die Schritte nach dem ersten Zugriff in den Taktiken von ATT&CK, mit der jeweiligen Begrenzung.

| Taktik | Was der Angreifer tut | Was ihn begrenzt |
|---|---|---|
| Discovery | sieht sich um: Welcher Benutzer bin ich, welche Dateien, Prozesse, Netzverbindungen und Nachbarn gibt es? | minimale Images ohne Werkzeuge, wenig erreichbare Nachbarn |
| Credential Access | sucht Zugangsdaten: in Konfigurationsdateien, Umgebungsvariablen, im Speicher, beim Metadatendienst | Geheimnisse nicht im Container ablegen, kurzlebige Zugangsdaten |
| Privilege Escalation | versucht, höhere Rechte auf dem Rechner oder in der Plattform zu erlangen | minimale Rechte, gehärtete Container, aktuelle Systeme |
| Lateral Movement | bewegt sich mit den gefundenen Zugangsdaten und Verbindungen zu weiteren Systemen | Netzsegmentierung, Authentifizierung zwischen Services, eigene Identität je Service |
| Persistence | richtet sich so ein, dass der Zugang einen Neustart oder eine Korrektur der Lücke übersteht | unveränderliche Infrastruktur, Überwachung von Änderungen |
| Stealth, Defense Impairment | vermeidet Entdeckung, schwächt Überwachung und Schutzfunktionen | Protokolle außerhalb der Reichweite des betroffenen Systems |
| Collection, Exfiltration | sammelt die gesuchten Daten und bringt sie hinaus | Beschränkung ausgehender Verbindungen, Erkennung ungewöhnlicher Datenmengen |
| Impact | verändert, verschlüsselt oder zerstört Daten und Systeme | Sicherungen außer Reichweite, Wiederherstellungsübungen |

Die rechte Spalte von Tab. 36 liest sich wie eine Liste aus deinen Foliensätzen. Das ist kein Zufall. Jede Maßnahme dort ist eine Architektur- oder Betriebsentscheidung, und keine davon verhindert den ersten Zugriff. Sie alle wirken danach. Die Sicherheitstaktiken in deinem Foliensatz zu den Architekturtaktiken teilen sich in erkennen, widerstehen, reagieren und wiederherstellen. Der größte Teil dieses Kapitels handelt vom Widerstehen in der Tiefe und vom Erkennen.

In einem Pentest geht der Tester diesen Weg nur so weit, wie der Auftrag es erlaubt. Im Scope aus Kapitel 2 stand, dass er nach der Übernahme eines Service nicht in die Cloud-Plattform vordringt. Er wird also feststellen und belegen, dass er im Container Zugangsdaten für die Plattform gefunden hat, und er wird sie nicht verwenden. Im Bericht steht dann ein Satz wie „Die weitere Ausnutzung wurde gemäß Vereinbarung nicht durchgeführt“. Dieser Satz bedeutet nicht, dass es dahinter harmlos wäre.

## 11.3 Rechteausweitung

> **DEFINITION 11.1 · RECHTEAUSWEITUNG**
> Rechteausweitung (*privilege escalation*) ist jeder Schritt, mit dem ein Angreifer von den Rechten, die er hat, zu höheren Rechten gelangt. Auf einem einzelnen Rechner heißt das meist: vom Konto eines Dienstes zum Administrator. In einer Plattform heißt es: von den Rechten eines Dienstes zu denen, die andere Dienste oder die Verwaltung der Plattform betreffen.

Kapitel 9 hat die Rechteausweitung innerhalb der Anwendung behandelt, vom Kunden zur Kassenkraft. Hier geht es um die Schicht darunter. Die Wege dorthin lassen sich in drei Gruppen ordnen, und jede hat eine andere Ursache.

Die erste und häufigste Gruppe sind **zu weite Rechte von Anfang an**. Der Dienst läuft bereits als Administrator, weil das bei der Einrichtung am einfachsten war. Dann gibt es nichts auszuweiten. Dein Foliensatz zu den sicheren Programmierpraktiken behandelt die Varianten bei Containern: der Betrieb als Root-Benutzer, der privilegierte Modus, zusätzliche Fähigkeiten des Betriebssystems, Zugriff auf Geräte des Wirtsrechners und der in den Container gereichte Zugang zur Container-Verwaltung. Jede davon

macht aus der Übernahme eines Containers die Übernahme des Rechners, auf dem er läuft. Die Ursache ist eine Fehlkonfiguration im Sinn von Kapitel 10.

Die zweite Gruppe sind **auffindbare Zugangsdaten** eines mächtigeren Kontos. Sie leitet zum nächsten Abschnitt über.

Die dritte Gruppe sind **Schwachstellen in der Schicht darunter**: im Betriebssystem, in der Laufzeitumgebung der Container, in einem mit hohen Rechten laufenden Hilfsprogramm. Sie tragen CVE-Nummern, und ihre Ursache ist ein fehlender Aktualisierungsprozess.

Für die Arbeit mit Befunden ist die Unterscheidung nützlich, weil sie die Ebene der Behebung anzeigt. Bei der ersten Gruppe liegt sie in den Vorgaben für den Betrieb von Diensten, bei der zweiten im Umgang mit Geheimnissen, bei der dritten in der Pflege der Plattform. In keinem Fall liegt sie im Code der Anwendung.

> IN DER REIHE
> Die einzelnen Einstellungen, mit denen ein Container ohne unnötige Rechte läuft, stehen in *Docker von Grund auf*, Kapitel 12.

## 11.4 Zugangsdaten

Der ergiebigste Fund nach einem ersten Zugriff sind Zugangsdaten. Mit ihnen braucht ein Angreifer keine weiteren Schwachstellen. Er meldet sich an. In der ATT&CK-Matrix ist die Verwendung gültiger Konten eine eigene, sehr häufig beobachtete Technik [35].

### Wo sie liegen

Ein Dienst braucht Zugangsdaten für alles, womit er spricht: Datenbank, Broker, andere Services, Dienste der Cloud-Plattform, externe Partner. Wo er sie herbekommt, entscheidet, was ein Angreifer findet.

- In **Konfigurationsdateien** und im **Image** selbst. Wer das Image hat, hat die Zugangsdaten, auch ohne Einbruch.
- In **Umgebungsvariablen**. Besser als im Image, aber für jeden lesbar, der im Container Befehle ausführen kann, und sie tauchen in Fehlerberichten und Diagnoseausgaben auf.
- Im **Arbeitsspeicher** des laufenden Prozesses. Dein Foliensatz beginnt mit diesem Fall.
- Beim **Metadatendienst** der Cloud-Plattform, wie im Fall Capital One aus Kapitel 8.
- In der **Pipeline**: Die Variablen des Build-Systems enthalten oft die mächtigsten Zugangsdaten überhaupt, nämlich die zum Ausliefern in die Produktion.
- In **Sicherungen**, **Protokollen** und **Versionsgeschichten**.

Die Qualität der Zugangsdaten zählt ebenso wie ihr Ort. Ein langlebiges Passwort eines Kontos mit weiten Rechten, das mehrere Services teilen, ist der schlechteste Fall. Ein kurzlebiges Zugangsmerkmal, das nur für einen Service gilt, nur dessen Aufgaben erlaubt und in Minuten abläuft, ist der beste. Dazwischen liegt eine Architekturentscheidung: ob Geheimnisse statisch verteilt oder zur Laufzeit von einem zentralen Tresor an eine nachgewiesene Identität ausgegeben werden.

### Hashes knacken

Ein eigener Begriff aus diesem Umfeld verdient eine Erklärung, weil er in Berichten und in der Pentesting-Lehrveranstaltung vorkommt und oft falsch verstanden wird.

Gelangt ein Angreifer an die Benutzertabelle einer Anwendung, findet er dort keine Passwörter, sondern deren Hashes, sofern die Anwendung richtig gebaut ist. Ein Hash lässt sich nicht zurückrechnen. Er lässt sich aber nachrechnen: Der Angreifer nimmt ein mögliches Passwort, berechnet dessen Hash und vergleicht ihn mit dem erbeuteten. Stimmen sie überein, kennt er das Passwort. Das nennt man *Knacken* (*cracking*), und es ist ein Angriff, der vollständig auf den Rechnern des Angreifers stattfindet. Die Anwendung bemerkt nichts davon, und keine Begrenzung von Anmeldeversuchen hilft.

Wie aussichtsreich das ist, hängt von drei Dingen ab, die dein Foliensatz zur Passwortspeicherung nennt.

**Die Geschwindigkeit des Hash-Verfahrens.** Allgemeine Hash-Verfahren sind auf Tempo gebaut, mit ihnen lassen sich auf geeigneter Hardware sehr viele Kandidaten pro Sekunde prüfen. Verfahren für Passwörter sind absichtlich langsam und speicherhungrig, sodass jeder Versuch spürbar kostet.

**Das Salz.** Ohne einen zufälligen, pro Benutzer verschiedenen Zusatz ergibt dasselbe Passwort überall denselben Hash. Dann lassen sich vorberechnete Tabellen verwenden, und ein geknacktes Passwort gilt für alle Konten mit demselben Hash. Mit Salz muss jedes Konto einzeln angegriffen werden.

**Die Qualität der Passwörter.** Angreifer probieren nicht alle Zeichenfolgen durch. Sie verwenden Listen bekannter Passwörter aus früheren Datenabflüssen und Regeln für die üblichen Abwandlungen. Ein Passwort, das in einer solchen Liste steht, fällt unabhängig vom Verfahren schnell.

Im Bericht steht dazu ein Satz wie „Von den 4.000 erbeuteten Hashes konnten innerhalb eines Tages 60 Prozent geknackt werden“. Diese Zahl ist eine Messung von zwei Dingen zugleich: der Speicherung und der Passwortrichtlinie. Sie ist auch der Grund, warum ein Datenabfluss mit Passwort-Hashes zu behandeln ist, als wären die Passwörter selbst abgeflossen, sobald das Verfahren schwach ist.

Geknackte Passwörter sind über die eine Anwendung hinaus wertvoll, weil Menschen sie wiederverwenden. Das Passwort des Administrators der Buchungsplattform passt möglicherweise auch für die Verwaltung der Cloud-Plattform. Damit schließt sich der Kreis zum Credential Stuffing aus Kapitel 9.

## 11.5 Seitwärtsbewegung

> **DEFINITION 11.2 · SEITWÄRTSBEWEGUNG**
> Seitwärtsbewegung (*lateral movement*) ist der Übergang eines Angreifers von einem übernommenen System zu einem weiteren System im selben Netz, unter Verwendung von Zugangsdaten, Vertrauensbeziehungen oder Schwachstellen, die erst von innen erreichbar sind.

Wie weit sich ein Angreifer seitwärts bewegen kann, hängt von zwei Eigenschaften der Architektur ab.

Die erste ist die **Erreichbarkeit**. In einem flachen Netz kann jeder Rechner jeden anderen ansprechen. Der übernommene Buchungsservice erreicht dann die Datenbank der Abrechnung, die Verwaltungsoberfläche des Brokers und die Überwachung. In einem segmentierten Netz erreicht er nur, was er für seine Aufgabe braucht. Segmentierung hieß früher Firewalls zwischen Netzbereichen. In Container-Plattformen heißt sie Netzrichtlinien, die festlegen, welcher Dienst mit welchem sprechen darf.

Die zweite ist das **Vertrauen**. Selbst wenn der Buchungsservice den Abrechnungsservice erreicht, muss dieser die Anfrage nicht annehmen. Hier wirkt, was die Kapitel 6 und 9 beschrieben haben: Authentifizierung zwischen Services, eine eigene Identität je Service und minimale Rechte an jeder Verbindung. Implizites Vertrauen im internen Netz ist die Einladung zur Seitwärtsbewegung.

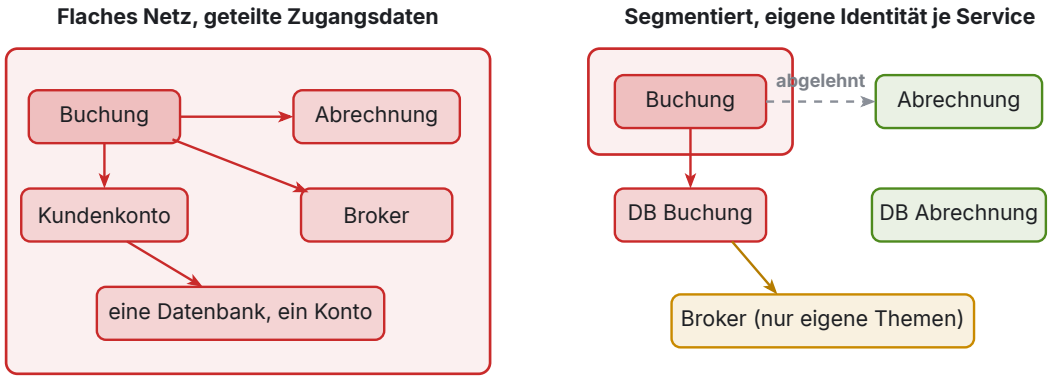



**Abb. 21:** Derselbe Einbruch in den Buchungsservice in zwei Architekturen. Rot ist, was der Angreifer erreicht.

Abb. 21 zeigt den Begriff, auf den dieses Kapitel hinausläuft. Dein Foliensatz zu den Grundlagen nennt ihn bei der Analyse der großen Softwarefehler: den *Blast Radius*, den Wirkungsradius. Dort ging es um einen Fehler, der sich durch das ganze System fortpflanzt. Hier ist es ein Angreifer, der dasselbe tut. Die Maßnahmen dagegen sind dieselben, und einige kennst du als Verfügbarkeitstaktiken: Das Bulkhead-Muster, das einen überlasteten Teil vom Rest trennt, ist dieselbe Idee wie die Segmentierung, die einen übernommenen Teil vom Rest trennt.

### Die zentrale Identität

In Unternehmensnetzen gibt es ein System, das für Angreifer mehr wert ist als jedes andere: den zentralen Verzeichnisdienst, der Benutzer, Rechner, Gruppen und Berechtigungen verwaltet. In Windows-

Umgebungen ist das das *Active Directory*. Wer es beherrscht, beherrscht alles, was ihm vertraut, also praktisch das gesamte Netz. Die Pentesting-Lehrveranstaltung im Bachelor behandelt Angriffe darauf ausführlich.

Für dieses Buch genügt das Prinzip, denn es ist dasselbe wie im Kleinen. Die typischen Schwächen sind keine Programmierfehler. Es sind Konten mit zu weiten Rechten, Dienstkonten mit schwachen Passwörtern, alte Anmeldeverfahren, die aus Gründen der Verträglichkeit eingeschaltet blieben, und Vertrauensbeziehungen, die niemand mehr überblickt. Ein Angreifer bewegt sich darin von Konto zu Konto, bis er eines erreicht, das alles darf. In der Cloud nimmt die Verwaltung von Identitäten und Berechtigungen der Plattform diese Rolle ein, mit denselben Fehlerarten.

Bei einem Pentest einer einzelnen Anwendung ist der Verzeichnisdienst fast nie im Scope. Wichtig ist er hier als Ziel am Ende des Weges: Wenn im Bericht steht, dass im übernommenen Service Zugangsdaten eines Kontos aus dem Verzeichnis gefunden wurden, ist das die Stelle, an der aus einem Einbruch in eine Anwendung ein Einbruch in das Unternehmen werden kann.

## 11.6 Unentdeckt bleiben

Die letzte Gruppe von Begriffen betrifft das Verhältnis zwischen Angreifer und Überwachung. In der Pentesting-Lehrveranstaltung steht dazu das Stichwort der Verschleierung von Schadcode.

Schutzprogramme auf Rechnern arbeiten mit zwei Verfahren. Das ältere erkennt bekannte Schadprogramme an ihrem Aussehen, an charakteristischen Folgen von Bytes. Das neuere beobachtet das Verhalten: Ein Programm, das massenhaft Dateien verschlüsselt oder aus einem Büroprogramm heraus eine Kommandozeile startet, fällt auf, egal wie es aussieht. Verschleierung richtet sich gegen das erste Verfahren. Ändert man das Aussehen eines Programms, ohne sein Verhalten zu ändern, passt das bekannte Muster nicht mehr.

Für die Verteidigung folgt daraus, dass Erkennung nach dem Aussehen eine notwendige und keine hinreichende Schicht ist. Wer sich auf sie allein verlässt, erkennt nur Angreifer, die sich keine Mühe geben. Das Erkennen von Verhalten braucht Daten: Protokolle der Rechner, der Anwendungen und des Netzes, an einer Stelle zusammengeführt, die der Angreifer vom übernommenen System aus nicht erreicht. Dein Foliensatz zu den Architekturtaktiken nennt das bei den Taktiken zum Erkennen und zum Wiederherstellen.

Für den Pentest einer Anwendung ist dieser Teil weniger eine Technik als eine Frage an den Betrieb, dieselbe wie in Kapitel 7: Wurde etwas bemerkt? Für einen Red-Team-Einsatz ist er der Kern.

## 11.7 Der Wirkungsradius als Architektureigenschaft

Fasst man das Kapitel zusammen, bleibt eine Liste von Entscheidungen, die bestimmen, wie weit ein Einbruch reicht. Keine davon steht im Code einer Funktion. Alle stehen, wenn sie überhaupt irgendwo stehen, in Architekturentscheidungen.

**Tab. 37:** Architekturentscheidungen, die den Wirkungsradius eines Einbruchs bestimmen.

| Entscheidung | Vergrößert den Radius | Verkleinert den Radius |
|---|---|---|
| Rechte des Prozesses | Dienst läuft als Administrator | eigener Benutzer ohne besondere Rechte |
| Container | privilegiert, mit Zugriff auf den Wirt | gehärtet, schreibgeschütztes Dateisystem |
| Geheimnisse | langlebig, im Image oder in Variablen, zwischen Services geteilt | kurzlebig, zur Laufzeit aus einem Tresor, je Service |
| Datenhaltung | eine Datenbank und ein Konto für alle | Datenbank und Konto je Service, minimale Rechte |
| Netz | flach, jeder erreicht jeden | segmentiert nach Bedarf |
| Vertrauen zwischen Services | implizit, nach Netzposition | Authentifizierung und Autorisierung an jeder Verbindung |
| Ausgehende Verbindungen | unbeschränkt ins Internet | nur zu benannten Zielen |
| Berechtigungen in der Plattform | weite Rollen für Dienstkonten | Rollen genau auf die Aufgabe zugeschnitten |
| Überwachung | Protokolle nur lokal auf dem Dienst | zentrale Sammlung außer Reichweite, Alarmregeln |

Jede Zeile von Tab. 37 ist eine Anwendung desselben Prinzips, das schon Saltzer und Schroeder formuliert haben: *Least Privilege*, die minimale Berechtigung. Jeder Teil eines Systems soll genau die Rechte haben, die er für seine Aufgabe braucht, und keine weiteren [41]. Das zweite Prinzip dahinter ist die gestaffelte Verteidigung, *Defense in Depth*: Keine einzelne Schicht muss halten, weil hinter ihr eine weitere steht. Dein Foliensatz zeigt das am Container mit sechs Schichten.

Die linke Spalte der Tabelle ist nicht aus Nachlässigkeit so verbreitet. Jede Zeile dort ist die bequemere, billigere und in der Entwicklung schnellere Wahl. Eine Datenbank für alle ist einfacher zu betreiben als fünf. Ein flaches Netz braucht keine Regeln. Geteilte Zugangsdaten muss man nur einmal einrichten. Der kleine Wirkungsradius hat einen Preis in Betriebsaufwand, und ob ein Team ihn bezahlen kann, ist eine echte Abwägung. Dein Foliensatz zur Architekturdokumentation würde sagen: ein Tradeoff Point, der in ein Architecture Decision Record gehört.

> **BRÜCKE ZU SOFTWARE SECURITY**
> Für die Architekturanalyse in deiner Pflicht-Lehrveranstaltung ergibt sich aus diesem Kapitel eine Übung, die ganz ohne Pentest-Wissen auskommt: „Angenommen, Service X ist übernommen. Was erreicht der Angreifer von dort, und woran liegt das?“ Die Studierenden beantworten sie am Container- und am Deployment-Diagramm mit der Tabelle oben. In der Spezialisierung steht die Antwort des Testers im Bericht, und die Studierenden müssen entscheiden, welche Zeilen der Tabelle sie ändern, um den Radius zu verkleinern, und was das kostet.

## 11.8 Was davon im Bericht steht

In einem guten Bericht steht das Ergebnis dieser Phase an drei Stellen.

Im einzelnen Befund steht ein Abschnitt zur **Auswirkung**: was der Tester von dieser Stelle aus erreicht hat, mit Belegen. Dort entscheidet sich die Bewertung. Die Formulierung verdient Aufmerksamkeit. „Ein Angreifer könnte möglicherweise auf weitere Systeme zugreifen“ ist eine Vermutung. „Vom übernommenen Service aus wurde mit den dort gefundenen Zugangsdaten die Datenbank der Abrechnung gelesen“ ist ein Nachweis.

In einem eigenen Abschnitt steht, wenn es einen gab, der **Angriffspfad**: die Kette von der ersten Schwäche bis zum weitesten erreichten Punkt, als Erzählung in Schritten. Das ist oft der wertvollste Teil des Berichts, weil er zeigt, wie Befunde zusammenwirken, die einzeln unscheinbar aussehen. Kapitel 15 behandelt, wie solche Ketten zu bewerten sind.

Und es steht die **Grenze**: bis wohin der Tester gegangen ist und warum er dort aufgehört hat. Ohne diese Angabe lässt sich nicht unterscheiden, ob hinter dem letzten Schritt nichts mehr war oder ob dort der Auftrag endete.

FRAGE AUS DEM HÖRSAAL

***Wenn die Architektur so viel ausmacht, warum bewertet CVSS dann nur die einzelne Lücke?***

Weil CVSS in seiner Grundform eine Lücke unabhängig von der Umgebung beschreiben soll, damit ein Hersteller einen Wert veröffentlichen kann, der für alle Kunden gilt. Die Umgebung kommt über einen eigenen Teil der Bewertung hinein, den jeder Betreiber selbst ausfüllen muss, und das tut kaum jemand. Kapitel 14 zeigt, wie es geht. Der Pentest hat hier einen Vorteil gegenüber jeder Datenbank: Der Tester war in der Umgebung und hat gesehen, wie weit man kommt.

FRAGE AUS DEM HÖRSAAL

***Ist es nicht übertrieben, vom Einbruch auszugehen? Dann könnte man sich die Absicherung nach außen ja sparen.***

Beides gehört zusammen. Die Absicherung nach außen senkt die Wahrscheinlichkeit eines Einbruchs, die Begrenzung im Inneren senkt den Schaden. Wer nur das Erste tut, verliert bei einem einzigen Fehler alles. Wer nur das Zweite tut, hat ständig Einbrüche zu behandeln. Die Erfahrung der großen Vorfälle zeigt, dass der erste Zugang oft über etwas Banales gelang, eine vergessene Einstellung oder ein gestohlenes Passwort, und dass der Schaden von dem bestimmt wurde, was danach möglich war.

FRAGE AUS DEM HÖRSAAL

***Sind Container nicht von sich aus eine Isolationsschicht?***

Sie sind eine, aber keine so starke wie eine eigene virtuelle Maschine, und sie ist nur so gut wie ihre Konfiguration. Ein Container teilt sich den Kern des Betriebssystems mit dem Wirt. Ein privilegierter Container oder einer mit Zugriff auf die Container-Verwaltung ist praktisch keine Grenze. Für Lasten, bei denen fremder Code läuft oder der Schutzbedarf hoch ist, gibt es stärker isolierende Laufzeitumgebungen. Dein Foliensatz nennt sie am Ende des Abschnitts zur Container-Härtung.

FÜR DIE LEHRE

In der Spezialisierung ist dieses Kapitel die Grundlage für alle Behebungen auf der Ebene Architektur, die nicht die Schwäche selbst betreffen, sondern ihren Schaden. Bau in den Pentest-Bericht der Case Study einen Angriffspfad ein: ein Einstieg über eine Code-Schwäche, dann Zugangsdaten im Container, dann eine geteilte Datenbank. Die Studierenden sollen entscheiden, an welcher Stelle der Kette sie mit welchem Aufwand ansetzen. Die beste Antwort schließt den Einstieg und unterbricht die Kette an einer zweiten Stelle, weil der nächste Einstieg woanders sein wird.

Achte in der Beurteilung darauf, ob die Studierenden die Kosten nennen. Wer „alles segmentieren, Tresor einführen, Service Mesh aufbauen“ schreibt, ohne die Folgen für ein kleines Betriebsteam zu bedenken, hat die Stakeholder-Position des Betriebs aus Kapitel 18 nicht gelesen.

Typische Fehlvorstellungen: dass ein Befund ohne nachgewiesene Folgen harmlos sei, dass das interne Netz ein sicherer Bereich sei, und dass sich aus Passwort-Hashes die Passwörter nicht gewinnen ließen.

## Selbsttest

1. Warum kann dieselbe Schwäche in zwei Umgebungen ein sehr unterschiedliches Risiko bedeuten?
2. Was bedeutet „Assume Breach“ für den Entwurf eines Systems?
3. Nenne drei Gruppen von Wegen zur Rechteausweitung auf einem Rechner und die jeweilige Ebene der Ursache.
4. An welchen Orten findet ein Angreifer in einem übernommenen Service typischerweise Zugangsdaten?
5. Erkläre, was das Knacken von Passwort-Hashes ist und welche drei Eigenschaften seine Erfolgsaussicht bestimmen.

6. Welche zwei Eigenschaften der Architektur bestimmen, wie weit eine Seitwärtsbewegung reicht?
7. Im Bericht steht: „Die weitere Ausnutzung wurde gemäß Vereinbarung nicht durchgeführt.“ Wie liest du diesen Satz?
8. Nenne fünf Architekturentscheidungen, die den Wirkungsradius verkleinern, und zu einer davon den Preis.

# 12 KI-Systeme testen

Die Tabelle der Interpreter in Kapitel 8 endete mit einer Zeile, die aus der Reihe fiel: dem Sprachmodell, das Anweisungen in natürlicher Sprache versteht. Seit Anwendungen Sprachmodelle einbauen, als Assistenten, als Suchhilfe oder als Agenten, die selbst Funktionen aufrufen, gibt es eine neue Angriffsfläche mit alten Mustern. Die Pentesting-Lehrveranstaltung behandelt Angriffe auf KI-Systeme als eigenen Punkt. Du kennst das Gebiet aus deiner Forschung besser als die meisten. Dieses Kapitel ist deshalb kurz gehalten und hat einen engen Zweck: die Schwächen von Anwendungen mit Sprachmodellen so in das Raster dieses Buchs einzuordnen, dass du sie in der Spezialisierung neben den klassischen Befunden unterrichten kannst, mit denselben Fragen nach Vertrauensgrenze, Ursache und Ebene.

**IN DIESEM KAPITEL**

- den Aufbau einer Anwendung mit Sprachmodell beschreiben und ihre Vertrauensgrenzen einzeichnen
- die Risiken der OWASP-Liste für LLM-Anwendungen nennen
- direkte und indirekte Prompt Injection erklären und begründen, warum es dafür keine Parametrisierung gibt
- unsichere Ausgabebehandlung und übermäßige Handlungsmacht als bekannte Muster wiedererkennen
- sagen, was am Testen solcher Systeme anders ist und wie Befunde zu lesen sind

## 12.1 Zwei Rollen der KI

Zuerst eine Abgrenzung, die im Hörsaal regelmäßig durcheinandergeht. KI kommt in diesem Buch in zwei Rollen vor.

In der ersten ist sie **Werkzeug**: Tester verwenden KI-Assistenten bei Aufklärung, Analyse und Bericht, Entwickler beim Schreiben und Prüfen von Code, die Studierenden bei der Case Study. Dazu haben die Kapitel 2 und 3 das Nötige gesagt, und die Frage ist immer dieselbe: Wer prüft das Ergebnis und steht dafür ein?

In der zweiten ist sie **Ziel**: Die getestete Anwendung enthält selbst ein Sprachmodell. Davon handelt dieses Kapitel.

Die beiden Rollen berühren sich an einer Stelle. Dein Foliensatz zu den Grundlagen zitiert Studien, nach denen Entwickler mit KI-Assistenten mehr unsicheren Code schrieben und sich dabei sicherer fühlten, und nach denen ein erheblicher Teil der vorgeschlagenen Lösungen in sicherheitsrelevanten Szenarien verwundbar war [64], [65]. Code, den ein Sprachmodell geschrieben hat, wird im Pentest wie jeder andere Code geprüft. Die Schwächen darin sind die aus den Kapiteln 8 bis 10.

## 12.2 Aufbau einer Anwendung mit Sprachmodell

Als Beispiel bekommt die Plattform *Kurbad* einen Assistenten. Kunden können ihm Fragen stellen, er kennt die Hausordnung und die Preisliste, er kann die Buchungen des Kunden nachschlagen und auf Wunsch eine Buchung stornieren.

Technisch besteht so ein Assistent aus mehr als dem Modell. Das Modell bekommt bei jeder Anfrage einen einzigen langen Text, den *Prompt*, den die Anwendung aus mehreren Teilen zusammensetzt.

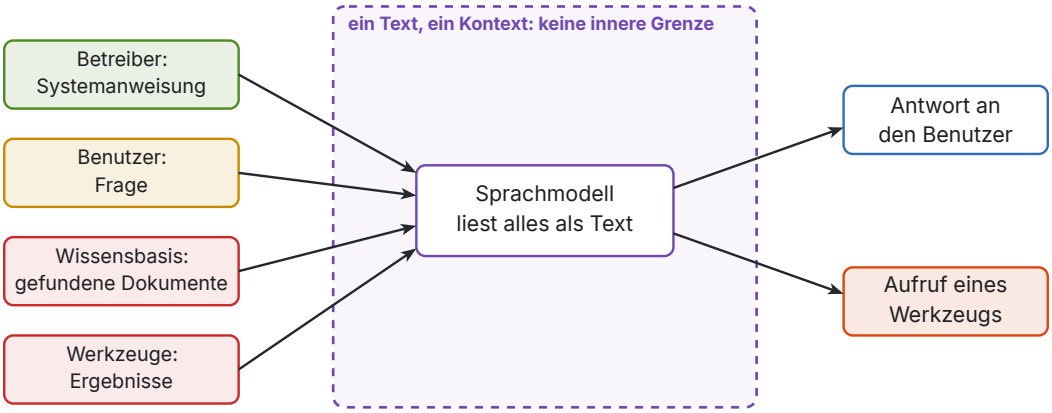



**Abb. 22:** Die Eingänge eines Sprachmodells in einer Anwendung. Grün stammt vom Betreiber, gelb vom Benutzer, rot aus Quellen, die Dritte beeinflussen können.

Die Teile in Abb. 22 haben sehr verschiedene Vertrauenswürdigkeit. Die **Systemanweisung** schreibt der Betreiber: wie sich der Assistent verhalten soll und was er nicht tun darf. Die **Frage** kommt vom Benutzer. Die **gefundenen Dokumente** kommen aus einer Wissensbasis, die bei jeder Frage nach passenden Textstellen durchsucht wird. Das Verfahren heißt *Retrieval-Augmented Generation*, kurz RAG. Die **Ergebnisse von Werkzeugen** sind das, was zurückkommt, wenn das Modell eine Funktion aufgerufen hat, etwa den Inhalt einer Buchung oder einer abgerufenen Webseite.

Das Modell gibt Text aus. Die Anwendung zeigt ihn dem Benutzer an oder liest daraus, dass das Modell ein **Werkzeug** aufrufen will, und führt den Aufruf aus. Ein System, das in dieser Schleife selbstständig mehrere Schritte geht, heißt *Agent*.

Die entscheidende Beobachtung steht in der Abbildung als Beschriftung. Für das Modell gibt es keinen technischen Unterschied zwischen den vier Eingängen. Alles ist Text in einem gemeinsamen Kontext. Welcher Teil eine Anweisung ist, der es folgen soll, und welcher bloß Inhalt, über den es Auskunft geben soll, entscheidet das Modell aus dem Text selbst, mit der Unschärfe, mit der es alles entscheidet. Das ist die Vertrauensgrenze zwischen Daten und Anweisungen aus Kapitel 6, und hier fehlt sie von der Bauart her.

## 12.3 Die OWASP-Liste für LLM-Anwendungen

OWASP führt für Anwendungen mit Sprachmodellen eine eigene Liste der wichtigsten Risiken. Die Fassung 2025 wurde im Sommer 2026 von einer neuen abgelöst [66], [67].

**Tab. 38:** Die OWASP Top 10 für LLM-Anwendungen in der Fassung 2026 [67]. Gegenüber 2025 ist vor allem die übermäßige Handlungsmacht aufgestiegen.

| Nr. | Risiko (Fassung 2026) | Worum es geht |
|---|---|---|
| LLM01 | Prompt Injection | Eingaben bringen das Modell dazu, anderen Anweisungen zu folgen als denen des Betreibers |
| LLM02 | Sensitive Information Disclosure | das Modell gibt vertrauliche Daten preis |
| LLM03 | Excessive Agency | das Modell kann mehr tun, als für seine Aufgabe nötig und vertretbar ist |
| LLM04 | Supply Chain | Modelle, Daten, Erweiterungen aus fremder Hand |
| LLM05 | Data and Model Poisoning | manipulierte Trainings- oder Wissensdaten verändern das Verhalten |
| LLM06 | Unbounded Consumption | unbegrenzter Verbrauch von Rechenleistung und Geld |
| LLM07 | Misinformation | falsche Ausgaben, denen vertraut wird |
| LLM08 | Hidden Context Exposure | Systemanweisung und anderer verborgener Kontext lassen sich auslesen |
| LLM09 | Vector and Embedding Weaknesses | Schwächen in der Wissensbasis und ihrer Suche |
| LLM10 | Improper Output Handling | Ausgaben des Modells werden ungeprüft weiterverarbeitet |

Für das Vorgehen von Angreifern gegen KI-Systeme gibt es mit ATLAS eine Wissensbasis nach dem Muster von ATT&CK [38], und ATT&CK selbst führt seit Version 20 Techniken wie direkte und indirekte Prompt Injection [37].

Die Liste sieht aus wie zehn neue Themen. Bei näherem Hinsehen sind die meisten alte Bekannte, und so behandeln die folgenden Abschnitte sie auch.

## 12.4 Prompt Injection

**DEFINITION 12.1 · PROMPT INJECTION**
Bei der Prompt Injection gelangt Text in den Kontext eines Sprachmodells, den das Modell als Anweisung befolgt, obwohl er aus einer Quelle stammt, die keine Anweisungen geben dürfte. Bei der **direkten** Form schreibt der Benutzer selbst solche Anweisungen in seine Eingabe. Bei der **indirekten** Form stehen sie in Inhalten, die das Modell im Zuge seiner Arbeit liest: in einem Dokument der Wissensbasis, einer abgerufenen Webseite, einer E-Mail, dem Ergebnis eines Werkzeugs [68].

Die direkte Form ist die bekanntere und die harmlosere. Ein Benutzer, der den Assistenten überredet, seine Vorgaben zu ignorieren, schadet zunächst nur der eigenen Sitzung. Ernst wird es, wenn der Assistent dabei etwas preisgibt oder tut, was dieser Benutzer nicht dürfte.

Die indirekte Form ist die gefährliche, weil Angreifer und Opfer verschiedene Personen sind. Angenommen, der Assistent von *Kurbad* bezieht bei Fragen zu einer Therme auch die Bewertungen anderer Kunden ein. Jemand schreibt eine Bewertung, die neben dem sichtbaren Text Anweisungen an den Assistenten enthält. Die nächste Kundin, die nach dieser Therme fragt, bekommt eine Antwort, die von diesen Anweisungen geformt ist: mit einem falschen Hinweis, einem fremden Verweis oder, wenn der Assistent Werkzeuge hat, mit einer Handlung in ihrem Namen. Sie hat nichts Auffälliges getan.

Das ist dem gespeicherten Cross-Site Scripting aus Kapitel 8 Zug um Zug gleich: Fremde Daten werden abgelegt und später im Kontext eines anderen Benutzers als Anweisungen ausgeführt. Der Unterschied liegt in der Behebung. Für die SQL-Injection gibt es die Parametrisierung, die Anweisung und Daten strukturell trennt. Für XSS gibt es die Kodierung für einen genau bekannten Kontext. Für Sprachmodelle gibt es nichts Vergleichbares. Es gibt keinen Kanal, auf dem man einem Modell Daten übergeben könnte, die es garantiert nicht als Anweisung liest. Jede Abwehr, die im Prompt selbst ansetzt, etwa Markierungen um die fremden Inhalte oder die Mahnung, darin keine Anweisungen zu befolgen, senkt die Erfolgsquote und hebt sie nicht auf.

**MERKE**
Prompt Injection ist nach heutigem Stand nicht behebbar, sondern nur eindämmbar. Die Sicherheit einer Anwendung mit Sprachmodell darf deshalb nicht davon abhängen, dass das Modell sich an seine Anweisungen hält.

Daraus folgt, wo die Maßnahmen liegen müssen. Sie liegen nicht im Modell, sondern um es herum, und sie sind dieselben wie in Kapitel 11: annehmen, dass der Einbruch gelingt, und den Wirkungsradius klein halten. Die nächsten beiden Abschnitte zeigen die zwei Stellen, an denen das geschieht.

## 12.5 Die Ausgabe ist eine nicht vertrauenswürdige Eingabe

Wenn der Kontext eines Modells von außen beeinflusst werden kann, dann auch seine Ausgabe. Die Ausgabe eines Sprachmodells ist für den Rest der Anwendung deshalb, was jede Benutzereingabe ist: nicht vertrauenswürdig.

Das Risiko *Improper Output Handling* beschreibt, was passiert, wenn das übersehen wird. Die Ausgabe des Modells wird in eine Webseite eingesetzt, und der Browser führt darin enthaltene Skripte aus. Sie wird Teil einer Datenbankabfrage, eines Befehls für die Kommandozeile, eines Dateipfads, einer Adresse, die der Server abruft. Jede Senke aus Tab. 27 ist auch hier eine Senke, mit dem Modell als Quelle.

Die Behebung ist die aus Kapitel 8, unverändert: Kodierung passend zum Kontext, Parametrisierung, Erlaubnislisten. Neu ist allein, dass Entwickler die Ausgabe eines Bausteins, den sie selbst eingebaut

haben, für vertrauenswürdig halten. Es ist derselbe Irrtum wie bei Daten aus der eigenen Datenbank und bei Nachrichten vom eigenen Broker.

Die Fassung 2026 der Liste nennt ausdrücklich Senken, an die früher wenige dachten: Terminals, die Steuerzeichen in der Ausgabe auswerten, und Oberflächen, die Verweise und Bilder aus der Ausgabe automatisch laden. Das zweite ist ein Weg, Daten hinauszubringen. Bringt eine eingeschleuste Anweisung das Modell dazu, ein Bild von einer fremden Adresse einzubinden, in deren Parametern der Gesprächsinhalt steht, verlässt dieser Inhalt beim Laden des Bildes das System, ohne dass die Benutzerin etwas anklickt.

## 12.6 Übermäßige Handlungsmacht

Die zweite Stelle sind die Werkzeuge. Ein Assistent, der nur redet, kann Falsches sagen. Ein Assistent, der handeln kann, kann Falsches tun.

> **DEFINITION 12.2 · EXCESSIVE AGENCY**
> Übermäßige Handlungsmacht liegt vor, wenn ein System mit Sprachmodell mehr Funktionen, weitere Berechtigungen oder mehr Selbstständigkeit hat, als seine Aufgabe erfordert, sodass eine Fehlentscheidung des Modells Schaden anrichten kann, sei sie durch einen Angriff ausgelöst oder durch einen Irrtum.

Die drei Ausprägungen sind in der Definition genannt. Zu viele **Funktionen**: Der Assistent hat ein Werkzeug zum Lesen und Schreiben von Buchungen, obwohl er nur lesen müsste. Zu weite **Berechtigungen**: Das Werkzeug arbeitet mit einem Dienstkonto, das alle Buchungen aller Kunden sieht. Zu viel **Selbstständigkeit**: Der Assistent storniert, ohne dass ein Mensch die Stornierung bestätigt.

Die zweite Ausprägung ist die häufigste, und sie ist der verwirrte Stellvertreter aus Kapitel 9. Der Assistent handelt im Auftrag einer Kundin mit den Rechten eines mächtigen Dienstkontos. Fragt die Kundin nach der Buchung mit einer fremden Nummer, oder bringt eine eingeschleuste Anweisung den Assistenten dazu, ruft das Werkzeug sie ab, weil das Dienstkonto es darf. Die Zugriffskontrolle wurde dem Modell überlassen, also der Systemanweisung, in der steht, es möge nur die eigenen Buchungen des Kunden zeigen. Das ist eine Berechtigungsprüfung in natürlicher Sprache, und sie hält so gut wie eine Prüfung im Browser.

Die Behebung liegt in der Architektur und ist mit dem Vokabular der vorigen Kapitel vollständig zu beschreiben.

- Jedes Werkzeug läuft mit der Identität und den Rechten des Benutzers, in dessen Auftrag der Assistent arbeitet, nicht mit einem eigenen mächtigen Konto.
- Die Autorisierung wird im Werkzeug erzwungen, also in gewöhnlichem Code, und nicht im Prompt.
- Der Assistent bekommt nur die Werkzeuge, die er braucht, und jedes nur mit den nötigen Operationen.
- Handlungen, die sich nicht rückgängig machen lassen oder Geld bewegen, brauchen die Bestätigung eines Menschen außerhalb des Gesprächs.
- Inhalte aus fremden Quellen und mächtige Werkzeuge kommen nicht im selben Kontext zusammen.

Der letzte Punkt ist der wirksamste und der unbequemste. Ein Assistent ist dann gefährdet, wenn drei Dinge zusammentreffen: Er liest Inhalte, die Dritte beeinflussen können, er hat Zugriff auf vertrauliche Daten, und er kann nach außen wirken. Nimmt man eines der drei weg, bricht die Kette. Das ist dieselbe Überlegung wie beim Wirkungsradius in Kapitel 11.

## 12.7 Preisgabe, Vergiftung, Verbrauch

Die übrigen Einträge der Liste lassen sich knapper einordnen.

**Preisgabe** hat mehrere Quellen. Die Systemanweisung und anderer verborgener Kontext lassen sich dem Modell mit Geduld entlocken. Daraus folgt eine einfache Regel: In der Systemanweisung steht nichts, was nicht bekannt werden dürfte, also keine Zugangsdaten und keine Geschäftsgeheimnisse. Schwerer wiegt die Preisgabe über die Wissensbasis. Durchsucht der Assistent bei jeder Frage alle Dokumente, findet er auch solche, die der fragende Benutzer nicht sehen dürfte, und fasst sie ihm zusammen. Die Suche in der Wissensbasis muss die Berechtigungen des Benutzers beachten. Das ist ein Fehler der Zugriffskontrolle auf Objektebene, an einer Stelle, an der ihn niemand vermutet.

**Vergiftung** betrifft die Daten, aus denen das Verhalten entsteht. Wer Inhalte in die Wissensbasis bringen kann, etwa über Bewertungen, hochgeladene Dokumente oder eine öffentlich bearbeitbare Quelle, beeinflusst die Antworten für alle. Bei Modellen, die mit eigenen Daten nachtrainiert werden, gilt dasselbe für die Trainingsdaten. In STRIDE ist das eine Verfälschung des Datenspeichers.

Die **Lieferkette** umfasst hier Modelle, Datensätze, Bibliotheken und Erweiterungen, über die ein Assistent an Werkzeuge kommt. Für alles gilt Kapitel 10, mit dem Zusatz, dass sich einem Modell nicht ansehen lässt, was es gelernt hat. NIST hat das SSDF für diesen Bereich um eigene Praktiken ergänzt [45].

**Unbegrenzter Verbrauch** ist die Ressourcenschwäche aus Kapitel 10 mit einer direkten Rechnung: Jede Anfrage an ein Modell kostet Geld, und ein Agent in einer Schleife kann sehr viele stellen.

**Fehlinformation** ist kein Angriff, sondern eine Eigenschaft: Modelle geben Falsches in überzeugender Form aus. Zur Schwachstelle wird das, wenn die Anwendung oder ihre Benutzer sich darauf verlassen, ohne zu prüfen. Für die Lehre ist das die Stelle, an der das Thema dieses Kapitels auf die Studierenden selbst zurückfällt.

## 12.8 Was am Testen anders ist

Der Test einer Anwendung mit Sprachmodell folgt denselben Phasen wie jeder andere. Drei Dinge sind anders, und sie verändern, wie Befunde zu lesen sind.

**Das Verhalten ist nicht deterministisch.** Dieselbe Eingabe führt nicht jedes Mal zum selben Ergebnis. Ein Angriff, der bei einem von zwanzig Versuchen gelingt, ist ein gültiger Befund, denn ein Angreifer kann beliebig oft versuchen. Ein guter Bericht nennt deshalb eine Erfolgsquote und die Zahl der Versuche statt eines einzelnen Belegs. Umgekehrt beweist ein Nachtest, bei dem der Angriff dreimal nicht gelingt, nicht, dass er behoben ist.

**Die Behebung im Modell ist keine.** Wird ein Befund zur Prompt Injection dadurch „behoben“, dass die Systemanweisung um ein Verbot ergänzt oder ein Filter für bestimmte Formulierungen vorgeschaltet wird, ist das eine Abmilderung im Sinn von Kapitel 17. Sie hält, bis jemand eine andere Formulierung findet. Eine Behebung liegt erst vor, wenn die Folge des Angriffs durch die Architektur verhindert ist.

**Die Bewertung passt schlecht.** CVSS ist für Schwachstellen in Software gebaut, deren Ausnutzung entweder gelingt oder nicht. Für einen Angriff mit einer Erfolgsquote und einer Auswirkung, die davon abhängt, welche Werkzeuge gerade angebunden sind, liefert der Vektor nur eine grobe Näherung. Auch die CWE hat für vieles noch keine genauen Einträge. Die ehrlichste Bewertung beschreibt das Szenario: Wer kann was einschleusen, was kann der Assistent dann tun, und was begrenzt es.

In diesem Feld hat sich für das gezielte Suchen nach Fehlverhalten der Begriff *Red Teaming* eingebürgert, in einer weiteren Bedeutung als in Kapitel 1. Gemeint ist dort oft auch die Suche nach unerwünschten Inhalten, nicht nur nach Sicherheitslücken. Wenn in einem Angebot „KI Red Teaming“ steht, lohnt die Nachfrage, was genau geprüft wird.

**Tab. 39:** Behebungen für Anwendungen mit Sprachmodell auf drei Ebenen.

| Risiko | Code | Architektur | Lifecycle |
|---|---|---|---|
| Prompt Injection | fremde Inhalte kennzeichnen, Eingaben und Ausgaben prüfen (nur Abmilderung) | fremde Inhalte, vertrauliche Daten und Wirkung nach außen trennen | wiederholte Tests mit Erfolgsquote, Überwachung der Gespräche |
| Improper Output Handling | Ausgabe wie Benutzereingabe behandeln: kodieren, parametrisieren | kein automatisches Nachladen fremder Inhalte in der Oberfläche | Prüfregel: Modellausgabe ist eine Quelle |
| Excessive Agency | Autorisierung im Werkzeug prüfen | Werkzeuge mit den Rechten des Benutzers, Bestätigung für kritische Handlungen, minimale Werkzeuge | Bedrohungsmodell für jedes neue Werkzeug |
| Preisgabe über die Wissensbasis | Berechtigung bei der Suche filtern | Wissensbasen nach Schutzbedarf trennen | Regeln, was in die Wissensbasis darf |
| Hidden Context Exposure | keine Geheimnisse in der Systemanweisung | Regeln im Code statt im Prompt durchsetzen | Prüfung der Systemanweisungen im Review |
| Vergiftung | Herkunft von Inhalten festhalten | getrennte Wissensbasen für geprüfte und fremde Inhalte | Freigabeprozess für Inhalte und Modelle |
| Unbounded Consumption | Obergrenzen für Länge, Schritte und Aufrufe | Kontingente je Benutzer, Abbruch von Schleifen | Überwachung der Kosten |

Die Tabelle zeigt, worauf das Kapitel hinauswollte. Was an Anwendungen mit Sprachmodellen neu ist, steht in der ersten Zeile, und dort steht in der Spalte Code nur eine Abmilderung. Alles andere sind die Muster der Kapitel 8 bis 11 an einem neuen Ort: die Senke, die Zugriffskontrolle, der verwirrte Stellvertreter, die minimale Berechtigung, der Wirkungsradius.

**BRÜCKE ZU SOFTWARE SECURITY**

Der Studienplan führt im ersten Semester AI for Security und im dritten Secure and Trustworthy AI. Für die Spezialisierung im zweiten Semester reicht es, wenn die Studierenden einen Befund zu einem Assistenten mit denselben Fragen behandeln wie jeden anderen: Wo ist die Vertrauensgrenze, auf welcher Ebene liegt die Ursache, was ist Behebung und was Abmilderung? Ein Befund dieser Art in der Case Study ist ein guter Prüfstein, weil KI-Assistenten als Behebung fast immer eine strengere Systemanweisung vorschlagen.

**FRAGE AUS DEM HÖRSAAL**

***Kann man Prompt Injection nicht mit einem zweiten Modell erkennen, das die Eingaben prüft?***

Man kann die Erfolgsquote damit senken, und das wird auch gemacht. Das prüfende Modell ist aber selbst ein Sprachmodell und selbst täuschbar, und es entscheidet mit einer Fehlerquote in beide Richtungen. Eine Sicherheitsfunktion, die in einem Teil der Fälle versagt, taugt als zusätzliche Schicht und nicht als Grenze. Bei der SQL-Injection wäre das, als würde man die Abfragen weiter als Text zusammensetzen und einen Filter davorschalten. Die verlässliche Antwort liegt in der Begrenzung dessen, was nach einer gelungenen Injection möglich ist.

**FRAGE AUS DEM HÖRSAAL**

***Warum ist die Systemanweisung keine Zugriffskontrolle? Dort steht doch, was der Assistent nicht darf.***

Weil sie eine Bitte an ein System ist, das Bitten verschiedener Absender nicht verlässlich auseinanderhält. Eine Zugriffskontrolle muss in jedem Fall gelten, auch gegen einen Benutzer, der es darauf anlegt. Die Systemanweisung gilt in den meisten Fällen. Der Unterschied zwischen „immer“ und „meistens“ ist der Unterschied zwischen einer Sicherheitsfunktion und einer Voreinstellung. Was der Assistent nicht dürfen soll, darf das Werkzeug nicht können.

**FRAGE AUS DEM HÖRSAAL**

***Ist das nicht alles nur ein Problem von Chatbots?***

Chatbots sind der harmloseste Fall, weil sie nur reden. Das Problem wächst mit der Handlungsmacht. Ein Agent, der E-Mails liest, im Web sucht, Code ausführt und im Namen eines Benutzers Systeme bedient, vereint genau die drei Eigenschaften, die zusammen gefährlich sind: fremde Inhalte, vertrauliche Daten und Wirkung nach außen. Deshalb ist die übermäßige Handlungsmacht in der Liste von 2026 so weit aufgestiegen.

**FÜR DIE LEHRE**

Du kannst dieses Kapitel in der Spezialisierung als Transferaufgabe verwenden. Gib den Studierenden die Beschreibung eines Assistenten mit zwei Werkzeugen und lass sie ohne weitere Vorgabe ein Datenflussdiagramm mit Vertrauensgrenzen zeichnen und STRIDE darauf anwenden. Wer die Kapitel 6 bis 11 verstanden hat, findet die wesentlichen Bedrohungen ohne ein einziges KI-spezifisches Stichwort: die fehlende Grenze im Kontext, die ungeprüfte Ausgabe, das mächtige Dienstkonto.

Eine zweite, sehr wirksame Übung ist der Vergleich von Behebungen. Lege einen Befund zur indirekten Prompt Injection mit Datenabfluss vor und drei Vorschläge: eine verschärfte Systemanweisung, einen Filter für die Eingabe und eine Architekturänderung, bei der die Oberfläche keine fremden Bilder nachlädt und das Werkzeug mit den Rechten des Benutzers läuft. Die Studierenden sollen einordnen, was Behebung und was Abmilderung ist.

Typische Fehlvorstellungen: dass ein besseres Modell das Problem löse, dass die Systemanweisung geheim bleibe, und dass ein Nachtest ohne Erfolg die Behebung belege.

## Selbsttest

1. Welche vier Arten von Eingängen hat das Sprachmodell in einer typischen Anwendung, und wie vertrauenswürdig ist jede?
2. Erkläre den Unterschied zwischen direkter und indirekter Prompt Injection an einem Beispiel.
3. Warum gibt es für Prompt Injection kein Gegenstück zur parametrisierten Abfrage?
4. Inwiefern ist die Ausgabe eines Sprachmodells eine Quelle im Sinn von Kapitel 8?
5. Beschreibe die drei Ausprägungen übermäßiger Handlungsmacht.
6. Der Assistent zeigt einer Kundin eine fremde Buchung, weil sie nach deren Nummer gefragt hat. Welches Muster aus Kapitel 9 liegt vor, und wo gehört die Behebung hin?
7. Ein Bericht belegt einen Angriff mit „in 3 von 40 Versuchen erfolgreich“. Ist das ein gültiger Befund? Begründe.
8. Ein Befund zur Prompt Injection wird durch einen zusätzlichen Satz in der Systemanweisung geschlossen. Beurteile das.

# 13 Der Pentest-Bericht

Zwei Tester haben acht Tage an der Plattform *Kurbad* gearbeitet. Sie haben Hunderte Anfragen geschickt, Notizen gemacht, Bildschirmfotos gesammelt, Vermutungen verworfen und einige bestätigt. Von alldem bleibt beim Auftraggeber ein einziges Dokument. Was nicht im Bericht steht, ist für die Organisation nicht geschehen, und was unverständlich darin steht, wird nicht behoben.

Die Teile I und II haben dir das Vokabular und die Mechanismen gegeben. Mit diesem Kapitel wechselt die Blickrichtung: Du bist jetzt der Empfänger. In der Spezialisierung bekommen die Studierenden einen Bericht mit rund 25 Befunden in die Hand, und alles, was sie danach tun, hängt davon ab, wie gut sie ihn lesen. Dieses Kapitel zeigt, wie ein Bericht aufgebaut ist, was in jedem Teil stehen muss, woran man einen guten von einem schwachen unterscheidet und in welcher Reihenfolge man ihn liest. Die Bewertung der Schwere bekommt ein eigenes Kapitel, das nächste.

**IN DIESEM KAPITEL**

- die Teile eines Pentest-Berichts nennen und sagen, für wen jeder geschrieben ist
- die Felder eines einzelnen Befunds erklären und beurteilen, ob einer vollständig ist
- aus Scope, Einschränkungen und Abdeckung ableiten, was ein Bericht aussagt und was nicht
- einen Bericht nach Qualitätskriterien beurteilen und Nachfragen an die Tester formulieren
- einen Bericht in einer sinnvollen Reihenfolge lesen und für die Triage vorbereiten

## 13.1 Drei Leser, ein Dokument

Ein Bericht hat mindestens drei Leser mit verschiedenen Fragen, und er muss allen dreien dienen.

Die **Leitung** will wissen, wie es steht, was auf dem Spiel steht und was es kostet, das zu ändern. Sie liest zwei Seiten und entscheidet über Geld und Prioritäten.

Die **Technik**, also Entwicklung, Architektur und Betrieb, will wissen, was genau wo falsch ist, wie man es nachvollzieht und was zu tun ist. Sie liest die Befunde und überfliegt den Rest.

Die **Prüfung**, also Informationssicherheit, Revision, manchmal ein Kunde oder eine Aufsichtsbehörde, will wissen, was getestet wurde, nach welcher Methodik, von wem und mit welchem Ergebnis. Sie liest Scope, Methodik und Abdeckung.

Ein Bericht, der nur einem dieser Leser dient, hat seinen Zweck verfehlt. Eine Liste technischer Befunde ohne Zusammenfassung erreicht die Leitung nicht, und dann gibt es kein Geld für die Behebung. Eine schöne Zusammenfassung mit dünnen Befunden lässt die Technik raten. Und ein Bericht ohne Angaben zur Abdeckung lässt sich gegenüber Dritten nicht verwenden.

In der Spezialisierung nehmen die Studierenden die zweite Rolle ein und müssen für die erste und dritte schreiben. Das ist die Lage jedes Architekten nach einem Pentest.

## 13.2 Der Aufbau

Berichte verschiedener Anbieter sehen verschieden aus, enthalten aber dieselben Teile. PTES und die Praxisleitfäden beschreiben sie ähnlich [13], [17].

**Tab. 40:** Die Teile eines Pentest-Berichts.

| Teil | Inhalt | Hauptleser |
|---|---|---|
| Zusammenfassung für die Leitung | Gesamturteil in Alltagssprache, die wichtigsten Risiken, die wichtigsten Maßnahmen, Zahl der Befunde je Schweregrad | Leitung |
| Auftrag und Scope | Ziele, Umgebung, Zeitraum, Vorwissen, Konten, Ausschlüsse, beteiligte Personen | Prüfung |
| Methodik | verwendete Methodik und Kataloge, Bewertungsverfahren mit Fassung | Prüfung |
| Einschränkungen | was nicht oder nur teilweise geprüft werden konnte und warum | alle |
| Befundübersicht | Tabelle aller Befunde mit Kennung, Titel und Schwere | alle |
| Angriffspfade | Ketten aus mehreren Befunden als Erzählung in Schritten | Leitung, Technik |
| Befunde im Einzelnen | je Befund: Ort, Beschreibung, Nachweis, Auswirkung, Einordnung, Empfehlung | Technik |
| Positive Feststellungen | was geprüft wurde und standgehalten hat | alle |
| Abdeckung | geprüfte Bereiche und Testkategorien mit Ergebnis | Prüfung |
| Anhang | Bestandsaufnahme, Werkzeuge, Rohdaten, Testkonten, Aufräumhinweise | Technik |

Die Reihenfolge in Tab. 40 ist die, in der Berichte meist gedruckt sind. Sie ist nicht die, in der du lesen solltest. Dazu kommt der Abschnitt über das Lesen weiter unten.

## 13.3 Die Zusammenfassung für die Leitung

Die Zusammenfassung ist der schwierigste Teil, weil sie in einer Sprache geschrieben sein muss, die ohne Fachbegriffe auskommt, und trotzdem nichts Falsches sagen darf.

Eine gute Zusammenfassung beantwortet vier Fragen. Was wurde geprüft, in einem Satz? Wie ist das Gesamtbild? Was sind die zwei oder drei Dinge, die jemand von außen tatsächlich anrichten könnte? Und was sind die zwei oder drei Maßnahmen, die am meisten bewirken?

So könnte der Kern für *Kurbad* lauten:

> Geprüft wurden die Buchungsplattform mit Web-Oberfläche, App, Schnittstelle und Verwaltungsoberfläche in der Testumgebung, an acht Tagen mit Kenntnis der Schnittstellenbeschreibung und mit Testkonten.
>
> Jeder angemeldete Kunde konnte die Buchungen und Rechnungen aller anderen Kunden lesen, auf zwei voneinander unabhängigen Wegen. Über einen der beiden Wege ließ sich die Kontrolle über einen Service übernehmen und von dort aus auf die Daten der Abrechnung zugreifen, weil die Services einander im internen Netz ohne Prüfung vertrauen.
>
> Die Befunde gehen auf wenige Ursachen zurück: Berechtigungen werden an einzelnen Stellen im Code geprüft statt an einer zentralen, Eingaben werden an mehreren Stellen ungeprüft weiterverarbeitet, und das interne Netz gilt als vertrauenswürdig. Wir empfehlen, diese drei Ursachen zu beheben und nicht nur die 16 einzelnen Stellen.

Der Text enthält keine Kürzel und keine Zahl aus einem Bewertungssystem. Er sagt, was jemand tun konnte, und nennt Ursachen statt Symptome. Der letzte Absatz ist selten. Viele Berichte zählen in der Zusammenfassung nur auf, wie viele Befunde welcher Stufe es gab. Diese Zählung gehört dazu, sie ist aber keine Zusammenfassung.

> **FALLSTRICK**
> Die Tortengrafik mit der Verteilung der Schweregrade wird gern zitiert und ist das am wenigsten aussagekräftige Element eines Berichts. „Ein kritischer, sechs hohe, sieben mittlere“ sagt nichts darüber, ob die sechs hohen sechs Probleme sind oder eines an sechs Stellen, und nichts darüber, wie viel Arbeit die Behebung ist. Wer nur die Grafik an die Leitung weitergibt, hat den Bericht nicht gelesen.

## 13.4 Auftrag, Scope und Einschränkungen

Dieser Teil wiederholt, was in Kapitel 2 vereinbart wurde, und ergänzt, was tatsächlich geschah. Der zweite Teil ist der wichtige. Zwischen Plan und Durchführung liegen fast immer Abweichungen.

Typische Einträge unter *Einschränkungen*:

- Die Konten für die Rolle Kassenkraft standen erst am dritten Tag zur Verfügung.
- Die Testumgebung war an einem halben Tag nicht erreichbar.
- Die mobile App konnte nur auf einer der beiden Plattformen geprüft werden.
- Die Anbindung an den Zahlungsdienstleister war in der Testumgebung durch eine Attrappe ersetzt.
- Auf Wunsch des Auftraggebers wurde die Funktion zur Stornierung nicht mit echten Rückzahlungen geprüft.

Jeder dieser Sätze schränkt eine Aussage des Berichts ein, und jeder ist ein Zeichen für einen ehrlichen Bericht. Ein Bericht ohne Einschränkungen ist verdächtig, denn es gibt keinen Test ohne sie.

Der vierte Eintrag verdient einen zweiten Blick. Die Rückmeldungen des Zahlungsdienstleisters sind eine der Stellen, an denen Daten von außen in das System kommen. Kapitel 6 hat sie zur Angriffsfläche gezählt. Wenn dort in der Testumgebung eine Attrappe steht, ist diese Vertrauensgrenze ungeprüft, und kein Befund dazu heißt nichts.

## 13.5 Die Abdeckung

> **DEFINITION 13.1 · ABDECKUNGSMATRIX**
> Eine Abdeckungsmatrix stellt die geprüften Bereiche eines Systems den Testkategorien einer Methodik gegenüber und hält für jede Kombination fest, ob sie geprüft wurde und mit welchem Ergebnis.

Kapitel 3 hat angekündigt, dass erst die Abdeckung aus „kein Befund“ eine Aussage macht. So sieht sie für *Kurbad* aus, verkürzt auf die Kategorien des WSTG [14].

**Tab. 41:** Abdeckungsmatrix für *Kurbad*. **g** geprüft ohne Befund, **B** geprüft mit Befund, **t** teilweise geprüft, - nicht geprüft oder nicht anwendbar.

| Kategorie | Web | App | API | Admin | Services |
|---|---|---|---|---|---|
| Konfiguration und Bereitstellung | g | t | B | B | B |
| Identitätsverwaltung | g | g | g | t | - |
| Authentifizierung | B | g | B | B | B |
| Autorisierung | B | t | B | t | B |
| Sitzungsverwaltung | g | g | B | t | - |
| Eingabevalidierung | B | t | B | t | t |
| Fehlerbehandlung | g | g | B | g | t |
| Kryptografie | g | t | g | g | t |
| Fachlogik | B | t | B | - | - |
| Clientseite | B | t | - | t | - |

Lies Tab. 41 von den gelben und grauen Feldern her. Die Spalte *Admin* ist fast durchgehend nur teilweise geprüft, die Fachlogik dort gar nicht. Die mobile App wurde in den meisten Kategorien nur teilweise geprüft. Daraus folgt, was der Bericht über diese Teile aussagt: wenig. Wer aus den wenigen Befunden zur Admin-Oberfläche schließt, sie sei der sicherste Teil des Systems, liest die Matrix falsch.

Viele Berichte enthalten keine solche Matrix. Dann lohnt die Frage an die Tester, welche Bereiche wie viel Zeit bekommen haben. Die Antwort ist oft eine einfache Tabelle mit Tagen je Bereich, und sie reicht für denselben Zweck.

## 13.6 Die Befundübersicht

Die Übersicht listet alle Befunde in einer Tabelle. Für *Kurbad* sieht sie so aus. Die Befunde F-03, F-07 und F-12 kennst du aus den Kapiteln 8 bis 10. Dieser Bericht begleitet dich durch die Teile III und IV.

**Tab. 42:** Befundübersicht des Berichts zu *Kurbad*. Die CVSS-Werte sind Basiswerte nach Version 3.1.

| Nr. | Titel | Schwere | CVSS | CWE |
|---|---|---|---|---|
| F-01 | Admin-Oberfläche aus dem Internet erreichbar, Anmeldung nur mit Passwort | Hoch | 7,4 | CWE-308 |
| F-02 | Keine Begrenzung der Anmeldeversuche | Mittel | 4,8 | CWE-307 |
| F-03 | SQL-Injection in der Buchungssuche | Kritisch | 9,9 | CWE-89 |
| F-04 | Gespeichertes Cross-Site Scripting im Bemerkungsfeld einer Buchung | Mittel | 5,4 | CWE-79 |
| F-05 | Server-Side Request Forgery beim Abruf eines Profilbilds über eine Adresse | Hoch | 7,7 | CWE-918 |
| F-06 | Tokens bleiben nach der Abmeldung 30 Tage gültig | Mittel | 5,9 | CWE-613 |
| F-07 | Abruf fremder Rechnungen über fortlaufende Kennung | Hoch | 6,5 | CWE-639 |
| F-08 | Bonuspunkte über das Profil veränderbar | Mittel | 6,5 | CWE-915 |
| F-09 | Gutschein durch gleichzeitige Anfragen mehrfach einlösbar | Mittel | 5,3 | CWE-362 |
| F-10 | Services nehmen Anfragen aus dem internen Netz ohne Authentifizierung an | Hoch | 8,1 | CWE-306 |
| F-11 | Zugangsdaten des Nachrichten-Brokers im Klartext im Abbild des Service | Hoch | 8,4 | CWE-798 |
| F-12 | Bibliothek mit bekannter Schwachstelle | Hoch | 8,8 | CWE-1395 |
| F-13 | Ausführliche Fehlermeldungen mit internen Angaben | Niedrig | 5,3 | CWE-209 |
| F-14 | Abgelehnte Zugriffe werden nicht protokolliert | Mittel | ohne | CWE-778 |
| F-15 | Fehlende Sicherheits-Header in der Web-Oberfläche | Niedrig | 3,1 | CWE-693 |
| F-16 | Datenbankkonto des Buchungsservice mit Rechten auf alle Schemata | Mittel | ohne | CWE-250 |

Schon die Übersicht wirft Fragen auf, und ein aufmerksamer Leser stellt sie, bevor er einen einzigen Befund im Detail liest.

- F-07 hat den Wert 6,5 und die Stufe *Hoch*, F-08 denselben Wert und die Stufe *Mittel*. F-13 hat 5,3 und die Stufe *Niedrig*. Die Tester haben also an drei Stellen von der Zahl abgewichen. Steht im Befund, warum?
- F-14 und F-16 haben keinen Wert. Sind das Schwachstellen oder Beobachtungen?
- F-03 und F-16 betreffen beide die Datenbank des Buchungsservice. Hängen sie zusammen?
- F-05, F-10 und F-11 betreffen alle das Innere des Systems. Gibt es einen Angriffspfad, der sie verbindet?

Kapitel 14 klärt die ersten beiden Fragen, Kapitel 15 die anderen.

## 13.7 Anatomie eines Befunds

Der einzelne Befund ist die Einheit, mit der später gearbeitet wird. Er wird zu einem Ticket, er wird priorisiert, behoben und nachgetestet. Die verkürzten Befundkarten der früheren Kapitel haben fünf Felder gezeigt. Ein vollständiger Befund hat mehr.

**Tab. 43:** Die Felder eines vollständigen Befunds.

| Feld | Was dort steht | Woran du Qualität erkennst |
|---|---|---|
| Kennung und Titel | eindeutige Nummer, Titel aus Fehlerart und Ort | der Titel sagt, was wo falsch ist, nicht nur die Fehlerart |
| Schwere | Stufe, Wert und Vektor mit Fassung | Abweichungen zwischen Wert und Stufe sind begründet |
| Ort | Endpunkt, Parameter, Komponente, Umgebung | so genau, dass die Entwicklung die Stelle im Code findet |
| Beschreibung | was falsch ist, in zwei bis fünf Sätzen | beschreibt diese Anwendung, kein Textbaustein über die Fehlerart |
| Nachweis | Schritte zum Nachvollziehen, mit Anfrage, Antwort und Zeitpunkt | ein Dritter kann es mit den Angaben wiederholen |
| Auswirkung | was der Tester erreicht hat und was daraus folgt | trennt Belegtes von Vermutetem |
| Voraussetzungen | welches Konto, welcher Netzzugang, welche Mitwirkung nötig war | vollständig, denn sie bestimmen die Bewertung |
| Einordnung | CWE, Testkategorie, bei Fremdkomponenten CVE | CWE auf der Ebene Base oder Variant |
| Empfehlung | Behebung an dieser Stelle und darüber hinaus | unterscheidet Sofortmaßnahme und Ursachenbehebung |
| Verweise | Quellen zur Fehlerart und zur Behebung | passend zur eingesetzten Technik |

Als Beispiel der Befund F-05 in voller Länge.

**F-05 Server-Side Request Forgery beim Abruf eines Profilbilds** HOCH

| | |
|---|---|
| **Ort** | `POST /api/v3/profil/bild`, Feld `quelle`, Kundenkontoservice, Testumgebung |
| **Beschreibung** | Kunden können ein Profilbild hochladen oder die Adresse eines Bildes angeben. Im zweiten Fall ruft der Kundenkontoservice die Adresse selbst ab. Er prüft dabei nicht, wohin die Adresse zeigt. |
| **Voraussetzungen** | ein gewöhnliches Kundenkonto, Zugriff aus dem Internet |
| **Nachweis** | Am zweiten Testtag wurde als Quelle eine Adresse im internen Netz angegeben. Die Antwort des Service unterschied sich je nachdem, ob unter der Adresse ein Dienst erreichbar war. Für eine interne Adresse des Abrechnungsservice wurde der Inhalt der Antwort im Fehlertext zurückgegeben. Anfragen und Antworten in Anhang C, Abschnitt 5. |
| **Auswirkung** | Belegt: Ein Kunde kann über den Kundenkontoservice Dienste im internen Netz ansprechen und Antworten lesen. In Verbindung mit F-10 wurden auf diesem Weg Rechnungsdaten anderer Kunden gelesen. Nicht geprüft: Zugriff auf Verwaltungsdienste der Cloud-Plattform, da ausgeschlossen. |
| **Einordnung** | CWE-918, WSTG-INPV-19, CVSS 3.1: 7.7 (AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N) |
| **Empfehlung** | Kurzfristig: Abruf über Adressen abschalten, bis die Prüfung ergänzt ist. Behebung: Ziele auf eine Erlaubnisliste öffentlicher Adressen beschränken, nach der Namensauflösung prüfen, Weiterleitungen nicht folgen. Darüber hinaus: ausgehende Verbindungen des Service auf Netzebene beschränken. |

An diesem Befund lässt sich ablesen, was die rechte Spalte von Tab. 43 meint. Der Titel nennt Fehlerart und Ort. Die Beschreibung handelt von dieser Funktion und nicht von SSRF im Allgemeinen. Der Nachweis sagt, was beobachtet wurde, und verweist auf die Rohdaten. Die Auswirkung trennt in drei Teile: was belegt ist, was durch die Verbindung mit einem anderen Befund entstand, und was nicht geprüft wurde. Die Empfehlung hat drei Stufen, und die dritte liegt auf einer anderen Ebene als die zweite: Die Erlaubnisliste ist Code, die Beschränkung ausgehender Verbindungen ist Architektur.

### Der Nachweis

Der Nachweis hat zwei Aufgaben. Er belegt, dass der Befund echt ist, und er macht ihn wiederholbar. Die zweite Aufgabe wird beim Nachtest wichtig: Wer in drei Monaten prüfen soll, ob die Lücke geschlossen ist, braucht dieselben Schritte.

Ein guter Nachweis enthält die Anfrage, die relevante Stelle der Antwort, das verwendete Konto und den Zeitpunkt. Der Zeitpunkt erlaubt dem Betrieb, den Vorgang in den eigenen Protokollen zu suchen. Findet er ihn dort nicht, ist das ein eigener Befund, nämlich der mit der Nummer F-14.

Der Nachweis zeigt, *dass* etwas geht, und nicht mehr. Ein seriöser Bericht belegt den Zugriff auf fremde Daten mit einem Datensatz eines zweiten Testkontos und nicht mit einem Abzug der Datenbank. Kapitel 2 hat begründet, warum: Die Tester gehen so weit, wie für den Beleg nötig ist.

### Die Auswirkung

Die Auswirkung ist das Feld, an dem die Bewertung hängt, und das Feld, in dem am meisten vermutet wird. Kapitel 11 hat den Unterschied benannt. „Ein Angreifer könnte möglicherweise“ ist eine Vermutung, „es wurde gelesen“ ein Nachweis.

Beides hat seinen Platz. Tester dürfen und sollen sagen, was sie für möglich halten und nicht geprüft haben, weil die Zeit fehlte oder der Auftrag es ausschloss. Sie müssen es nur kennzeichnen. Beim Lesen markierst du dir in jedem Befund, welcher Teil der Auswirkung belegt ist. Bei der Triage in Kapitel 15 entscheidet das darüber, wie viel Gewicht du dem Befund gibst.

### Die Empfehlung

Die Empfehlung ist der Teil, den Tester am wenigsten gut schreiben können, und das liegt nicht an ihnen. Sie kennen die Anwendung seit acht Tagen und von außen. Sie wissen nicht, warum etwas so gebaut wurde, welche Abhängigkeiten eine Änderung hat und was sie kostet. Ihre Empfehlung ist ein Vorschlag aus der Sicht dessen, der die Lücke gefunden hat.

Daraus folgt eine Haltung, die für die ganze Spezialisierung trägt.

> **MERKE**
> Die Empfehlung im Bericht ist der Anfang der Überlegung, nicht ihr Ergebnis. Sie nennt fast immer die Korrektur an der gefundenen Stelle. Ob das die richtige Ebene ist, ob dieselbe Ursache noch woanders wirkt und was die Behebung im System bedeutet, muss jemand entscheiden, der das System kennt.

Die Empfehlungen in einem Bericht lassen sich danach sortieren, wie weit sie reichen. Die schwächste Form nennt das Ziel: „Eingaben sollten validiert werden.“ Die übliche Form nennt die Korrektur an der Stelle: „Im Endpunkt eine Besitzprüfung ergänzen.“ Die gute Form nennt zusätzlich die Klasse: „Alle Endpunkte prüfen, die Objekte über eine Kennung ansprechen.“ Die beste Form nennt die Ursache: „Die Besitzprüfung zentral erzwingen.“ Der Befund F-07 aus Kapitel 9 hatte alle drei Stufen. Viele Befunde haben nur die erste oder zweite, und die anderen zu ergänzen ist die Arbeit von Teil IV.

## 13.8 Angriffspfade

Der Abschnitt zu den Angriffspfaden erzählt, wie die Tester Befunde verbunden haben. Für *Kurbad* könnte dort stehen:

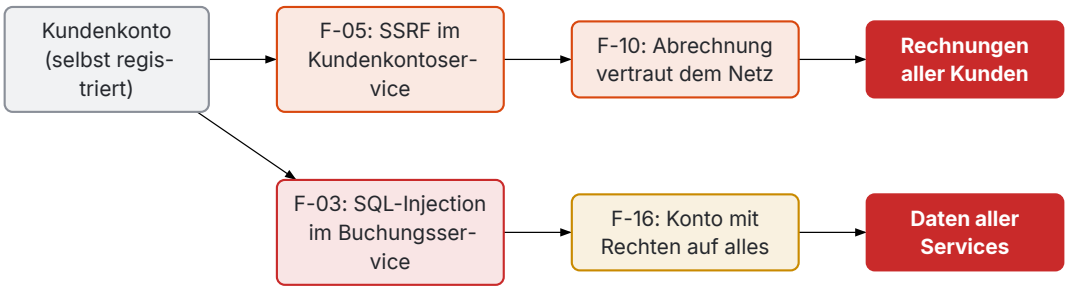



**Abb. 23:** Zwei Angriffspfade aus dem Bericht zu *Kurbad*. Jeder verbindet einen Befund am Rand mit einem im Inneren.

In beiden Pfaden aus Abb. 23 steht ein Befund, der für sich allein unscheinbar wirkt. F-16 hat nicht einmal einen CVSS-Wert. Erst zusammen mit F-03 wird daraus, dass eine Lücke in einem Service die Daten aller

Services öffnet. Der Angriffspfad ist deshalb der Teil des Berichts, der am meisten über die Architektur sagt, und er ist der Teil, den die Leitung am besten versteht, weil er eine Geschichte erzählt.

Nicht jeder Bericht hat diesen Abschnitt. Fehlt er, ist offen, ob es keine Ketten gab oder ob niemand danach gesucht hat. Das ist eine Frage für die Abschlussbesprechung, und in Kapitel 15 lernst du, mögliche Ketten selbst zu erkennen.

## 13.9 Was standgehalten hat

Ein vollständiger Bericht nennt auch, was die Tester versucht haben und was nicht gelang. Diese positiven Feststellungen haben drei Funktionen.

Sie sind die einzige Stelle, an der der Bericht etwas über Stärken sagt. Für *Kurbad* etwa: Die Anmeldung über den Identitätsdienst war korrekt eingebunden, die Signaturen der Tokens wurden an allen geprüften Stellen validiert, die Passwörter in der Datenbank waren mit einem geeigneten Verfahren gespeichert.

Sie schützen vor Verschlechterung. Wer weiß, dass die Validierung der Tokens geprüft und für gut befunden wurde, baut sie beim nächsten Umbau nicht versehentlich aus.

Und sie sagen etwas über die Ursachen. Wenn die Besitzprüfung an zwanzig Endpunkten vorhanden ist und an einem fehlt, ist das ein Versehen. Wenn sie an einem vorhanden ist und an zwanzig fehlt, gibt es kein Konzept. Der einzelne Befund sieht in beiden Fällen gleich aus. Erst die positiven Feststellungen zeigen, welcher Fall vorliegt, und davon hängt ab, auf welcher Ebene die Behebung liegt.

## 13.10 Einen Bericht beurteilen

Nicht jeder Bericht ist gut, und du musst das beurteilen können, bevor du dich auf ihn stützt. Die folgenden Fragen lassen sich in einer halben Stunde beantworten.

**Tab. 44:** Qualitätskriterien für einen Pentest-Bericht.

| Kriterium | Guter Bericht | Schwacher Bericht |
|---|---|---|
| Nachvollziehbarkeit | jeder Befund lässt sich mit den Angaben wiederholen | Bildschirmfotos ohne Anfrage, „siehe Werkzeugausgabe“ |
| Bezug zur Anwendung | Beschreibungen handeln von diesem System | Textbausteine, die in jedem Bericht stehen könnten |
| Geprüft statt gemeldet | Scannerfunde sind von Hand bestätigt oder als ungeprüft gekennzeichnet | Werkzeugausgabe ist ungeprüft übernommen |
| Bewertung | Vektor angegeben, Abweichungen begründet | nur eine Stufe, oder alles ist hoch |
| Auswirkung | Belegtes und Vermutetes sind getrennt | durchgehend „könnte“ |
| Zusammenhang | Ketten und gemeinsame Ursachen sind benannt | Befunde stehen unverbunden nebeneinander |
| Abdeckung | geprüfte und ungeprüfte Bereiche sind ausgewiesen | keine Angabe, was nicht geprüft wurde |
| Fachlogik | es gibt Befunde, die nur ein Mensch findet | nur Befunde, die ein Scanner auch meldet |
| Empfehlungen | konkret, mit Sofortmaßnahme und Ursache | „sollte abgesichert werden“ |
| Ehrlichkeit | Einschränkungen und Grenzen sind genannt | keine Einschränkungen, keine positiven Feststellungen |

Das achte Kriterium ist das schnellste. Schau in die Befundübersicht und frage dich bei jedem Befund, ob ein Werkzeug ihn allein hätte finden können. F-07, F-08 und F-09 aus Tab. 42 kann kein Scanner finden, weil sie Wissen über Besitz und Geschäftsregeln voraussetzen. Ein Bericht, der nur fehlende Header, veraltete Versionen und schwache Verschlüsselungseinstellungen enthält, ist ein Scan mit Deckblatt. Kapitel 1 hat diesen Unterschied als den zwischen Schwachstellenscan und Pentest beschrieben.

**AUS DER PRAXIS**

Eine verbreitete Schwäche von Berichten sind unbestätigte Scannerfunde mit dem Schweregrad aus der Datenbank. Ein Bericht meldet dann etwa vierzig Befunde, davon dreißig zu veralteten Komponenten, jeder mit der Stufe der zugehörigen Lücke. Die Entwicklung beginnt oben in der Liste und verbringt Wochen damit, Bibliotheken zu aktualisieren, deren verwundbare Funktion nie aufgerufen wird. Der eine Befund zur Berechtigung, der weiter unten steht, weil sein Wert „nur" mittel ist, bleibt liegen. Die vier Fragen aus Kapitel 10 zu Befunden über Komponenten sind das Mittel dagegen.

## 13.11 Einen Bericht lesen

Aus alldem ergibt sich eine Reihenfolge, die von der gedruckten abweicht. Sie folgt aus dem, was jeder Teil leistet, und du kannst sie den Studierenden als Arbeitsanweisung geben.

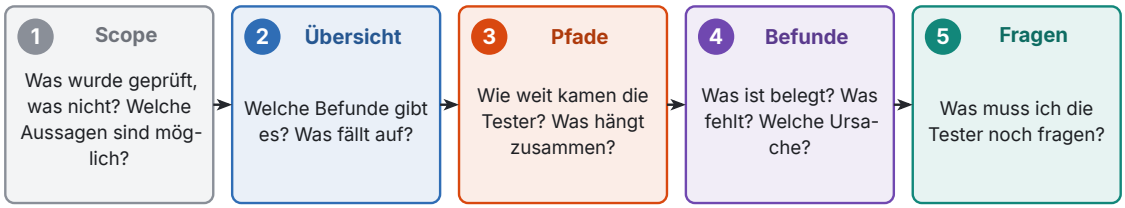



**Abb. 24:** Lesereihenfolge für einen Pentest-Bericht.

**Erstens Scope, Einschränkungen und Abdeckung.** Bevor du weißt, was gefunden wurde, musst du wissen, wo gesucht wurde. Notiere dir die Bereiche, über die der Bericht nichts sagen kann.

**Zweitens die Befundübersicht.** Lies nur die Tabelle und stelle Fragen an sie, wie oben gezeigt. Welche Befunde betreffen dieselbe Komponente? Welche haben dieselbe CWE? Wo weichen Wert und Stufe voneinander ab?

**Drittens die Angriffspfade.** Sie zeigen, was im schlimmsten Fall erreicht wurde, und geben den Rahmen für alles Weitere.

**Viertens die einzelnen Befunde**, und zwar mit dem Stift. Für jeden Befund hältst du fest: Ist er nachvollziehbar? Was an der Auswirkung ist belegt? Welche Voraussetzungen hat der Angriff? Und als erste Vermutung: Auf welcher Ebene liegt die Ursache?

**Fünftens die Liste der Nachfragen.** Was du nach dem Lesen nicht weißt, fragst du die Tester.

Die Zusammenfassung für die Leitung fehlt in der Reihenfolge mit Absicht. Wer sie zuerst liest, übernimmt das Urteil der Tester, bevor er sich ein eigenes gebildet hat. Lies sie am Ende und vergleiche.

Das Ergebnis des Lesens ist eine Arbeitsliste: je Befund eine Zeile mit deinen Anmerkungen. Diese Liste ist der Eingang für die Triage in Kapitel 15.

## 13.12 Die Abschlussbesprechung

Zu einem ordentlichen Auftrag gehört ein Gespräch, in dem die Tester den Bericht vorstellen und Fragen beantworten. Es ist die billigste Gelegenheit, den Wert des Tests zu erhöhen, und sie wird oft vertan, weil nur die Leitung teilnimmt und nur die Zusammenfassung besprochen wird.

Gute Fragen an die Tester sind solche, deren Antwort nicht im Bericht steht:

- Wo hätten Sie mit zwei Tagen mehr weitergesucht?
- Welcher Befund hat Sie überrascht, und welchen hatten Sie erwartet?
- Was haben Sie versucht, das nicht ging, und warum ging es nicht?
- Bei welchen Befunden sind Sie unsicher, ob die Bewertung stimmt?
- Welche Befunde gehören aus Ihrer Sicht zusammen?
- Was würden Sie an unserer Stelle zuerst tun?

Die Antwort auf die erste Frage ist die Liste der ungeprüften Risiken. Die Antwort auf die dritte ergänzt die positiven Feststellungen. Die Antwort auf die letzte ist eine Meinung, und zwar die von jemandem, der das System gerade aus der Sicht eines Angreifers gesehen hat. Sie ersetzt die eigene Entscheidung nicht, aber man sollte sie kennen.

## 13.13 Der Bericht als vertrauliches Dokument

Kapitel 2 hat es schon gesagt, hier noch einmal aus der Sicht des Empfängers: Ein Pentest-Bericht ist eine Anleitung zum Angriff auf das eigene System, solange die Befunde offen sind. Er gehört nicht in ein allgemein lesbares Wiki und nicht als Anhang in eine Mail an einen großen Verteiler.

Daraus entsteht ein praktischer Zielkonflikt. Die Entwickler, die beheben sollen, brauchen die Details. Üblich ist, den Bericht selbst eng zu halten und die Befunde einzeln in das Ticketsystem zu übertragen, mit eingeschränkter Sichtbarkeit für Sicherheitsthemen. Viele Anbieter liefern die Befunde dafür zusätzlich in einem maschinenlesbaren Format.

Für KI-Assistenten gilt, was Kapitel 2 für die Tester gesagt hat, nun für dich: Wer einen Bericht in einen Assistenten lädt, um ihn auswerten zu lassen, gibt ihn an einen Dritten weiter. Ob das zulässig ist, hängt vom Vertrag mit dem Anbieter des Assistenten und von den Regeln der eigenen Organisation ab. In der Lehrveranstaltung ist das kein Problem, weil System und Bericht erfunden sind. Es ist aber ein guter Anlass, die Frage zu stellen.

## 13.14 Berichte aus der Maschine

Berichte werden heute selten ganz von Hand geschrieben. Tester lassen Beschreibungen und Empfehlungen von KI-Assistenten vorformulieren, und in der Spezialisierung erzeugst du selbst den Bericht mit Hilfe eines Assistenten. Dem Leser hilft es, die typischen Spuren zu kennen.

Maschinell erzeugte Befunde sind sprachlich glatt und inhaltlich allgemein. Die Beschreibung erklärt die Fehlerart, statt die Stelle zu beschreiben. Die Empfehlung zählt alle denkbaren Maßnahmen auf, statt die passende zu nennen. Einordnungen stimmen im Groben und irren im Detail: eine CWE, die es gibt, die aber eine andere Schwäche bezeichnet, ein Vektor, der nicht zum beschriebenen Angriff passt, ein Wert, der nicht zum Vektor passt. Und gelegentlich steht ein Befund im Bericht, den es nicht gibt, weil ein Assistent aus einer Werkzeugausgabe mehr herausgelesen hat, als darin stand.

Keine dieser Spuren ist neu. Textbausteine und ungeprüfte Scannerfunde gab es vorher. Neu ist, dass sie besser klingen. Die Gegenmittel sind die Kriterien aus Tab. 44, vor allem die ersten drei.

**BRÜCKE ZU SOFTWARE SECURITY**

Der Pentest-Bericht ist ein Gegenstück zur Architekturdokumentation aus deinem Foliensatz 04. Beide beschreiben dasselbe System aus einer bestimmten Sicht für bestimmte Leser, und für beide gilt, dass Aussagen ohne Angabe ihrer Grenzen wertlos sind. Wer eine Architekturdokumentation nach arc42 kennt, findet die Befunde am natürlichsten in zwei Abschnitten wieder: bei den Risiken und technischen Schulden und, sobald über die Behebung entschieden ist, bei den Architekturentscheidungen. Kapitel 18 nimmt das auf.

**FRAGE AUS DEM HÖRSAAL**

***Warum liefern die Tester nicht einfach die Korrektur mit, wenn sie die Lücke schon gefunden haben?***

Manche Anbieter tun das für einfache Fälle, und bei einer fehlenden Prüfung in einem Endpunkt wäre es auch nicht schwer. Es hat aber Gründe, dass es nicht die Regel ist. Die Tester kennen den Code oft nicht. Sie tragen keine Verantwortung für das, was ihre Korrektur sonst noch verändert. Und es ist ein Interessenkonflikt, wenn derselbe Anbieter prüft, behebt und danach bestätigt, dass behoben wurde. Wichtiger ist der fachliche Grund: Die Korrektur an der gefundenen Stelle ist selten die ganze Behebung.

**FRAGE AUS DEM HÖRSAAL**

***Ein Bericht mit 16 Befunden und einer mit 60: Welcher Test war gründlicher?***

Das lässt sich aus der Zahl nicht sagen. Sechzig Befunde entstehen leicht, wenn jede betroffene Stelle einzeln gezählt und jede Werkzeugmeldung übernommen wird. Sechzehn Befunde können das Ergebnis sorgfältiger Bündelung sein. Aussagekräftiger sind die Abdeckung, der Anteil der Befunde, die nur ein Mensch findet, und die Tiefe der Angriffspfade.

**FRAGE AUS DEM HÖRSAAL**

***Muss ich als Empfänger jeden Befund glauben?***

Nein, und du sollst es auch nicht. Ein Befund ist eine Behauptung mit Beleg. Du prüfst den Beleg. Ist er nachvollziehbar, gilt der Befund, auch wenn er unbequem ist. Ist er es nicht, fragst du nach. Tester irren sich, vor allem bei der Auswirkung und bei der Bewertung, seltener bei der Beobachtung selbst. Was du nicht tun solltest, ist einen Befund ablehnen, weil die Entwicklung sagt, das könne nicht sein. Dann lässt man es sich zeigen.

**FÜR DIE LEHRE**

Lass die Studierenden in der ersten Einheit zum Bericht noch nichts beheben, sondern nur lesen. Eine geeignete Aufgabe: Jede Gruppe bekommt denselben Bericht und eine Stunde Zeit und liefert drei Dinge ab. Erstens die Liste der Bereiche, über die der Bericht nichts aussagt. Zweitens für jeden Befund ein Urteil, ob er nachvollziehbar ist. Drittens fünf Fragen an die Tester. Du spielst in der Abschlussbesprechung die Tester und beantwortest die Fragen. Was niemand fragt, erfährt niemand.

Für den Bericht der Case Study lohnt es sich, Schwächen absichtlich einzubauen: einen Befund ohne brauchbaren Nachweis, einen mit einer CWE, die nicht passt, einen mit einer Auswirkung, die nur vermutet ist, einen ungeprüften Scannerfund und zwei Befunde, die dieselbe Ursache haben. Wer mit einem KI-Assistenten arbeitet, bekommt diese Schwächen nicht geschenkt, denn der Assistent übernimmt die Angaben des Berichts meist ungeprüft.

Typische Fehlvorstellungen: dass die Zusammenfassung das Wichtigste enthalte, dass die Empfehlung im Bericht die Lösung sei, und dass Bereiche ohne Befund sicher seien.

## Selbsttest

1. Nenne die drei Leser eines Berichts und für jeden den Teil, den er vor allem liest.
2. Welche vier Fragen beantwortet eine gute Zusammenfassung für die Leitung?
3. Was ist eine Abdeckungsmatrix, und was folgt aus einem Feld „teilweise geprüft“?
4. Welche Felder hat ein vollständiger Befund? Nenne mindestens acht.
5. Warum ist der Zeitpunkt im Nachweis nützlich?
6. Woran erkennst du in der Auswirkung, was belegt und was vermutet ist, und warum ist das wichtig?
7. Ordne diese vier Empfehlungen nach ihrer Reichweite: Ursache zentral beheben, Stelle korrigieren, Ziel nennen, Klasse prüfen.
8. Was sagen positive Feststellungen über die Ursache eines Befunds aus?
9. Nenne fünf Kriterien, an denen du einen schwachen Bericht erkennst.
10. In welcher Reihenfolge liest du einen Bericht, und warum kommt die Zusammenfassung zuletzt?

# 14 Schweregrade: CVSS rechnen und einordnen

In der Befundübersicht zu *Kurbad* stehen drei Zeilen, die nicht zusammenpassen. Der Abruf fremder Rechnungen hat den Wert 6,5 und die Stufe *Hoch*. Die veränderbaren Bonuspunkte haben denselben Wert und die Stufe *Mittel*. Die ausführlichen Fehlermeldungen haben 5,3 und die Stufe *Niedrig*, obwohl 5,3 nach der Tabelle in die Mitte gehört. Entweder haben die Tester geschlampt, oder die Zahl ist nicht das, wofür man sie hält.

Kapitel 4 hat CVSS vorgestellt und gewarnt, dass die Zahl objektiver aussieht, als sie ist. Dieses Kapitel öffnet die Maschine. Du lernst, einen Vektor Buchstabe für Buchstabe zu lesen, einen Wert von Hand nachzurechnen und zu erkennen, welche Annahme hinter welchem Buchstaben steckt. Am Ende kannst du eine Bewertung prüfen, statt sie zu glauben, und begründen, wann die Stufe vom Wert abweichen darf. Für die Spezialisierung ist das eine Kernfertigkeit: Die Studierenden bekommen bewertete Befunde und müssen entscheiden, in welcher Reihenfolge sie beheben.

**IN DIESEM KAPITEL**

- Schwere, Risiko und Dringlichkeit unterscheiden
- die acht Basismetriken von CVSS 3.1 erklären und einen Vektor lesen
- einen Basiswert von Hand nachrechnen und sagen, welche Metrik ihn am stärksten verschiebt
- mit den Umgebungsmetriken einen Wert an das eigene System anpassen
- die wichtigsten Änderungen in CVSS 4.0 nennen
- eine Bewertung im Bericht prüfen und eine Abweichung zwischen Wert und Stufe begründen

## 14.1 Drei Dinge, die oft verwechselt werden

Bevor es an die Buchstaben geht, eine Unterscheidung, an der die meisten Missverständnisse hängen.

Die **Schwere** beschreibt eine Schwachstelle: wie leicht sie technisch auszunutzen ist und wie viel dabei im schlimmsten Fall verloren geht. Sie ist eine Eigenschaft der Lücke.

Das **Risiko** beschreibt eine Lage: wie wahrscheinlich es ist, dass jemand die Lücke in dieser Organisation ausnutzt, und was das diese Organisation kostet. Es ist eine Eigenschaft von Lücke, System, Angreifern und Geschäft zusammen.

Die **Dringlichkeit** beschreibt eine Entscheidung: in welcher Reihenfolge und bis wann behoben wird. In sie gehen das Risiko ein und dazu der Aufwand, die Abhängigkeiten zwischen den Befunden und was sonst gerade ansteht.

CVSS misst in seiner Grundform die Schwere. Die Stufe im Bericht ist das Urteil der Tester, und es liegt irgendwo zwischen Schwere und Risiko, weil die Tester das System gesehen haben, das Geschäft aber nur von außen kennen. Die Dringlichkeit legt niemand anderer fest als die Organisation selbst. Dieses Kapitel behandelt die ersten beiden, Kapitel 18 die dritte.

## 14.2 Der Aufbau von CVSS 3.1

Version 3.1 ist die Fassung, die in bestehenden Berichten und Datenbanken am häufigsten steht [27]. Sie besteht aus drei Gruppen von Merkmalen, den *Metriken*.




**Abb. 25:** Die drei Metrikgruppen von CVSS 3.1 und wer sie ausfüllt.

Die Zahl, die im Bericht steht, ist fast immer der **Basiswert**. Die beiden anderen Gruppen sind freiwillig und werden selten ausgefüllt. Darin liegt die Erklärung für einen großen Teil des Rätsels vom Anfang: Der Basiswert weiß nichts über *Kurbad*. Er wäre für dieselbe Lücke in einer Anwendung für Rezeptvorschläge derselbe.

## 14.3 Die acht Basismetriken

Vier Metriken beschreiben, wie schwer die Ausnutzung ist. Drei beschreiben, was dabei verloren geht. Eine, der Scope, verbindet beides.

**Tab. 45:** Die Basismetriken von CVSS 3.1 [27].

| Metrik | Kürzel | Werte | Die Frage dazu |
|---|---|---|---|
| Attack Vector | AV | Network (N), Adjacent (A), Local (L), Physical (P) | Von wo aus muss der Angreifer handeln? Über das Netz, aus dem benachbarten Netz, auf dem Rechner, mit der Hand am Gerät? |
| Attack Complexity | AC | Low (L), High (H) | Hängt der Erfolg von Umständen ab, die der Angreifer nicht in der Hand hat? |
| Privileges Required | PR | None (N), Low (L), High (H) | Welche Rechte braucht der Angreifer vorher? Keine, die eines gewöhnlichen Benutzers, die eines Administrators? |
| User Interaction | UI | None (N), Required (R) | Muss ein anderer Mensch mitwirken, etwa einen Verweis öffnen? |
| Scope | S | Unchanged (U), Changed (C) | Bleibt der Schaden in der verwundbaren Komponente, oder trifft er eine andere, die eigene Schutzmechanismen hat? |
| Confidentiality | C | High (H), Low (L), None (N) | Wie viel Vertrauliches wird lesbar? Alles, ein Teil ohne Kontrolle darüber, nichts? |
| Integrity | I | High (H), Low (L), None (N) | Wie viel lässt sich verändern? |
| Availability | A | High (H), Low (L), None (N) | Wie stark lässt sich die Verfügbarkeit stören? |

Drei Anmerkungen zu Tab. 45, weil die Namen in die Irre führen.

Der **Scope** der Bewertung hat nichts mit dem Scope des Auftrags aus Kapitel 2 zu tun. Gemeint ist die Frage, ob der Angriff eine Grenze zwischen zwei Verantwortungsbereichen überschreitet. Beim Cross-Site Scripting liegt die Lücke in der Web-Anwendung, der Schaden entsteht im Browser des Opfers: Scope *Changed*. Bei einer Lücke, die aus einem Container auf den Rechner darunter führt, ebenso. Der Scope ist die Metrik, über die sich Fachleute am häufigsten uneinig sind.

Die **Privileges Required** werden vor dem Angriff gemessen. Eine Lücke, die ein Kundenkonto voraussetzt, hat *Low*, auch wenn jeder Mensch auf der Welt sich in zwei Minuten ein solches Konto anlegen kann. Dieser Punkt wird gleich wichtig.

Die drei Metriken zur **Auswirkung** messen den Anteil, nicht die Menge. *High* bei der Vertraulichkeit heißt, dass der Angreifer alles lesen kann, was die Komponente hat, oder dass das Gelesene für sich schwer wiegt. Ob das zehn Datensätze sind oder zehn Millionen, steht nirgends.

## 14.4 Einen Vektor lesen

Der Vektor ist die Zeichenkette, in der alle acht Entscheidungen stehen. Der des Befunds F-07 lautet:

```
CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N
```

Übersetzt in einen Satz: Ein Angreifer kann über das Netz (AV:N), ohne auf besondere Umstände angewiesen zu sein (AC:L), mit den Rechten eines gewöhnlichen Benutzers (PR:L) und ohne Mitwirkung eines anderen Menschen (UI:N) alle vertraulichen Daten der betroffenen Komponente lesen (C:H), aber nichts verändern (I:N) und nichts stören (A:N). Der Schaden bleibt in der Komponente (S:U).

Dieser Satz ist die eigentliche Aussage. Die Zahl ist nur seine Verdichtung. Wenn du einen Bericht prüfst, liest du den Vektor als Satz und vergleichst ihn mit dem, was im Befund unter Voraussetzungen, Nachweis und Auswirkung steht. Passt der Satz nicht zum Text, stimmt eines von beiden nicht.

## 14.5 Wie aus dem Vektor die Zahl wird

Jedem Wert jeder Metrik ist eine Gewichtszahl zugeordnet. Aus ihnen entstehen zwei Teilwerte, einer für die Ausnutzbarkeit und einer für die Auswirkung, die zum Basiswert zusammengefasst werden.

**Tab. 46:** Die Gewichte der Basismetriken in CVSS 3.1 [27].

| Metrik | Gewichte | Metrik | Gewichte |
|---|---|---|---|
| AV | N 0,85 A 0,62 L 0,55 P 0,20 | UI | N 0,85 R 0,62 |
| AC | L 0,77 H 0,44 | C, I, A | H 0,56 L 0,22 N 0 |
| PR (S:U) | N 0,85 L 0,62 H 0,27 | PR (S:C) | N 0,85 L 0,68 H 0,50 |

**RECHENBEISPIEL 14.1 · DER BASISWERT VON F-07**

Der Vektor ist AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N.

**Ausnutzbarkeit.** Die vier Gewichte werden miteinander und mit dem festen Faktor 8,22 multipliziert:

$$8{,}22 \times 0{,}85 \times 0{,}77 \times 0{,}62 \times 0{,}85 = 2{,}84$$

**Auswirkung.** Zuerst werden die drei Auswirkungen zu einem Wert zusammengefasst, der sagt, wie viel von Vertraulichkeit, Integrität und Verfügbarkeit insgesamt verloren geht:

$$1 - (1 - 0{,}56) \times (1 - 0) \times (1 - 0) = 0{,}56$$

Bei unverändertem Scope wird dieser Wert mit 6,42 multipliziert:

$$6{,}42 \times 0{,}56 = 3{,}60$$

**Basiswert.** Die Summe der beiden Teilwerte ist 6,44. Sie wird auf eine Nachkommastelle *aufgerundet* und bei 10 gedeckelt:

$$2{,}84 + 3{,}60 = 6{,}44 \quad \text{->} \quad \mathbf{6{,}5}$$

Drei Dinge lassen sich an der Rechnung ablesen.

Die Ausnutzbarkeit kann höchstens 3,9 erreichen, die Auswirkung bei unverändertem Scope höchstens 5,9. Die Auswirkung wiegt also schwerer, aber eine Lücke, die nur die Vertraulichkeit betrifft, kommt über 3,6 bei der Auswirkung nicht hinaus und damit ohne Scope-Wechsel nie über 7,5. Das ist der Wert der berühmten Lücke Heartbleed [19]. Ein reines Datenleck kann nach dieser Rechnung nicht *kritisch* sein, gleich wie groß es ist.

Bei verändertem Scope gilt eine andere Formel für die Auswirkung, und die Summe wird zusätzlich mit 1,08 multipliziert. Deshalb erreicht die SQL-Injection F-03 mit ihrem Vektor den Wert 9,9. Mit S:U wären es 8,8.

Und es wird immer aufgerundet. Aus 6,44 wird 6,5, aus 6,91 wird 7,0 und damit *hoch*. An den Grenzen der Stufen entscheidet die zweite Nachkommastelle über die Farbe im Bericht.

## 14.6 Was die Zahl verschiebt

Wie empfindlich der Wert auf einzelne Entscheidungen reagiert, zeigt sich, wenn man am Vektor von F-07 jeweils einen Buchstaben ändert.

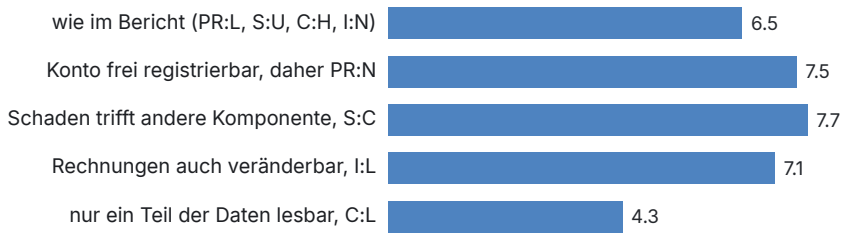



**Abb. 26:** Der Basiswert von F-07 nach CVSS 3.1, wenn je eine Metrik anders beurteilt wird.

Jede der Varianten in Abb. 26 lässt sich vertreten. Ob ein Konto, das sich jeder anlegen kann, ein Privileg ist, beurteilen Tester verschieden. Ob die Rechnungsdaten „alle“ Daten der Komponente sind oder ein Teil, hängt davon ab, was man als Komponente ansieht. Zwischen der niedrigsten und der höchsten Lesart liegen 3,4 Punkte und zwei Stufen.

**STUDIE 14.2 · WIE EINHEITLICH BEWERTEN FACHLEUTE?**

Wunder und andere legten 196 Personen, die beruflich mit CVSS arbeiten, dieselben Schwachstellen zur Bewertung vor [69]. Für verbreitete Arten von Schwachstellen, darunter Cross-Site Scripting, wurden vor allem Attack Vector, User Interaction und Scope uneinheitlich beurteilt. Neun Monate später bewerteten 59 der Teilnehmenden dieselben Schwachstellen noch einmal. 68 Prozent von ihnen kamen dabei zu einem anderen Schweregrad als beim ersten Mal.

Die Bewertung streut also nicht nur zwischen Personen, sondern auch bei derselben Person über die Zeit. Für den Umgang mit einem Bericht folgt daraus, dass ein halber Punkt Unterschied nichts bedeutet und eine Stufe Unterschied wenig.

**MERKE**

Ein CVSS-Wert ist das Ergebnis von acht Entscheidungen, und mindestens zwei davon sind Ermessenssache. Wer den Wert ohne den Vektor bekommt, kann die Entscheidungen nicht sehen. Ein Wert ohne Vektor ist deshalb nicht prüfbar.

## 14.7 Warum 6,5 „hoch“ sein kann

Jetzt lässt sich das Rätsel um F-07 auflösen. Die Tester haben den Basiswert korrekt berechnet und die Stufe trotzdem angehoben. Sie hatten dafür drei Gründe, und alle drei liegen außerhalb dessen, was der Basiswert sehen kann.

**Das Privileg ist keines.** Der Angriff braucht ein Kundenkonto, und ein Kundenkonto bekommt jeder, der sich registriert. Der Basiswert behandelt das wie ein Konto, das einem erst jemand einrichten muss.

**Die Menge zählt nicht.** Der Vektor sagt C:H, und mehr als H gibt es nicht. Dass es um die Rechnungen sämtlicher Kunden geht, mit Namen, Anschriften und gebuchten Leistungen, macht den Wert nicht höher.

**Der Schutzbedarf zählt nicht.** Unter den gebuchten Leistungen eines Thermenbetreibers sind Behandlungen, aus denen sich auf die Gesundheit eines Menschen schließen lässt. Solche Daten sind nach der Datenschutz-Grundverordnung besonders geschützt [12]. Für den Basiswert sind es Daten wie alle anderen.

Der dritte Grund ist der, für den CVSS selbst ein Mittel vorsieht.

## 14.8 Die Umgebungsmetriken

Die Umgebungsmetriken erlauben dem Betreiber, den Basiswert an das eigene System anzupassen. Sie bestehen aus zwei Teilen.

Der erste Teil sind drei Angaben zum **Schutzbedarf**: *Confidentiality Requirement* (CR), *Integrity Requirement* (IR) und *Availability Requirement* (AR), jeweils mit den Werten *Low*, *Medium* und *High*. Sie gewichten

die drei Auswirkungen. Ist die Vertraulichkeit der Daten in dieser Komponente besonders wichtig, zählt ein Verlust dort mehr.

Der zweite Teil sind die **modifizierten Basismetriken**. Zu jeder der acht Basismetriken gibt es ein Gegenstück mit vorangestelltem M, also MAV, MAC, MPR und so weiter. Damit lässt sich festhalten, dass eine Metrik in dieser Umgebung einen anderen Wert hat als im allgemeinen Fall, weil eine Maßnahme vorhanden ist oder fehlt.

**RECHENBEISPIEL 14.3 · F-07 MIT SCHUTZBEDARF**

Der Betreiber von *Kurbad* stuft die Vertraulichkeit der Rechnungsdaten als hoch ein, wegen der Gesundheitsbezüge. Der Vektor wird ergänzt:

`AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N/CR:H`

Das Gewicht für CR:H ist 1,5. Der zusammengefasste Auswirkungswert steigt von 0,56 auf 0,56 × 1,5 = 0,84, die Auswirkung von 3,60 auf 6,42 × 0,84 = 5,39. Mit der unveränderten Ausnutzbarkeit von 2,84 ergibt sich 8,23, aufgerundet **8,3**.

Mit einer einzigen zusätzlichen Angabe, die nur der Betreiber machen kann, liegt der Wert in der Stufe *hoch*. Das Urteil der Tester und die Zahl stimmen wieder überein.

In die andere Richtung geht es ebenso. Der Befund F-12 trägt den Wert 8,8 aus der Schwachstellendatenbank, berechnet für den allgemeinen Fall: aus dem Netz erreichbar, ohne Rechte. Angenommen, die betroffene Bibliothek wird im Buchungsservice nur für Bilder verwendet, die Mitarbeiter über die Admin-Oberfläche hochladen, und diese ist nach der Behebung von F-01 nur noch aus dem internen Netz erreichbar. Dann gilt in dieser Umgebung MAV:A und MPR:L, und der Wert sinkt auf 7,4. Die Lücke ist dieselbe, ihre Lage im System ist eine andere.

**FALLSTRICK**

Die Umgebungsmetriken lassen sich missbrauchen, um unbequeme Befunde kleinzurechnen. „Das ist intern, also MAV:A“ ist die naheliegende Form. Kapitel 11 hat gezeigt, warum das interne Netz keine Grenze ist, wenn es einen Weg hinein gibt, und bei *Kurbad* gibt es mit F-05 einen. Eine modifizierte Metrik muss sich auf eine Maßnahme stützen, die es gibt und die geprüft ist, nicht auf eine Annahme über das Netz.

## 14.9 Die zeitlichen Metriken

Die zweite freiwillige Gruppe erfasst, was sich nach der Entdeckung einer Lücke ändert. Sie ist vor allem für Lücken in Fremdkomponenten von Belang.

*Exploit Code Maturity* (E) sagt, ob es Angriffscode gibt, von gar keinem über einen Machbarkeitsnachweis bis zu zuverlässig funktionierendem. *Remediation Level* (RL) sagt, ob es eine offizielle Korrektur, eine vorläufige oder gar keine gibt. *Report Confidence* (RC) sagt, wie gesichert die Angaben zur Lücke sind.

Die zeitlichen Metriken können den Wert nur senken. Für F-12 ohne bekannten Angriffscode und mit offizieller Korrektur (E:U/RL:O/RC:C) wird aus 8,8 der Wert 7,7. In der Praxis hat sich diese Gruppe kaum durchgesetzt, weil niemand die Angaben laufend pflegt. Die Frage, ob eine Lücke ausgenutzt wird, beantworten heute EPSS und KEV aus Kapitel 4 besser.

## 14.10 CVSS 4.0

Im November 2023 erschien Version 4.0 [28]. Sie setzt sich in Datenbanken und Berichten allmählich durch, und für einige Jahre wirst du beide Fassungen nebeneinander sehen. Du erkennst sie am Anfang des Vektors: `CVSS:4.0/`. Fünf Änderungen sind für das Lesen von Berichten wichtig.

**Der Scope ist abgeschafft.** An seine Stelle treten zwei Gruppen von Auswirkungen: auf das verwundbare System (VC, VI, VA) und auf nachgelagerte Systeme (SC, SI, SA). Damit wird die strittigste Metrik durch sechs klarere Fragen ersetzt.

**Es gibt eine neue Metrik für Voraussetzungen.** *Attack Requirements* (AT) erfasst Bedingungen des Angriffs, die außerhalb der Kontrolle des Angreifers liegen, etwa dass ein Wettlauf gewonnen werden muss. Die Attack Complexity beschreibt nur noch Schutzmechanismen, die zu überwinden sind.

**Die Mitwirkung hat drei Stufen.** User Interaction unterscheidet nun *None*, *Passive* (das Opfer tut etwas Alltägliches, etwa eine Seite öffnen) und *Active* (das Opfer muss bewusst etwas Ungewöhnliches tun).

**Aus der zeitlichen Gruppe wurde die Bedrohungsgruppe** mit nur noch einer Metrik, der *Exploit Maturity* (E) mit den Werten *Attacked*, *Proof-of-Concept* und *Unreported*.

**Der Name sagt, was eingeflossen ist.** Ein Wert heißt CVSS-B, wenn er nur die Basismetriken enthält, CVSS-BT mit Bedrohung, CVSS-BE mit Umgebung und CVSS-BTE mit beidem. Das macht sichtbar, was bisher verborgen blieb: dass die allermeisten veröffentlichten Werte reine Basiswerte sind.

Dazu kommen ergänzende Metriken ohne Einfluss auf die Zahl, etwa ob sich ein Angriff automatisieren lässt und ob Menschen körperlich zu Schaden kommen können. Und die Zahl wird nicht mehr mit einer Formel berechnet, sondern aus einer Tabelle abgelesen, in die Bewertungen von Fachleuten eingeflossen sind. Von Hand nachrechnen lässt sie sich nicht mehr, man braucht den Rechner von FIRST.

**Tab. 47:** Basiswerte ausgewählter Befunde nach beiden Fassungen, berechnet nach den Spezifikationen von FIRST. Für 4.0 wurde der jeweils naheliegende Vektor gewählt.

| Nr. | Befund | 3.1 | 4.0 | Warum der Unterschied |
|---|---|---|---|---|
| F-03 | SQL-Injection | 9,9 | 9,4 | nachgelagerte Systeme zählen anders als der Scope |
| F-07 | fremde Rechnungen | 6,5 | 7,1 | Verlust der Vertraulichkeit wiegt allein schwerer |
| F-04 | gespeichertes XSS | 5,4 | 5,1 | Auswirkung liegt nur im nachgelagerten System |
| F-09 | Gutschein mehrfach | 5,3 | 6,0 | Wettlauf ist Voraussetzung statt Komplexität |
| F-12 | Bibliothek | 8,8 | 8,6 | aktive Mitwirkung nötig |
| F-13 | Fehlermeldungen | 5,3 | 6,9 | geringe Auswirkung ohne Rechte wiegt schwerer |

Tab. 47 zeigt zweierlei. F-07 liegt nach Version 4.0 mit 7,1 in der Stufe *hoch*, ganz ohne Anhebung von Hand. Die neue Fassung beurteilt den Fall also so wie die Tester. Und F-13 steigt auf 6,9, obwohl niemand ausführliche Fehlermeldungen für beinahe hoch hält. Werte aus verschiedenen Fassungen lassen sich nicht miteinander vergleichen. Ein Bericht muss bei einer Fassung bleiben, und wer Berichte aus mehreren Jahren nebeneinanderlegt, muss darauf achten.

Bei F-12 lohnt ein Blick auf die Bedrohungsmetrik. Ist kein Angriff bekannt (E:U), sinkt der Wert von 8,6 auf 6,1. Gibt es einen Machbarkeitsnachweis (E:P), sind es 7,3. Version 4.0 nimmt die Frage, ob eine Lücke ausgenutzt wird, also deutlich ernster als ihre Vorgängerin.

## 14.11 Wo CVSS nicht passt

CVSS ist für Schwachstellen in Softwareprodukten gebaut. Ein Pentest liefert auch Befunde anderer Art, und dort wird die Bewertung gezwungen oder sinnlos.

**Fehlende Schutzmaßnahmen.** F-14, die fehlende Protokollierung abgelehnter Zugriffe, ist keine Lücke, durch die jemand eindringt. Sie sorgt dafür, dass ein Eindringen unbemerkt bleibt. Ein Vektor lässt sich dafür nicht ehrlich angeben, weil es keinen Angriff gibt, den er beschreiben würde. Die Tester haben zu Recht keinen Wert angegeben und die Stufe begründet. Dasselbe gilt für F-16, das Datenbankkonto mit zu weiten Rechten.

**Ketten.** CVSS bewertet eine Lücke für sich. Dass F-05 erst durch F-10 gefährlich wird und F-03 durch F-16 noch gefährlicher, steht in keinem der vier Vektoren. FIRST erlaubt ausdrücklich, eine Kette als Ganzes zu bewerten, aber dann bewertet man etwas anderes als die einzelnen Befunde. Kapitel 15 behandelt, wie man damit umgeht.

**Fachlogik.** Beim mehrfach einlösbaren Gutschein F-09 hängt der Schaden davon ab, wie viel ein Gutschein wert ist und wie oft sich der Angriff wiederholen lässt. Ob das I:L oder I:H ist, lässt sich nur mit Kenntnis des Geschäfts sagen, und der Wert schwankt entsprechend zwischen 3,1 und 5,3.

**Schwache Konfiguration.** Fehlende Header, ausführliche Fehlermeldungen und Versionsangaben erzeugen nach der Formel Werte um 5, weil sie aus dem Netz und ohne Rechte „ausnutzbar" sind. In der Studie von Wunder und anderen zweifelten die Befragten selbst daran, ob sich solche Befunde für CVSS eignen [69]. Die Tester von *Kurbad* haben F-13 deshalb von Hand auf *niedrig* gesetzt. Das ist die dritte Abweichung vom Anfang des Kapitels, und sie ist ebenso begründet wie die erste.

Bleibt die zweite: F-08, die veränderbaren Bonuspunkte, mit 6,5 und der Stufe *mittel*. Hier haben die Tester nicht abgewichen, sondern einen Vektor mit I:H gewählt, also einen vollständigen Verlust der Integrität angenommen. Verändert werden kann aber nur ein Feld des eigenen Kontos. Mit I:L ergäbe sich 4,3. Der Wert im Bericht ist an dieser Stelle zu hoch und die Stufe richtig. Auch das kommt vor.

## 14.12 Von der Schwere zur Dringlichkeit

Die Schwere ist eine Eingangsgröße für die Reihenfolge der Behebung, nicht die Reihenfolge selbst. Kapitel 4 hat mit EPSS und KEV die zweite Achse gezeigt: ob eine Lücke tatsächlich angegriffen wird. Für Befunde im eigenen Code gibt es diese Daten nicht, und die Frage muss anders beantwortet werden: Wie auffindbar ist die Stelle, und wie viel Können braucht der Angriff? F-07 findet jeder, der in der Adresse eine Zahl verändert. F-09 braucht jemanden, der gezielt gleichzeitige Anfragen schickt.

Einen anderen Weg geht ein Verfahren der Carnegie Mellon University, das die US-Behörde CISA übernommen hat.

> **DEFINITION 14.4 · SSVC**
> Die *Stakeholder-Specific Vulnerability Categorization* ist ein Verfahren, das statt einer Zahl eine Handlungsanweisung liefert. Wenige Fragen mit festen Antworten werden als Entscheidungsbaum durchlaufen, an dessen Ende eine von wenigen Stufen der Dringlichkeit steht [70], [71].

In der Fassung der CISA sind es fünf Fragen: ob die Lücke ausgenutzt wird, wie groß die technische Auswirkung ist, ob sich der Angriff automatisieren lässt, wie wichtig das betroffene System für die Aufgabe der Organisation ist und ob das Wohl der Öffentlichkeit berührt ist. Am Ende stehen vier Ergebnisse.

**Tab. 48:** Die vier Ergebnisse von SSVC in der Fassung der CISA [71].

| Ergebnis | Bedeutung | Behebung |
|---|---|---|
| Track | keine Handlung nötig, weiter beobachten | im üblichen Rhythmus |
| Track* | genauer beobachten, weil Merkmale auf Veränderung hindeuten | im üblichen Rhythmus |
| Attend | Aufmerksamkeit der Vorgesetzten nötig, gegebenenfalls Unterstützung anfordern und benachrichtigen | früher als üblich |
| Act | Leitung einbeziehen, abgestimmte Reaktion der ganzen Organisation | so schnell wie möglich |

Der Wert des Verfahrens liegt weniger im Baum als in der Haltung. Es fragt nicht „wie schlimm ist das auf einer Skala", sondern „was tun wir jetzt", und es zwingt dazu, die Bedeutung des Systems für die eigene Aufgabe ausdrücklich einzubeziehen. Für die Triage eines Pentest-Berichts kannst du die Idee übernehmen, ohne den Baum zu verwenden.

Daneben gibt es Verfahren, die Wahrscheinlichkeit und Auswirkung getrennt schätzen und in einer Matrix zusammenführen, etwa die Risikobewertung von OWASP [57]. Viele Organisationen haben eine eigene Matrix aus ihrem Risikomanagement. Für alle gilt, was für CVSS gilt: Das Verfahren ordnet die Überlegung, es ersetzt sie nicht.

## 14.13 Eine Bewertung prüfen

Aus dem Kapitel ergibt sich ein Prüfgang, den du auf jeden bewerteten Befund anwenden kannst. Er dauert je Befund wenige Minuten.

1. **Ist ein Vektor angegeben, und mit welcher Fassung?** Ohne Vektor ist nichts prüfbar. Dann fragst du nach.
2. **Passt der Vektor zum Text?** Lies ihn als Satz. Steht unter Voraussetzungen „Kundenkonto", muss PR:L stehen. Muss ein Opfer etwas öffnen, muss UI:R stehen. Wurde nur gelesen, muss I:N stehen.

3. **Ist die Auswirkung belegt?** C:H, I:H und A:H dürfen sich nur auf das stützen, was der Nachweis zeigt, oder müssen als Annahme gekennzeichnet sein.
4. **Ist der Scope begründet?** Bei S:C muss erkennbar sein, welche zweite Komponente betroffen ist.
5. **Stimmt die Rechnung?** Gib den Vektor in den Rechner ein. Ein Wert, der nicht zum Vektor passt, ist ein Zeichen für einen nachlässig oder maschinell erstellten Bericht.
6. **Weicht die Stufe vom Wert ab, und ist das begründet?** Eine begründete Abweichung ist ein Qualitätsmerkmal, eine unbegründete eine Frage an die Tester.
7. **Was ändert sich in deiner Umgebung?** Trage Schutzbedarf und vorhandene Maßnahmen ein und halte den eigenen Wert neben dem des Berichts fest.

Der siebte Schritt ist dein Beitrag. Für *Kurbad* könnte das Ergebnis für einige Befunde so aussehen.

**Tab. 49:** Bewertung des Berichts und eigene Bewertung mit Umgebungsmetriken, nach CVSS 3.1.

| Nr. | Bericht | eigener Wert | Begründung |
|---|---|---|---|
| F-03 | 9,9 | 9,9 | keine Änderung, Schutzbedarf ändert am Höchstwert nichts |
| F-07 | 6,5 | 8,3 | CR:H wegen Gesundheitsbezug der Rechnungsdaten |
| F-08 | 6,5 | 4,3 | I:L, da nur ein Feld des eigenen Kontos veränderbar ist |
| F-12 | 8,8 | 8,8 | vorerst keine Änderung, da F-01 noch offen ist |
| F-13 | 5,3 | 4,6 | CR:L, die Angaben sind für sich nicht schutzbedürftig |

Die Zeile zu F-12 zeigt eine Abhängigkeit, die in Kapitel 15 wiederkehrt: Die Bewertung eines Befunds kann davon abhängen, ob ein anderer behoben ist.

**BRÜCKE ZU SOFTWARE SECURITY**

In deinen Foliensätzen zur Softwarearchitektur stehen Qualitätsszenarien: eine Quelle, ein Auslöser, eine Umgebung, ein betroffenes Artefakt, eine Antwort und ein Maß. Ein CVSS-Vektor ist ein stark verkürztes Sicherheitsszenario. AV und PR beschreiben die Quelle, AC und UI die Umgebung des Auslösers, C, I und A die Wirkung auf das Artefakt. Was fehlt, sind die Antwort des Systems und das Maß, also genau das, was eine Architektur leisten soll. Wer einen Befund in ein vollständiges Qualitätsszenario umschreibt, hat die Anforderung formuliert, an der sich die Behebung messen lässt.

**FRAGE AUS DEM HÖRSAAL**

***Wenn die Zahl so unsicher ist, warum verwendet man sie dann überhaupt?***

Weil die Alternative schlechter ist. Ohne ein gemeinsames Verfahren bewertet jeder nach Gefühl, und zwei Berichte lassen sich gar nicht mehr vergleichen. CVSS zwingt dazu, acht Fragen ausdrücklich zu beantworten, und macht die Antworten im Vektor sichtbar. Der Nutzen liegt im Vektor und nicht in der Zahl. Wer nur die Zahl verwendet, wirft den nützlichen Teil weg.

**FRAGE AUS DEM HÖRSAAL**

***Dürfen die Tester die Stufe einfach anders setzen, als die Zahl sagt?***

Ja, wenn sie es begründen. Die Zuordnung von Werten zu Stufen ist eine Empfehlung der Spezifikation, kein Gesetz, und die Tester haben Wissen, das die Formel nicht hat. Was nicht geht, ist eine Abweichung ohne Begründung, oder ein Vektor, der so lange verändert wird, bis die gewünschte Zahl herauskommt. Das Zweite erkennst du daran, dass der Vektor nicht mehr zum Text passt.

**FRAGE AUS DEM HÖRSAAL**

***Soll ich bei jedem Befund die Umgebungsmetriken ausfüllen?***

Nicht bei jedem. Es lohnt sich dort, wo der Schutzbedarf deutlich vom Durchschnitt abweicht, nach oben oder nach unten, und bei Befunden zu Fremdkomponenten, deren Wert aus einer Datenbank stammt und nichts über dein System weiß. Für einen Bericht mit 25 Befunden ist das eine Handvoll. Bei den anderen genügt die Feststellung, dass der Basiswert passt.

**FÜR DIE LEHRE**

Die wirksamste Übung zu diesem Kapitel dauert zwanzig Minuten. Gib allen Studierenden dieselbe Befundbeschreibung ohne Bewertung und lass jede Person für sich einen Vektor festlegen. Sammle die Werte an der Tafel. Die Streuung, die dabei nach den Ergebnissen der Studie zu erwarten ist, überzeugt mehr als jede Folie. Lass danach zwei Personen mit weit auseinanderliegenden Werten ihre Vektoren Buchstabe für Buchstabe vergleichen. Der Unterschied liegt meist in einer oder zwei Metriken, und über die lässt sich sachlich reden.

Eine zweite Übung richtet sich gegen die Gläubigkeit gegenüber KI-Assistenten: Lass einen Assistenten zu drei Befunden Vektor und Wert angeben und die Studierenden nachrechnen. Dabei kommen immer wieder Werte heraus, die nicht zum Vektor passen, oder Vektoren, die nicht zur Beschreibung passen.

In der Prüfung eignet sich die Bewertung gut für Fragen mit eindeutiger Antwort: einen Vektor in einen Satz übersetzen, zu einer Beschreibung den falschen Buchstaben finden, begründen, welche Umgebungsmetrik einen Wert hebt.

Typische Fehlvorstellungen: dass der Wert das Risiko sei, dass ein mittlerer Wert warten könne, dass PR:L „schwer auszunutzen“ heiße, und dass ein Wert ohne Vektor eine Information sei.

## Selbsttest

1. Grenze Schwere, Risiko und Dringlichkeit voneinander ab.
2. Übersetze den Vektor AV:N/AC:H/PR:N/UI:R/S:U/C:L/I:N/A:N in einen Satz.
3. Was bedeutet der Scope in CVSS 3.1, und wie unterscheidet er sich vom Scope eines Auftrags?
4. Rechne nach: Welchen Basiswert hat AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N? Verwende die Gewichte aus Tab. 46.
5. Warum kann ein reines Datenleck ohne Scope-Wechsel nach CVSS 3.1 nie kritisch sein?
6. Nenne die drei Gründe, aus denen die Tester F-07 trotz des Werts 6,5 als hoch eingestuft haben.
7. Was bewirkt die Angabe CR:H, und wer darf sie machen?
8. Nenne drei Änderungen von CVSS 4.0 gegenüber 3.1.
9. Warum lässt sich für F-14 kein ehrlicher Vektor angeben?
10. Ein Befund nennt „Voraussetzung: Administratorkonto“ und den Vektor AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H. Was stimmt nicht, und wie ändert sich der Wert ungefähr?

# 15 Triage: prüfen, bündeln, verketten

Der Bericht zu *Kurbad* hat sechzehn Befunde. Die naheliegende Reaktion ist, sechzehn Tickets anzulegen, sie nach Schweregrad zu sortieren und oben anzufangen. So wird es in vielen Organisationen gemacht, und so entsteht die Erfahrung aus Kapitel 1: Ein Jahr später steht dieselbe Art von Befund wieder im Bericht, nur an anderen Stellen.

Zwischen dem Lesen des Berichts und dem Beheben liegt ein Arbeitsschritt, der in den meisten Büchern über Pentesting fehlt, weil er nicht mehr Sache der Tester ist. Er heißt Triage, und er macht aus einer Liste von Befunden ein Bild der Lage: welche Befunde echt sind, welche zusammengehören, welche einander verstärken und was daraus für die Reihenfolge folgt. In der Spezialisierung ist das der erste Schritt der Case Study, und an ihm entscheidet sich, ob eine Gruppe danach 25 Einzelkorrekturen abliefert oder einen Plan.

**IN DIESEM KAPITEL**

- Triage definieren und ihre fünf Schritte nennen
- Befunde prüfen und die Arten falsch positiver Befunde unterscheiden
- Befunde aus einer Testumgebung auf die Produktion übertragen
- Befunde nach gemeinsamer Ursache bündeln und das Kriterium dafür angeben
- Ketten aus Befunden systematisch finden und bewerten
- ein Triage-Ergebnis in einer Form festhalten, mit der sich weiterarbeiten lässt

## 15.1 Was Triage ist

Das Wort kommt aus der Notfallmedizin und bezeichnet dort das Sichten vieler Verletzter, bevor jemand behandelt wird. Der Vergleich trägt weit: Es geht nicht darum, den Ersten in der Reihe zu versorgen, sondern zuerst alle anzusehen.

**DEFINITION 15.1 · TRIAGE**
Triage ist die Sichtung der Befunde eines Sicherheitstests mit dem Ziel, für jeden Befund festzustellen, ob er zutrifft, wie er im eigenen System zu bewerten ist, mit welchen anderen Befunden er eine Ursache oder einen Angriffspfad teilt, und in welcher Reihenfolge die daraus entstehenden Aufgaben anzugehen sind.

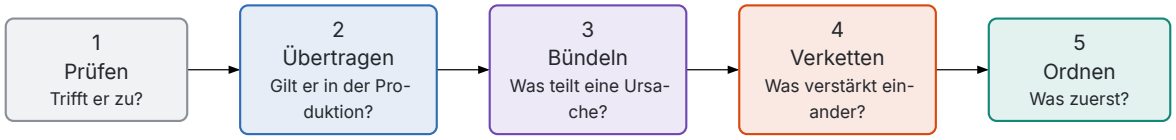



**Abb. 27:** Die fünf Schritte der Triage.

Die ersten beiden Schritte betrachten jeden Befund einzeln, die nächsten beiden die Befunde im Zusammenhang, der letzte fasst zusammen. Wer die Arbeit leistet, ist in der Praxis ein kleines Team aus jemandem mit Sicherheitswissen, jemandem, der die Architektur kennt, und jemandem aus der Entwicklung des betroffenen Teils. Allein geht es schlecht, weil niemand alle drei Sichten hat.

Triage endet mit einer Einschätzung, nicht mit einer Entscheidung. Was tatsächlich getan wird und was man hinnimmt, entscheidet am Ende jemand mit Verantwortung für Budget und Risiko. Davon handelt Kapitel 18.

## 15.2 Schritt 1: Trifft der Befund zu?

Ein Befund ist eine Behauptung mit Beleg, hieß es in Kapitel 13. Der erste Schritt prüft den Beleg. Am Ende bekommt jeder Befund einen von wenigen Zuständen.

**Tab. 50:** Zustände eines Befunds nach der Prüfung.

| Zustand | Bedeutung | Was folgt |
|---|---|---|
| bestätigt | der Nachweis wurde nachvollzogen, der Befund trifft zu | geht in die weiteren Schritte |
| bestätigt, anders bewertet | trifft zu, die eigene Bewertung weicht ab | eigene Bewertung mit Begründung festhalten |
| nicht nachvollziehbar | mit den Angaben nicht zu wiederholen | Rückfrage an die Tester, bleibt offen |
| falsch positiv | die Beobachtung stimmt nicht oder bedeutet nicht, was behauptet wird | mit Begründung schließen |
| Duplikat | dieselbe Stelle wie ein anderer Befund | zusammenführen |
| gewolltes Verhalten | die Beobachtung stimmt, das Verhalten ist so entschieden | Entscheidung belegen oder nachholen |

### Falsch positive Befunde

Ein falsch positiver Befund meldet eine Schwachstelle, die es nicht gibt. Es gibt vier Arten, und sie zu unterscheiden spart Streit.

Die erste ist die **ungeprüfte Werkzeugmeldung**. Ein Scanner schließt aus einer Versionsnummer auf eine Lücke, aber die Version ist vom Hersteller der Distribution nachträglich korrigiert worden, ohne dass sich die Nummer geändert hat. Oder ein Werkzeug meldet eine Injection, weil eine Antwort anders aussah als erwartet.

Die zweite ist die **nicht erreichbare Stelle**. Die Komponente ist verwundbar, aber der betroffene Teil wird nicht verwendet. Das waren die ersten beiden der vier Fragen zu Komponenten aus Kapitel 10.

Die dritte ist die **falsch gedeutete Beobachtung**. Die Tester sehen in einer Antwort Daten, die sie für fremde halten, die aber öffentlich oder dem Testkonto zugeordnet sind.

Die vierte ist keine: das **gewollte Verhalten**. Die Tester melden, dass Kassenkräfte die Buchungen aller Kunden ihrer Therme sehen. Das ist so vorgesehen. Solche Befunde verdienen trotzdem einen zweiten Blick, denn „das ist so gewollt“ ist eine Behauptung, die sich belegen lassen sollte. Gibt es eine Anforderung oder eine dokumentierte Entscheidung dazu? Wenn niemand sie findet, hat der Befund eine Lücke in der Dokumentation aufgedeckt, und manchmal eine Entscheidung, die nie jemand getroffen hat.

> **FALLSTRICK**
> „Falsch positiv“ ist das bequemste Urteil, das eine Entwicklung über einen Befund fällen kann, und es wird zu oft gefällt. Die Beweislast liegt bei dem, der schließen will. Ein Befund wird nicht geschlossen, weil jemand sagt, das könne nicht sein, sondern weil jemand zeigt, warum der Nachweis nicht trägt. Das Umgekehrte gilt ebenso: Ein Befund wird nicht dadurch echt, dass er im Bericht steht.

### Nachvollziehen

Den Nachweis nachzuvollziehen heißt, die beschriebenen Schritte in der Testumgebung zu wiederholen, mit einem eigenen Testkonto und im Rahmen dessen, was der Betreiber in seinem eigenen System darf. Das ist keine Wiederholung des Pentests. Es ist die Prüfung, ob die Beschreibung stimmt und ob das Team verstanden hat, was gemeint ist.

Der Nutzen geht über die Prüfung hinaus. Wer einen Befund selbst nachvollzogen hat, versteht ihn besser als nach dem Lesen, sieht meist sofort, wo im Code die Ursache sitzt, und hat den Test in der Hand, mit dem sich später die Behebung prüfen lässt. In Kapitel 17 wird aus diesem Nachvollziehen der Regressionstest.

Für *Kurbad* ergibt die Prüfung, dass fünfzehn Befunde zutreffen. Bei F-12 beantwortet das Team die vier Fragen aus Kapitel 10: Die betroffene Funktion der Bibliothek wird verwendet, und zwar für Profilbilder, die Kunden hochladen. Der Befund trifft also zu, und er ist von außen erreichbar. Offen bleibt F-02.

## 15.3 Schritt 2: Gilt der Befund in der Produktion?

Der Test lief in einer Testumgebung. Kapitel 2 hat gewarnt, dass sie der Produktion selten genau gleicht. Für jeden Befund ist deshalb zu fragen, was in der Produktion anders ist. Es gibt drei mögliche Antworten.

**Dasselbe.** Fehler im Code und in der Architektur sind in beiden Umgebungen gleich, wenn derselbe Stand läuft. F-03, F-07 und F-08 gehören dazu.

**In der Produktion besser.** Dort gibt es eine Schutzschicht, die in der Testumgebung fehlt. Bei F-02, den unbegrenzten Anmeldeversuchen, sagt der Betrieb, dass in der Produktion das API-Gateway die Anfragen je Adresse begrenzt. Das ändert die Bewertung, sofern es stimmt. Es ist zu belegen, etwa mit der Konfiguration des Gateways, und es ändert nichts daran, dass der Service selbst keine Begrenzung hat. Der Befund wird damit zu einem Hinweis auf eine Maßnahme, die an einer einzigen Stelle hängt.

**In der Produktion schlechter.** Die Testumgebung enthält erzeugte Daten, die Produktion echte. Die Testumgebung hat drei Instanzen, die Produktion dreißig, mit entsprechend mehr Gelegenheit für Wettläufe wie bei F-09. Und Konfiguration, Zugangsdaten und Netzregeln der Produktion hat niemand getestet. Dass in der Testumgebung keine Fehlkonfiguration der Cloud-Plattform gefunden wurde, sagt über die Produktion nichts.

> **MERKE**
> Ein Befund aus der Testumgebung gilt für die Produktion, bis das Gegenteil belegt ist. „In der Produktion ist das anders“ ist eine Aussage, die einen Beleg braucht wie jeder Befund.

## 15.4 Schritt 3: Bündeln

Nach den ersten beiden Schritten liegt eine geprüfte Liste vor. Jetzt beginnt die eigentliche Denkarbeit. Sechzehn Befunde sind nicht sechzehn Probleme.

Es gibt drei Arten, wie Befunde zusammengehören können, und sie werden oft verwechselt.

Ein **Duplikat** ist dieselbe Schwäche an derselben Stelle, zweimal gemeldet, etwa einmal über die Web-Oberfläche und einmal über die App, die beide denselben Endpunkt verwenden.

Dieselbe **Klasse** liegt vor, wenn dieselbe Fehlerart an verschiedenen Stellen auftritt: eine fehlende Besitzprüfung bei den Rechnungen und eine bei den Buchungen. Tester fassen solche Fälle manchmal zu einem Befund mit mehreren Orten zusammen, manchmal melden sie jeden einzeln. Deshalb sagt die Zahl der Befunde so wenig.

Dieselbe **Ursache** liegt vor, wenn verschiedene Fehlerarten auf dieselbe Entscheidung oder dasselbe Versäumnis zurückgehen. Das ist die wertvollste Art der Zusammengehörigkeit und die am schwersten zu sehende.

> **DEFINITION 15.2 · BÜNDEL**
> Ein Bündel ist eine Gruppe von Befunden, die durch eine gemeinsame Maßnahme behoben oder künftig verhindert werden können. Das Kriterium ist die gemeinsame Behebung, nicht die gemeinsame Fehlerart.

Das Kriterium in der Definition ist mit Bedacht gewählt. Die CWE-Hierarchie aus Kapitel 4 hilft beim Bündeln, aber nur ein Stück weit. F-03 und F-04, die SQL-Injection und das Cross-Site Scripting, haben mit `CWE-74` einen gemeinsamen Vorfahren, die Injection im weiten Sinn. F-07 und F-08, der Abruf fremder Rechnungen und die veränderbaren Bonuspunkte, stehen im Katalog in verschiedenen Ästen, obwohl beide dieselbe Ursache haben: Die Anwendung prüft nicht, ob dieser Benutzer dieses Objekt und dieses Feld anfassen darf. Der Katalog ordnet nach der Art des Fehlers im Code. Gebündelt wird nach der Stelle, an der man ihn verhindert.

Für *Kurbad* ergeben sich sechs Bündel.

**Tab. 51:** Die sechzehn Befunde zu *Kurbad* in sechs Bündeln.

| | Bündel | Befunde | Gemeinsame Ursache |
|---|---|---|---|
| A | Zugriffskontrolle je Objekt und Feld | F-07, F-08 | Berechtigung wird in einzelnen Endpunkten geprüft oder vergessen, es gibt keine zentrale Stelle |
| B | Eingaben erreichen eine Senke | F-03, F-04, F-05 | fremde Daten werden ohne Trennung, Kodierung oder Erlaubnisliste weiterverarbeitet |
| C | Vertrauen im Inneren | F-10, F-11, F-16 | das interne Netz gilt als sicher, Services und Konten haben mehr Rechte als nötig |
| D | Anmeldung und Sitzung | F-01, F-02, F-06 | Schutz der Anmeldung ist je Oberfläche verschieden gelöst, Tokens lassen sich nicht widerrufen |
| E | Betrieb und Pflege | F-12, F-13, F-14, F-15 | keine Vorgaben für Konfiguration, keine laufende Prüfung von Abhängigkeiten, keine Auswertung von Protokollen |
| F | Fachlogik | F-09 | Zustandsübergang ohne Absicherung gegen Gleichzeitigkeit |

Aus sechzehn Befunden sind sechs Themen geworden, und die Themen lesen sich anders als die Liste. Bündel A ist ein Thema für die Architektur der Services. Bündel C ist ein Thema für Plattform und Betrieb. Bündel E ist ein Thema für den Entwicklungsprozess. Die Befundübersicht mit ihren Schweregraden hat das nicht gezeigt.

Drei Hinweise für das Bündeln in der Praxis.

**Prüfe das Bündel mit der Frage nach der Maßnahme.** Gibt es eine Maßnahme, die alle Mitglieder behebt oder ihr Wiederauftreten verhindert? Für Bündel A ist das eine zentral erzwungene Besitzprüfung. Für Bündel B ist die Antwort weniger glatt: Die drei Befunde brauchen nach Kapitel 8 drei verschiedene Gegenmaßnahmen, nämlich Parametrisierung, Kodierung und eine Erlaubnisliste. Gemeinsam ist ihnen die Prüfregel und das Review, das solche Stellen findet. Das Bündel trägt auf der Ebene Lifecycle, nicht auf der Ebene Code.

**Ein Befund kann in zwei Bündel gehören.** F-16, das Datenbankkonto mit zu weiten Rechten, steht bei C. Man könnte es auch als Verstärker von F-03 zu B stellen. Entscheide dich für das Bündel, in dem die Behebung liegt, und halte die Verbindung zum anderen als Kette fest.

**Ein Bündel mit einem Mitglied ist erlaubt.** F-09 steht allein. Das kann heißen, dass es ein einzelner Fehler ist, oder dass die Tester für die Fachlogik wenig Zeit hatten. Die Abdeckungsmatrix aus Kapitel 13 legt das Zweite nahe.

> **AUS DER PRAXIS**
> Der häufigste Fehler beim Bündeln ist, es nach dem Schweregrad zu tun: alle kritischen und hohen in den ersten Korb, die mittleren in den zweiten. Das fühlt sich nach Priorisierung an und zerreißt genau die Zusammenhänge, um die es geht. F-08 mit der Stufe *mittel* und F-07 mit der Stufe *hoch* würden von verschiedenen Leuten zu verschiedenen Zeiten behoben, jede Stelle für sich, und die gemeinsame Ursache bliebe.

## 15.5 Schritt 4: Verketten

Bündel fragen, was Befunde gemeinsam *haben*. Ketten fragen, was Befunde gemeinsam *ermöglichen*.

> **DEFINITION 15.3 · KETTE**
> Eine Kette ist eine Folge von Befunden, bei der die Ausnutzung des einen die Voraussetzung für die Ausnutzung des nächsten schafft, sodass am Ende eine Auswirkung steht, die keiner der Befunde allein hat.

Der Bericht zu *Kurbad* hat zwei Ketten beschrieben, die Abb. 23 gezeigt hat. Tester finden nicht alle Ketten, weil ihre Zeit begrenzt ist und weil sie nach dem ersten Erfolg oft nicht nach einem zweiten Weg suchen. Wer das System kennt, kann weitere finden, und zwar am Schreibtisch.

### Was ein Befund braucht und was er liefert

Das Verfahren ist einfach. Für jeden Befund hältst du zwei Dinge fest: was ein Angreifer braucht, um ihn auszunutzen, und was er danach hat. Das erste steht im Bericht unter Voraussetzungen und im Vektor unter AV, PR und UI. Das zweite steht unter Auswirkung.

**Tab. 52:** Voraussetzungen und Ergebnis ausgewählter Befunde.

| Nr. | Braucht | Liefert |
|---|---|---|
| F-13 | Zugang aus dem Internet | Wissen über Aufbau, Versionen, Namen von Tabellen |
| F-03 | Kundenkonto | Lesen und Schreiben mit den Rechten des Datenbankkontos |
| F-16 | Zugriff mit dem Datenbankkonto des Buchungsservice | Zugriff auf die Daten aller Services |
| F-05 | Kundenkonto | Anfragen aus dem Inneren des Netzes |
| F-10 | Anfragen aus dem Inneren des Netzes | Aufruf beliebiger Funktionen der Services ohne Anmeldung |
| F-11 | Lesezugriff auf das Abbild oder den laufenden Service | Zugangsdaten des Brokers, damit Lesen und Senden von Nachrichten |
| F-04 | Kundenkonto, ein Mitarbeiter öffnet die Buchung | Handeln im Browser des Mitarbeiters in der Admin-Oberfläche |
| F-06 | ein einmal erlangtes Token | 30 Tage Zugang mit den Rechten des Kontos |
| F-07 | Kundenkonto | Rechnungen aller Kunden |

Eine Kette entsteht überall dort, wo das, was ein Befund liefert, das ist, was ein anderer braucht. F-05 liefert Anfragen aus dem Inneren, F-10 braucht genau das. F-03 liefert Zugriff mit dem Datenbankkonto, F-16 macht daraus Zugriff auf alles. Das sind die beiden Ketten aus dem Bericht.

Die Tabelle zeigt zwei weitere. F-04 liefert das Handeln im Browser eines Mitarbeiters, der an der Admin-Oberfläche angemeldet ist. Das gespeicherte Skript im Bemerkungsfeld einer Buchung wird ausgeführt, wenn eine Kassenkraft oder ein Administrator die Buchung öffnet. Der Bericht hat F-04 mit 5,4 als *mittel* bewertet, mit geringer Auswirkung auf Vertraulichkeit und Integrität. Im Kontext der Admin-Oberfläche ist die Auswirkung alles, was dieser Mitarbeiter darf. Mit hoher Auswirkung auf beide ergibt derselbe Vektor 8,7. Und wenn dabei ein Token abgegriffen werden kann, sorgt F-06 dafür, dass es dreißig Tage gilt.

Die zweite betrifft F-13. Die ausführlichen Fehlermeldungen liefern nichts als Wissen. Aber dieses Wissen verkürzt den Weg zu F-03 erheblich, weil der Angreifer nicht raten muss, wie die Datenbank aufgebaut ist.

### Der Angriffsbaum vom Ziel her

Kapitel 6 hat den Angriffsbaum als Werkzeug der Bedrohungsmodellierung vorgestellt: oben ein Ziel des Angreifers, darunter die Wege dorthin. Bei der Triage wird er in umgekehrter Richtung nützlich. Du nimmst ein Ziel, das für die Organisation besonders schmerzhaft wäre, und trägst ein, über welche Befunde es erreichbar ist.

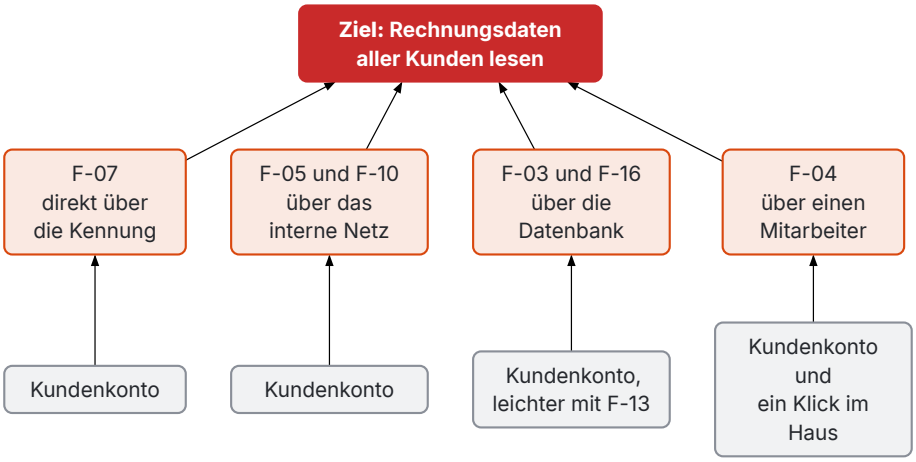



**Abb. 28:** Angriffsbaum für ein Ziel bei *Kurbad*. Jeder der vier Äste führt für sich zum Ziel. Grau steht, was der Angreifer dafür braucht.

Abb. 28 enthält die wichtigste Einsicht dieses Kapitels. Das Ziel ist auf vier voneinander unabhängigen Wegen erreichbar. Wer F-07 behebt, den auffälligsten und einfachsten, hat einen von vier Wegen geschlossen. Die Rechnungsdaten sind danach so erreichbar wie vorher, nur mit etwas mehr Aufwand. Eine Liste, die nach Schweregrad sortiert ist, zeigt das nicht. Sie würde nach F-03 und einigen hohen Befunden F-04 und F-16 weit hinten einreihen.

Daraus folgt eine andere Art zu priorisieren: nicht nach Befunden, sondern nach Zielen. Die Frage lautet, welche Befunde behoben sein müssen, damit ein bestimmtes Ziel nicht mehr erreichbar ist. Für das Ziel in der Abbildung sind es mindestens vier, aus jedem Ast einer.

### Welches Glied man bricht

Eine Kette ist unterbrochen, wenn ein Glied fehlt. Bei der Kette aus F-05 und F-10 genügt es rechnerisch, einen der beiden Befunde zu beheben. Welchen?

Drei Überlegungen helfen. Die erste ist die **Reichweite**: Ein Glied, das in mehreren Ketten vorkommt, zu beheben, unterbricht alle auf einmal. F-10, das Vertrauen im internen Netz, ist ein solches Glied. Jeder künftige Weg ins Innere, nicht nur F-05, führt daran vorbei. Die zweite ist der **Aufwand**: F-05 ist eine Korrektur an einer Stelle, F-10 ein Umbau der Kommunikation zwischen den Services. Die dritte ist die **Tiefe**: Wer nur ein Glied bricht, verlässt sich darauf, dass es nie wieder entsteht.

Die übliche Antwort ist deshalb beides, in verschiedener Geschwindigkeit: das billige Glied sofort, das weitreichende geplant. Das ist die Unterscheidung zwischen Sofortmaßnahme und Ursachenbehebung, die Kapitel 17 ausführt.

### Wie man eine Kette bewertet

Für die Bewertung einer Kette gibt es keine anerkannte Formel, wie Kapitel 14 schon angemerkt hat. Als Faustregel taugt: Die Kette bekommt die Voraussetzungen ihres ersten Glieds und die Auswirkung ihres letzten. Die Kette aus F-05 und F-10 braucht ein Kundenkonto und führt zum Lesen und Verändern von Daten der Abrechnung. Als ein Befund betrachtet wäre das kritisch.

Jedes Glied einer Kette erbt deren Dringlichkeit, solange die Kette besteht. F-16 ohne eigenen CVSS-Wert gehört damit zu den dringenden Befunden, solange F-03 offen ist, und umgekehrt bleibt nach der Behebung von F-03 ein Befund, der weiter gilt, aber warten kann.

## 15.6 Schritt 5: Ordnen

Der letzte Schritt fasst zusammen, was die ersten vier ergeben haben. Das Ergebnis ist keine sortierte Liste von Befunden mehr, sondern eine Übersicht der Bündel mit ihren Ketten und einer ersten Einschätzung der Dringlichkeit.

**Tab. 53:** Ergebnis der Triage für *Kurbad*.

| | Bündel | Lage | Einschätzung |
|---|---|---|---|
| B | Eingaben erreichen eine Senke | enthält den kritischen Befund F-03 und den Einstieg in zwei Ketten (F-05, F-04); von außen mit Kundenkonto erreichbar | sofort an den Stellen, Ursache geplant |
| A | Zugriffskontrolle je Objekt und Feld | F-07 von jedem Kunden auffindbar, Daten mit hohem Schutzbedarf; weitere Stellen derselben Klasse wahrscheinlich | sofort an den Stellen, Ursache als Nächstes |
| C | Vertrauen im Inneren | macht aus jedem Einbruch in einen Service einen Einbruch in alle; Glied in zwei Ketten | geplant, mit hoher Priorität |
| D | Anmeldung und Sitzung | Admin-Oberfläche aus dem Internet erreichbar; F-06 verlängert die Wirkung von F-04 | F-01 sofort, Rest geplant |
| E | Betrieb und Pflege | F-12 von außen erreichbar; F-14 lässt alle anderen Angriffe unbemerkt | F-12 sofort, Rest als Prozess |
| F | Fachlogik | begrenzter Schaden je Vorfall, wiederholbar | geplant, Fachlogik nachtesten lassen |

Vergleiche Tab. 53 mit der Befundübersicht in Tab. 42. Die Übersicht hätte zu sechzehn Tickets geführt, davon eines kritisch und sechs hoch. Die Triage führt zu sechs Themen, in denen eine Handvoll Sofortmaßnahmen und sechs Ursachen stecken, und zu einer Aussage, die in der Übersicht nicht zu finden war: dass die mittleren Befunde F-04 und F-16 zu den dringenden gehören und dass die Fachlogik noch nicht ausreichend geprüft ist.

Zum Ergebnis gehört auch, was offen geblieben ist: die Rückfragen an die Tester, die Belege, die der Betrieb für F-02 noch liefern muss, und die Bereiche, in denen nach weiteren Stellen derselben Klasse gesucht werden soll.

## 15.7 Die Suche nach weiteren Stellen

Dieser letzte Punkt verdient einen eigenen Abschnitt, weil er aus der Triage eine Aufgabe für die Entwicklung macht, die über den Bericht hinausgeht.

Ein Pentest ist eine Stichprobe. Wenn die Tester in acht Tagen eine fehlende Besitzprüfung bei den Rechnungen gefunden haben, ist die Annahme, es sei die einzige, unbegründet. Die Tester haben nicht alle Endpunkte geprüft, und die Entwickler, die an dieser Stelle die Prüfung vergessen haben, haben an anderen Stellen unter denselben Bedingungen gearbeitet.

Für jedes Bündel, das auf eine Klasse von Fehlern zurückgeht, folgt deshalb eine Suche im eigenen Code nach derselben Art von Stelle. Das Team hat dafür Mittel, die den Testern fehlten: den Quellcode, die Kenntnis aller Endpunkte, statische Analyse mit einer eigens geschriebenen Regel. Für Bündel A heißt das, alle Endpunkte aufzulisten, die ein Objekt über eine Kennung ansprechen, und bei jedem nachzusehen, ob der Besitz geprüft wird. Die Ergebnisse dieser Suche sind eigene Befunde, die im Bericht nicht stehen, und sie sind oft zahlreicher als die des Berichts.

**BRÜCKE ZU SOFTWARE SECURITY**

Die Tabelle mit „braucht“ und „liefert“ ist eine Analyse von Vor- und Nachbedingungen, wie du sie aus der Spezifikation von Operationen kennst, nur für Angriffe. Und das Bündeln nach der Stelle, an der man einen Fehler verhindert, ist die Frage nach der Zuständigkeit im Entwurf: Welche Komponente ist für diese Eigenschaft verantwortlich? Wenn die Antwort „jeder Endpunkt für sich“ lautet, ist die Eigenschaft über das System verstreut, und verstreute Zuständigkeiten sind in deinen Folien zur Modularität der Grund für Fehler bei jeder Änderung. Für die Sicherheit gilt dasselbe.

**FRAGE AUS DEM HÖRSAAL**

***Ist Triage nicht einfach eine höfliche Art, Befunde wegzudiskutieren?***

Sie kann dazu missbraucht werden, und deshalb liegt die Beweislast bei dem, der einen Befund schließen oder abstufen will. In einer ehrlichen Triage wird die Lage in der Regel ernster und nicht harmloser, weil Ketten und gemeinsame Ursachen sichtbar werden, die im Bericht nicht standen. Bei *Kurbad* wurde ein Befund in Frage gestellt, und zwei wurden dringender.

**FRAGE AUS DEM HÖRSAAL**

***Woher weiß ich, ob ich alle Ketten gefunden habe?***

Du weißt es nicht. Die Tabelle mit Voraussetzungen und Ergebnissen findet die Ketten zwischen den Befunden, die im Bericht stehen. Ketten, die über eine Schwäche führen, die niemand gefunden hat, findet sie nicht. Deshalb ist die Behebung der Ursachen wichtiger als das Durchtrennen der bekannten Ketten: Ein internes Netz, in dem Services einander prüfen, hält auch gegen den Weg, den noch keiner kennt.

**FRAGE AUS DEM HÖRSAAL**

***Wer sollte die Triage machen, die Sicherheitsabteilung oder die Entwicklung?***

Beide zusammen, und jemand mit Überblick über die Architektur dazu. Die Sicherheitsabteilung allein kann Befunde bewerten, aber nicht sagen, wo im System die Ursache sitzt. Die Entwicklung allein kennt den Code, neigt aber dazu, jeden Befund als einzelne Korrektur zu sehen. Die Bündel entstehen im Gespräch zwischen beiden. In der Lehrveranstaltung übernimmt die Gruppe alle drei Rollen, und es lohnt sich, sie ausdrücklich zu verteilen.

**FÜR DIE LEHRE**

Triage ist der Schritt, den KI-Assistenten am schlechtesten können und bei dem sie am überzeugendsten wirken. Ein Assistent gruppiert Befunde bereitwillig, und zwar nach der Ähnlichkeit der Texte. Er findet die Injection-Befunde zusammen, weil sie ähnlich beschrieben sind, und übersieht, dass F-16 zu F-03 gehört, weil die Texte nichts gemeinsam haben. Ketten, die er vorschlägt, klingen plausibel und setzen manchmal Verbindungen voraus, die es in der Architektur nicht gibt. Und er zweifelt keinen Befund an, den der Bericht behauptet.

Baue die Case Study deshalb so, dass die naheliegende Gruppierung falsch ist. Dafür eignen sich: ein Befund mit niedriger Stufe, der das fehlende Glied einer Kette ist; zwei Befunde mit verschiedener CWE und derselben Ursache; zwei Befunde mit derselben CWE und verschiedener Ursache; ein falsch positiver Befund, der sich nur mit einem Blick in die Architekturbeschreibung erkennen lässt; und ein Befund, der in der Produktion wegen einer dokumentierten Schutzschicht anders zu bewerten ist.

Als Abgabe für diesen Schritt eignet sich genau die Form aus diesem Kapitel: die Tabelle der Zustände, die Bündel mit Begründung, ein Angriffsbaum für ein vorgegebenes Ziel und das geordnete Ergebnis. In der mündlichen Verteidigung fragst du nach einem Bündel: „Welche eine Maßnahme behebt alle Mitglieder?“ Wer nach Textähnlichkeit gebündelt hat, kann das nicht beantworten.

Typische Fehlvorstellungen: dass Bündeln dasselbe sei wie Sortieren nach Schweregrad, dass eine unterbrochene Kette ein behobenes Problem sei, und dass niedrige Befunde warten können.

## Selbsttest

1. Nenne die fünf Schritte der Triage und sage, welche den einzelnen Befund betrachten und welche den Zusammenhang.
2. Welche vier Arten falsch positiver Befunde gibt es, und welche davon ist keine?
3. Bei wem liegt die Beweislast, wenn ein Befund als falsch positiv geschlossen werden soll?
4. Der Betrieb sagt zu einem Befund: „In der Produktion gibt es dafür eine Schutzschicht.“ Was verlangst du?
5. Grenze Duplikat, Klasse und Ursache voneinander ab.
6. Was ist das Kriterium für ein Bündel, und warum ist es nicht die gemeinsame CWE?

7. Wie findest du Ketten am Schreibtisch? Beschreibe das Verfahren.
8. Das Ziel „Rechnungsdaten aller Kunden lesen“ ist bei *Kurbad* auf vier Wegen erreichbar. Was folgt daraus für die Behebung von F-07?
9. Nach welcher Regel bewertest du eine Kette, und was folgt für ihre Glieder?
10. Warum folgt aus einem bestätigten Befund eine Suche nach weiteren Stellen?

# 16 Ursachenanalyse auf drei Ebenen

Im Praxiskasten von Kapitel 1 stand eine Geschichte in drei Jahren: fremde Rechnungen, dann fremde Gutschriften, dann fremde Mahnungen. Jedes Mal wurde an der gemeldeten Stelle eine Prüfung ergänzt, jedes Mal wurde das Ticket geschlossen, und jedes Mal war die Arbeit sauber gemacht. Trotzdem war nach drei Jahren nichts gelöst.

Dieses Kapitel ist das Herz von Teil IV und, wenn man so will, des ganzen Buchs. Es behandelt die Frage, die zwischen dem Befund und seiner Behebung steht: *Warum* gibt es diese Schwäche? Die Antwort hat immer mehrere Schichten, und je nachdem, auf welcher man ansetzt, behebt man eine Stelle, eine Klasse oder eine Gewohnheit. Du lernst ein Raster mit drei Ebenen kennen, das seit dem Vorwort in den farbigen Marken mitläuft, und ein Verfahren, es auf einen Befund anzuwenden. Es ist auch das Raster, nach dem die Case Study der Spezialisierung bewertet wird.

**IN DIESEM KAPITEL**

- Symptom, Fehler und Ursache unterscheiden
- die drei Ebenen Code, Architektur und Lifecycle definieren und die Leitfrage jeder Ebene nennen
- einen Befund mit der Warum-Kette bis zur Ursache auf jeder Ebene zurückverfolgen
- die CWE-Hierarchie und die Praktiken des SSDF als Hilfsmittel der Ursachenanalyse verwenden
- begründen, wann eine Behebung auf nur einer Ebene nicht ausreicht
- eine Ursachenanalyse in einer prüfbaren Form festhalten

## 16.1 Symptom, Fehler, Ursache

Die Lehre vom Softwaretest unterscheidet drei Dinge, die im Alltag alle „Fehler“ heißen [7]. Ein Mensch begeht einen **Irrtum**. Dadurch entsteht im Produkt ein **Fehlerzustand**, etwa eine falsche Zeile. Wird sie ausgeführt, zeigt sich nach außen eine **Fehlerwirkung**. Dein Foliensatz zu Test und Evolution verwendet dieselbe Unterscheidung.

Auf einen Pentest übertragen heißt das: Der Befund beschreibt eine Fehlerwirkung, nämlich das, was der Tester von außen beobachtet hat. Dahinter steht ein Fehlerzustand im Code, in der Konfiguration oder im Entwurf. Und dahinter steht, warum dieser Zustand entstanden ist und warum ihn niemand bemerkt hat.

**Tab. 54:** Symptom, Fehler und Ursache.

| Begriff | Im Softwaretest | Beim Befund F-07 |
|---|---|---|
| Symptom | Fehlerwirkung: beobachtetes Fehlverhalten | ein Kunde kann die Rechnung eines anderen abrufen |
| Fehler | Fehlerzustand: die fehlerhafte Stelle im Produkt | im Endpunkt für Rechnungen fehlt die Prüfung des Besitzers |
| Ursache | warum der Fehlerzustand entstand und bestehen blieb | nichts im System erzwingt die Prüfung, und nichts im Prozess bemerkt ihr Fehlen |

Die Empfehlung im Bericht richtet sich fast immer auf die mittlere Zeile. Das ist folgerichtig, denn weiter können Tester von außen selten sehen. Die Ursachenanalyse ist der Schritt von der mittleren zur unteren Zeile.

Für Sicherheitsfehler gibt es eine ältere Unterscheidung, die in dieselbe Richtung zeigt. Gary McGraw trennt *Bugs*, also Fehler in der Umsetzung, von *Flaws*, Fehlern im Entwurf, und schätzt aus seiner Praxis, dass sich Sicherheitsprobleme etwa je zur Hälfte auf beide verteilen [43]. Ein Bug lässt sich im Code

finden und beheben. Ein Flaw ist im Code nicht zu sehen, weil der Code genau das tut, was entworfen wurde. Die drei Ebenen dieses Buchs führen diese Unterscheidung fort und ergänzen eine dritte.

## 16.2 Die drei Ebenen

**DEFINITION 16.1 · DIE DREI EBENEN DER URSACHE**
Die Ursache auf der Ebene **Code** ist der konkrete Fehlerzustand: die Zeile, die Abfrage, die Einstellung, die falsch ist. Die Ursache auf der Ebene **Architektur** ist die Struktur des Systems, die diesen Fehler möglich oder wahrscheinlich gemacht hat oder seine Auswirkung vergrößert. Die Ursache auf der Ebene **Lifecycle** ist der Umstand im Entwicklungs- und Betriebsprozess, der den Fehler entstehen ließ und nicht entdeckt hat.

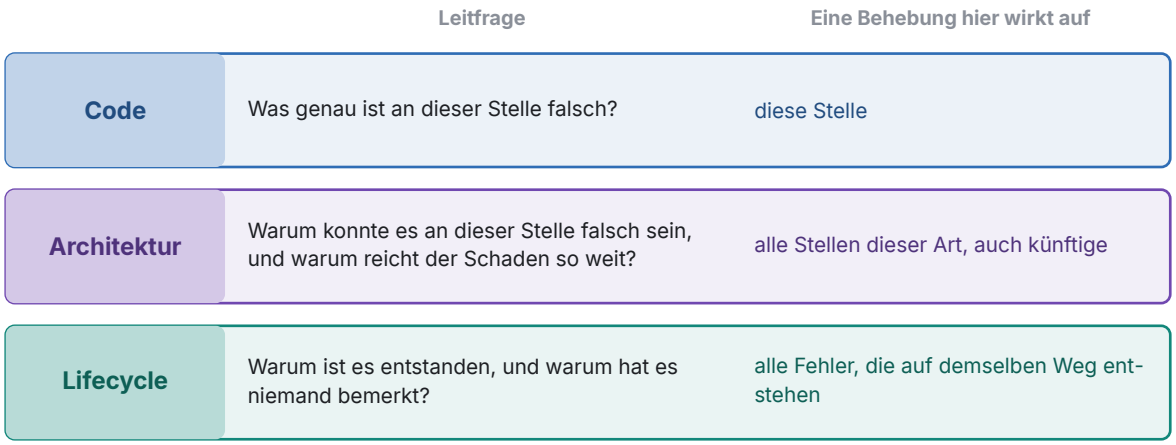



**Abb. 29:** Die drei Ebenen mit ihrer Leitfrage und der Reichweite einer Behebung.

Zwei Dinge an Abb. 29 sind wichtig.

Die Ebenen sind keine Schubladen, in die ein Befund einsortiert wird. **Jeder Befund hat auf jeder der drei Ebenen eine Ursache.** Die drei Leitfragen lassen sich immer stellen. Was sich von Befund zu Befund unterscheidet, ist, auf welcher Ebene die Antwort am meisten Gewicht hat und auf welcher eine Behebung am meisten bewirkt.

Die Reichweite wächst nach unten und der Aufwand mit ihr. Eine Korrektur im Code ist in Stunden gemacht und wirkt auf eine Stelle. Eine Änderung der Architektur dauert Wochen und wirkt auf eine Klasse. Eine Änderung im Prozess wirkt auf alles, was künftig gebaut wird, und zeigt ihre Wirkung erst nach Monaten. Keine der drei ersetzt die anderen.

## 16.3 Die Warum-Kette

Das einfachste Werkzeug der Ursachenanalyse stammt aus der Fertigung und ist unter dem Namen *Five Whys* bekannt: Man fragt so lange „Warum?“, bis die Antwort nicht mehr ein Ereignis ist, sondern ein Zustand, den man ändern kann. Die Zahl fünf ist eine Faustregel.

Am Befund F-07 sieht das so aus.

**Befund:** Ein Kunde kann fremde Rechnungen abrufen.
**Warum?** Der Endpunkt für Rechnungen prüft nicht, ob die Rechnung dem angemeldeten Kunden gehört. Code
**Warum prüft er das nicht?** Die Entwicklerin hat die Prüfung nicht eingebaut. Bei den Buchungen, die ein Kollege gebaut hat, ist sie vorhanden.
**Warum hing es an der einzelnen Entwicklerin?** Jeder Endpunkt lädt sein Objekt selbst über die Kennung aus der Datenbank. Die Besitzprüfung ist eine Zeile, die man dazuschreiben muss. Lässt man sie weg, funktioniert alles. Architektur
**Warum ist das so gebaut?** Bei der Aufteilung in Services wurde festgelegt, dass das API-Gateway die Anmeldung prüft. Was ein angemeldeter Benutzer sehen darf, wurde nicht als eigene Anforderung formuliert. Ein Bedrohungsmodell gab es nicht. Lifecycle

> **Warum ist es niemandem aufgefallen?** Die Tests prüfen jeden Endpunkt mit dem Konto, dem das Objekt gehört. Im Review wird auf das geachtet, was im Code steht, nicht auf das, was fehlt. Die statische Analyse kennt keine Besitzverhältnisse. Lifecycle

Die Kette durchläuft alle drei Ebenen, und an jeder Stelle hätte man aufhören können. Wer nach dem ersten Warum aufhört, ergänzt die Prüfung. Wer nach dem dritten aufhört, baut die Datenzugriffsschicht um. Wer bis zum Ende geht, ändert zusätzlich, wie Anforderungen formuliert und wie Tests geschrieben werden.

Drei Regeln machen die Warum-Kette brauchbar.

**Höre nicht bei einem Menschen auf.** „Die Entwicklerin hat es vergessen" ist nie eine Ursache, sondern die Stelle, an der die nächste Frage beginnt: Warum war es möglich, das zu vergessen, ohne dass etwas auffiel? Menschen vergessen Dinge. Ein System, das sich darauf verlässt, dass niemand etwas vergisst, ist falsch entworfen. Die zweite Antwort in der Kette oben wäre als Ende wertlos und als Durchgang nützlich: Sie zeigt mit dem Vergleich zu den Buchungen, dass es vom Zufall der Person abhing.

**Verzweige, wo es mehrere Antworten gibt.** Die Kette oben verzweigt am Ende in „warum entstanden" und „warum nicht bemerkt". Das sind zwei verschiedene Fragen mit verschiedenen Antworten, und beide gehören zur Ebene Lifecycle. Eine Kette, die immer nur einem Ast folgt, findet eine Ursache und hält sie für die einzige.

**Belege jede Antwort.** Jede Stufe ist eine Behauptung über das System oder den Prozess. „Die Tests prüfen nur mit dem Besitzer" lässt sich nachsehen. Eine Ursachenanalyse, die aus plausiblen Vermutungen besteht, ist eine Erzählung. Dieser Punkt wird in der Lehrveranstaltung wichtig, weil KI-Assistenten sehr plausible Erzählungen liefern.

## 16.4 Die CWE als Leiter

Kapitel 4 hat die Hierarchie der CWE beschrieben: unten die genauen Einträge der Stufen Variant und Base, darüber die allgemeineren der Stufe Class, ganz oben die Pillars. Für die Ursachenanalyse ist diese Hierarchie eine Leiter. Wer von der CWE eines Befunds aus nach oben steigt, kommt von der Stelle zur Klasse.

Für F-07 führt der Weg von `CWE-639`, der Umgehung der Autorisierung über einen vom Benutzer gesteuerten Schlüssel, über `CWE-863`, die fehlerhafte Autorisierung, und `CWE-285` zu `CWE-284`, der unzureichenden Zugriffskontrolle. Der unterste Eintrag sagt, was an der Stelle zu korrigieren ist. Die mittleren benennen das Thema, das in der Architektur zu lösen ist.

Die Katalogeinträge enthalten zwei weitere Angaben, die selten gelesen werden. Unter *Modes of Introduction* steht, in welcher Phase eine Schwäche typischerweise entsteht: in Architektur und Entwurf, in der Umsetzung oder im Betrieb. Unter *Potential Mitigations* stehen Gegenmaßnahmen, ebenfalls nach Phasen geordnet. Beides sind erste Hinweise darauf, auf welcher Ebene die Ursache am meisten Gewicht hat.

Und es gibt eine Sicht auf den Katalog, die für dich besonders nahe liegt. Die Sicht `CWE-1008` ordnet Schwächen nach den Sicherheitstaktiken der Architektur, die sie verletzen: Akteure authentifizieren, Akteure autorisieren, Eingaben validieren, Zugriff begrenzen und weitere [24]. Das ist dieselbe Denkweise wie in deinem Foliensatz zu den Architekturtaktiken [55]. Ein Befund lässt sich damit direkt auf die Taktik zurückführen, die im System fehlt oder lückenhaft umgesetzt ist.

> **MERKE**
> Die CWE eines Befunds nennt den Fehler. Der Weg nach oben in der Hierarchie nennt das Thema. Die verletzte Taktik nennt, was in der Architektur zu tun ist. Keine der drei Angaben ersetzt den Blick ins eigene System, aber zusammen geben sie der Analyse eine Richtung.

## 16.5 Wie die Architektur Fehler wahrscheinlich macht

Die mittlere Ebene ist die, die am schwersten zu sehen ist, weil dort nichts „falsch" ist. Eine Architektur enthält keine Sicherheitslücke in dem Sinn, in dem eine Zeile Code eine enthält. Sie macht Lücken

wahrscheinlich oder unwahrscheinlich, und sie bestimmt, wie weit ein Fehler reicht. Es gibt drei Arten, auf die sie das tut.

**Sie verteilt die Verantwortung.** Wenn eine Sicherheitseigenschaft an hundert Stellen von Hand hergestellt werden muss, wird sie an einigen fehlen. Das ist der Fall F-07. Das Gegenstück ist eine Stelle, an der die Eigenschaft für alle erzwungen wird und an der niemand vorbeikommt. Saltzer und Schroeder haben das 1975 als Prinzip der vollständigen Vermittlung formuliert: Jeder Zugriff auf jedes Objekt wird geprüft [41].

**Sie legt den bequemen Weg fest.** In jedem System gibt es eine Art, etwas zu tun, die am wenigsten Mühe macht. Ist der bequeme Weg der sichere, entstehen Fehler nur durch Absicht oder besondere Umstände. Ist der bequeme Weg der unsichere, entstehen sie durch Alltag. Ein Datenbankzugriff, bei dem Abfragen nur parametrisiert möglich sind, macht die SQL-Injection zur Ausnahme. Ein Zugriff, bei dem das Zusammensetzen von Text die kürzeste Schreibweise ist, macht sie zur Regel. Dasselbe gilt für Vorlagen, die von selbst kodieren, und für Voreinstellungen, die geschlossen statt offen sind.

**Sie bestimmt den Wirkungsradius.** Kapitel 11 hat das ausgeführt. Dass aus der SQL-Injection F-03 ein Zugriff auf die Daten aller Services wird, liegt nicht an der Injection, sondern an F-16: einem Datenbankkonto mit Rechten auf alles. Die Architektur hat hier keinen Fehler verursacht, sie hat einen vergrößert.

Die klassischen Entwurfsprinzipien für sichere Systeme lassen sich als Liste der Dinge lesen, die eine Architektur richtig machen kann. Jedes Bündel bei *Kurbad* verletzt mindestens eines.

**Tab. 55:** Entwurfsprinzipien für sichere Systeme und die Bündel aus Tab. 51, die sie verletzen. Die ersten Prinzipien gehen auf Saltzer und Schroeder zurück [41].

| Prinzip | Aussage | Verletzt in |
|---|---|---|
| Vollständige Vermittlung | jeder Zugriff auf jedes Objekt wird geprüft | A |
| Geringste Berechtigung | jeder Teil hat nur die Rechte, die er für seine Aufgabe braucht | C, D |
| Sichere Voreinstellung | ohne ausdrückliche Erlaubnis ist der Zugriff verwehrt | C, E |
| Trennung von Daten und Anweisungen | fremde Daten werden nie als Anweisung ausgewertet | B |
| Tiefenstaffelung | keine Sicherheitseigenschaft hängt an einer einzigen Maßnahme | C, D |
| Einfachheit des Mechanismus | Schutzmechanismen sind klein genug, um sie zu prüfen | A, D |
| Nachvollziehbarkeit | sicherheitsrelevante Vorgänge hinterlassen Spuren | E |

## 16.6 Wie der Lifecycle Fehler durchlässt

Die dritte Ebene fragt nach dem Prozess. Kapitel 5 hat ihn beschrieben und in Tab. 19 schon angedeutet, dass jeder Befund eine Rückmeldung an ihn ist. Hier wird daraus ein Verfahren.

Die Frage nach dem Lifecycle hat zwei Hälften, die getrennt zu beantworten sind.

**Warum ist der Fehler entstanden?** Mögliche Antworten: Es gab keine Anforderung dazu. Es gab kein Bedrohungsmodell, das die Frage aufgeworfen hätte. Die Entwickler kannten die Fehlerart nicht. Es gab keine Vorgabe, wie so etwas zu bauen ist. Die Vorgabe gab es, aber sie war unbequemer als der Fehler.

**Warum wurde er nicht bemerkt?** Mögliche Antworten: Kein Test prüft diese Eigenschaft. Das Review achtet nicht darauf. Die automatische Prüfung kann diese Fehlerart nicht erkennen oder ist nicht eingerichtet. Sie hat den Fehler gemeldet, und die Meldung ging in hundert anderen unter. Im Betrieb überwacht niemand die Anzeichen.

Für die zweite Hälfte ist das SSDF aus Kapitel 5 ein nützliches Raster, weil seine Praktiken aufzählen, welche Prüfungen es geben kann [44].

**Tab. 56:** Praktiken des SSDF als Fragen an einen Befund [44].

| Praktik | Inhalt | Frage an den Befund |
|---|---|---|
| PO.1 | Sicherheitsanforderungen festlegen | Gab es eine Anforderung, gegen die der Fehler verstößt? |
| PW.1 | sicher entwerfen, Risiken im Entwurf behandeln | Hätte ein Bedrohungsmodell die Frage gestellt? |
| PW.2 | den Entwurf prüfen | Hat jemand den Entwurf auf Sicherheit durchgesehen? |
| PW.4 | geprüfte Bausteine wiederverwenden | Gab es einen fertigen sicheren Baustein, und wurde er verwendet? |
| PW.5 | sicher programmieren | Gibt es Vorgaben für diese Art von Code, und sind sie bekannt? |
| PW.7 | Code prüfen und analysieren | Hätten Review oder statische Analyse den Fehler finden können? |
| PW.8 | das lauffähige Programm testen | Gibt es einen Test, der diese Eigenschaft prüft? |
| PW.9 | sichere Voreinstellungen | Ist die sichere Einstellung der Auslieferungszustand? |
| PS.3 | Auslieferungen und ihre Bestandteile festhalten | Ist bekannt, welche Komponenten in welcher Version laufen? |
| RV.1 | Schwachstellen laufend erkennen | Hätte eine laufende Prüfung den Fehler vor dem Pentest gemeldet? |

Für F-07 fallen die Antworten so aus: keine Anforderung (PO.1), kein Bedrohungsmodell (PW.1), kein Baustein für die Besitzprüfung (PW.4), kein Test mit einem zweiten Konto (PW.8). Für F-03 ganz anders: Es gibt eine Vorgabe für parametrisierte Abfragen, aber die statische Analyse läuft nur im Hauptzweig und ihre Meldungen blockieren nichts (PW.7). Für F-12: Niemand weiß, welche Bibliotheken eingesetzt sind (PS.3), und nichts prüft sie laufend (RV.1).

Die Antworten sehen für jeden Befund anders aus, und genau das ist ihr Wert. „Mehr Schulung“ und „mehr Tests“ sind keine Ergebnisse einer Ursachenanalyse. „Tests der Endpunkte verwenden nur ein Konto je Rolle“ ist eines.

Die Praktik RV.3 des SSDF verlangt genau diesen Gang: die Ursache jeder Schwachstelle ermitteln, über die Zeit nach Mustern suchen, die Software auf ähnliche Schwachstellen durchsehen und den Prozess so ändern, dass sie nicht wieder entstehen. Was dieses Kapitel beschreibt, ist also keine Besonderheit dieses Buchs, sondern anerkannte Praxis mit einer Kennung, auf die du dich berufen kannst.

## 16.7 Die Ursachen bei Kurbad

Mit den drei Leitfragen lassen sich die sechs Bündel aus Kapitel 15 durchgehen. Die Tabelle nennt Ursachen, noch keine Maßnahmen. Die folgen in Kapitel 17.

**Tab. 57:** Ursachen der sechs Bündel auf drei Ebenen.

| Bündel | Code | Architektur | Lifecycle |
|---|---|---|---|
| A<br>Zugriff je Objekt und Feld | Besitzprüfung fehlt bei Rechnungen; Profil übernimmt alle gesendeten Felder | jeder Endpunkt lädt Objekte selbst; keine Stelle erzwingt Besitz und erlaubte Felder | keine Anforderung zur Autorisierung je Objekt; Tests nur mit dem Besitzer |
| B<br>Eingaben erreichen eine Senke | Abfrage als Text zusammengesetzt; Bemerkung unkodiert ausgegeben; Adresse ungeprüft abgerufen | Datenzugriff erlaubt rohen Text; Admin-Oberfläche kodiert nicht von selbst; ausgehende Verbindungen unbeschränkt | statische Analyse blockiert nicht; Admin-Oberfläche von Prüfungen ausgenommen |
| C<br>Vertrauen im Inneren | Services prüfen keinen Absender; Zugangsdaten im Abbild; ein Konto für alle Schemata | Netzgrenze als einzige Grenze; keine Identität je Service; keine Verwaltung von Geheimnissen | Vertrauensgrenzen nie festgelegt; Betrieb und Entwicklung haben je angenommen, der andere sichere ab |
| D<br>Anmeldung und Sitzung | Admin-Anmeldung ohne zweiten Faktor; keine Begrenzung; lange Laufzeit | Admin-Oberfläche hat eigene Anmeldung statt des Identitätsdienstes; Tokens ohne Widerruf | Admin-Oberfläche als internes Werkzeug entstanden und nie neu bewertet |
| E<br>Betrieb und Pflege | alte Bibliothek; Fehlerdetails in Antworten; fehlende Header; keine Protokollierung von Ablehnungen | keine gemeinsame Grundkonfiguration der Services; Protokollierung je Service verschieden | keine Stückliste, keine laufende Prüfung, niemand wertet Protokolle aus |
| F<br>Fachlogik | Prüfen und Entwerten des Gutscheins sind zwei getrennte Schritte | kein Muster für Zustandsübergänge unter Gleichzeitigkeit | Missbrauchsfälle fehlen in Anforderungen und Tests |

Lies Tab. 57 einmal spaltenweise. Die Spalte Code ist eine Liste von sechzehn Stellen. Die Spalte Architektur enthält sechs Aussagen über das System, und zwei davon wiederholen sich in anderen Worten: Es gibt keine Stelle, die etwas erzwingt, und es gibt keine gemeinsame Grundlage, auf der alle Services aufbauen. Die Spalte Lifecycle enthält im Kern drei Aussagen: Sicherheitsanforderungen fehlen, die Tests prüfen nur den guten Fall, und die Admin-Oberfläche läuft außerhalb aller Regeln.

Aus sechzehn Befunden sind über sechs Bündel zwei Architekturthemen und drei Prozessthemen geworden. Das ist die Verdichtung, auf die die Triage und dieses Kapitel hinarbeiten, und sie ist der Inhalt, den eine Leitung versteht und über den sie entscheiden kann.

Eine Zeile verdient einen besonderen Hinweis. Bei Bündel C steht in der Spalte Lifecycle, dass Betrieb und Entwicklung je angenommen haben, der andere sichere ab. Die Entwicklung baute Services ohne Prüfung des Absenders, weil das Netz als geschützt galt. Der Betrieb ließ das Netz flach, weil die Services sich selbst schützen sollten. Solche Lücken zwischen zwei Verantwortungen sind eine typische Ursache in verteilten Systemen, und sie sind in keinem Code zu sehen.

## 16.8 Nur im Code, nur in der Architektur

Jeder Befund hat auf jeder Ebene eine Ursache. Daraus folgt nicht, dass jede Ebene gleich gut als Ansatz für die Behebung taugt. Für die Spezialisierung sind die Fälle lehrreich, in denen eine Ebene allein nicht reicht oder gar nicht greift.

**Tab. 58:** Befunde, bei denen eine Ebene allein nicht genügt. Ein Strich heißt, dass diese Ebene hier ausreicht oder keinen Beitrag leistet.

| Nr. | Warum Code allein nicht reicht | Warum Architektur allein nicht reicht |
|---|---|---|
| F-07 | die Prüfung an dieser Stelle behebt die nächste Stelle nicht | eine zentrale Prüfung muss wissen, wem ein Objekt gehört, und das steht im Code des Service |
| F-10 | es gibt keine einzelne Stelle; jeder Service müsste jeden Aufruf prüfen, ohne dass es Identitäten gäbe | Identitäten und Regeln lösen es, aber jeder Service muss sie auch auswerten |
| F-06 | die Laufzeit zu kürzen mildert nur; ein Token ohne Zustand lässt sich nicht widerrufen | ein Widerruf braucht eine Entwurfsentscheidung, die Umsetzung liegt dann in jedem Service |
| F-09 | - | kein Umbau hilft, wenn Prüfen und Entwerten zwei Schritte bleiben; hier muss die Operation selbst unteilbar werden |
| F-03 | - | geringere Rechte des Datenbankkontos begrenzen den Schaden, die Injection bleibt |
| F-12 | die Aktualisierung behebt diese Bibliothek, nicht die nächste veraltete | - |

Drei Muster stecken in Tab. 58.

Bei F-10 und F-06 gibt es im Code **keine Stelle**, an der sich der Fehler beheben ließe, weil der Fehler eine fehlende Fähigkeit des Systems ist. Ein Token, das nach seinem Entwurf bis zum Ablauf gilt, lässt sich nicht durch eine Zeile widerrufbar machen. Wer hier auf Code-Ebene sucht, findet nur Abmilderungen.

Bei F-09 und F-03 ist es umgekehrt. Der Fehler sitzt an einer Stelle und muss dort behoben werden. Die Architektur kann den Schaden begrenzen, und das soll sie auch, aber sie kann den Fehler nicht ersetzen. Wer einer SQL-Injection nur mit einer Schutzschicht davor und engeren Rechten dahinter begegnet, hat sie eingedämmt und nicht behoben.

Bei F-12 liegt das Gewicht ganz auf dem Lifecycle. Im Code ist nichts falsch, und die Architektur trägt nur insofern bei, als sie bestimmt, wer die verwundbare Funktion erreichen kann.

Eine gute Case Study enthält Befunde aus allen drei Mustern, und eine gute Lösung erkennt, welcher Befund zu welchem gehört.

## 16.9 Die Analyse aufschreiben

Eine Ursachenanalyse, die nur im Kopf stattfindet, lässt sich weder prüfen noch weitergeben. Für jedes Bündel genügt eine halbe Seite in einer festen Form.

**Ursachenanalyse Bündel A**

```
Bündel:        A - Zugriffskontrolle je Objekt und Feld
Befunde:       F-07, F-08

Fehler (Code)
  F-07: Endpunkt Rechnungen lädt über die Kennung, ohne Besitzer zu prüfen.
  F-08: Endpunkt Profil übernimmt alle gesendeten Felder in das Objekt.

Begünstigende Struktur (Architektur)
  Jeder Endpunkt greift direkt auf die Datenbank zu. Besitz und erlaubte
  Felder werden je Endpunkt von Hand geprüft. Verletzt: vollständige
  Vermittlung. Fehlende Taktik: Akteure autorisieren.
  Beleg: 4 von 23 Endpunkten mit Kennung prüfen den Besitz (eigene Suche).

Durchlassender Prozess (Lifecycle)
  Entstanden: keine Anforderung zur Autorisierung je Objekt (PO.1),
  kein Bedrohungsmodell (PW.1).
  Nicht bemerkt: Tests verwenden nur das Konto des Besitzers (PW.8),
  Review hat keine Prüfliste (PW.7).
  Beleg: Testfälle der Services Abrechnung und Kundenkonto durchgesehen.

Weitere Stellen
```

```
  19 Endpunkte ohne Besitzprüfung, davon 6 mit personenbezogenen Daten.

Offen
  Gilt dasselbe für die Zugriffe der Kassenkräfte auf fremde Thermen?
```

Das Formular erzwingt drei Dinge, die sonst verloren gehen. Jede Ebene hat ein eigenes Feld, sodass keine übersprungen wird. Jede Aussage über Architektur und Prozess hat einen Beleg. Und es gibt ein Feld für die weiteren Stellen, also das Ergebnis der Suche aus Kapitel 15. Die Zahl dort, neunzehn Endpunkte, ist das stärkste Argument der ganzen Analyse: Sie zeigt, dass der Befund des Berichts die Stichprobe war und nicht das Problem.

## 16.10 Grenzen der Ursachenanalyse

Drei Vorbehalte gehören dazu.

**Der Rückschaufehler.** Im Nachhinein sieht jede Ursache offensichtlich aus. Dass ein Bedrohungsmodell den Fehler gefunden hätte, weiß man, weil man den Fehler kennt. Ob es ihn unter den Bedingungen von damals gefunden hätte, ist eine andere Frage. Eine ehrliche Analyse behauptet nicht, dass alles vermeidbar war.

**Nicht jeder Befund hat eine tiefe Ursache.** Ein einzelner fehlender Header in einer einzelnen Antwort kann schlicht ein Versehen sein. Wer bei jedem niedrigen Befund fünfmal „Warum?“ fragt, verschwendet Zeit. Die Analyse lohnt für Bündel, für alles mit hoher Schwere und für alles, was sich wiederholt.

**Die Analyse sucht Zustände, keine Schuldigen.** In dem Moment, in dem eine Ursachenanalyse dazu dient, jemanden verantwortlich zu machen, erfährt sie nichts mehr. Die Menschen, die erklären könnten, warum etwas so gebaut wurde, schweigen dann. Im Betrieb großer Systeme hat sich dafür der Grundsatz der Nachbetrachtung ohne Schuldzuweisung durchgesetzt. Er gilt hier genauso.

BRÜCKE ZU SOFTWARE SECURITY

Hier schließt sich der Kreis zu deiner Pflicht-Lehrveranstaltung. Dort analysieren die Studierenden eine fertige Architektur und finden Schwächen, bevor ein Angreifer sie findet: von der Struktur zum möglichen Fehler. Hier gehen sie den umgekehrten Weg: vom gefundenen Fehler zur Struktur, die ihn möglich gemacht hat. Die Taktiken aus deinem Foliensatz 05 sind in beiden Richtungen das Vokabular. Eine Taktik, die in Software Security als Entwurfsmittel gelehrt wird, erscheint hier als das, was im System gefehlt hat.

FRAGE AUS DEM HÖRSAAL

***Ist „Architektur“ nicht eine Ausrede, um alles neu bauen zu wollen?***

Die Gefahr besteht, und Kapitel 18 behandelt, wie man damit umgeht. Die Ursachenanalyse sagt nur, wo die Ursache liegt. Sie sagt nicht, dass man dort auch beheben muss. Es kann die richtige Entscheidung sein, eine Ursache in der Architektur zu kennen und trotzdem vorerst die Stellen einzeln zu korrigieren, weil der Umbau zu teuer ist. Falsch ist nur, die Ursache nicht zu kennen und die Einzelkorrektur für die Lösung zu halten.

FRAGE AUS DEM HÖRSAAL

***Wie weit muss ich „Warum?“ fragen? Irgendwann lande ich beim Budget oder beim Management.***

Frage so weit, wie du etwas ändern kannst. Die Kette endet dort, wo die Antwort ein Zustand ist, den die Beteiligten beeinflussen können: eine fehlende Anforderung, ein fehlender Test, eine fehlende zentrale Stelle. „Zu wenig Budget“ ist manchmal wahr, aber als Ergebnis unbrauchbar, weil daraus keine Maßnahme folgt. Wenn die Kette dort ankommt, geh einen Schritt zurück und nimm die letzte Antwort, mit der sich arbeiten lässt.

FRAGE AUS DEM HÖRSAAL

***Was mache ich, wenn zwei Leute verschiedene Ursachen für denselben Befund nennen?***

Meistens haben beide recht, weil sie auf verschiedenen Ebenen antworten oder an der Verzweigung verschiedenen Ästen gefolgt sind. Schreibt beide auf und prüft, auf welcher Ebene jede liegt. Ein echter Widerspruch liegt nur vor, wenn sich zwei Aussagen über dieselbe Tatsache ausschließen, und dann entscheidet der Beleg.

**FÜR DIE LEHRE**

Die drei Ebenen sind das Bewertungsraster der Case Study. Eine Lösung, die jeden Befund an seiner Stelle im Code korrigiert, hat die Aufgabe nur zu einem Drittel bearbeitet, gleich wie sauber die Korrekturen sind. Sag das den Studierenden am Anfang, und zeig ihnen Tab. 57 als Muster dafür, was du erwartest.

KI-Assistenten liefern auf die Frage nach der Ursache eines Befunds zuverlässig die Ebene Code und auf Nachfrage allgemeine Sätze zu den beiden anderen: Man solle Sicherheit von Anfang an bedenken, Schulungen durchführen, Reviews einführen. Solche Sätze passen auf jeden Befund jedes Systems. Verlange deshalb zu jeder Aussage über Architektur und Lifecycle einen Beleg aus den Unterlagen der Case Study: eine Stelle in der Architekturbeschreibung, ein Testfall, ein Abschnitt der Entwicklungsrichtlinie. Wenn die Unterlagen so gebaut sind, dass die Belege darin zu finden sind, trennt diese eine Forderung die Gruppen, die das System untersucht haben, von denen, die den Assistenten gefragt haben.

Für die mündliche Verteidigung eignen sich zwei Fragen besonders. „Wenn Sie nur eine einzige Änderung an der Architektur vornehmen dürften, welche, und welche Befunde würde sie beheben?“ Und: „Dieser Befund ist behoben. Zeigen Sie mir, warum der nächste dieser Art nicht mehr entstehen kann.“

Typische Fehlvorstellungen: dass ein Befund genau eine Ursache habe, dass „menschliches Versagen“ eine Ursache sei, dass die Architekturebene immer die richtige sei, und dass die Lifecycle-Ebene nur aus Schulung bestehe.

## Selbsttest

1. Ordne zu: „Kunde sieht fremde Rechnung“, „Besitzprüfung fehlt im Endpunkt“, „keine Stelle erzwingt die Prüfung“. Was ist Symptom, was Fehler, was Ursache?
2. Nenne die Leitfrage jeder der drei Ebenen.
3. Warum ist „die Entwicklerin hat es vergessen“ keine Ursache?
4. Welche zwei Hälften hat die Frage nach dem Lifecycle?
5. Wie hilft die CWE-Hierarchie bei der Ursachenanalyse? Zeige es an einem Beispiel.
6. Nenne drei Arten, auf die eine Architektur Fehler wahrscheinlich macht oder vergrößert.
7. Welches Entwurfsprinzip verletzt ein Datenbankkonto mit Rechten auf alle Schemata, und welchen Befund vergrößert es bei *Kurbad*?
8. Warum lässt sich F-06 im Code nicht beheben, sondern nur abmildern?
9. Warum kann die Architektur F-09 nicht beheben?
10. Was verlangt die Praktik RV.3 des SSDF?
11. Welche Felder hat das Formular für die Ursachenanalyse, und wozu dient das Feld „Weitere Stellen“?

# 17 Beheben, abmildern, nachtesten

Drei Wochen nach dem Bericht meldet das Team von *Kurbad*, die SQL-Injection F-03 sei behoben. Die Buchungssuche weist jetzt Eingaben zurück, die bestimmte Sonderzeichen enthalten. Der Nachweis aus dem Bericht funktioniert nicht mehr. Beim Nachtest brauchen die Tester eine Viertelstunde, um die Daten auf einem leicht veränderten Weg wieder zu lesen. Das Team hat nicht die Schwäche behoben, sondern den Nachweis.

Teil II hat für jede Klasse von Schwächen gezeigt, was auf jeder der drei Ebenen dagegen hilft. Dieses Kapitel wiederholt das nicht. Es behandelt, was allen Behebungen gemeinsam ist: den Unterschied zwischen einer Behebung und einer Abmilderung, die Reihenfolge von der Sofortmaßnahme bis zur Ursache, die Prüfung, ob eine Behebung vollständig ist, ihre Nebenwirkungen und den Nachtest. Am Ende steht der Maßnahmenplan für *Kurbad*, über den in Kapitel 18 entschieden wird.

**IN DIESEM KAPITEL**

- die vier Arten nennen, auf einen Befund zu antworten
- Behebung und Abmilderung unterscheiden und an Beispielen prüfen, was vorliegt
- Maßnahmen nach Sofortmaßnahme, Korrektur der Klasse und Behebung der Ursache ordnen
- aus dem Nachweis eines Befunds einen Regressionstest ableiten
- die Vollständigkeit einer Behebung mit dem Dreispaltentest prüfen
- Zweck und Grenzen eines Nachtests erklären und sein Ergebnis lesen

## 17.1 Vier Antworten auf einen Befund

Auf einen bestätigten Befund gibt es vier mögliche Antworten. Das Risikomanagement kennt sie unter verschiedenen Namen [72].

**Tab. 59:** Vier Antworten auf einen Befund.

| Antwort | Bedeutung | Beispiel bei *Kurbad* |
|---|---|---|
| Beheben | die Schwäche wird beseitigt | die Abfrage in der Buchungssuche wird parametrisiert |
| Abmildern | die Schwäche bleibt, ihre Ausnutzung wird erschwert oder ihr Schaden begrenzt | die Admin-Oberfläche ist nur noch aus dem Firmennetz erreichbar |
| Vermeiden | die betroffene Funktion wird entfernt | der Abruf von Profilbildern über eine Adresse wird abgeschafft |
| Akzeptieren | die Schwäche bleibt bewusst bestehen | die fehlenden Header in einer Oberfläche, die demnächst abgelöst wird |

Eine fünfte, das Übertragen des Risikos auf einen Dritten, etwa durch eine Versicherung, ändert nichts an der Schwäche und spielt hier keine Rolle.

Das Vermeiden wird zu selten erwogen. Die sicherste Funktion ist die, die es nicht gibt. Bei jedem Befund lohnt die Frage, ob die betroffene Funktion gebraucht wird. Wie viele Kunden geben für ihr Profilbild eine Adresse an, statt eine Datei hochzuladen? Wenn es eine Handvoll ist, ist das Entfernen billiger und sicherer als jede Behebung.

Das Akzeptieren ist eine vollwertige Antwort und kein Versäumnis, sofern es eine Entscheidung ist. Kapitel 18 behandelt, wer sie treffen darf und wie man sie festhält.

## 17.2 Behebung oder Abmilderung

Die wichtigste Unterscheidung des Kapitels ist die zwischen den ersten beiden Zeilen.

**DEFINITION 17.1 · BEHEBUNG UND ABMILDERUNG**
Eine **Behebung** beseitigt den Fehlerzustand, auf den ein Befund zurückgeht. Danach gibt es die Schwäche nicht mehr. Eine **Abmilderung** lässt den Fehlerzustand bestehen und setzt eine Maßnahme davor, daneben oder dahinter, die seine Ausnutzung erschwert oder seine Auswirkung begrenzt.

Es gibt eine einfache Probe. Stell dir vor, die Maßnahme fällt weg oder wird umgangen. Ist die Schwäche dann wieder da? Wenn ja, war es eine Abmilderung.

**Tab. 60:** Abmilderung und Behebung für ausgewählte Befunde.

| Befund | Abmilderung | Behebung |
|---|---|---|
| F-03 SQL-Injection | Sonderzeichen in der Eingabe ablehnen; Regel in einer vorgeschalteten Schutzschicht; Rechte des Datenbankkontos einschränken | Abfrage parametrisieren |
| F-07 fremde Rechnungen | Kennungen zufällig statt fortlaufend; Zahl der Abrufe je Konto begrenzen | Besitz prüfen |
| F-05 SSRF | ausgehende Verbindungen auf Netzebene beschränken | Ziele auf eine Erlaubnisliste prüfen, oder die Funktion entfernen |
| F-06 lange gültige Tokens | Laufzeit verkürzen | Widerruf ermöglichen |
| F-10 Vertrauen im Netz | Netz segmentieren | Aufrufer authentifizieren und autorisieren |
| F-01 Admin-Oberfläche | Zugriff auf das Firmennetz beschränken | zweiten Faktor einführen |
| F-12 Bibliothek | betroffene Funktion abschalten, Eingaben davor filtern | aktualisieren |

Drei Beobachtungen zu Tab. 60.

Abmilderungen sind nicht schlecht. Die eingeschränkten Rechte des Datenbankkontos bei F-03 und die Beschränkung ausgehender Verbindungen bei F-05 sind Maßnahmen, die man in jedem Fall haben will, auch nach der Behebung. Sie sind Tiefenstaffelung. Schlecht ist nur, sie mit der Behebung zu verwechseln.

Manche Abmilderungen sehen der Behebung zum Verwechseln ähnlich. Das Ablehnen von Sonderzeichen bei F-03 ist das Beispiel vom Anfang. Es setzt an derselben Stelle an wie die Behebung, es ist im Code, und der Nachweis aus dem Bericht funktioniert danach nicht mehr. Aber es ist eine Sperrliste, und Kapitel 8 hat erklärt, warum Sperrlisten gegen Injection nicht tragen: Sie zählen auf, was verboten ist, und der Interpreter versteht mehr, als die Liste kennt. Die Struktur der Schwäche, dass Daten als Anweisung ausgewertet werden, besteht fort.

Und was auf einer Ebene Abmilderung ist, kann auf einer anderen Behebung sein. Die Segmentierung des Netzes mildert F-10 ab. Zugleich behebt sie einen Teil dessen, was in Kapitel 16 als Ursache von Bündel C auf der Ebene der Architektur stand.

**MERKE**
Ein Befund ist behoben, wenn die Schwäche beseitigt ist, nicht wenn der Nachweis nicht mehr funktioniert. Eine Abmilderung wird als Abmilderung benannt, bekommt ein Datum, bis zu dem die Behebung folgt, und der Befund bleibt bis dahin offen.

### Die vorgeschaltete Schutzschicht

Ein Werkzeug der Abmilderung verdient einen eigenen Absatz, weil es in fast jeder Diskussion nach einem Pentest auftaucht.

**DEFINITION 17.2 · WEB APPLICATION FIREWALL**
Eine Web Application Firewall, kurz WAF, ist eine Komponente, die vor einer Web-Anwendung sitzt, jede Anfrage untersucht und solche blockiert, die bekannten Angriffsmustern entsprechen oder gegen festgelegte Regeln verstoßen.

Die WAF hat einen echten Nutzen. Sie lässt sich in Stunden einrichten, sie braucht keine Änderung am Code, und sie kann eine Lücke abschirmen, bis die Behebung fertig ist. Dieses Vorgehen heißt *virtuelles Patchen*. Bei einer Lücke in einer Fremdkomponente, für die es noch keine Korrektur gibt, ist es manchmal das Einzige, was man tun kann.

Sie hat drei Grenzen, die man kennen muss, bevor man sich auf sie stützt. Sie sieht Anfragen und versteht keine Bedeutung. Dass ein Kunde eine fremde Rechnung anfragt, sieht für sie aus wie jede andere gültige Anfrage, gegen F-07 und die ganze Klasse der Autorisierungsfehler hilft sie also nicht, ebenso wenig gegen Fehler der Fachlogik. Sie arbeitet mit Mustern, und Muster lassen sich umgehen. Und sie sitzt an einer Stelle: Was nicht durch sie hindurchgeht, etwa der Verkehr zwischen den Services, sieht sie nicht.

Die ehrliche Einordnung ist deshalb: Eine WAF ist eine brauchbare Sofortmaßnahme gegen Fehler der Eingabeverarbeitung und eine sinnvolle zusätzliche Schicht. Sie ist keine Antwort auf einen Pentest-Bericht.

## 17.3 Drei Zeithorizonte

Eine Behebung, die an die Ursache geht, braucht Wochen. Eine offene kritische Lücke kann nicht Wochen warten. Der Ausweg ist, in drei Horizonten zu denken, die nebeneinander laufen.

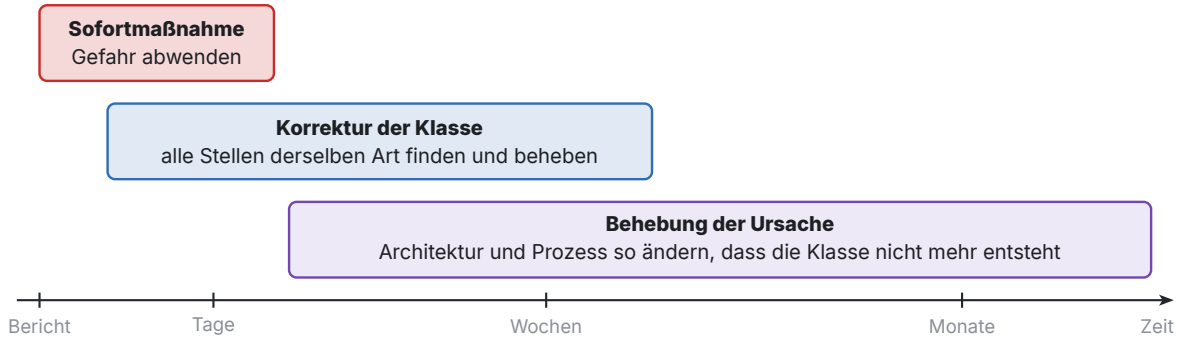



**Abb. 30:** Drei Zeithorizonte der Behebung. Sie beginnen nacheinander und überlappen.

Die **Sofortmaßnahme** wendet die unmittelbare Gefahr ab. Sie darf eine Abmilderung sein, sie darf unschön sein, und sie darf Funktion kosten. Typische Sofortmaßnahmen sind die Korrektur an der gemeldeten Stelle, das Abschalten einer Funktion, eine Regel in der Schutzschicht, das Einschränken der Erreichbarkeit und das Tauschen von Zugangsdaten.

Die **Korrektur der Klasse** behebt alle Stellen derselben Art, die die Suche aus Kapitel 15 gefunden hat. Sie arbeitet noch mit den vorhandenen Mitteln, Stelle für Stelle.

Die **Behebung der Ursache** ändert Architektur und Prozess. Sie macht die Korrekturen der zweiten Stufe zum Teil überflüssig, weil die zentrale Stelle übernimmt, was vorher an jeder einzelnen stand.

Der naheliegende Fehler in der Praxis ist, nach dem ersten Horizont aufzuhören, weil die Gefahr abgewendet scheint und das nächste Vorhaben wartet. In der Lehrveranstaltung ist eher der umgekehrte Fehler zu erwarten: Studierende planen den großen Umbau und lassen die kritische Lücke solange offen. Beides ist falsch. Ein guter Plan nennt für jedes Bündel alle drei Horizonte.

## 17.4 Die Behebung im Code prüfbar machen

Kapitel 15 hat empfohlen, jeden Befund selbst nachzuvollziehen. Daraus entsteht jetzt das wichtigste Hilfsmittel der Behebung.

**DEFINITION 17.3 · REGRESSIONSTEST ZU EINEM BEFUND**
Ein Regressionstest zu einem Befund ist ein automatisierter Test, der die im Nachweis beschriebene Handlung ausführt und erwartet, dass sie abgewiesen wird. Er schlägt vor der Behebung fehl und gelingt danach, und er läuft von da an bei jeder Änderung mit.

Für F-07 lautet der Test in Worten: Melde dich als Kundin A an, frage die Rechnung von Kunde B ab, erwarte eine Ablehnung. In der Sprache deines Foliensatzes zum Testen ist das ein gewöhnlicher Test auf der Ebene der Schnittstelle mit einer negativen Erwartung.

RechnungZugriffTest.cs

```
[Fact]
public async Task Kundin_bekommt_keine_fremde_Rechnung()
{
    var kundinA = await AnmeldenAls("kundin-a");
    var rechnungVonB = TestDaten.RechnungVon("kunde-b");

    var antwort = await kundinA.GetAsync($"/api/v3/rechnungen/{rechnungVonB.Id}");

    Assert.Equal(HttpStatusCode.NotFound, antwort.StatusCode);
}
```

Drei Regeln machen aus so einem Test ein verlässliches Mittel.

**Der Test muss zuerst rot sein.** Er wird geschrieben und ausgeführt, bevor die Korrektur eingebaut ist, und muss fehlschlagen. Ein Test, der von Anfang an grün ist, prüft nicht, was er prüfen soll. Das ist das Vorgehen der testgetriebenen Entwicklung, angewandt auf einen Sicherheitsfehler.

**Der Test prüft die Eigenschaft, nicht den Nachweis.** Ein Test, der genau die Anfrage aus dem Bericht wiederholt, hätte die Scheinbehebung vom Anfang des Kapitels für gut befunden. Bei F-03 prüft ein brauchbarer Test deshalb nicht eine bestimmte Eingabe, sondern mehrere, und zusätzlich prüft eine statische Regel, dass in diesem Service keine Abfrage mehr aus Text zusammengesetzt wird.

**Der Test wird verallgemeinert.** Aus dem einen Test für die Rechnungen wird ein Test, der für jeden Endpunkt mit einer Kennung dasselbe prüft: Zugriff mit einem zweiten Konto über Kreuz. Kapitel 9 hat das die Prüfung über Kreuz genannt. Damit wird aus dem Regressionstest für einen Befund eine Prüfung für die Klasse, und ein neuer Endpunkt ohne Besitzprüfung fällt beim nächsten Lauf der Pipeline auf.

Mit dem dritten Schritt hat der Test die Ebene gewechselt. Er ist keine Absicherung einer Korrektur mehr, sondern eine Maßnahme auf der Ebene Lifecycle, und zwar genau die, deren Fehlen in Kapitel 16 als Ursache stand.

## 17.5 Der Dreispaltentest

Ob eine Behebung vollständig ist, lässt sich mit drei Fragen prüfen. Sie folgen aus der Unterscheidung von Stelle, Klasse und Ursache.

**Tab. 61:** Der Dreispaltentest für die Vollständigkeit einer Behebung, mit Bündel A als Beispiel.

| | Stelle | Klasse | Ursache |
|---|---|---|---|
| Frage | Ist die gemeldete Stelle behoben? | Sind alle Stellen derselben Art gefunden und behoben? | Kann die Art von Fehler künftig noch entstehen, ohne aufzufallen? |
| Nachweis | Regressionstest zum Befund ist grün | Liste der geprüften Stellen, Test für jede | zentrale Stelle oder Prüfung in der Pipeline, die es verhindert oder meldet |
| Bündel A | Besitzprüfung bei Rechnungen, erlaubte Felder im Profil | 19 weitere Endpunkte korrigiert | Zugriff nur über die Datenzugriffsschicht mit Besitzer; Test über Kreuz für alle Endpunkte |

Ein Befund, bei dem nur die erste Spalte ausgefüllt ist, ist an der Stelle behoben. Das ist der Zustand der meisten geschlossenen Tickets. Die zweite Spalte verlangt die Suche, die dritte eine Antwort auf der Ebene der Architektur oder des Lifecycle.

Nicht jeder Befund braucht alle drei Spalten. Für einen Einzelfall ohne Klasse genügt die erste. Die Entscheidung, eine Spalte leer zu lassen, sollte aber eine Entscheidung sein und im Plan stehen: „Klasse: keine weiteren Stellen gefunden“ ist eine Aussage, ein leeres Feld ist keine.

## 17.6 Der Maßnahmenplan für Kurbad

Aus den Ursachen in Tab. 57 und den Behebungen aus Teil II ergibt sich für *Kurbad* ein Plan in zwei Teilen. Der erste Teil sind die Sofortmaßnahmen. Sie sind zusammen in etwa einer Woche zu schaffen und stehen nicht zur Diskussion.

**Tab. 62:** Sofortmaßnahmen für *Kurbad*. Die Aufwände in Personentagen sind Annahmen zur Veranschaulichung.

| | Sofortmaßnahme | Befunde | Art | Tage |
|---|---|---|---|---|
| S1 | Abfrage der Buchungssuche parametrisieren | F-03 | Behebung | 0,5 |
| S2 | Besitzprüfung bei Rechnungen; erlaubte Felder im Profil festlegen | F-07, F-08 | Behebung | 1 |
| S3 | Abruf von Profilbildern über eine Adresse abschalten | F-05 | Vermeiden | 0,5 |
| S4 | Bemerkungsfeld in der Admin-Oberfläche kodiert ausgeben | F-04 | Behebung | 0,5 |
| S5 | Admin-Oberfläche nur noch aus dem Firmennetz erreichbar | F-01 | Abmilderung | 0,5 |
| S6 | Zugangsdaten des Brokers tauschen | F-11 | Abmilderung | 0,5 |
| S7 | Bibliothek aktualisieren | F-12 | Behebung | 1 |

Zwei Zeilen zeigen, warum die Spalte *Art* nötig ist. S5 und S6 sind Abmilderungen. Nach S6 stehen im Abbild des Service neue Zugangsdaten im Klartext, und die Schwäche F-11 besteht unverändert. Der Tausch war trotzdem nötig, denn die alten Zugangsdaten standen in einem Bericht und waren den Testern bekannt. Die Befunde F-01 und F-11 bleiben offen.

Der zweite Teil sind die Maßnahmen, die an Klassen und Ursachen gehen.

**Tab. 63:** Maßnahmen an Klassen und Ursachen für *Kurbad*. Die Aufwände sind Annahmen zur Veranschaulichung, die Summe beträgt 107 Personentage.

| | Maßnahme | Bündel | Ebene | Tage |
|---|---|---|---|---|
| M1 | Objekte nur über eine Datenzugriffsschicht mit Besitzer laden, Verbot als Voreinstellung; alle Endpunkte umstellen | A | Architektur | 15 |
| M2 | Datenzugriff nur parametrisiert; Vorlagen der Admin-Oberfläche kodieren von selbst; ausgehende Verbindungen nur über eine Erlaubnisliste | B | Architektur | 10 |
| M3 | eigene Identität je Service, Authentifizierung und Autorisierung zwischen Services, Netzrichtlinien | C | Architektur | 25 |
| M4 | Geheimnisse zur Laufzeit aus einem Tresor; eigenes Datenbankkonto je Service | C | Architektur | 8 |
| M5 | Admin-Oberfläche an den Identitätsdienst anbinden, zweiter Faktor | D | Architektur | 6 |
| M6 | kurzlebige Tokens mit Erneuerung und Widerruf | D | Architektur | 12 |
| M7 | Gutschein in einem unteilbaren Schritt einlösen; Muster für Zustandsübergänge | F | Code | 3 |
| M8 | Pipeline: Stückliste und Prüfung der Abhängigkeiten, statische Analyse blockierend, Suche nach Geheimnissen | B, C, E | Lifecycle | 8 |
| M9 | Berechtigungstests über Kreuz für alle Endpunkte in der Pipeline | A | Lifecycle | 5 |
| M10 | abgelehnte Zugriffe zentral protokollieren und bei Häufung alarmieren | E | Architektur | 6 |
| M11 | gemeinsame Grundkonfiguration der Services für Header und Fehlerbehandlung | E | Architektur | 4 |
| M12 | Sicherheitsanforderungen und Bedrohungsmodell für neue Funktionen; Admin-Oberfläche in den Prozess aufnehmen | alle | Lifecycle | 5 |

Der Plan in Tab. 63 hat drei Eigenschaften, an denen du einen brauchbaren Plan erkennst.

Jede Maßnahme nennt, welche Bündel sie behebt, und jedes Bündel hat mindestens eine Maßnahme. Die Zuordnung lässt sich in beide Richtungen prüfen: Gibt es eine Maßnahme ohne Bündel, ist sie überflüssig oder die Triage unvollständig. Gibt es ein Bündel ohne Maßnahme, fehlt etwas oder es wurde bewusst akzeptiert.

Die Ebenen sind gemischt. Acht Maßnahmen liegen in der Architektur, drei im Lifecycle, eine im Code. Das Verhältnis kehrt sich gegenüber den Sofortmaßnahmen um, die fast alle im Code liegen.

Und der Plan ist zu groß. 107 Personentage sind für ein Team von sechs Leuten fast ein Monat ohne jede andere Arbeit. Kein Unternehmen setzt so einen Plan in einem Zug um. Welche Maßnahmen zuerst kommen und welche gar nicht, ist keine technische Frage mehr. Sie ist der Gegenstand des nächsten Kapitels.

## 17.7 Nebenwirkungen

Jede Behebung ist eine Änderung, und jede Änderung kann etwas kaputt machen. Sicherheitsmaßnahmen haben dabei eine eigene Art von Nebenwirkungen, weil sie fast immer etwas verbieten, das vorher ging.

**Bestehende Nutzer verlassen sich auf das Fehlverhalten.** Wenn M1 den Zugriff auf fremde Objekte unterbindet, hören vielleicht Dinge auf zu funktionieren, die nie hätten funktionieren dürfen und trotzdem verwendet wurden. Ein Bericht in der Buchhaltung, der sich mit einem gewöhnlichen Konto alle Rechnungen holt, ist ein klassisches Beispiel.

**Alte Clients senden, was nicht mehr erlaubt ist.** Die mobile App ist auf den Geräten der Kunden in mehreren Versionen im Umlauf. Eine strengere Prüfung der Eingaben oder ein neues Verfahren für Tokens (M6) trifft Versionen, die niemand mehr aktualisieren kann. Die Behebung braucht dann einen Übergang: eine neue Version der Schnittstelle, eine Frist, eine erzwungene Aktualisierung.

**Sicherheit kostet andere Qualitäten.** Der Widerruf von Tokens verlangt, dass jeder Service bei jeder Anfrage nachsieht, ob das Token noch gilt. Das kostet Zeit und schafft eine Abhängigkeit: Fällt die Stelle aus, die das beantwortet, steht alles. Authentifizierung zwischen den Services (M3) kostet Rechenzeit und macht die Fehlersuche schwerer. Der zweite Faktor kostet die Mitarbeiter täglich einige Sekunden.

Der dritte Punkt ist für dich vertrautes Gelände. Es sind Zielkonflikte zwischen Qualitätsmerkmalen, wie sie deine Foliensätze zur Architektur behandeln, und die Antwort ist dieselbe: Man macht sie sichtbar, wägt ab und entscheidet. Eine Maßnahme, deren Nebenwirkungen niemand bedacht hat, wird im Betrieb beim ersten Problem abgeschaltet, und danach ist die Lage schlechter als vorher, weil die Lücke als behoben gilt.

> **AUS DER PRAXIS**
> Die unangenehmste Nebenwirkung ist die auf das Fehlerverhalten. Eine Maßnahme, die bei einem Ausfall „offen“ versagt, also im Zweifel durchlässt, schützt genau dann nicht, wenn ein Angreifer den Ausfall herbeiführen kann. Eine Maßnahme, die „geschlossen“ versagt, legt bei einem Ausfall den Betrieb lahm. Kapitel 10 hat die beiden Arten beschrieben. Bei jeder zentralen Sicherheitsfunktion, die du einführst, also bei M1, M3 und M6, gehört die Frage in den Entwurf: Was geschieht, wenn sie nicht antwortet? Die Antwort ist eine Architekturentscheidung und keine Einstellung.

## 17.8 Was sonst noch zu tun ist

Zur Behebung gehören drei Dinge, die in keinem Ticket stehen und oft vergessen werden.

**Zugangsdaten tauschen.** Alles, was die Tester an Geheimnissen gesehen haben, gilt als bekannt: die Zugangsdaten aus F-11, Tokens aus Testkonten, Schlüssel aus Konfigurationsdateien. Sie werden getauscht, auch wenn die Tester vertrauenswürdig sind, denn sie stehen nun in einem Bericht, der Jahre aufbewahrt wird.

**Aufräumen.** Die Tester haben Konten angelegt, Daten verändert und vielleicht Dateien hinterlassen. Ein guter Bericht listet das im Anhang auf. Es wird entfernt, in der Testumgebung und überall, wohin die Testumgebung kopiert wird.

**Nachsehen, ob schon jemand da war.** Das ist die unbequemste Frage. Eine Lücke, die Tester in acht Tagen gefunden haben, kann vor ihnen ein anderer gefunden haben. F-07 braucht kein Können, nur Neugier. Bevor die Lücke geschlossen wird, lohnt der Blick in die Protokolle der Produktion: Gibt es Konten, die auffällig viele verschiedene Rechnungen abgerufen haben? Bei *Kurbad* führt die Frage unmittelbar zu F-14. Abgelehnte Zugriffe werden nicht protokolliert, und erfolgreiche Zugriffe auf fremde Rechnungen sahen für das System aus wie gewöhnliche. Ob jemand die Lücke ausgenutzt hat, lässt sich möglicherweise nicht mehr feststellen.

Findet sich ein Hinweis auf eine tatsächliche Ausnutzung, ist aus dem Befund ein Sicherheitsvorfall geworden, mit eigenen Pflichten. Bei personenbezogenen Daten kann eine Meldung an die Datenschutzbehörde nötig sein, für die die Datenschutz-Grundverordnung eine Frist von 72 Stunden ab Kenntnis setzt [12]. Für wichtige und wesentliche Einrichtungen kommen die Meldepflichten aus dem NISG 2026 dazu [9], für Hersteller von Produkten mit digitalen Elementen die aus dem Cyber Resilience Act [10]. Das ist nicht mehr Sache des Entwicklungsteams, und es ist der Moment, die Rechtsabteilung und die Leitung einzubeziehen.

## 17.9 Der Nachtest

> **DEFINITION 17.4 · NACHTEST**
> Ein Nachtest (*Retest*) ist eine erneute Prüfung der Befunde eines Pentests nach deren Behebung, in der Regel durch dieselben Tester, mit dem Ziel festzustellen, ob jeder einzelne Befund behoben ist.

Der Nachtest ist ein kleiner Auftrag, oft ein bis zwei Tage, und er sollte schon im ursprünglichen Auftrag vereinbart sein. Die Tester wiederholen ihre Nachweise und versuchen, die Korrektur zu umgehen. Das Ergebnis ist eine Tabelle mit einem Zustand je Befund.

**Tab. 64:** Zustände eines Befunds im Nachtest.

| Zustand | Bedeutung | Typischer Grund |
|---|---|---|
| behoben | der Nachweis und Varianten davon gelingen nicht mehr, die Schwäche ist beseitigt | - |
| abgemildert | die Ausnutzung ist erschwert, die Schwäche besteht | Schutzschicht, Sperrliste, eingeschränkte Erreichbarkeit |
| teilweise behoben | an einigen Stellen behoben, an anderen nicht | Korrektur in der Web-Oberfläche, nicht in der Schnittstelle der App; ein Endpunkt von mehreren |
| nicht behoben | der Nachweis gelingt unverändert oder leicht verändert | Korrektur nicht ausgerollt; Nachweis statt Schwäche behoben |
| nicht prüfbar | die Voraussetzungen für die Prüfung fehlten | Konto fehlt, Umgebung nicht erreichbar |

Die Gründe in der rechten Spalte von Tab. 64 sind eine Liste dessen, was bei Behebungen schiefgeht, und es lohnt, sie vor dem Nachtest selbst durchzugehen. Ist die Korrektur in allen Umgebungen ausgerollt, auch in der Produktion? Gilt sie für alle Wege zu derselben Funktion, also Web, App und Schnittstelle? Gilt sie für alle Rollen? Prüft sie die Eigenschaft oder den Nachweis?

Zwei Grenzen des Nachtests sind wichtig.

Er prüft die Befunde und sonst nichts. Ob bei der Behebung neue Schwächen entstanden sind, etwa in der neuen Datenzugriffsschicht oder im neuen Verfahren für Tokens, prüft er nicht. Nach einem größeren Umbau ist ein neuer Test der umgebauten Teile fällig, kein Nachtest.

Und er prüft Stellen, nicht Ursachen. Ein Nachtest, der für F-07 „behoben“ meldet, sagt, dass die Rechnungen nun geschützt sind. Über die anderen neunzehn Endpunkte sagt er nur dann etwas, wenn sie Teil des Auftrags waren. Wer den Testern die Liste der weiteren Stellen aus der eigenen Suche mitgibt, bekommt einen deutlich wertvolleren Nachtest für fast dasselbe Geld.

Für *Kurbad* meldet der Nachtest nach den Sofortmaßnahmen: F-04, F-05, F-07, F-08 und F-12 behoben, F-01 und F-11 abgemildert, die übrigen noch offen wie geplant, und F-03 nicht behoben. Das ist die Geschichte vom Anfang des Kapitels. Das Team hatte S1 nicht so umgesetzt, wie es im Plan stand, sondern

die Eingabe gefiltert, weil das schneller ging. Der Regressionstest, der nur die Anfrage aus dem Bericht wiederholte, war grün.

## 17.10 Behebungen von der Maschine

KI-Assistenten schreiben Korrekturen für Sicherheitsfehler, und für die Ebene Code tun sie das oft gut. Wer einem Assistenten den Endpunkt aus Kapitel 9 und den Befund F-07 gibt, bekommt mit hoher Wahrscheinlichkeit die richtige Korrektur an dieser Stelle. In der Spezialisierung werden die Studierenden genau so arbeiten, und das ist erwünscht.

Vier Schwächen solltest du kennen, weil sie die Aufgaben der Case Study bestimmen.

Assistenten beheben, was man ihnen zeigt. Sie sehen eine Datei, nicht das System. Die Korrektur der Klasse und die Ursache in der Architektur liegen außerhalb dessen, was sie sehen, solange niemand sie ihnen vorlegt.

Sie neigen zur Abmilderung, die wie eine Behebung aussieht: eine zusätzliche Prüfung der Eingabe, eine Sperrliste, ein Abfangen der Ausnahme. Solche Korrekturen sind lokal, schnell geschrieben und lassen den Nachweis verschwinden.

Sie bedenken keine Nebenwirkungen im System. Dass die App in alten Versionen im Umlauf ist, steht in keiner Datei.

Und sie prüfen ihre Korrektur nicht, wenn man es nicht verlangt. Hier gelten dieselben Befunde wie für erzeugten Code überhaupt: Er wird für sicherer gehalten, als er ist [64].

Die Gegenmittel stehen in diesem Kapitel: der Test, der zuerst rot sein muss, die Probe, ob es eine Behebung oder eine Abmilderung ist, und der Dreispaltentest.

**BRÜCKE ZU SOFTWARE SECURITY**

Fast jede Maßnahme in Tab. 63 ist eine Sicherheitstaktik aus deinem Foliensatz 05, wie Bass, Clements und Kazman sie ordnen [55]. M1 und M3 sind *Akteure autorisieren* und *Akteure authentifizieren*. M4 und die engeren Rechte der Konten sind *Zugriff begrenzen*. M2 ist *Eingaben validieren* im weiten Sinn. M10 gehört zu den Taktiken, die Angriffe *erkennen*, und der Widerruf in M6 zu denen, die auf Angriffe *reagieren*. Wenn die Studierenden einen Maßnahmenplan schreiben, können sie jede Maßnahme mit der Taktik benennen, die sie umsetzt. Das verbindet die Spezialisierung sichtbar mit der Pflicht-Lehrveranstaltung und gibt dir ein Kriterium für die Bewertung: Ein Plan ohne Maßnahmen zum Erkennen und Reagieren hat die Hälfte des Katalogs übersehen.

**FRAGE AUS DEM HÖRSAAL**

***Ist eine Abmilderung nicht manchmal die bessere Wahl als eine Behebung?***

Als erste Maßnahme oft, als einzige selten. Es gibt Fälle, in denen die Behebung unverhältnismäßig wäre, etwa bei einem Altsystem, das in einem halben Jahr abgeschaltet wird. Dann ist die Abmilderung zusammen mit einer bewussten Annahme des Restrisikos eine vertretbare Entscheidung. Sie muss nur als das benannt sein, was sie ist, und der Befund darf nicht als behoben gelten. Was nicht vertretbar ist, ist die Abmilderung aus Bequemlichkeit, die im Ticket als Behebung steht.

**FRAGE AUS DEM HÖRSAAL**

***Warum sollen dieselben Tester den Nachtest machen? Wäre ein anderer Blick nicht besser?***

Für den Nachtest sind dieselben Tester besser, weil sie wissen, was sie beim ersten Mal getan haben, und weil sie die Korrektur gezielt zu umgehen versuchen können. Der andere Blick ist beim nächsten vollen Pentest wertvoll. Manche Organisationen wechseln den Anbieter deshalb regelmäßig, lassen aber den Nachtest immer vom Anbieter des jeweiligen Tests machen.

**FRAGE AUS DEM HÖRSAAL**

***Wie lange darf die Behebung dauern?***

Dafür gibt es keine allgemeine Zahl, aber jede Organisation sollte eigene haben. Üblich sind Fristen je Schweregrad, etwa wenige Tage für kritische Befunde, wenige Wochen für hohe und ein Quartal für mittlere. Wichtiger als die genauen Zahlen ist, dass es sie gibt, dass sie für die Sofortmaßnahme gelten und dass eine Überschreitung jemandem auffällt. Für die Behebung der Ursache gelten keine Fristen dieser Art, sie wird wie jedes andere Vorhaben geplant.

**FÜR DIE LEHRE**

In der Case Study ist der Maßnahmenplan das Hauptergebnis, und der Dreispaltentest ist dein Korrekturschlüssel. Du kannst für jedes Bündel der Musterlösung festlegen, was in jeder der drei Spalten stehen muss, und die Abgaben dagegen prüfen. Das geht schnell und ist nachvollziehbar.

Baue in den Bericht der Case Study mindestens zwei Befunde ein, für die die naheliegende Korrektur eine Abmilderung ist, und frage in der mündlichen Verteidigung gezielt danach: „Angenommen, diese Maßnahme wird umgangen. Ist die Schwäche dann noch da?“ Wer die Korrektur vom Assistenten übernommen hat, ohne sie zu prüfen, merkt es an dieser Stelle selbst.

Eine gute kleine Übung für eine Präsenzeinheit ist das Lesen eines Nachtest-Ergebnisses. Gib den Gruppen die Tabelle eines Nachtests mit zwei Befunden „teilweise behoben“ und einem „nicht behoben“ und lass sie für jeden die wahrscheinlichste Erklärung und den nächsten Schritt nennen.

Wenn du erzeugten Code mit Schwachstellen ausgibst, gilt deine eigene Regel: Im verwundbaren Code stehen keine Hinweise auf die Schwachstelle, weder in Kommentaren noch in Namen. Die Erklärung gehört in die Musterlösung.

Typische Fehlvorstellungen: dass ein Befund behoben sei, wenn der Nachweis nicht mehr geht, dass eine Schutzschicht vor der Anwendung eine Behebung sei, dass der Nachtest die Sicherheit des Systems bestätige, und dass nach der Korrektur im Code nichts mehr zu tun sei.

## Selbsttest

1. Nenne die vier Antworten auf einen Befund mit je einem Beispiel.
2. Mit welcher Probe unterscheidest du eine Behebung von einer Abmilderung?
3. Warum ist das Ablehnen von Sonderzeichen keine Behebung einer SQL-Injection?
4. Gegen welche Arten von Befunden hilft eine Web Application Firewall nicht, und warum?
5. Beschreibe die drei Zeithorizonte der Behebung.
6. Welche drei Regeln gelten für einen Regressionstest zu einem Befund?
7. Was prüfen die drei Spalten des Dreispaltentests, und was ist jeweils der Nachweis?
8. Die Zugangsdaten aus F-11 wurden getauscht. Ist der Befund behoben? Begründe.
9. Nenne drei Arten von Nebenwirkungen einer Sicherheitsmaßnahme.
10. Was prüft ein Nachtest nicht?
11. Ein Nachtest meldet „teilweise behoben“. Nenne zwei wahrscheinliche Gründe.

# 18 Entscheiden unter Konflikt

Der Maßnahmenplan für *Kurbad* umfasst 107 Personentage. Das Team hat in diesem Quartal 45 für Sicherheitsthemen frei, und das auch nur, wenn die Arbeit an der Familienkarte, die zum Beginn der Wintersaison fertig sein soll, nicht leidet. In der Besprechung sitzen vier Leute, die den Bericht gelesen haben, und jeder hält etwas anderes für das Wichtigste. Alle vier haben gute Gründe.

Bis hierher ließ sich jede Frage dieses Buchs mit Fachwissen beantworten. Ob ein Befund zutrifft, wo seine Ursache liegt und was ihn behebt, kann man richtig oder falsch beantworten. Die Frage, was von einem zu großen Plan zuerst getan wird und was liegen bleibt, hat keine richtige Antwort. Sie hat begründete und unbegründete Antworten. In der Spezialisierung ist das der Teil der Case Study, in dem die Studierenden widersprüchliche Stellungnahmen bekommen und eine Entscheidung vorlegen und verteidigen müssen. Dieses Kapitel gibt dir die Begriffe, die typischen Positionen und eine Form, in der sich eine solche Entscheidung aufschreiben und prüfen lässt.

**IN DIESEM KAPITEL**

- erklären, warum die Reihenfolge der Behebung keine rein technische Entscheidung ist und wer sie trifft
- die typischen Positionen von Sicherheit, Produkt, Betrieb und Compliance beschreiben, mit ihren berechtigten Anliegen und blinden Flecken
- Maßnahmen nach nachvollziehbaren Kriterien in eine Reihenfolge bringen
- eine Risikoakzeptanz formulieren, die den Namen verdient
- eine Entscheidungsvorlage mit Optionen aufbauen und eine Entscheidung als Architekturentscheidung festhalten

## 18.1 Wer entscheidet

Die erste Frage ist nicht, was entschieden wird, sondern von wem.

**DEFINITION 18.1 · RISIKOEIGNER**
Der Risikoeigner ist die Person, die für ein Risiko verantwortlich ist und die Befugnis hat, über seine Behandlung zu entscheiden. Er trägt die Folgen, wenn das Risiko eintritt, und er verfügt über die Mittel, es zu behandeln [72].

Beide Hälften der Definition sind wichtig. Wer die Folgen trägt, aber keine Mittel hat, kann nur warnen. Wer die Mittel hat, aber die Folgen nicht trägt, entscheidet zu leichtfertig. Bei einer Plattform wie *Kurbad* ist der Risikoeigner in der Regel die Person, die das Geschäft mit der Plattform verantwortet, also jemand aus der Geschäftsleitung oder die Leitung des Bereichs.

Daraus folgt, wer es nicht ist. Die Entwicklung ist es nicht: Sie kann schätzen, was eine Maßnahme kostet, und sagen, was sie bewirkt. Die Informationssicherheit ist es nicht: Sie berät, bewertet und warnt. Die Tester sind es schon gar nicht. In vielen Organisationen wird die Entscheidung trotzdem stillschweigend von der Entwicklung getroffen, indem Tickets liegen bleiben. Das ist eine Risikoakzeptanz ohne Befugnis und ohne dass der Risikoeigner davon weiß.

Für wichtige und wesentliche Einrichtungen ist das seit der Umsetzung der NIS-2-Richtlinie auch rechtlich klargestellt: Die Leitungsorgane müssen die Maßnahmen des Risikomanagements billigen und ihre Umsetzung überwachen, und sie können dafür haftbar gemacht werden [8], [9]. Die Aufgabe der Fachleute ist es deshalb, die Entscheidung so vorzubereiten, dass jemand ohne technisches Wissen sie verantwortlich treffen kann.

## 18.2 Vier Positionen

In fast jeder Organisation treffen nach einem Pentest dieselben vier Sichten aufeinander. Die Namen der Rollen wechseln, die Positionen sind erstaunlich stabil.

**Tab. 65:** Vier typische Positionen nach einem Pentest.

| Rolle | Will | Berechtigtes Anliegen | Blinder Fleck |
|---|---|---|---|
| Sicherheit (CISO) | alles Hohe schnell beheben, Ursachen angehen | kennt die Bedrohungslage und haftet für Vorfälle mit | unterschätzt Aufwand und Nebenwirkungen, behandelt alle Befunde gleich |
| Produkt (Product Owner) | Funktionen liefern, Termine halten, Kunden nicht verärgern | ohne Produkt kein Geschäft, das zu schützen wäre | hält Sicherheit für eine Eigenschaft, die man später ergänzt |
| Betrieb | Stabilität, wenige und kleine Änderungen | jede Änderung ist ein Risiko für die Verfügbarkeit | zählt den Ausfall durch eine Änderung, nicht den durch einen Angriff |
| Compliance und Datenschutz | Nachweise, dokumentierte Entscheidungen, Fristen | Pflichten gegenüber Behörden, Kunden und Betroffenen | hält dokumentiert für behoben |

Für *Kurbad* könnten die Stellungnahmen so lauten. In der Case Study bekommen die Studierenden vier solche Texte.

**INFORMATIONSSICHERHEIT**

Der Bericht zeigt, dass ein Einbruch in einen Service ein Einbruch in alle ist. Solange die Services einander blind vertrauen, ist jede weitere Lücke kritisch, auch die, die wir noch nicht kennen. M3 muss sofort beginnen. Alle Befunde der Stufen kritisch und hoch sind nach unserer Richtlinie in 30 Tagen zu beheben. Abmilderungen akzeptiere ich nicht als Behebung.

**PRODUKT**

Die Familienkarte muss in sechs Wochen fertig sein, sie ist mit drei Thermen vertraglich zugesagt. Zur Saison läuft außerdem eine Gutscheinaktion. Ich kann in diesem Quartal 45 Tage freimachen, nicht 107. M6 schließe ich vor der Saison aus: Ein neues Anmeldeverfahren sperrt alle Kunden mit alter App aus, mitten im stärksten Geschäft. Die kritische Lücke ist geschlossen. Für den Rest schlage ich eine Schutzschicht vor der Anwendung vor und die Ursachen im Frühjahr.

**BETRIEB**

Vier Wochen vor Saisonbeginn gilt bei uns ein Änderungsstopp für alles, was die Kommunikation zwischen den Services betrifft. M3 in dieser Zeit ist ausgeschlossen. Ein abgelaufenes Zertifikat zwischen zwei Services, und die Buchung steht. M4 und M10 können wir sofort umsetzen, das liegt in unserer Hand und stört niemanden. Im Übrigen ist das interne Netz von außen nicht erreichbar.

**DATENSCHUTZ UND COMPLIANCE**

Die Rechnungen enthalten Angaben zu Behandlungen. Wir müssen nachweisen können, dass der Zugriff auf fremde Rechnungen nicht nur an der gemeldeten Stelle unterbunden ist. Im ersten Quartal steht die Prüfung durch einen Großkunden an, der den Bericht und unseren Maßnahmenplan sehen will. Ich brauche für jeden offenen Befund eine dokumentierte Entscheidung mit Verantwortlichem und Datum. Und ich muss wissen, ob die Lücke ausgenutzt wurde.

## 18.3 Die Positionen lesen

Die erste Aufgabe ist, in jeder Stellungnahme drei Dinge zu trennen: was eine Tatsache ist, was eine Bewertung und was eine Forderung.

Der Änderungsstopp ist eine Tatsache, mit der zu planen ist. Dass die Familienkarte vertraglich zugesagt ist, ebenfalls. Dass alle hohen Befunde in 30 Tagen zu beheben sind, ist eine Regel der Organisation, also

auch eine Tatsache, an der der Plan zu messen ist. Wenn der Plan sie nicht einhält, muss das jemand mit Befugnis ausdrücklich genehmigen.

„Die kritische Lücke ist geschlossen“ ist eine Bewertung, und sie ist falsch. Kapitel 17 hat erzählt, dass der Nachtest F-03 als nicht behoben meldete. „Das interne Netz ist von außen nicht erreichbar“ ist ebenfalls eine Bewertung, und der Angriffspfad über F-05 hat sie widerlegt. Beide Aussagen sind typisch. Sie sind nicht böswillig, sie stammen aus einem Bild des Systems, das vor dem Bericht galt.

Die Forderungen schließlich sind das, worüber verhandelt wird. Hinter jeder steht ein Anliegen, und das Anliegen lässt sich oft auf mehr als eine Art erfüllen. Die Sicherheit fordert M3 sofort. Ihr Anliegen ist, dass ein Einbruch in einen Service nicht alle öffnet. Der Betrieb lehnt M3 vor der Saison ab. Sein Anliegen ist, dass die Buchung nicht ausfällt. Beide Anliegen lassen sich eine Zeit lang zugleich bedienen, wenn man die Ketten aus Kapitel 15 an anderen Gliedern unterbricht.

> **MERKE**
> Stellungnahmen enthalten Tatsachen, Bewertungen und Forderungen. Tatsachen werden eingeplant, Bewertungen werden am Bericht geprüft, und hinter Forderungen wird das Anliegen gesucht. Wer nur zwischen den Forderungen vermittelt, bekommt einen Kompromiss, den niemand begründen kann.

Die zweite Aufgabe ist, zu sehen, was in keiner Stellungnahme steht. Niemand erwähnt Bündel A außer dem Datenschutz, und der nur als Nachweisfrage. Niemand erwähnt die Fachlogik, obwohl zur Saison eine Gutscheinaktion läuft und F-09 genau die Einlösung von Gutscheinen betrifft. Und niemand spricht für die Kunden, deren Daten betroffen sind und die an der Besprechung nicht teilnehmen.

## 18.4 Kriterien für die Reihenfolge

Eine Reihenfolge lässt sich begründen, wenn sie auf Kriterien beruht, die man nennen kann. Fünf sind hier nützlich.

**Wirkung auf die Ziele.** Welche Maßnahme schließt die meisten Wege zu den Zielen, die am meisten schmerzen würden? Das ist die Sicht aus dem Angriffsbaum in Kapitel 15. Eine Maßnahme, die in mehreren Ketten ein Glied bricht, zählt mehr als eine, die einen einzelnen Befund schließt.

**Aufwand.** Bei gleicher Wirkung kommt das Billigere zuerst. Wichtiger ist das Umgekehrte: Eine billige Maßnahme mit mäßiger Wirkung kann vor einer teuren mit großer Wirkung stehen, wenn sie die Zeit bis zur teuren überbrückt.

**Nebenwirkungen und Zeitfenster.** Manche Maßnahmen haben Zeiten, in denen sie nicht gehen, wie M3 im Änderungsstopp und M6 vor der Saison. Andere haben Zeiten, in denen sie besonders nötig sind, wie M7 vor der Gutscheinaktion.

**Abhängigkeiten.** Manche Maßnahmen setzen andere voraus oder werden durch sie billiger. Die Berechtigungstests M9 zeigen, welche Endpunkte M1 umstellen muss, und belegen danach, dass es geschehen ist. Der Tresor für Geheimnisse M4 ist eine Voraussetzung für die Identitäten der Services in M3.

**Pflichten.** Was rechtlich oder vertraglich geboten ist, steht nicht zur Abwägung, sondern nur zur Planung. Die Datenschutz-Grundverordnung verlangt für personenbezogene Daten Maßnahmen, die dem Risiko angemessen sind, und für besonders schutzwürdige Daten liegt die Schwelle höher [12].

**Tab. 66:** Die Maßnahmen aus Tab. 63 mit den Kriterien für die Reihenfolge. Die Einstufung der Wirkung ist ein Urteil und steht zur Diskussion.

| | Maßnahme | Wirkung | Tage | Fenster und Abhängigkeiten |
|---|---|---|---|---|
| M1 | Zugriff zentral | hoch | 15 | Pflicht wegen Datenschutz; wird durch M9 gestützt |
| M9 | Tests über Kreuz | hoch | 5 | vor oder mit M1; liefert den Nachweis für die Prüfung |
| M4 | Tresor, Konten je Service | hoch | 8 | bricht die Kette F-03 und F-16, behebt F-11; Voraussetzung für M3 |
| M5 | Admin am Identitäts-dienst | mittel | 6 | löst die Abmilderung S5 ab |
| M10 | Protokoll und Alarm | mittel | 6 | wirkt auf alle Bündel; beantwortet künftig die Frage nach der Ausnutzung |
| M7 | Gutschein unteilbar | mittel | 3 | vor der Gutscheinaktion |
| M3 | Identität je Service | sehr hoch | 25 | nicht im Änderungsstopp; braucht M4 |
| M2 | sichere Wege für Eingaben | hoch | 10 | Stellen sind durch S1, S3, S4 behoben |
| M8 | Pipeline | mittel | 8 | verhindert Wiederkehr in B, C, E |
| M6 | Tokens mit Widerruf | mittel | 12 | nicht vor der Saison; braucht neue App-Version |
| M11 | Grundkonfiguration | niedrig | 4 | keine |
| M12 | Anforderungen, Be-drohungsmodell | hoch, spät | 5 | wirkt nur auf Neues |

Die Spalte *Wirkung* in Tab. 66 enthält Worte und keine Zahlen, und das mit Absicht. Es gibt Verfahren, die jede Maßnahme mit Punkten bewerten und eine Rangfolge ausrechnen. Sie geben einem Urteil den Anschein einer Messung. Für zwölf Maßnahmen reicht eine Tabelle, über die man reden kann, und der Wert liegt im Gespräch über die Zeilen, in denen man sich uneinig ist.

## 18.5 Ein Vorschlag in Stufen

Aus den Kriterien ergibt sich für *Kurbad* ein Vorschlag in drei Stufen. Er ist vertretbar, und er ist nicht der einzige vertretbare.

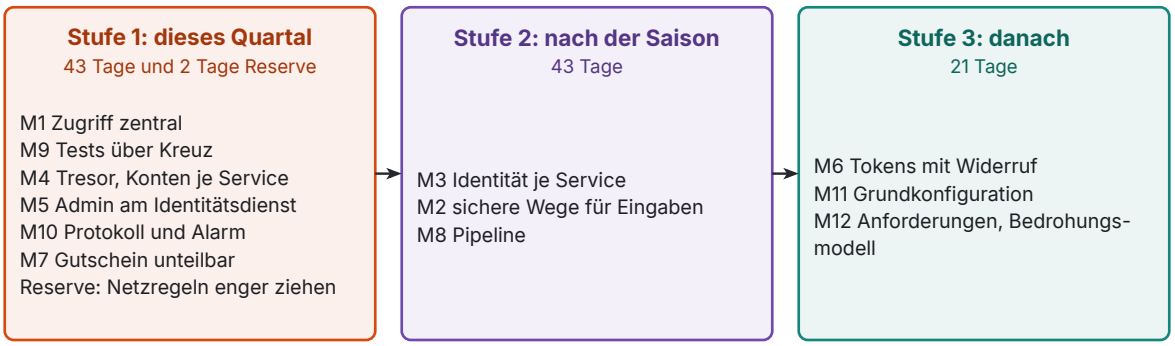



**Abb. 31:** Ein Vorschlag für die Reihenfolge der Maßnahmen bei *Kurbad*.

Die Begründung für Abb. 31 in der Sprache der Kriterien:

Stufe 1 schließt Bündel A an Stelle, Klasse und Ursache und liefert mit M9 den Nachweis dafür. Das erfüllt die Pflicht aus dem Datenschutz und das Anliegen der Compliance. M4 bricht die Kette von der SQL-Injection zu den Daten aller Services und beseitigt die Zugangsdaten im Abbild. Es ist die Voraussetzung für Stufe 2 und liegt nach eigener Aussage in der Hand des Betriebs. M5 löst eine Abmilderung durch eine Behebung ab. M10 sorgt dafür, dass ein Angriff in der Saison wenigstens bemerkt wird. M7 kommt vor der Gutscheinaktion. Nichts davon berührt den Änderungsstopp oder die App.

Der Preis von Stufe 1 ist, dass F-10 ein Quartal lang offen bleibt. Das ist der Punkt, an dem die Sicherheit recht hat: Ein Einbruch in einen Service erreicht in dieser Zeit weiterhin die anderen. Der Vorschlag begegnet dem mit einer Abmilderung, den enger gezogenen Netzregeln aus der Reserve, und mit den gebrochenen Ketten. Das Restrisiko muss der Risikoeigner ausdrücklich annehmen.

Stufe 2 holt M3 nach der Saison nach, zusammen mit den Maßnahmen, die die Wiederkehr von Bündel B verhindern. Stufe 3 enthält M6, das an den Veröffentlichungszyklus der App gebunden ist.

Ein anderer vertretbarer Vorschlag würde M3 vorziehen, den Änderungsstopp für diesen einen Umbau aufheben und dafür M1 auf die Endpunkte mit personenbezogenen Daten beschränken. Er setzt mehr auf die Architektur und nimmt ein höheres Risiko für die Verfügbarkeit in der Saison in Kauf. Welcher der beiden besser ist, hängt davon ab, wie man einen Ausfall der Buchung gegen einen Abfluss von Daten gewichtet. Das ist die Abwägung, die der Risikoeigner treffen muss.

**FALLSTRICK**
Der bequemste Kompromiss ist, von jeder Forderung ein wenig zu erfüllen: ein bisschen M3, ein bisschen M1, eine Schutzschicht für das Produkt, ein Dokument für die Compliance. Das Ergebnis sind vier angefangene Maßnahmen, von denen keine eine Kette bricht oder ein Bündel schließt. Eine halb eingeführte zentrale Zugriffskontrolle ist schlechter als keine, weil sich alle darauf verlassen. Lieber wenige Maßnahmen ganz als viele halb.

## 18.6 Die Risikoakzeptanz

Was nicht in Stufe 1 steht, bleibt vorerst offen. Für jeden dieser Befunde ist damit entschieden, ein Risiko für eine Zeit hinzunehmen. Das ist zulässig, wenn es richtig gemacht wird.

**DEFINITION 18.2 · RISIKOAKZEPTANZ**
Eine Risikoakzeptanz ist die dokumentierte Entscheidung des Risikoeigners, ein bestimmtes Risiko für einen bestimmten Zeitraum unter bestimmten Bedingungen nicht weiter zu behandeln.

Jedes Wort der Definition schließt eine verbreitete Unsitte aus. *Dokumentiert* schließt das stillschweigende Liegenlassen aus. *Des Risikoeigners* schließt aus, dass die Entwicklung oder die Sicherheit für das Geschäft entscheidet. *Bestimmtes Risiko* schließt Pauschalen wie „die mittleren Befunde werden akzeptiert“ aus. *Bestimmter Zeitraum* schließt die Akzeptanz auf ewig aus. *Bestimmte Bedingungen* verlangt, dass klar ist, wann die Entscheidung neu zu treffen ist.

**Risikoakzeptanz F-10**

```
Befund:         F-10 - Services nehmen Anfragen aus dem internen Netz
                ohne Authentifizierung an (hoch)
Risiko:         Wer einen Service übernimmt oder Anfragen aus dem Inneren
                senden kann, erreicht die Funktionen aller Services.
Bekannte Wege:  F-05 (behoben durch S3), F-03 (S1, Nachtest ausständig)
Entscheidung:   Behebung (M3) wird auf das Quartal nach der Saison
                verschoben.
Grund:          Änderungsstopp vor der Saison; M3 berührt jede Verbindung
                zwischen den Services.
Abmilderung:    Netzregeln zwischen den Services enger gezogen;
                M4 trennt die Datenbankkonten; M10 meldet auffällige
                Zugriffe.
Gilt bis:       31. März, danach neue Entscheidung
Neu zu treffen, wenn: ein weiterer Weg ins Innere bekannt wird;
                F-03 im zweiten Nachtest nicht behoben ist
Entschieden:    Bereichsleitung Digital, nach Stellungnahme der
                Informationssicherheit (abweichend: fordert M3 sofort)
```

Zwei Zeilen dieses Formulars sind die wichtigsten. Die Zeile *Neu zu treffen, wenn* macht aus der Akzeptanz eine Entscheidung unter Annahmen. Wird ein neuer Weg ins Innere bekannt, gilt sie nicht mehr. Und die letzte Zeile hält fest, dass die Sicherheit anderer Meinung war. Eine abweichende Stellungnahme zu dokumentieren ist kein Zeichen von Streit, sondern von Sorgfalt. Sie zeigt, dass die Entscheidung in Kenntnis des Einwands getroffen wurde.

Es gibt Risiken, die sich nicht akzeptieren lassen. Eine Organisation kann ein Risiko für sich selbst annehmen. Sie kann nicht im Namen ihrer Kunden annehmen, dass deren Gesundheitsdaten für andere

lesbar sind, und sie kann keine gesetzliche Pflicht durch eine interne Entscheidung aufheben. Bei *Kurbad* ist F-07 deshalb kein Kandidat für eine Akzeptanz, gleich was es kostet.

## 18.7 Die Vorlage für die Entscheidung

Der Risikoeigner kann nicht über zwölf Maßnahmen und sechzehn Befunde entscheiden, und er soll es auch nicht. Er entscheidet zwischen wenigen Optionen, die für ihn vorbereitet sind. Eine gute Vorlage hat eine Seite und vier Teile.

**Die Lage** in drei Sätzen, in der Sprache der Zusammenfassung aus Kapitel 13: was jemand von außen tun konnte, auf welche Ursachen es zurückgeht, was schon getan ist.

**Die Optionen**, meist drei, mit Kosten, Wirkung und dem Risiko, das bleibt.

**Die Empfehlung** mit Begründung und mit den abweichenden Stimmen.

**Was entschieden werden muss:** die Wahl der Option und die Annahme der genannten Restrisiken.

**Tab. 67:** Drei Optionen für die Entscheidungsvorlage zu *Kurbad*.

| | Option A: Minimum | Option B: gestuft | Option C: vollständig |
|---|---|---|---|
| Inhalt | Sofortmaßnahmen, Schutzschicht vor der Anwendung, Ursachen im Frühjahr | Stufe 1 jetzt, Stufen 2 und 3 in den Folgequartalen | alle Maßnahmen ab sofort |
| Aufwand jetzt | etwa 8 Tage | 45 Tage | 107 Tage |
| Familienkarte | nicht berührt | nicht berührt | verschiebt sich um ein Quartal |
| Was offen bleibt | alle Ursachen; weitere 19 Endpunkte ohne Besitzprüfung | Vertrauen im Inneren und Tokens für ein bis zwei Quartale | nichts Bekanntes |
| Hauptrisiko | Abfluss von Kundendaten auf einem der offenen Wege; Prüfung durch Großkunden nicht bestanden | Einbruch in einen Service erreicht in der Saison die anderen | Ausfälle in der Saison durch große Umbauten; Vertragsstrafe |
| Stimmen | Produkt dafür | Betrieb und Compliance dafür, Sicherheit mit Vorbehalt | Sicherheit dafür |

Die Vorlage in Tab. 67 nennt für jede Option ein Hauptrisiko, auch für die vollständige. Das ist ehrlich und es ist klug. Eine Vorlage, in der eine Option keine Nachteile hat, wird als Werbung gelesen. Und sie nennt Option A, obwohl die Verfasser sie für falsch halten. Der Risikoeigner soll sehen, dass sie erwogen wurde und warum sie nicht empfohlen wird: Sie lässt neunzehn Endpunkte offen, von denen man weiß.

## 18.8 Die Entscheidung festhalten

Entscheidungen über die Architektur werden in vielen Teams als *Architecture Decision Records* festgehalten, kurze Texte mit Kontext, Entscheidung, erwogenen Alternativen und Folgen. Dein Foliensatz zur Architekturdokumentation behandelt sie. Für die Maßnahmen an der Architektur aus dem Plan ist das die passende Form, und sie hat einen besonderen Wert für die Sicherheit.

Ein Entscheidungsdokument zu M1 hält fest, dass Objekte nur über die Datenzugriffsschicht mit Besitzer geladen werden, und warum: wegen der Befunde F-07 und F-08 und der neunzehn weiteren Stellen. Wer in zwei Jahren einen Endpunkt baut und sich fragt, warum er nicht direkt auf die Datenbank zugreifen darf, findet die Antwort. Ohne das Dokument sieht die Regel wie eine Schikane aus und wird bei der nächsten Gelegenheit umgangen. Der Pentest-Bericht selbst taugt dafür nicht, denn er ist vertraulich und in zwei Jahren vergessen.

Die offenen Befunde gehören an eine zweite Stelle. In einer Dokumentation nach arc42 ist das der Abschnitt zu Risiken und technischen Schulden. Ein akzeptiertes Sicherheitsrisiko ist eine Schuld mit einem Fälligkeitsdatum, und sie gehört dorthin, wo die anderen Schulden stehen, damit sie bei der Planung gesehen wird.

**BRÜCKE ZU SOFTWARE SECURITY**

Was dieses Kapitel beschreibt, ist eine Bewertung von Architekturalternativen unter widerstreitenden Qualitätszielen. Dein Foliensatz zur Architektur kennt dafür ein Verfahren: Stakeholder nennen ihre Szenarien, die Szenarien werden gewichtet, und für jede Alternative wird sichtbar gemacht, wo sie ein Ziel auf Kosten eines anderen erreicht. Die vier Stellungnahmen sind solche Szenarien in roher Form. Die Sicherheit beschreibt ein Sicherheitsszenario, der Betrieb eines zur Verfügbarkeit, das Produkt eines zur Änderbarkeit und Lieferfähigkeit. Die Stelle, an der M3 die Sicherheit erhöht und die Verfügbarkeit gefährdet, ist ein Zielkonflikt im Sinn dieses Verfahrens. Du kannst die Studierenden die Entscheidung mit genau diesem Vokabular begründen lassen.

**FRAGE AUS DEM HÖRSAAL**

***Was mache ich, wenn der Risikoeigner eine Entscheidung trifft, die ich für falsch halte?***

Du hältst deine abweichende Einschätzung schriftlich fest, sorgst dafür, dass sie Teil der Entscheidung ist, und setzt die Entscheidung um. Das ist die Rollenverteilung: Fachleute beraten, Verantwortliche entscheiden. Die Grenze liegt dort, wo eine Entscheidung gegen Gesetze verstößt oder Dritte gefährdet, die nicht zustimmen konnten. Dann gibt es Wege nach oben und in vielen Organisationen eigene Meldestellen. Für den gewöhnlichen Fall, in dem jemand ein Risiko anders gewichtet als du, ist die dokumentierte abweichende Meinung das richtige Mittel.

**FRAGE AUS DEM HÖRSAAL**

***Ist es nicht zynisch, Sicherheit gegen einen Liefertermin abzuwägen?***

Es wäre zynisch, es zu verschweigen. Abgewogen wird immer, in jeder Organisation, bei jedem Befund. Die Frage ist nur, ob es offen geschieht, durch jemanden mit Verantwortung und mit einem Datum, oder verdeckt, indem ein Ticket liegen bleibt. Das offene Verfahren zwingt dazu, das Risiko zu benennen, und führt in der Praxis dazu, dass weniger hingenommen wird, weil niemand gern seinen Namen unter eine schlechte Begründung setzt.

**FRAGE AUS DEM HÖRSAAL**

***Wer vertritt in dieser Runde eigentlich die Kunden?***

Niemand von sich aus, und das ist eine Schwäche des Verfahrens. Am nächsten kommt der Datenschutz, dessen gesetzlicher Auftrag der Schutz der Betroffenen ist. Es hilft, die Frage in der Vorlage ausdrücklich zu stellen: Wessen Schaden ist das Hauptrisiko jeder Option? Bei Option A in der Tabelle tragen ihn die Kunden, bei Option C das Unternehmen. Das sichtbar zu machen verändert die Abwägung.

**FÜR DIE LEHRE**

Die widersprüchlichen Stellungnahmen sind der Teil der Case Study, bei dem ein KI-Assistent am wenigsten hilft, und deshalb der Teil, der am besten zwischen den Gruppen trennt. Ein Assistent fasst vier Stellungnahmen bereitwillig zusammen und schlägt einen ausgewogenen Kompromiss vor, der jedem etwas gibt. Das ist der Fallstrick von oben. Damit die Aufgabe trägt, brauchen die Stellungnahmen drei Eigenschaften.

Sie enthalten mindestens eine Bewertung, die dem Bericht widerspricht, wie hier „die kritische Lücke ist geschlossen“ und „das interne Netz ist nicht erreichbar“. Wer die Stellungnahmen ungeprüft übernimmt, baut auf einer falschen Annahme.

Sie enthalten echte Tatsachen, die den Plan einschränken, wie den Änderungsstopp und den Vertragstermin. Wer sie übergeht, legt einen Plan vor, der nicht umsetzbar ist.

Und sie lassen etwas Wichtiges aus, das nur findet, wer den Bericht und die Unterlagen kennt, wie hier die Gutscheinaktion zusammen mit F-09.

In der mündlichen Verteidigung kannst du die Rolle des Risikoeigners oder eines der vier Stakeholder übernehmen und nachfragen: „Sie verschieben M3. Was sagen Sie der Sicherheit, wenn in der Saison etwas passiert?“ Oder umgekehrt: „Sie ziehen M3 vor. Wer trägt die Vertragsstrafe?“ Bewertet wird nicht,

welche Option die Gruppe wählt, sondern ob sie das Restrisiko benennen kann, ob ihre Reihenfolge den genannten Kriterien folgt und ob sie unterscheiden kann, was sie weiß und was sie annimmt.

Wenn du mit mehreren „Richtern“ arbeiten willst, also mit mehreren Personen oder KI-Rollen, die die Abgabe aus je einer Sicht beurteilen, dann gib jeder Sicht ein eigenes kurzes Raster. Eine Abgabe, die bei allen vier Sichten mittelmäßig abschneidet, ist meist der Kompromiss ohne Begründung. Eine gute Abgabe enttäuscht mindestens eine Sicht und sagt, warum.

Typische Fehlvorstellungen: dass es eine richtige Reihenfolge gebe, dass die Sicherheit immer recht habe, dass ein akzeptiertes Risiko ein erledigtes sei, und dass der Kompromiss die neutrale Lösung sei.

## Selbsttest

1. Was macht jemanden zum Risikoeigner, und warum ist es weder die Entwicklung noch die Informationssicherheit?
2. Nenne für jede der vier Positionen das berechtigte Anliegen und den blinden Fleck.
3. Unterscheide in dieser Aussage Tatsache, Bewertung und Forderung: „Vier Wochen vor der Saison gilt ein Änderungsstopp. Das interne Netz ist ohnehin sicher. M3 kommt nicht in Frage.“
4. Nenne die fünf Kriterien für die Reihenfolge von Maßnahmen.
5. Warum steht M9 im Vorschlag in derselben Stufe wie M1?
6. Warum ist ein Kompromiss, der von jeder Forderung ein wenig erfüllt, meist die schlechteste Lösung?
7. Welche fünf Bestandteile der Definition machen eine Risikoakzeptanz aus?
8. Welche Risiken kann eine Organisation nicht akzeptieren?
9. Aus welchen vier Teilen besteht eine Entscheidungsvorlage, und warum nennt sie auch für die vollständige Option ein Hauptrisiko?
10. Wozu dient ein Entscheidungsdokument zu einer Sicherheitsmaßnahme in zwei Jahren?

# 19 Die Spezialisierung unterrichten

Achtzehn Kapitel lang hast du gelernt, was in der Spezialisierung vorkommt. Die Kästen „Für die Lehre“ haben an jeder Stelle angemerkt, wie sich der Stoff vermitteln lässt. Dieses Kapitel setzt die Stücke zusammen. Es behandelt, wie aus dem Inhalt eine Lehrveranstaltung wird: wie sie sich von deiner Pflicht-Lehrveranstaltung und vom Pentesting abgrenzt, wie du eine Case Study baust, die schwer genug ist und sich trotzdem fair bewerten lässt, was der Einsatz von KI-Assistenten daran ändert und wie die mündliche Verteidigung abläuft.

Das Kapitel ist als Arbeitsunterlage gedacht. Es wiederholt nicht die Festlegungen aus dem Studienplan und den internen Datenblättern, sondern gibt dir das Handwerk, um sie zu füllen. Wo Zahlen stehen, etwa zur Aufteilung von Befunden oder zur Dauer einer Prüfung, sind es Vorschläge.

**IN DIESEM KAPITEL**

- die Spezialisierung gegen Software Security und Pentesting abgrenzen und ihre Lernergebnisse an den Kapiteln dieses Buchs festmachen
- eine Case Study vom Lösungsschlüssel her entwerfen
- ein Dossier zusammenstellen, in dem die Belege für Ursachen zu finden sind
- festlegen, was KI-Assistenten in der Case Study leisten dürfen und was die Studierenden selbst zeigen müssen
- Abgabe, Bewertungsraster und mündliche Verteidigung aufeinander abstimmen

## 19.1 Was die Spezialisierung leisten soll

Die drei Lehrveranstaltungen, um die es geht, betrachten dasselbe System aus drei Richtungen.

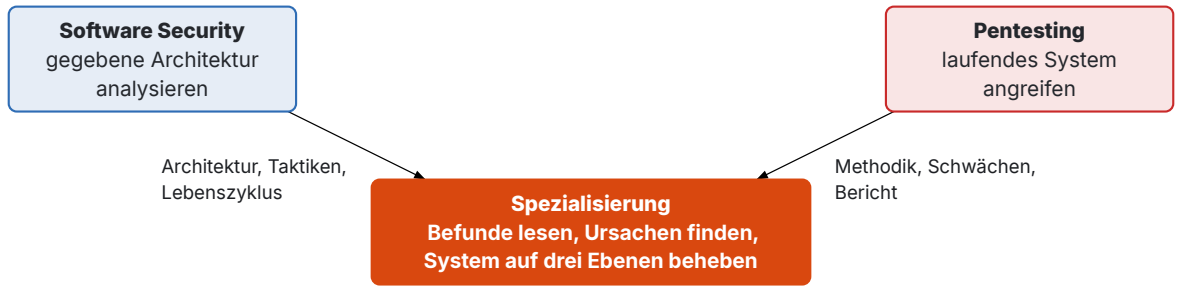



**Abb. 32:** Die Spezialisierung führt die beiden Pflicht-Lehrveranstaltungen zusammen.

In **Software Security** bekommen die Studierenden eine fertige Architektur und suchen darin nach Schwächen. Die Richtung ist vom Entwurf zum möglichen Fehler. Im **Pentesting** bekommen sie ein laufendes System und suchen nach Schwächen, die sich tatsächlich ausnutzen lassen. Die Richtung ist von außen nach innen. In der **Spezialisierung** bekommen sie das Ergebnis eines solchen Tests und gehen den Weg zurück: vom Befund zur Ursache in Code, Architektur und Lebenszyklus, und von dort zu einem Plan, der beides verbindet.

Daraus ergeben sich die Grenzen. Die Spezialisierung lehrt nicht noch einmal, wie man eine Architektur analysiert, und nicht noch einmal, wie man angreift. Sie setzt beides voraus und verlangt die Verbindung. Wer nur eine der beiden Pflicht-Lehrveranstaltungen wirklich verstanden hat, scheitert an ihr auf eine von zwei kennzeichnenden Arten: Er korrigiert Stellen im Code, ohne die Struktur zu sehen, oder er entwirft eine neue Architektur, ohne zu prüfen, ob sie die Befunde behebt.

Was die Studierenden am Ende können sollen, lässt sich an den Teilen III und IV dieses Buchs festmachen.

**Tab. 68:** Lernergebnisse der Spezialisierung und ihre Nachweise.

| Lernergebnis | Kapitel | Zeigt sich in |
|---|---|---|
| einen Pentest-Bericht kritisch lesen und beurteilen | 13, 14 | Liste der Aussagegrenzen, geprüfte Bewertungen |
| Befunde prüfen, bündeln und verketten | 15 | Triage-Tabelle, Bündel, Angriffsbaum |
| Ursachen auf drei Ebenen ermitteln und belegen | 16 | Ursachenanalyse je Bündel mit Belegen aus dem Dossier |
| Behebungen entwerfen und von Abmilderungen unterscheiden | 17, Teil II | Maßnahmenplan mit Dreispaltentest |
| unter widersprüchlichen Anforderungen eine Reihenfolge begründen | 18 | Entscheidungsvorlage, Risikoakzeptanzen |
| die eigene Lösung erklären und gegen Einwände vertreten | alle | mündliche Verteidigung |

Die rechte Spalte von Tab. 68 ist die wichtige. Jedes Lernergebnis hat ein Artefakt, an dem es sich zeigt, und die Prüfung besteht aus genau diesen Artefakten. Das ist der Gedanke der konstruktiven Abstimmung von Lernergebnis, Lernaktivität und Prüfung [73]: Geprüft wird, was gelernt werden soll, in der Form, in der es später gebraucht wird.

## 19.2 Die Case Study vom Ende her bauen

Die naheliegende Art, eine Case Study zu bauen, ist: ein System erfinden, Schwächen hineinlegen, einen Bericht schreiben. So entsteht eine Liste unverbundener Befunde, und die Aufgabe wird zu dem, was Kapitel 15 als Fehler beschrieben hat: fünfundzwanzig Tickets.

Die bessere Art ist, vom Lösungsschlüssel auszugehen und rückwärts zu arbeiten.

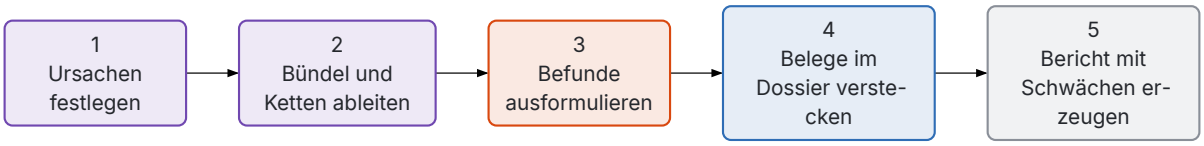



**Abb. 33:** Der Bau einer Case Study in fünf Schritten, vom Lösungsschlüssel zum Bericht.

### Schritt 1: Ursachen festlegen

Am Anfang stehen vier bis sechs Ursachen auf den Ebenen Architektur und Lifecycle. Sie sind das, was die Studierenden am Ende finden sollen. Für *Kurbad* waren es zwei Architekturthemen und drei Prozessthemen: keine Stelle erzwingt Sicherheitseigenschaften, es gibt keine gemeinsame Grundlage der Services, Sicherheitsanforderungen fehlen, die Tests prüfen nur den guten Fall, und ein Teil des Systems läuft außerhalb aller Regeln.

Gute Ursachen für eine Case Study haben drei Eigenschaften. Sie sind realistisch, also in echten Systemen häufig. Sie sind im Dossier belegbar. Und jede erzeugt mehrere Befunde verschiedener Art, damit das Bündeln etwas zu tun hat.

### Schritt 2: Bündel und Ketten ableiten

Aus jeder Ursache leitest du drei bis sechs Befunde ab und legst fest, welche davon Ketten bilden. Hier entsteht die Schwierigkeit der Aufgabe, und du kannst sie gezielt einstellen. Die folgende Verteilung ist ein Vorschlag für einen Bericht mit etwa fünfundzwanzig Befunden.

**Tab. 69:** Zutaten für einen Bericht mit etwa 25 Befunden. Ein Befund kann mehrere Eigenschaften haben.

| Zutat | Anzahl | Wozu |
|---|---|---|
| Bündel mit gemeinsamer Ursache | 5 bis 6 | Kern der Aufgabe |
| Ketten aus zwei oder drei Befunden | 3 | Triage, Bewertung, Reihenfolge |
| davon mit einem Glied der Stufe niedrig oder mittel | 2 | gegen das Sortieren nach Schweregrad |
| Befunde mit verschiedener CWE und gleicher Ursache | 2 Paare | gegen das Bündeln nach Textähnlichkeit |
| Befunde mit gleicher CWE und verschiedener Ursache | 1 Paar | dasselbe in der anderen Richtung |
| falsch positive Befunde | 2 | Prüfen statt glauben |
| zu hoch oder zu niedrig bewertete Befunde | 3 | Kapitel 14 |
| Befund, der in der Produktion anders zu bewerten ist | 1 | Übertragung |
| Befunde, die nur im Code behebbar sind | 2 bis 3 | gegen den Reflex zum großen Umbau |
| Befunde, die im Code nicht behebbar sind | 2 bis 3 | gegen die reine Stellenkorrektur |
| Befunde, deren naheliegende Korrektur eine Abmilderung ist | 2 | Kapitel 17 |
| Befunde ohne tiefere Ursache | 3 bis 4 | Rauschen, wie in echten Berichten |

Die letzte Zeile wird leicht vergessen. Ein Bericht, in dem jeder Befund Teil eines Musters ist, ist unrealistisch und verrät sich. Echte Berichte enthalten Einzelfälle, und zu erkennen, dass ein Befund keiner weiteren Analyse bedarf, ist auch eine Leistung.

**Schritt 3: Befunde ausformulieren**
Erst jetzt schreibst du die Befunde, in der Form aus Kapitel 13. Halte in deinem Lösungsschlüssel für jeden Befund fest: Zustand nach der Prüfung, eigene Bewertung, Bündel, Ketten, Ursache auf jeder Ebene, Behebung und Abmilderung. Diese Tabelle ist dein Korrekturschlüssel, und sie zu haben, bevor der Bericht existiert, ist der ganze Vorteil des Vorgehens.

**Schritt 4: Belege im Dossier verstecken**
Die Studierenden bekommen nicht nur den Bericht. Sie bekommen ein Dossier über das System, so wie ein Team in einem Unternehmen seine eigenen Unterlagen hat. Das Dossier ist das, was die Aufgabe gegen bloßes Abfragen eines KI-Assistenten schützt, denn in ihm stehen die Belege für die Ursachen, und nirgends sonst.

**Tab. 70:** Bestandteile des Dossiers und was sie für die Aufgabe leisten.

| Unterlage | Inhalt | Was sich darin verbirgt |
|---|---|---|
| Architekturbeschreibung | Kontext, Bausteine, Laufzeitsicht, Verteilung, Entscheidungen | fehlende zentrale Stellen, geteilte Konten, Vertrauensgrenzen, die nie gezogen wurden |
| Ausschnitte aus dem Code | einige Endpunkte, Datenzugriff, Konfiguration | die Stelle des Befunds und drei weitere derselben Klasse |
| Beschreibung der Pipeline | Stufen, Prüfungen, was blockiert | Prüfung, die nur meldet; Teil des Systems, der nicht durchläuft |
| Entwicklungsrichtlinie | Vorgaben für Code, Review, Tests | Vorgabe, die fehlt; Vorgabe, die es gibt und die nicht eingehalten wird |
| Testfälle | Auszug aus den automatisierten Tests | nur gute Fälle, nur ein Konto je Rolle |
| Betriebsunterlagen | Netzplan, Konten, Protokollierung, Änderungsregeln | flaches Netz, Schutzschicht nur in der Produktion, Änderungsstopp |
| Geschäftlicher Rahmen | Termine, Verträge, Aktionen, Kundengruppen | was die Stellungnahmen verschweigen |

Für jede Ursache aus Schritt 1 muss es im Dossier mindestens eine Stelle geben, an der sie sich belegen lässt, und du notierst diese Stelle im Lösungsschlüssel. Für mindestens ein Bündel sollte der Code so viel zeigen, dass die Suche nach weiteren Stellen aus Kapitel 15 Erfolg haben kann.

Zwei Dinge sind beim Code zu beachten. Deine Studierenden programmieren wenig, also müssen die Ausschnitte kurz und lesbar sein, und die Aufgabe darf nicht verlangen, größere Mengen Code zu schreiben. Und für absichtlich verwundbaren Code gilt deine Regel: keine Hinweise im Code selbst, weder in Kommentaren noch in Namen. Die Erklärung steht im Lösungsschlüssel.

**Schritt 5: Den Bericht mit Schwächen erzeugen**
Den Bericht kannst du mit einem KI-Assistenten aus deinen ausformulierten Befunden erzeugen lassen. Das geht schnell und hat einen nützlichen Nebeneffekt: Der Bericht bekommt von selbst einige der Schwächen, die Kapitel 13 beschrieben hat, etwa allgemeine Textbausteine und allzu vollständige Empfehlungen. Die übrigen baust du gezielt ein: einen Befund ohne brauchbaren Nachweis, eine CWE, die nicht passt, einen Wert, der nicht zum Vektor passt, eine Auswirkung, die nur vermutet ist.

Danach kommen die vier Stellungnahmen aus Kapitel 18 mit ihren drei Eigenschaften: eine Bewertung, die dem Bericht widerspricht, echte Einschränkungen, und eine Auslassung.

**Der Probelauf**
Bevor du die Case Study ausgibst, bearbeitest du sie selbst so, wie es eine bequeme Gruppe täte: Bericht und Stellungnahmen in einen Assistenten laden, um einen Maßnahmenplan bitten, das Ergebnis abgeben. Was dabei herauskommt, ist deine Messlatte nach unten.

Halte das Ergebnis gegen den Lösungsschlüssel. Findet der Assistent die Bündel? Erkennt er die falsch positiven Befunde? Nennt er Ursachen mit Beleg? Unterscheidet er Behebung und Abmilderung? Wenn die bequeme Abgabe mehr als ein Genügend erreicht, ist die Case Study zu leicht, und du verschärfst sie an den Stellen aus Tab. 69. Diesen Probelauf solltest du jedes Jahr wiederholen, denn die Assistenten werden besser.

> **MERKE**
> Eine Case Study ist dann gut gebaut, wenn die Antwort eines KI-Assistenten auf die bloße Aufgabenstellung plausibel aussieht und am Lösungsschlüssel scheitert, und wenn sie mit dem Dossier und eigenem Urteil gut lösbar ist.

## 19.3 Was KI in der Case Study leistet

Die Studierenden dürfen und sollen KI-Assistenten verwenden. Das ist kein Zugeständnis, sondern entspricht der Arbeit, auf die der Studiengang vorbereitet. Es verschiebt nur, was geprüft wird: nicht mehr das Hervorbringen eines Textes, sondern das Urteil darüber.

**Tab. 71:** Stärken und Schwächen von KI-Assistenten in den Schritten der Case Study und was daraus für die Aufgabenstellung folgt.

| Schritt | Was ein Assistent gut kann | Wo er versagt | Was du deshalb verlangst |
|---|---|---|---|
| Bericht lesen | zusammenfassen, Begriffe erklären, Vektoren übersetzen | übernimmt Behauptungen ungeprüft, übersieht Aussagegrenzen | Liste dessen, worüber der Bericht nichts sagt |
| Bewertung | Formel anwenden, wenn man sie verlangt | Werte, die nicht zum Vektor passen; kennt den Schutzbedarf nicht | eigener Wert mit Begründung aus dem Dossier |
| Triage | nach Ähnlichkeit gruppieren | Bündel nach Ursache, Ketten über die Architektur, falsch positive Befunde | die eine Maßnahme je Bündel; Angriffsbaum |
| Ursachen | Ebene Code | Architektur und Lifecycle nur als Gemeinplatz | Beleg aus dem Dossier zu jeder Aussage |
| Behebung | Korrektur einer gezeigten Stelle | Klasse, Nebenwirkungen; Abmilderung als Behebung | Dreispaltentest; Probe auf Abmilderung |
| Entscheidung | Positionen zusammenfassen | ausgewogener Kompromiss ohne Begründung; prüft Stellungnahmen nicht | Restrisiko je Option; Kriterien für die Reihenfolge |

Die rechte Spalte von Tab. 71 ist die Aufgabenstellung in Kurzform. Jede Forderung dort ist so gewählt, dass sie sich nur mit dem Dossier und eigenem Nachdenken erfüllen lässt.

Zwei Regeln für den Umgang mit KI gehören in die Aufgabenstellung selbst. Die Gruppe legt offen, wofür sie Assistenten verwendet hat. Und sie steht für jede Aussage ihrer Abgabe ein, gleich wer sie formuliert hat. „Das hat der Assistent so geschrieben" ist in der Verteidigung keine Antwort. Dieselbe Regel galt in Kapitel 2 für die Tester.

## 19.4 Die Abgabe

Die Abgabe besteht aus den Artefakten, die die Kapitel 13 bis 18 als Ergebnis jedes Schritts beschrieben haben. Sie in fester Form zu verlangen hat zwei Vorteile: Die Gruppen wissen, was erwartet wird, und du kannst gegen den Lösungsschlüssel korrigieren.

**Tab. 72:** Artefakte der Abgabe.

| | Artefakt | Form | Vorbild |
|---|---|---|---|
| 1 | Beurteilung des Berichts | Aussagegrenzen, Urteil zur Qualität, fünf Fragen an die Tester | Kapitel 13 |
| 2 | Triage-Tabelle | je Befund: Zustand, eigene Bewertung mit Begründung, Bündel, Ketten | Tab. 50 |
| 3 | Bündel und Angriffsbaum | Bündel mit der einen Maßnahme; ein Baum für ein vorgegebenes Ziel | Tab. 51 |
| 4 | Ursachenanalysen | je Bündel ein Formular mit drei Ebenen, Belegen und weiteren Stellen | Kapitel 16 |
| 5 | Maßnahmenplan | Sofortmaßnahmen und Maßnahmen mit Ebene, Art, Aufwand; Dreispaltentest je Bündel | Tab. 63 |
| 6 | Zwei Korrekturen im Code | mit Regressionstest, der vorher fehlschlägt | Kapitel 17 |
| 7 | Entscheidungsvorlage | eine Seite mit drei Optionen, Empfehlung und Restrisiko | Tab. 67 |
| 8 | Risikoakzeptanzen | je verschobenem hohen Befund ein Formular | Kapitel 18 |
| 9 | Erklärung zum Einsatz von KI | wofür, womit, was verworfen wurde | - |

Das sechste Artefakt ist bewusst klein gehalten. Zwei Korrekturen genügen, um zu zeigen, dass die Gruppe eine Behebung im Code prüfen kann, und sie dürfen mit einem Assistenten geschrieben sein.

Verlangt ist der Test, der zuerst rot war. Die Wahl der beiden Befunde kannst du vorgeben: einen, dessen naheliegende Korrektur richtig ist, und einen, bei dem sie eine Abmilderung wäre.

Das neunte Artefakt hat einen Zweck, der über die Kontrolle hinausgeht. Die Frage „was haben Sie vom Vorschlag des Assistenten verworfen und warum“ ist eine der aufschlussreichsten in der Verteidigung.

## 19.5 Bewerten

Die Bewertung folgt den Lernergebnissen. Für jedes Kriterium beschreibst du, woran die Stufen zu erkennen sind. Das Raster ist für die Studierenden öffentlich, der Lösungsschlüssel nicht.

**Tab. 73:** Bewertungsraster in drei Stufen. Die Stufen dazwischen ergeben sich sinngemäß.

| Kriterium | Nicht genügend | Genügend | Sehr gut |
|---|---|---|---|
| Triage | alle Befunde übernommen, nach Schweregrad sortiert | die meisten Bündel erkannt, eine Kette | alle Bündel mit tragender Maßnahme, falsch positive erkannt, Ketten vom Ziel her |
| Bewertung | Werte des Berichts übernommen | Fehler im Bericht erkannt | eigene Werte mit Schutzbedarf begründet, Abweichungen erklärt |
| Ursachen | nur Ebene Code | alle Ebenen benannt, teils ohne Beleg | jede Aussage belegt, weitere Stellen gefunden |
| Maßnahmen | Liste von Stellenkorrekturen | Maßnahmen auf mehreren Ebenen, Abmilderungen nicht erkannt | Dreispaltentest erfüllt, Abmilderungen benannt, Nebenwirkungen bedacht |
| Entscheidung | alles sofort, oder Kompromiss ohne Begründung | Reihenfolge mit Kriterien, Einschränkungen beachtet | Optionen mit Restrisiko, falsche Annahmen der Stellungnahmen erkannt |
| Verteidigung | kann die eigene Abgabe nicht erklären | erklärt, was getan wurde | begründet, warum, und hält Einwänden stand |

Das Raster in Tab. 73 ist streng, und das ist gewollt. Die Stufe *Genügend* verlangt in jeder Zeile mehr, als ein Assistent auf die bloße Aufgabenstellung liefert. Wer die linke Spalte erreicht, hat die Lehrveranstaltung nicht bestanden. Für die Gewichtung zwischen schriftlicher Abgabe und mündlichem Teil und für die Frage, ob beide Teile für sich bestanden sein müssen, gelten die Festlegungen aus deinem Datenblatt.

Für die Korrektur selbst empfiehlt es sich, mit dem Lösungsschlüssel in der Hand zuerst nur drei Dinge zu prüfen: Hat die Gruppe die falsch positiven Befunde erkannt? Hat sie für jedes Bündel eine Ursache mit Beleg? Hat sie die Abmilderungen benannt? Diese drei Punkte lassen sich rasch feststellen und zeigen früh, auf welchem Niveau eine Abgabe liegt.

## 19.6 Die mündliche Verteidigung

Die schriftliche Abgabe ist eine Gruppenleistung, entstanden mit KI-Assistenten. Was eine einzelne Person verstanden hat, lässt sich aus ihr nicht ablesen. Die mündliche Verteidigung ist deshalb kein Anhängsel, sondern der Teil der Prüfung, der die individuelle Leistung feststellt.

Vier Arten von Fragen decken ab, was zu prüfen ist. Jede Person bekommt aus jeder Art mindestens eine.

**Tab. 74:** Vier Arten von Fragen in der mündlichen Verteidigung.

| Art | Was sie prüft | Beispiel |
|---|---|---|
| Erklären | Verständnis der Begriffe aus den Teilen I und II | „Lesen Sie mir diesen Vektor als Satz vor.“ „Was ist der Unterschied zwischen CVE und CWE?“ |
| Begründen | ob die Person hinter der Abgabe steht | „Warum haben Sie diese beiden Befunde in ein Bündel gelegt? Welche eine Maßnahme behebt beide?“ |
| Übertragen | ob das Verständnis über den Fall hinausreicht | „Angenommen, der Service läge hinter einer weiteren Schutzschicht. Was ändert sich an Ihrer Bewertung?“ |
| Angreifen | ob die Person einem Einwand standhält | „Ich bin der Betrieb. Ihre Maßnahme gefährdet die Verfügbarkeit in der Saison. Überzeugen Sie mich.“ |

Für die Durchführung einige Vorschläge. Zehn bis fünfzehn Minuten je Person sollten reichen. Die Fragen stammen aus einem vorbereiteten Vorrat, damit alle vergleichbar geprüft werden, und richten sich auf Teile der Abgabe, die die Person nicht selbst vorstellt. Die erste Frage zielt immer auf das Verständnis und nicht auf die Abgabe: Wer einen Vektor nicht lesen und eine Abmilderung nicht von einer Behebung unterscheiden kann, hat die Grundlage nicht, gleich wie gut das Papier der Gruppe ist. Ein kurzes Protokoll mit den gestellten Fragen und einem Stichwort zur Antwort macht die Bewertung nachvollziehbar.

Die Fragen der Art *Angreifen* sind die ergiebigsten und die, die am meisten Vorbereitung brauchen. Du nimmst dabei eine der vier Rollen aus Kapitel 18 ein. Bewertet wird nicht, ob die Person nachgibt oder beharrt, sondern ob sie das Anliegen hinter dem Einwand erkennt, ob sie Tatsachen von Bewertungen trennt und ob sie das Restrisiko ihrer Position benennen kann.

## 19.7 Der Ablauf des Blocks

Die Lehrveranstaltung läuft geblockt. Wie viele Einheiten zur Verfügung stehen, legt der Studienplan fest. Die folgende Gliederung in fünf Abschnitte passt sich an und ordnet jedem Abschnitt die Kapitel dieses Buchs zu.

**Tab. 75:** Ein möglicher Ablauf der Spezialisierung in fünf Abschnitten.

| | Abschnitt | Präsenz | Arbeit der Gruppen | Kapitel |
|---|---|---|---|---|
| 1 | Den Bericht lesen | Auffrischung der Landkarte; Aufbau eines Berichts; Lesen eines Übungsberichts | Beurteilung des Berichts der Case Study | 4, 13 |
| 2 | Bewerten und sichten | Vektoren lesen und rechnen; Übung zur Streuung; Triage am Übungsbericht | Triage-Tabelle, Bündel, Angriffsbaum | 14, 15 |
| 3 | Ursachen finden | drei Ebenen; Warum-Kette an einem Beispiel; Arbeit mit dem Dossier | Ursachenanalysen | 16, 5, 6 |
| 4 | Beheben und planen | Behebung und Abmilderung; Regressionstest; Nachtest lesen | Maßnahmenplan, zwei Korrekturen | 17, Teil II |
| 5 | Entscheiden und verteidigen | Stellungnahmen lesen; Optionen bauen; Probeverteidigung | Entscheidungsvorlage, Risikoakzeptanzen | 18 |

Drei Hinweise dazu. Der **Übungsbericht** in den ersten beiden Abschnitten ist ein kleiner, anderer Fall mit sechs bis acht Befunden, an dem du die Verfahren gemeinsam vorführst. Der Bericht zu *Kurbad* aus diesem Buch eignet sich dafür, und Kapitel 20 enthält ihn in gesammelter Form. Die Case Study selbst ist ein anderes System, damit niemand die Lösung abschreiben kann.

Die **Probeverteidigung** im letzten Abschnitt nimmt den Studierenden die Angst und zeigt ihnen, welche Art von Fragen kommt. Lass eine Gruppe freiwillig vor allen fünf Minuten verteidigen und stelle je eine Frage jeder Art.

Und plane **Zeit zwischen den Abschnitten** ein, soweit der Block es erlaubt. Die Ursachenanalyse braucht Stunden mit dem Dossier, und sie lässt sich nicht in einer Präsenzeinheit erledigen.

## 19.8 Abstimmung mit der Pentesting-Lehrveranstaltung

Die Verzahnung mit dem Pentesting läuft über die Lernergebnisse, und zwei Lehrpersonen müssen dafür nicht denselben Fall verwenden. Sie müssen aber dieselben Wörter verwenden. Aus den Teilen I und II lässt sich eine kurze Liste ableiten, die du mit der Kollegin oder dem Kollegen durchgehst.

- Welche Methodik und welcher Prüfkatalog werden gelehrt, und heißen die Phasen so wie in Kapitel 3?
- Schreiben die Studierenden dort einen Bericht, und in welcher Form? Wenn ja, kennen sie die Felder eines Befunds aus eigener Arbeit.
- Welches Bewertungsverfahren in welcher Fassung wird verwendet?
- Welche Klassen von Schwächen kommen vor, und unter welchen Namen? Die Zugriffskontrolle auf Objektebene etwa heißt je nach Quelle BOLA oder IDOR.
- Wie weit geht die Lehrveranstaltung nach dem ersten Zugriff, und kommen Angriffe auf KI-Systeme vor?

Das Ergebnis ist im besten Fall eine halbe Seite mit gemeinsamen Begriffen, die beide Lehrpersonen ihren Unterlagen voranstellen. Für die Studierenden ist das der sichtbarste Teil der Verzahnung: In der Spezialisierung lesen sie einen Bericht in genau der Form, in der sie im Pentesting einen geschrieben haben.

## 19.9 Was schiefgehen kann

**Die Case Study ist zu groß.** Fünfundzwanzig Befunde mit Dossier sind viel. Wenn die Gruppen in der verfügbaren Zeit nicht über die Triage hinauskommen, kürze den Bericht und nicht die Schritte. Fünfzehn Befunde, die alle Zutaten enthalten, sind besser als fünfundzwanzig, bei denen die Entscheidung fehlt.

**Eine Person macht die Arbeit.** Die mündliche Verteidigung fängt das für die Note auf. Für das Lernen hilft, die Rollen aus Kapitel 15 in der Gruppe ausdrücklich zu verteilen und in der Verteidigung jede Person zu einem Teil zu befragen, den laut Aufteilung jemand anderer bearbeitet hat.

**Die Abgaben gleichen einander.** Wenn alle Gruppen denselben Assistenten mit derselben Aufgabenstellung verwenden, sehen die Texte ähnlich aus. Das ist kein Beweis für Abschreiben. Unterscheiden werden sich die Abgaben in den Belegen aus dem Dossier und in der Entscheidung, und daran bewertest du.

**Der große Umbau.** Manche Gruppen entwerfen eine neue Architektur und verlieren die Befunde aus dem Blick. Der Dreispaltentest holt sie zurück: Für jedes Bündel muss in der ersten Spalte etwas stehen, das sofort wirkt.

**Der Fall wird bekannt.** Lösungsschlüssel wandern von Jahrgang zu Jahrgang. Da du vom Ende her baust, kannst du mit geringem Aufwand Varianten erzeugen: dieselben Ursachen in einem anderen System, oder dasselbe System mit zwei ausgetauschten Ursachen. Ein Assistent hilft beim Umschreiben von Bericht und Dossier, der Lösungsschlüssel bleibt deine Arbeit.

> **BRÜCKE ZU SOFTWARE SECURITY**
> Die Case Study der Pflicht-Lehrveranstaltung und die der Spezialisierung lassen sich nach demselben Verfahren bauen. Dort legst du Schwächen in eine Architektur und lässt sie finden. Hier legst du Ursachen in ein System, leitest Befunde ab und lässt den Weg zurück finden. In beiden Fällen entsteht zuerst der Lösungsschlüssel. Wenn du für beide dasselbe Format für Architekturbeschreibungen verwendest, etwa die Sichten aus deinem Foliensatz 04, erkennen die Studierenden in der Spezialisierung die Art der Unterlage wieder und können sich auf den neuen Teil konzentrieren.

> **FRAGE AUS DEM HÖRSAAL**
> ***Wenn wir ohnehin KI verwenden dürfen, was wird dann eigentlich bewertet?***
>
> Ihr Urteil. Der Assistent liefert Vorschläge, und viele davon sind brauchbar. Bewertet wird, ob Sie erkennen, welche es sind: ob ein Befund zutrifft, ob eine Ursache belegt ist, ob eine Korrektur die Schwäche

beseitigt oder nur verdeckt, ob eine Reihenfolge begründet ist. Dafür brauchen Sie die Unterlagen zum System, die der Assistent nicht kennt, und das Wissen, um seine Antwort zu prüfen. In der Verteidigung stehen Sie allein für Ihre Abgabe ein.

FRAGE AUS DEM HÖRSAAL

***Warum wird hier strenger benotet als im Bachelor?***

Weil die Hilfsmittel stärker sind. Was vor wenigen Jahren eine gute Leistung war, nämlich einen strukturierten Maßnahmenplan zu einem Bericht zu schreiben, liefert heute ein Assistent in einer Minute. Die Messlatte liegt deshalb dort, wo die Arbeit des Assistenten aufhört. Das Raster ist öffentlich, Sie wissen also, was verlangt ist.

FRAGE AUS DEM HÖRSAAL

***Müssen wir programmieren können?***

Sie müssen Code lesen können, in kurzen Ausschnitten, und eine Korrektur beurteilen. Schreiben dürfen Sie die beiden verlangten Korrekturen mit einem Assistenten. Sie müssen aber zeigen, dass der Test vorher fehlschlug und danach gelingt, und erklären können, warum die Korrektur die Schwäche beseitigt. Der größere Teil der Arbeit liegt in Architektur und Prozess, nicht im Code.

## Selbsttest

1. Worin unterscheiden sich die Blickrichtungen von Software Security, Pentesting und der Spezialisierung?
2. Warum baust du eine Case Study vom Lösungsschlüssel her und nicht vom System her?
3. Nenne sechs Zutaten eines Berichts für die Case Study und wozu jede dient.
4. Welche Unterlagen gehören in das Dossier, und warum schützt es die Aufgabe gegen bloßes Abfragen eines Assistenten?
5. Wozu dient der Probelauf, und wann ist die Case Study zu leicht?
6. Was kann ein Assistent bei der Ursachenanalyse, was nicht, und was verlangst du deshalb?
7. Welche neun Artefakte umfasst die Abgabe?
8. Welche drei Punkte prüfst du bei der Korrektur zuerst?
9. Nenne die vier Arten von Fragen in der Verteidigung mit je einem Beispiel.
10. Was stimmst du mit der Lehrperson des Pentestings ab?

# 20 Ein Fall von Anfang bis Ende

Die Plattform *Kurbad* hat dich durch das ganze Buch begleitet, in Stücken: hier ein Scope, dort ein Befund, später Bündel, Ursachen, ein Plan und vier Stellungnahmen. Dieses Kapitel setzt die Stücke in der Reihenfolge zusammen, in der sie geschehen sind, von der Beauftragung des Tests bis zum zweiten Test ein Jahr später. Es enthält wenig Neues und verweist für die Einzelheiten zurück. Sein Zweck ist ein anderer: Du sollst den ganzen Bogen einmal am Stück sehen, so wie ihn die Studierenden in der Case Study durchlaufen, und du bekommst den Fall in einer Form, die du als Übungsfall in der Lehre verwenden kannst.

Der Fall ist erfunden. Er ist so gebaut, wie Kapitel 19 es beschreibt, also vom Ende her, und er ist absichtlich etwas ordentlicher als die Wirklichkeit.

## 20.1 Das System und seine Befunde

Zur Erinnerung das System: Kunden buchen über eine Web-Oberfläche und eine App, Mitarbeiter arbeiten in einer Admin-Oberfläche. Ein API-Gateway prüft die Anmeldung und leitet an drei Services weiter. Die Services sprechen untereinander direkt und über einen Nachrichten-Broker. Anmeldung und Zahlung liegen bei externen Diensten.

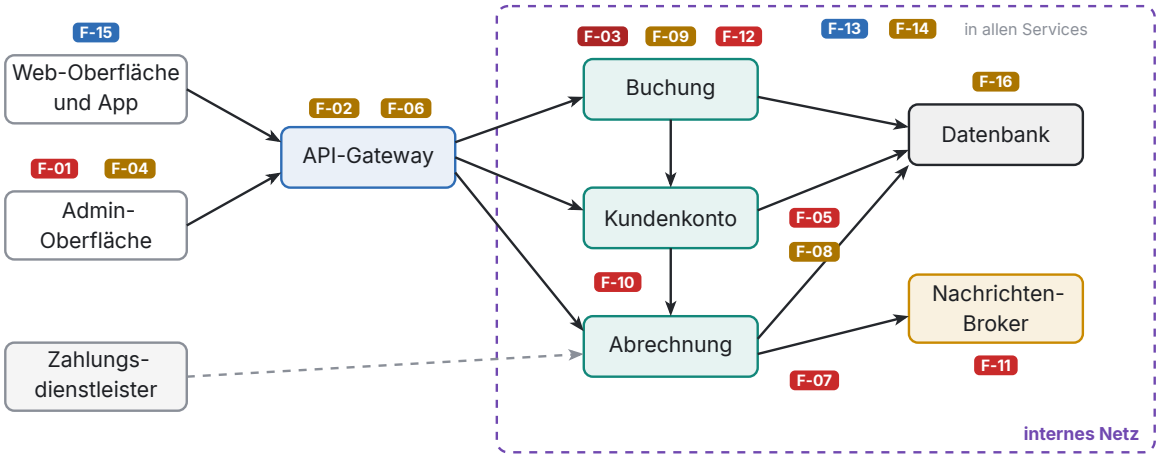



**Abb. 34:** Das System *Kurbad* mit den Orten der sechzehn Befunde. Dunkelrot kritisch, rot hoch, gelb mittel, blau niedrig. Der gestrichelte Weg vom Zahlungsdienstleister war in der Testumgebung durch eine Attrappe ersetzt.

Schon die Verteilung in Abb. 34 sagt etwas. Am Rand, wo die Tester begonnen haben, sitzen die Befunde zu Anmeldung und Eingaben. Im Inneren sitzen die, die aus einem Einbruch einen größeren machen. Und an einem Weg, dem gestrichelten, sitzt gar nichts. Dort wurde nicht getestet.

## 20.2 Woche 0: Der Auftrag

Der Betreiber beauftragt einen Test der Plattform vor der Wintersaison. Vereinbart werden acht Testtage in der Testumgebung, als Grey-Box-Test mit je zwei Konten für Kunde, Kassenkraft und Administrator und mit der Beschreibung der Schnittstelle. Ausgeschlossen sind der Identitätsdienst, der Zahlungsdienstleister, die Cloud-Plattform, die Kassen in den Thermen, Lasttests und die Täuschung von Mitarbeitern. Ein Nachtest von zwei Tagen ist Teil des Auftrags. Die Einzelheiten stehen in Tab. 3.

Zwei Entscheidungen aus dieser Woche wirken bis zum Ende nach. Die Testumgebung ersetzt die Anbindung an den Zahlungsdienstleister durch eine Attrappe, und niemand hält das für wichtig. Und der Nachtest wird gleich mit beauftragt, was sich in Woche 6 bezahlt macht.

## 20.3 Wochen 1 und 2: Der Test

Was die Tester in acht Tagen getan haben, lässt sich aus dem Bericht rekonstruieren.

**Tab. 76:** Der Verlauf des Tests, aus dem Bericht rekonstruiert.

| Tage | Tätigkeit | Befunde |
|---|---|---|
| 1 | Aufklärung, Kartieren der Anwendung, Abgleich mit der Schnittstellenbeschreibung | F-01, F-13, F-15 |
| 2 bis 3 | Zugriffskontrolle mit je zwei Konten über Kreuz; Verarbeitung von Eingaben | F-07, F-08, F-03, F-04, F-05 |
| 4 bis 5 | nach dem Zugriff: was ist von F-05 und F-03 aus erreichbar | F-10, F-16, F-11 |
| 6 | Anmeldung und Sitzung; ein halber Tag Fachlogik | F-02, F-06, F-09 |
| 7 | Komponenten, Konfiguration; Rückfrage an den Betrieb nach den eigenen Spuren | F-12, F-14 |
| 8 | Bericht | |

Der Verlauf erklärt die Abdeckungsmatrix in Tab. 41. Zwei Tage gingen in die Verfolgung der Angriffspfade, und sie haben die wertvollsten Befunde gebracht. Dafür blieb für die Fachlogik ein halber Tag und für die Admin-Oberfläche kaum etwas. F-14 entstand aus einer Rückfrage: Die Tester baten den Betrieb, ihre abgelehnten Zugriffe vom dritten Tag in den Protokollen zu suchen. Es gab keine.

## 20.4 Woche 3: Der Bericht

Der Bericht enthält sechzehn Befunde, einen kritischen, sechs hohe, sieben mittlere und zwei niedrige, dazu zwei Angriffspfade und eine Abdeckungsmatrix. Die Übersicht steht in Tab. 42.

Das Team liest in der Reihenfolge aus Kapitel 13 und hält als Erstes fest, worüber der Bericht nichts sagt:

- die Admin-Oberfläche, die fast durchgehend nur teilweise geprüft wurde
- die mobile App in den meisten Kategorien
- die Fachlogik über den einen geprüften Ablauf hinaus
- die Rückmeldungen des Zahlungsdienstleisters, weil dort eine Attrappe stand
- alles, was nur in der Produktion gilt: Konfiguration, Zugangsdaten, Netzregeln

In der Abschlussbesprechung fragt das Team, wo die Tester mit zwei Tagen mehr weitergesucht hätten. Die Antwort: bei der Fachlogik und in der Admin-Oberfläche.

## 20.5 Woche 3: Sofortmaßnahmen

Noch in derselben Woche setzt das Team die sieben Sofortmaßnahmen aus Tab. 62 um und tauscht alle Zugangsdaten, die im Bericht vorkommen.

Dann die unbequeme Frage, ob schon jemand vor den Testern da war. Für F-07 lässt sie sich teilweise beantworten. Die Protokolle des Gateways halten fest, welches Konto welche Adresse abgerufen hat, und reichen dreißig Tage zurück. In dieser Zeit hat kein Konto mehr Rechnungen abgerufen als die eigenen. Über die Zeit davor lässt sich nichts sagen. Der Datenschutz hält die Prüfung und ihr Ergebnis schriftlich fest, mit dieser Einschränkung.

## 20.6 Woche 4: Triage

Die Triage nach Kapitel 15 verändert das Bild in fünf Punkten.

1. Fünfzehn Befunde werden bestätigt. Für F-02 belegt der Betrieb mit der Konfiguration des Gateways, dass die Anmeldeversuche in der Produktion begrenzt sind. Der Befund bleibt als Hinweis bestehen, dass der Schutz an einer einzigen Stelle hängt.
2. Die eigene Bewertung weicht an vier Stellen ab, wie in Tab. 49: F-07 steigt wegen des Schutzbedarfs, F-08 und F-13 sinken. F-04 wird höher eingestuft, weil das gespeicherte Skript in der Admin-Oberfläche ausgeführt wird.
3. Aus sechzehn Befunden werden sechs Bündel, siehe Tab. 51.

4. Zu den zwei Ketten des Berichts kommen zwei weitere. Das Ziel „Rechnungsdaten aller Kunden lesen“ ist auf vier unabhängigen Wegen erreichbar, siehe Abb. 28.
5. Die Befunde F-04 und F-16, im Bericht beide *mittel*, gehören zu den dringenden.

## 20.7 Wochen 4 und 5: Ursachen

Die Ursachenanalyse nach Kapitel 16 führt die sechs Bündel auf fünf Themen zurück, siehe Tab. 57. In der Architektur: Keine Stelle erzwingt Sicherheitseigenschaften, und die Services haben keine gemeinsame Grundlage. Im Lebenszyklus: Sicherheitsanforderungen fehlen, die Tests prüfen nur den guten Fall, und die Admin-Oberfläche läuft außerhalb aller Regeln.

Die Suche nach weiteren Stellen bringt mehr als der Bericht. Von 23 Endpunkten, die ein Objekt über eine Kennung ansprechen, prüfen vier den Besitz. Eine eigens geschriebene Regel der statischen Analyse findet drei weitere Abfragen, die aus Text zusammengesetzt sind, zwei davon in der Admin-Oberfläche. Die Tester hatten in acht Tagen je eine Stelle gefunden.

## 20.8 Woche 5: Plan und Entscheidung

Der Maßnahmenplan aus Tab. 63 umfasst zwölf Maßnahmen mit 107 Personentagen. Verfügbar sind 45. Die vier Stellungnahmen aus Kapitel 18 liegen vor, und das Team legt der Bereichsleitung eine Vorlage mit drei Optionen vor, siehe Tab. 67.

Die Bereichsleitung entscheidet sich für die gestufte Option und ändert sie an zwei Stellen. Der Entwurf für die Identitäten der Services, also die Vorarbeit zu M3, beginnt sofort und nicht erst nach der Saison, damit die Umsetzung am ersten Tag nach dem Änderungsstopp starten kann. Und vor der Gutscheinaktion werden zwei zusätzliche Testtage für die Fachlogik beauftragt, weil die Abdeckungsmatrix dort die größte Lücke zeigt. Für F-10 und F-06 unterschreibt sie je eine Risikoakzeptanz mit Ablaufdatum. Die abweichende Stellungnahme der Informationssicherheit wird festgehalten.

Beide Änderungen sind lehrreich. Sie stammen nicht aus den Stellungnahmen, sondern aus den Unterlagen, die das Team vorgelegt hat: der Abhängigkeit zwischen M4 und M3 und der Abdeckungsmatrix. Eine gute Vorlage macht es dem Risikoeigner möglich, besser zu entscheiden als jede der vorgeschlagenen Optionen.

## 20.9 Woche 6: Der erste Nachtest

Der Nachtest nach den Sofortmaßnahmen meldet fünf Befunde als behoben, zwei als abgemildert und einen als nicht behoben: F-03, den kritischen. Das Team hatte unter Zeitdruck die Eingabe gefiltert, statt die Abfrage zu parametrisieren, und der eigene Test wiederholte nur die Anfrage aus dem Bericht. Die Geschichte steht am Anfang von Kapitel 17.

Die Korrektur dauert einen halben Tag. Wichtiger ist, was das Team daraus ändert: Regressionstests zu Befunden prüfen von da an die Eigenschaft und nicht den Nachweis, und die Regel der statischen Analyse, die zusammengesetzte Abfragen findet, wird in die Pipeline aufgenommen und blockiert. Das ist ein vorgezogenes Stück von M8, und es kostet zwei Tage aus der Reserve.

Die zusätzlichen Testtage für die Fachlogik bringen drei neue Befunde, darunter einen zur Stornierung, mit dem sich eine Rückzahlung für eine nicht bezahlte Buchung auslösen ließ. Die Abdeckungsmatrix hatte recht.

## 20.10 Das Quartal: Stufe 1

Die Umsetzung von Stufe 1 verläuft nicht glatt, und die Störungen sind die, die Kapitel 17 angekündigt hat.

Die zentrale Zugriffskontrolle M1 unterbindet am ersten Tag einen Monatsbericht der Buchhaltung. Er hatte sich seit Jahren mit einem gewöhnlichen Kundenkonto alle Rechnungen geholt. Die Lösung ist eine eigene Rolle mit dem nötigen Recht, also eine Entscheidung, die bis dahin nie jemand getroffen hatte.

Die Berechtigungstests M9 schlagen in der dritten Woche bei zwei Endpunkten fehl, die auf keiner Liste standen. Sie waren im laufenden Quartal für die Familienkarte gebaut worden, noch auf die alte Art. Ohne M9 wären sie in die Produktion gegangen. Mit diesem Vorfall hört die Diskussion auf, ob die Tests den Aufwand wert sind.

Die Trennung der Datenbankkonten M4 deckt auf, dass der Kundenkontoservice an zwei Stellen direkt in Tabellen der Buchung liest. Das war niemandem bekannt und stand in keiner Architekturbeschreibung.

Am Ende des Quartals ist Bündel A an Stelle, Klasse und Ursache geschlossen, die Kette von F-03 zu den Daten aller Services gebrochen, die Admin-Oberfläche am Identitätsdienst, und abgelehnte Zugriffe werden protokolliert und gemeldet. Die Prüfung durch den Großkunden besteht das Unternehmen mit dem Maßnahmenplan, den Risikoakzeptanzen und den Ergebnissen von M9 als Nachweis.

## 20.11 Ein Jahr später

Der nächste Pentest findet nach der Umsetzung aller drei Stufen statt, diesmal mit einem anderen Anbieter und mit der echten Anbindung an den Zahlungsdienstleister in einer eigenen Umgebung. Er hat neun Befunde, keinen kritischen und zwei hohe.

Was fehlt, ist die Nachricht. Es gibt keinen Befund zur Zugriffskontrolle auf Objektebene, keinen zur Injection und keinen zum Vertrauen zwischen den Services. Der Bericht vermerkt unter den positiven Feststellungen, dass die Tester am ersten Tag einen Anruf vom Betrieb bekamen, weil die Häufung abgelehnter Zugriffe einen Alarm ausgelöst hatte.

Die beiden hohen Befunde sind die lehrreichen.

Der erste betrifft die Rückmeldungen des Zahlungsdienstleisters. Der Abrechnungsservice nimmt sie an, ohne zu prüfen, ob sie vom Zahlungsdienstleister stammen. Wer die Adresse kennt, kann eine Buchung als bezahlt markieren. Kapitel 6 hat diese Stelle als die genannt, an der ein Tester sofort stehen bleibt. Sie bestand die ganze Zeit. Der erste Test konnte sie nicht finden, weil dort eine Attrappe stand, und die Einschränkung stand im Bericht. Das Team hatte sie in Woche 3 sogar notiert. Es folgte nichts daraus, weil eine Aussagegrenze kein Ticket ist.

Der zweite betrifft eine neue Funktion. Im Lauf des Jahres hat die Plattform einen Assistenten bekommen, der Fragen von Kunden beantwortet und Buchungen nachschlägt. Er bezieht Bewertungen anderer Kunden ein, und sein Werkzeug arbeitet mit einem Dienstkonto, das alle Buchungen sieht. Das ist die Lage aus Kapitel 12. Die Maßnahme M12, Sicherheitsanforderungen und ein Bedrohungsmodell für neue Funktionen, stand in Stufe 3 und war erst seit wenigen Wochen in Kraft. Der Assistent war davor entworfen worden.

## 20.12 Was der Fall zeigt

Aus dem Bogen lassen sich sieben Sätze ziehen, die das Buch zusammenfassen.

1. **Der Bericht ist eine Stichprobe mit Grenzen.** Die folgenreichste Lücke des Systems stand nicht im ersten Bericht, sondern in seinen Einschränkungen.
2. **Die Zahl ist nicht das Urteil.** Zwei Befunde der Stufe *mittel* gehörten zu den dringendsten, und ein Wert von 6,5 beschrieb den Abfluss von Gesundheitsdaten.
3. **Sechzehn Befunde waren fünf Ursachen.** Erst das Bündeln hat gezeigt, worüber zu entscheiden war.
4. **Die Suche im eigenen Code findet mehr als der Test.** Auf eine gefundene Stelle kamen neunzehn weitere.
5. **Eine Behebung ist erst eine, wenn die Schwäche weg ist.** Der kritische Befund galt drei Wochen lang zu Unrecht als behoben.
6. **Die Reihenfolge ist eine Entscheidung mit Namen und Datum.** Zwei Risiken wurden bewusst ein Quartal lang getragen, und jemand hat dafür unterschrieben.
7. **Die Ebene Lifecycle wirkt am spätesten und am längsten.** Die Tests über Kreuz haben neue Fehler abgefangen, bevor sie ausgeliefert wurden. Die Regel für neue Funktionen kam für den Assistenten zu spät.

Der siebte Satz enthält eine Kritik an der Reihenfolge, die in Kapitel 18 als vertretbar vorgestellt wurde. M12 kostete fünf Tage und stand am Ende, weil es „nur auf Neues wirkt“. In einem Jahr, in dem viel Neues gebaut wird, ist genau das der Grund, es an den Anfang zu stellen. Auch ein begründeter Plan kann sich als falsch erweisen, und zu einer ehrlichen Nachbetrachtung gehört, das festzuhalten.

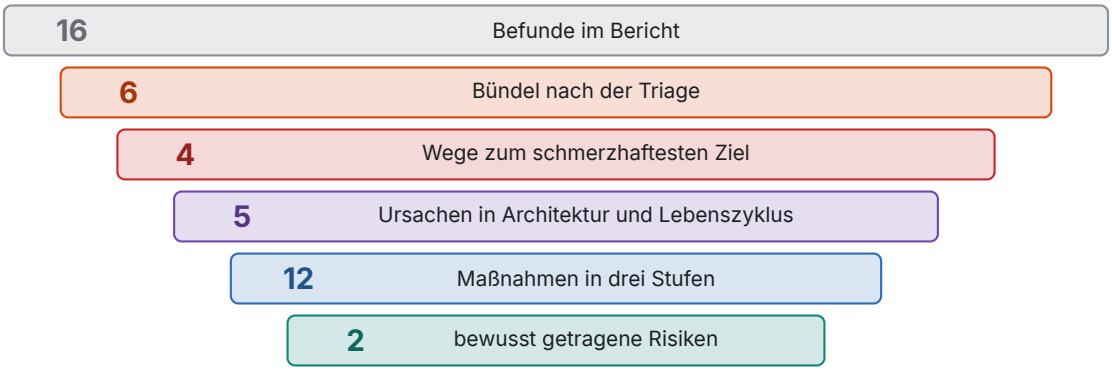



**Abb. 35:** Der Fall *Kurbad* in Zahlen.

## 20.13 Der Fall als Übungsfall

Du kannst *Kurbad* in den ersten Abschnitten der Lehrveranstaltung als gemeinsamen Übungsfall verwenden, an dem du die Verfahren vorführst, bevor die Gruppen an der eigentlichen Case Study arbeiten. Für eine Präsenzeinheit ist der ganze Bericht zu groß. Eine Auswahl von acht Befunden enthält alles, was sich zeigen lässt.

**Tab. 77:** Eine Auswahl von acht Befunden für die Arbeit in einer Präsenzeinheit.

| Auswahl | Was sich daran zeigen lässt |
|---|---|
| F-03, F-16 | Kette; ein Befund ohne CVSS-Wert wird dringend; Behebung an der Stelle und Begrenzung in der Architektur |
| F-05, F-10 | Kette; welches Glied man bricht; ein Befund, der im Code keine Stelle hat |
| F-07, F-08 | Bündel bei verschiedener CWE; Wert und Stufe; Schutzbedarf; weitere Stellen |
| F-04 | zu niedrige Bewertung; Bedeutung des Kontexts |
| F-13 | Wert höher als das Urteil; Informationsbefund als Trittstein |

Dazu passen vier Aufgaben, die sich in je zwanzig bis dreißig Minuten bearbeiten lassen:

- Übersetzt die Vektoren von F-03 und F-07 in Sätze und prüft, ob sie zum Text der Befunde passen.
- Legt für jeden der acht Befunde fest, was er braucht und was er liefert, und zeichnet den Angriffsbaum für ein Ziel eurer Wahl.
- Bildet Bündel und nennt für jedes die eine Maßnahme.
- Führt für F-07 die Warum-Kette bis zur Ebene Lifecycle und sagt bei jeder Stufe, welchen Beleg ihr dafür bräuchtet.

Die Lösungen stehen in den Kapiteln 14 bis 16. Die Studierenden sollten sie erst danach lesen.

> **FÜR DIE LEHRE**
> Der zweite Pentest ist das stärkste Stück des Falls für die Lehre, und es lohnt, ihn erst am Ende zu erzählen. Frage die Studierenden nach der Arbeit am Übungsfall, was sie in einem Jahr im nächsten Bericht erwarten. Die meisten nennen Befunde derselben Art wie im ersten. Dann zeigst du, was tatsächlich kam: die Stelle, die nie im Scope war, und die Funktion, die es noch nicht gab. Beides sind Ursachen auf der Ebene Lifecycle, und beides war aus den Unterlagen des ersten Jahres absehbar.
>
> Daran lässt sich die Frage anschließen, mit der du die Lehrveranstaltung beenden kannst: Was hätte in Woche 3 geschehen müssen, damit die notierte Aussagegrenze zu einer Maßnahme wird? Die Antwort, auf die die Studierenden kommen sollen, ist unspektakulär. Aussagegrenzen gehören in dieselbe Liste wie Befunde, mit einem Verantwortlichen und einem Datum.

## Selbsttest

1. Welche Entscheidung aus Woche 0 hat dazu geführt, dass die Lücke bei den Rückmeldungen des Zahlungsdienstleisters ein Jahr lang unentdeckt blieb?
2. Wie hat das Team die Frage beantwortet, ob F-07 schon ausgenutzt wurde, und wo lag die Grenze der Antwort?

3. Welche zwei Änderungen hat die Bereichsleitung an der gestuften Option vorgenommen, und worauf stützten sie sich?
4. Warum meldete der erste Nachtest F-03 als nicht behoben, und was hat das Team daraufhin geändert?
5. Nenne zwei Nebenwirkungen, die bei der Umsetzung von Stufe 1 aufgetreten sind.
6. Was fehlte im Bericht des zweiten Pentests, und warum ist das die eigentliche Nachricht?
7. Welche Kritik an der Reihenfolge der Maßnahmen ergibt sich aus dem Befund zum Assistenten?

ANHANG

# A Spickzettel

Dieser Anhang sammelt, was man beim Lesen eines Berichts und in der Lehre zum Nachschlagen braucht. Er erklärt nichts, sondern erinnert. Die Erklärungen stehen in den genannten Kapiteln.

## A.1 Kürzel auf einen Blick

| Kürzel | Steht für | Was es ist | Kap. |
|---|---|---|---|
| ASVS | Application Security Verification Standard | Katalog von Sicherheitsanforderungen an Anwendungen (OWASP) | 3 |
| ATLAS | Adversarial Threat Landscape for AI Systems | Wissensbasis zu Angriffen auf KI-Systeme (MITRE) | 12 |
| ATT&CK | Adversarial Tactics, Techniques and Common Knowledge | Wissensbasis zum Vorgehen von Angreifern (MITRE) | 4 |
| BFLA | Broken Function Level Authorization | fehlende Prüfung, ob eine Rolle eine Funktion aufrufen darf | 9 |
| BOLA | Broken Object Level Authorization | fehlende Prüfung, ob ein Benutzer dieses Objekt sehen darf; auch IDOR | 9 |
| BOPLA | Broken Object Property Level Authorization | fehlende Prüfung auf der Ebene einzelner Felder | 9 |
| BSIMM | Building Security In Maturity Model | beschreibendes Reifegradmodell | 5 |
| CAPEC | Common Attack Pattern Enumeration and Classification | Katalog von Angriffsmustern (MITRE) | 4 |
| CRA | Cyber Resilience Act | EU-Verordnung für Produkte mit digitalen Elementen | 5 |
| CSRF | Cross-Site Request Forgery | fremde Seite löst Anfragen mit der Sitzung des Opfers aus | 9 |
| CVE | Common Vulnerabilities and Exposures | Nummer für eine konkrete, öffentlich bekannte Lücke | 4 |
| CVSS | Common Vulnerability Scoring System | Verfahren zur Bewertung der technischen Schwere (FIRST) | 4, 14 |
| CWE | Common Weakness Enumeration | Katalog von Fehlerarten (MITRE) | 4 |
| DAST | Dynamic Application Security Testing | automatisierte Prüfung der laufenden Anwendung von außen | 5 |
| DFD | Datenflussdiagramm | Modell für die Bedrohungsmodellierung | 6 |
| EPSS | Exploit Prediction Scoring System | geschätzte Wahrscheinlichkeit der Ausnutzung in 30 Tagen (FIRST) | 4 |
| KEV | Known Exploited Vulnerabilities | Katalog nachweislich ausgenutzter Lücken (CISA) | 4 |

| Kürzel | Steht für | Was es ist | Kap. |
|---|---|---|---|
| MFA | Multi-Factor Authentication | Anmeldung mit mehr als einem Faktor | 9 |
| NVD | National Vulnerability Database | Datenbank, die CVE-Einträge anreichert (NIST) | 4 |
| OSINT | Open Source Intelligence | Aufklärung aus öffentlichen Quellen | 7 |
| PTES | Penetration Testing Execution Standard | Methodik in sieben Phasen | 3 |
| RAG | Retrieval-Augmented Generation | Sprachmodell mit Suche in einer Wissensbasis | 12 |
| RoE | Rules of Engagement | Spielregeln eines Tests | 2 |
| SAMM | Software Assurance Maturity Model | vorschreibendes Reifegradmodell (OWASP) | 5 |
| SAST | Static Application Security Testing | automatisierte Prüfung des Quelltexts | 5 |
| SBOM | Software Bill of Materials | Stückliste der Komponenten einer Software | 5 |
| SCA | Software Composition Analysis | Abgleich der Abhängigkeiten mit bekannten Lücken | 5 |
| SSDF | Secure Software Development Framework | Katalog von Praktiken sicherer Entwicklung (NIST SP 800-218) | 5, 16 |
| SSDLC | Secure Software Development Lifecycle | Entwicklungsprozess mit Sicherheit in jeder Phase | 5 |
| SSRF | Server-Side Request Forgery | der Server ruft eine vom Angreifer bestimmte Adresse ab | 8 |
| SSVC | Stakeholder-Specific Vulnerability Categorization | Entscheidungsbaum für die Dringlichkeit | 14 |
| STRIDE | Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege | Schema für Bedrohungen | 6 |
| VEX | Vulnerability Exploitability eXchange | Angabe, ob eine Lücke in einem Produkt ausnutzbar ist | 5 |
| WAF | Web Application Firewall | Schutzschicht vor einer Web-Anwendung | 17 |
| WSTG | Web Security Testing Guide | Prüfkatalog für Web-Anwendungen (OWASP) | 3 |
| XSS | Cross-Site Scripting | fremdes Skript läuft in der Seite der Anwendung | 8 |

## A.2 Einen CVSS-Vektor lesen (Version 3.1)

| Kürzel | Metrik | Werte, vom schwersten zum leichtesten | Die Frage |
|---|---|---|---|
| AV | Attack Vector | N Netz, A benachbartes Netz, L lokal, P physisch | Von wo aus? |
| AC | Attack Complexity | L niedrig, H hoch | Braucht es Umstände, die der Angreifer nicht steuert? |
| PR | Privileges Required | N keine, L gewöhnlicher Benutzer, H Administrator | Mit welchen Rechten vorher? |
| UI | User Interaction | N keine, R nötig | Muss ein anderer Mensch mitwirken? |
| S | Scope | C verändert, U unverändert | Trifft der Schaden eine andere Komponente? |
| C, I, A | Confidentiality, Integrity, Availability | H hoch, L gering, N keine | Wie viel wird lesbar, veränderbar, gestört? |

Stufen: 0,1 bis 3,9 niedrig, 4,0 bis 6,9 mittel, 7,0 bis 8,9 hoch, 9,0 bis 10,0 kritisch. Umgebungsmetriken: CR, IR, AR für den Schutzbedarf, dazu jede Basismetrik mit vorangestelltem M. Version 4.0 erkennst du an `CVSS:4.0/`, an AT, an VC, VI, VA und SC, SI, SA statt des Scope.

Prüfgang für eine Bewertung (Abschnitt 14): Vektor und Fassung angegeben? Passt der Vektor als Satz zum Text? Auswirkung belegt? Scope begründet? Stimmt die Rechnung? Abweichung von Wert und Stufe begründet? Was ändert sich in der eigenen Umgebung?

## A.3 STRIDE

| Kategorie | Verletzt | Frage |
|---|---|---|
| Spoofing | Authentizität | Kann sich jemand als ein anderer ausgeben? |
| Tampering | Integrität | Kann jemand Daten unbefugt verändern? |
| Repudiation | Nachweisbarkeit | Kann jemand eine Handlung abstreiten? |
| Information Disclosure | Vertraulichkeit | Kann jemand Daten sehen, die nicht für ihn sind? |
| Denial of Service | Verfügbarkeit | Kann jemand das System unbenutzbar machen? |
| Elevation of Privilege | Autorisierung | Kann jemand mehr tun, als ihm zusteht? |

An jeder Vertrauensgrenze: Wer steht auf der anderen Seite? Woran erkennt das System das? Was passiert, wenn es sich irrt?

## A.4 OWASP Top 10:2025

| Nr. | Kategorie | Nr. | Kategorie |
|---|---|---|---|
| A01 | Broken Access Control | A06 | Insecure Design |
| A02 | Security Misconfiguration | A07 | Authentication Failures |
| A03 | Software Supply Chain Failures | A08 | Software and Data Integrity Failures |
| A04 | Cryptographic Failures | A09 | Security Logging and Alerting Failures |
| A05 | Injection | A10 | Mishandling of Exceptional Conditions |

## A.5 Schwächen und ihre Behebung im Überblick

| Schwäche | CWE | Behebung an der Stelle | Behebung der Ursache | Kap. |
|---|---|---|---|---|
| SQL-Injection | 89 | Abfrage parametrisieren | Datenzugriff, der nur parametrisiert geht; blockierende Analyse | 8 |
| Cross-Site Scripting | 79 | passend zum Kontext kodieren | Vorlagen, die von selbst kodieren | 8 |
| SSRF | 918 | Ziel gegen Erlaubnisliste prüfen | ausgehende Verbindungen beschränken, Prüfung im Inneren | 8 |
| Fremdes Objekt abrufbar | 639 | Besitz prüfen | Besitzprüfung zentral erzwingen; Tests über Kreuz | 9 |
| Mass Assignment | 915 | erlaubte Felder benennen | getrennte Modelle für Eingang und Ausgang | 9 |
| Funktion ohne Rollenprüfung | 862 | Rolle am Endpunkt prüfen | zentrale Richtlinie, Verbot als Voreinstellung | 9 |
| Fehlende Authentifizierung zwischen Services | 306 | Aufrufer prüfen | Identität je Service, Prüfung an jeder Verbindung | 9, 11 |
| Zugangsdaten im Code oder Abbild | 798 | entfernen und tauschen | Geheimnisse zur Laufzeit aus einem Tresor | 9, 11 |
| Sitzung nicht widerrufbar | 613 | Laufzeit kürzen (mildert) | Verfahren mit Widerruf | 9 |
| Fehlkonfiguration | je nach Fall | Einstellung korrigieren | Härtungsstandard, Konfiguration als Code, Prüfung | 10 |
| Komponente mit bekannter Lücke | 1395 | aktualisieren | Stückliste, laufende Prüfung, Fristen | 10 |
| Wettlauf bei Zustandswechsel | 362 | Prüfen und Ändern in einem Schritt | Muster für Zustandsübergänge, Missbrauchsfälle | 10 |
| Fehlende Protokollierung | 778 | Ereignis protokollieren | zentrale Sammlung und Alarmregeln | 10 |
| Prompt Injection | - | nur Abmilderung | fremde Inhalte, Daten und Wirkung nach außen trennen | 12 |

## A.6 Die drei Ebenen

| Ebene | Leitfrage | Eine Behebung hier wirkt auf |
|---|---|---|
| Code | Was genau ist an dieser Stelle falsch? | diese Stelle |
| Architektur | Warum konnte es dort falsch sein, und warum reicht der Schaden so weit? | alle Stellen dieser Art, auch künftige |
| Lifecycle | Warum ist es entstanden, und warum hat es niemand bemerkt? | alle Fehler, die auf demselben Weg entstehen |

## A.7 Vom Bericht zum Plan in sechs Schritten

1. **Lesen** (Abschnitt 13): Scope, Einschränkungen und Abdeckung; Befundübersicht; Angriffspfade; einzelne Befunde; Nachfragen. Die Zusammenfassung zuletzt.
2. **Bewerten** (Abschnitt 14): Vektor als Satz lesen, mit dem Text vergleichen, eigenen Wert mit Schutzbedarf festhalten.
3. **Sichten** (Abschnitt 15): prüfen, auf die Produktion übertragen, nach der gemeinsamen Maßnahme bündeln, über „braucht“ und „liefert“ verketten, ordnen.
4. **Ursachen finden** (Abschnitt 16): je Bündel die drei Leitfragen, jede Antwort mit Beleg, Suche nach weiteren Stellen.
5. **Beheben** (Abschnitt 17): Sofortmaßnahme, Korrektur der Klasse, Behebung der Ursache. Probe auf Abmilderung. Regressionstest, der zuerst rot ist. Dreispaltentest.

6. **Entscheiden** (Abschnitt 18): Tatsachen, Bewertungen und Forderungen trennen. Reihenfolge nach Wirkung, Aufwand, Zeitfenster, Abhängigkeiten, Pflichten. Optionen mit Restrisiko. Risikoakzeptanz mit Namen und Datum.

## A.8 Zehn Fragen für die mündliche Verteidigung

1. Lesen Sie diesen Vektor als Satz vor. Passt er zum Befund?
2. Welche Aussagen lässt dieser Bericht nicht zu?
3. Warum haben Sie diese Befunde gebündelt? Welche eine Maßnahme behebt alle?
4. Auf welchen Wegen ist dieses Ziel erreichbar, und was folgt daraus für die Reihenfolge?
5. Wo steht in den Unterlagen der Beleg für diese Ursache?
6. Angenommen, diese Maßnahme wird umgangen. Ist die Schwäche dann noch da?
7. Dieser Befund ist behoben. Warum kann der nächste dieser Art nicht mehr entstehen?
8. Was kostet Ihre Maßnahme an Verfügbarkeit, Geschwindigkeit oder Bedienbarkeit?
9. Welches Risiko bleibt bei Ihrer Reihenfolge offen, wer trägt es, und bis wann?
10. Was haben Sie vom Vorschlag des Assistenten verworfen, und warum?

# B Lösungen zu den Selbsttests

Die Lösungen sind knapp gehalten. Sie nennen den Kern der Antwort und, wo es hilft, die Stelle im Buch. Bei Fragen nach einem eigenen Beispiel steht ein mögliches. Wenn deine Antwort anders lautet und sich begründen lässt, ist sie nicht deshalb falsch.

## B.1 Kapitel 1: Was ist ein Pentest?

1. Beauftragt (trennt vom Angriff), begrenzt (trennt von der dauerhaften Überwachung), zielgerichtet auf das Ausnutzen (trennt vom Schwachstellenscan), mit Nachweis (trennt von der bloßen Behauptung).
2. Einen Schwachstellenscan mit Deckblatt. Erkennbar an der großen Zahl, an Textbausteinen, am fehlenden Nachweis und daran, dass kein Befund dabei ist, den nur ein Mensch finden kann.
3. Weil der Tester Konten für mehrere Rollen und die Beschreibung der Schnittstelle hat. Er verliert keine Zeit mit dem Erraten und kann die Zugriffskontrolle über Rollen hinweg prüfen, wo die schwersten Befunde liegen.
4. Stichprobe (nur ein Teil wurde angesehen), Momentaufnahme (nur der Stand dieses Tages), Auftrag (nur der Scope mit den erlaubten Mitteln).
5. Der Pentest sucht in einem begrenzten Scope möglichst viele Schwächen und ist angekündigt. Der Red-Team-Einsatz verfolgt ein bestimmtes Ziel mit allen Mitteln, möglichst unbemerkt, und prüft damit auch Erkennung und Reaktion.
6. Die Besitzprüfung wird am gemeldeten Endpunkt für Rechnungen ergänzt. Die Ursache, dass jeder Endpunkt selbst an die Prüfung denken muss, bleibt, und im nächsten Jahr fehlt sie bei den Gutschriften.

## B.2 Kapitel 2: Auftrag, Recht und Rules of Engagement

1. Die Einwilligung des Berechtigten. Technisch sind die Anfragen gleich, rechtlich handelt nur die Testerin befugt.
2. Einwilligen kann nur, wer über das System verfügen darf. Der Zahlungsdienstleister ist ein Dritter. Geprüft werden kann die eigene Seite der Schnittstelle, die fremde nur mit dessen Erlaubnis oder in einer dafür vorgesehenen Testumgebung.
3. Welche Ziele? Welche Umgebung? Welche Rollen und welches Vorwissen? Welche Tiefe? Was ist ausgeschlossen?
4. Ob Lasttests und Angriffe auf die Verfügbarkeit überhaupt im Scope waren. Meist sind sie ausgeschlossen, dann sagt der Bericht dazu nichts.
5. Weil die Tester Konten, Daten und manchmal Dateien hinterlassen, die selbst Schwachstellen sind. Es muss vereinbart sein, wer was wann entfernt.
6. Personenbezogene Daten gehen ohne Rechtsgrundlage an einen Dritten, die Vertraulichkeitsvereinbarung wird verletzt, und Wissen über Schwächen des Auftraggebers verlässt den vereinbarten Kreis.
7. Sie gleicht der Produktion nie genau, also müssen Befunde übertragen werden. Und Fehler, die nur in Konfiguration, Daten oder Netz der Produktion stecken, werden nicht gefunden.

## B.3 Kapitel 3: Methodik

1. Abdeckung, Wiederholbarkeit, Vergleichbarkeit und Nachweis gegenüber Dritten.
2. Pre-engagement Interactions zu Planning. Intelligence Gathering, Threat Modeling und Vulnerability Analysis zu Discovery. Exploitation und Post Exploitation zu Attack. Reporting zu Reporting.

3. WSTG ist der Prüfkatalog, ATHZ die Kategorie der Autorisierungstests, 04 der vierte Test darin, die Prüfung auf direkte Objektreferenzen. Mit der Kennung lässt sich nachschlagen, was geprüft wurde, der Test lässt sich wiederholen, und die Abdeckung lässt sich darstellen.
4. Der ASVS nennt Anforderungen, die eine Anwendung erfüllen soll. Der WSTG nennt Tests, mit denen man prüft. Der eine gehört an den Anfang eines Projekts, der andere ans Ende.
5. Weil sie bestimmt, wo der Tester seine knappe Zeit einsetzt, und das hängt von Bausteinen, Datenflüssen und Vertrauensgrenzen ab. Wer die Architektur kennt, findet die lohnenden Stellen.
6. Die Abdeckung. Ohne sie ist offen, ob Bereiche ohne Befund geprüft wurden oder nicht.
7. Gut automatisierbar sind das Auffinden von Systemen und Diensten und der Abgleich mit bekannten Lücken. Nicht automatisierbar sind Fachlogik, Zugriffskontrolle über Rollen hinweg und Verkettungen, weil sie Verständnis des Systems verlangen.

## B.4 Kapitel 4: Die Landkarte

1. CVE: welche konkrete Lücke in welchem Produkt. CWE: welche Art von Fehler. CVSS: wie schwer technisch. EPSS: wie wahrscheinlich demnächst ausgenutzt. KEV: nachweislich schon ausgenutzt. CAPEC: nach welchem Muster angegriffen. ATT&CK: wie Angreifer insgesamt vorgehen.
2. Schwäche: „Objekte werden über eine Kennung geladen, ohne den Besitzer zu prüfen.“ Schwachstelle: „Der Endpunkt für Rechnungen von *Kurbad* in Version 3 gibt fremde Rechnungen heraus.“
3. CVE-Nummern gibt es für öffentlich verzeichnete Lücken in Produkten. EPSS und KEV hängen an CVE-Nummern und sagen über Fehler im eigenen Code nichts. Die Dringlichkeit muss dort jemand begründen.
4. CWE-20 ist ein sehr allgemeiner Eintrag. Die Zuordnung zeigt, dass die Fehlerart nicht bestimmt wurde. Verlangt wird eine Zuordnung auf der Ebene Base oder Variant.
5. Die Taktik ist das Ziel eines Schritts, etwa der erste Zugang. Die Technik ist der Weg dorthin, etwa das Ausnutzen einer öffentlich erreichbaren Anwendung.
6. In der Regel die mit dem Eintrag in KEV, weil sie tatsächlich angegriffen wird. Zu wissen wäre noch, ob beide Lücken im eigenen System erreichbar sind und wie wichtig die betroffenen Systeme sind.
7. Sie sind grobe Risikokategorien und ändern sich mit jeder Fassung. Für die Einordnung eines Befunds ist die CWE genauer und stabil.
8. Zwei von vier: lückenhafte Abdeckung (nicht Verzeichnetes gilt fälschlich als sicher), Ermessen bei der Zuordnung (Werte streuen), Ranglisten spiegeln ihre Datenquellen (sagen wenig über die eigene Anwendung), verletzliche Infrastruktur (Daten können verspätet sein oder fehlen).

## B.5 Kapitel 5: Der Lebenszyklus

1. Der SSDLC hat in jeder Phase Sicherheitstätigkeiten, von der Anforderung bis zum Betrieb. Der SDLC ohne diesen Zusatz prüft Sicherheit, wenn überhaupt, am Ende.
2. PO, etwa Sicherheitsanforderungen festlegen. PS, etwa die Echtheit einer Auslieferung prüfbar machen. PW, etwa den Entwurf prüfen oder das lauffähige Programm testen. RV, etwa Schwachstellen bewerten und beheben.
3. RV.3. Sie verlangt, die Ursache zu ermitteln, nach weiteren Vorkommen derselben Art zu suchen und den Prozess so zu ändern, dass die Art nicht wieder entsteht.
4. SAMM ist vorschreibend und sagt, was eine Organisation tun sollte. BSIMM ist beschreibend und erhebt, was Organisationen tatsächlich tun.
5. Ob der verwundbare Teil der Bibliothek in dieser Anwendung verwendet wird und erreichbar ist. Ausdrücken lässt sich das mit einer VEX-Angabe.
6. Weil es weder die gewollten Regeln des Geschäfts noch die Rollen kennt. Eine Anfrage, die die Regeln verletzt, sieht für das Werkzeug gültig aus.
7. Es gibt keinen Überblick über die eingesetzten Komponenten und keinen geregelten Weg, sie zu aktualisieren. Das ist eine Ursache auf der Ebene Lifecycle.
8. Ein bekannter, unbehandelter Befund ist für Hersteller im Geltungsbereich nicht mehr nur ein Risiko, sondern auch ein Verstoß gegen eine gesetzliche Pflicht. Aktiv ausgenutzte Schwachstellen sind binnen 24 Stunden zu melden.

## B.6 Kapitel 6: Angriffsfläche und STRIDE

1. Fünf aus: Endpunkte, die nur die App verwendet oder die aus alten Versionen stammen; die Admin-Oberfläche; Rückmeldungen des Zahlungsdienstleisters; Nachrichten auf dem Broker; Verweise in E-Mails; Schnittstellen für Überwachung und Verwaltung; der Weg neuer Software ins System; Menschen mit Zugang.
2. Eine Linie, an der sich ändert, wie weit Daten und Absender vertraut werden darf. Wer steht auf der anderen Seite? Woran erkennt das System das? Was passiert, wenn es sich irrt?
3. Services nehmen an, dass jede Anfrage aus dem Inneren schon geprüft wurde. Das ist ein Architekturfehler, weil ein einziger übernommener Service genügt, um alle anderen zu erreichen, und keine Zeile Code dabei falsch ist.
4. Woran arbeiten wir? Was kann schiefgehen? Was tun wir dagegen? Haben wir gute Arbeit geleistet? Der Pentest beantwortet die vierte.
5. Spoofing: Authentizität. Tampering: Integrität. Repudiation: Nachweisbarkeit. Information Disclosure: Vertraulichkeit. Denial of Service: Verfügbarkeit. Elevation of Privilege: Autorisierung.
6. Sie steht außerhalb der eigenen Kontrolle und hat im System keine Rechte, die sich ausweiten ließen. Bei ihr kommen nur Vortäuschen und Abstreiten in Frage.
7. Am Datenfluss: verfälschen, mitlesen, unterbrechen. Am gewichtigsten ist die gefälschte oder verfälschte Zahlungsmeldung. Die Gegenmaßnahme ist, die Herkunft jeder Meldung zu prüfen, etwa über eine Signatur, und sie nicht an der Absenderadresse zu erkennen.
8. Wie mehrere Schritte zu einem Ziel zusammenwirken und welche Wege es als Alternativen gibt. STRIDE betrachtet jedes Element für sich.
9. Zwei aus: findet Kategorien und keine Schwachstellen; ist so gut wie das Modell; erfasst Fachlogik schlecht; der Aufwand wächst bei großen Systemen stark.

## B.7 Kapitel 7: Aufklärung

1. Vier aus: Namenssystem und Zertifikatsprotokolle (Rechnernamen), Stellenanzeigen und Vorträge (eingesetzte Technik), öffentliche Code-Ablagen (Zugangsdaten, interne Adressen), Web-Archive (frühere Funktionen), berufliche Netzwerke (Namen, Rollen, Schema der Adressen).
2. Offen: Ein Dienst antwortet. Geschlossen: Der Rechner ist erreichbar, auf dem Port lauscht nichts. Gefiltert: Es kommt keine Antwort, weil etwas dazwischen die Anfrage verwirft.
3. Die Erkennung der Version kann irren, etwa bei nachträglich korrigierten Paketen, und der betroffene Teil wird vielleicht nicht verwendet oder ist nicht erreichbar. Es ist ein Verdacht, bis jemand ihn prüft.
4. Die vergessene Umgebung, die alte Version einer Schnittstelle und der Zweitzugang. Gefährlich sind sie, weil niemand sie aktualisiert, überwacht oder abschaltet.
5. Sie hält fest, welche Rolle welche Funktion aufrufen darf, und ist der Sollzustand für den Test. Sie zeigt nicht die Objektebene, also wessen Daten eine Rolle sehen darf.
6. Falsch positiv: Der Scanner meldet aus der Versionsnummer eine Lücke, die längst korrigiert ist. Falsch negativ: Er meldet nichts, obwohl jeder Kunde fremde Rechnungen abrufen kann.
7. Weil sie einem Angreifer den Weg verkürzen und als Trittsteine in Ketten dienen, und weil sechs davon auf fehlende Vorgaben für die Konfiguration hinweisen.
8. Dass die Überwachung Angriffe nicht erkennt. Ein echter Angriff bliebe unbemerkt. Das ist ein eigener Befund.

## B.8 Kapitel 8: Eingaben

1. Daten aus einer nicht vertrauenswürdigen Quelle erreichen eine Senke, an der ein Interpreter einen Text auswertet, in dem Anweisungen und Daten gemischt sind. Der Interpreter liest einen Teil der Daten als Anweisung.
2. Die Parametrisierung übergibt Anweisung und Daten auf getrennten Wegen, sodass Daten nie als Anweisung gelesen werden. Der Filter ist eine Sperrliste und kennt nicht alles, was der Interpreter versteht.
3. Ein Wert wird harmlos gespeichert und später ungeprüft in eine Abfrage eingesetzt. Der Denkfehler ist, Daten aus der eigenen Datenbank für vertrauenswürdig zu halten.

4. Skripte dürfen auf die Daten der Herkunft zugreifen, von der sie stammen. Ein eingeschleustes Skript läuft unter der Herkunft der Anwendung und hat damit alles, was der angemeldete Benutzer dort hat.
5. Gespeichert: Das Skript liegt auf dem Server und wird anderen ausgeliefert. Reflektiert: Es steht in der Anfrage und kommt in der Antwort zurück, das Opfer muss einen präparierten Verweis öffnen. DOM-basiert: Es entsteht erst im Browser durch die Verarbeitung im Skript der Seite.
6. Weil der Schaden davon abhängt, was der Server im Inneren erreichen kann und was ihm dort vertraut: Segmentierung, Authentifizierung zwischen Services, Verwaltungsdienste der Plattform.
7. Die Validierung setzt an der Quelle an und fragt, ob eine Eingabe der Erwartung entspricht. Kodierung und Parametrisierung setzen an der Senke an und sorgen dafür, dass Daten dort Daten bleiben.
8. Eine Sperrliste, also eine Abmilderung. Die Behebung ist, keinen Kommandointerpreter aufzurufen, sondern eine Schnittstelle mit getrennten Argumenten, und Werte gegen eine Erlaubnisliste zu prüfen.
9. Code: den aufgelösten Pfad prüfen oder gar keine Namen aus Eingaben verwenden. Architektur: Dateien außerhalb des Programmverzeichnisses oder in einem eigenen Speicher ablegen, mit minimalen Rechten. Lifecycle: Regel in der statischen Analyse und Punkt in der Prüfliste des Reviews.

## B.9 Kapitel 9: Identität und Zugriff

1. Authentifizierung stellt fest, wer jemand ist, wie der Ausweis am Eingang. Autorisierung stellt fest, was er darf, wie die Eintrittskarte für einen bestimmten Platz.
2. Das Durchprobieren von Paaren aus Benutzername und Passwort, die bei anderen Diensten abgeflossen sind. Die Richtlinie hilft nicht, weil das Passwort stark sein kann und trotzdem bekannt ist. Es helfen ein zweiter Faktor, der Abgleich mit bekannten Passwörtern und die Begrenzung der Versuche.
3. Drei aus Tab. 30, etwa: Signatur nicht geprüft (jeder stellt sich beliebige Angaben aus), Verfahren aus dem Token übernommen (Signatur lässt sich aushebeln), Gültigkeit nicht geprüft (alte und fremde Tokens werden angenommen).
4. Bei XSS läuft ein Skript des Angreifers in der Anwendung und kann lesen und handeln wie der Benutzer. Bei CSRF löst der Angreifer eine Anfrage mit der Sitzung des Opfers aus, sieht aber die Antwort nicht.
5. BFLA ist die Funktionsebene, etwa ein Kunde stellt Gutscheine aus. BOLA ist die Objektebene, etwa ein Kunde ruft eine fremde Rechnung ab. BOPLA ist die Eigenschaftsebene, etwa ein Kunde setzt seine Bonuspunkte.
6. Weil Anfrage und Antwort gültig sind. Dass die Antwort dem Falschen gegeben wurde, weiß nur, wer die Besitzverhältnisse kennt.
7. Eine Abmilderung. Kennungen tauchen in Verweisen und Protokollen auf, und wer eine kennt, bekommt weiter das Objekt. Der Befund ist nicht behoben.
8. Eine Komponente mit weiten Rechten handelt im Auftrag eines Aufrufers mit geringen Rechten und prüft dessen Rechte nicht. Bei SSRF ruft der Server mit seiner Stellung im Netz etwas ab, das der Benutzer selbst nicht erreichen dürfte.
9. Weil die Korrektur an einer Stelle das Vergessen an der nächsten nicht verhindert. Erst eine zentrale Durchsetzung macht das Vergessen unmöglich oder auffällig.

## B.10 Kapitel 10: Konfiguration, Kryptografie, Lieferkette, Fachlogik

1. Weil immer mehr Verhalten eines Systems durch Einstellungen bestimmt wird, in Frameworks, Containern und Cloud-Diensten, und damit mehr falsch eingestellt sein kann.
2. Warum war er erreichbar? Und warum hat es niemand bemerkt?
3. Sichtbar sind die Einstellungen der verschlüsselten Verbindung, Zertifikate und vorhersagbare Werte. Nicht sichtbar sind die Speicherung von Daten und Passwörtern und die Prüfung von Zertifikaten in Clients. Kein Befund zur Speicherung heißt deshalb nicht, dass sie in Ordnung ist.
4. Wird der betroffene Teil verwendet? Kommt ein Angreifer an die Stelle heran? Wird die Lücke ausgenutzt? Hat der Tester es geprüft oder nur das Werkzeug?
5. Typosquatting: Ein schädliches Paket trägt einen Namen, der einem beliebten zum Verwechseln ähnelt. Dependency Confusion: Ein öffentliches Paket trägt den Namen eines internen und wird vom Werkzeug bevorzugt geladen.

6. Eine Buchung darf nur aus dem Zustand „bezahlt“ in den Zustand „erstattet“ wechseln. Lässt sich eine Erstattung für eine unbezahlte Buchung auslösen, ist das ein verbotener Übergang.
7. Weil Prüfen und Schreiben zwei Schritte bleiben, zwischen die eine zweite Anfrage passt. Die Behebung macht beides zu einem unteilbaren Schritt.
8. Im Fehlerfall wird verweigert statt durchgelassen. Ein Circuit Breaker, der beim Ausfall der Berechtigungsprüfung die Anfragen ohne Prüfung weiterleitet, öffnet das System genau dann, wenn ein Angreifer den Ausfall herbeiführt.
9. In der Spalte Lifecycle. Die einzelne Einstellung oder Bibliothek zu korrigieren verhindert die nächste nicht, dafür braucht es Vorgaben, Konfiguration als Code und laufende Prüfung.

## B.11 Kapitel 11: Nach dem ersten Zugriff

1. Weil die Auswirkung vom Wirkungsradius abhängt: von den Rechten des betroffenen Teils, vom Netz, vom Vertrauen zwischen den Teilen und von den erreichbaren Zugangsdaten.
2. Das System wird so entworfen, als sei ein Teil schon übernommen: minimale Rechte, getrennte Bereiche, Prüfung an jeder Verbindung und Erkennung.
3. Zu weite Rechte von Anfang an (Vorgaben für den Betrieb von Diensten), auffindbare Zugangsdaten eines mächtigeren Kontos (Umgang mit Geheimnissen), Schwachstellen in der Schicht darunter (Pflege der Plattform). Keine liegt im Code der Anwendung.
4. In Umgebungsvariablen, Konfigurationsdateien, im Abbild und im Code, in eingehängten Geheimnissen, im Verwaltungsdienst der Plattform und in Protokollen.
5. Der Angreifer rechnet zu möglichen Passwörtern den Hash aus und vergleicht. Die Aussicht hängt von der Geschwindigkeit des Hash-Verfahrens, vom Salz und von der Qualität der Passwörter ab.
6. Die Erreichbarkeit, also ob das Netz flach oder segmentiert ist, und das Vertrauen, also ob Services einander ohne Prüfung annehmen.
7. Der Tester hat an einer vereinbarten Grenze aufgehört. Der Satz sagt nicht, dass dahinter nichts mehr möglich war. Die weitere Auswirkung ist offen und selbst einzuschätzen.
8. Fünf aus Tab. 37, etwa eigener Benutzer ohne besondere Rechte, Geheimnisse aus einem Tresor, Datenbank und Konto je Service, segmentiertes Netz, Authentifizierung an jeder Verbindung. Der Preis der letzten sind Rechenzeit und Aufwand im Betrieb.

## B.12 Kapitel 12: KI-Systeme testen

1. Die Systemanweisung des Betreibers (vertrauenswürdig), die Frage des Benutzers (nicht vertrauenswürdig), gefundene Dokumente der Wissensbasis und Ergebnisse von Werkzeugen (beides von Dritten beeinflussbar).
2. Direkt: Der Benutzer schreibt selbst Anweisungen in seine Eingabe. Indirekt: Die Anweisungen stehen in einer Bewertung, die der Assistent später für eine andere Kundin liest.
3. Weil es keinen Weg gibt, einem Modell Daten so zu übergeben, dass es sie sicher nicht als Anweisung liest. Alles ist ein gemeinsamer Text.
4. Der Kontext des Modells ist von außen beeinflussbar, also auch seine Ausgabe. Wird sie in eine Seite, eine Abfrage oder einen Befehl eingesetzt, ist sie eine nicht vertrauenswürdige Quelle wie jede Eingabe.
5. Zu viele Funktionen, zu weite Berechtigungen, zu viel Selbstständigkeit.
6. Der verwirrte Stellvertreter. Das Werkzeug muss mit den Rechten der Kundin arbeiten, und die Berechtigung wird im Werkzeug geprüft, nicht in der Systemanweisung.
7. Ja. Ein Angreifer kann beliebig oft versuchen. Ein guter Bericht nennt deshalb die Erfolgsquote statt eines einzelnen Belegs.
8. Eine Abmilderung. Sie hält, bis jemand eine andere Formulierung findet. Behoben ist der Befund erst, wenn die Folge des Angriffs durch die Architektur verhindert ist.

## B.13 Kapitel 13: Der Pentest-Bericht

1. Die Leitung liest die Zusammenfassung, die Technik die einzelnen Befunde, die Prüfung Scope, Methodik und Abdeckung.

2. Was wurde geprüft? Wie ist das Gesamtbild? Was könnte jemand von außen tatsächlich anrichten? Welche zwei oder drei Maßnahmen bewirken am meisten?
3. Eine Gegenüberstellung von Bereichen und Testkategorien mit dem Vermerk, was geprüft wurde. „Teilweise geprüft" heißt, dass der Bericht über dieses Feld wenig aussagt und kein Befund dort nichts bedeutet.
4. Kennung und Titel, Schwere, Ort, Beschreibung, Nachweis, Auswirkung, Voraussetzungen, Einordnung, Empfehlung, Verweise.
5. Der Betrieb kann den Vorgang in den eigenen Protokollen suchen. Findet er ihn nicht, ist das ein eigener Befund.
6. An der Formulierung und am Verweis auf Belege: „wurde gelesen" gegen „könnte". Wichtig ist es, weil die Bewertung an der Auswirkung hängt.
7. Ziel nennen, Stelle korrigieren, Klasse prüfen, Ursache zentral beheben.
8. Ob eine Prüfung im System üblich ist und an einer Stelle fehlt, oder ob sie fast überall fehlt. Im ersten Fall ist es ein Versehen, im zweiten fehlt das Konzept, und die Behebung liegt auf einer anderen Ebene.
9. Fünf aus Tab. 44, etwa Textbausteine, ungeprüfte Werkzeugausgabe, durchgehend „könnte", keine Angabe zur Abdeckung, nur Befunde, die auch ein Scanner meldet.
10. Scope mit Einschränkungen und Abdeckung, Befundübersicht, Angriffspfade, einzelne Befunde, Nachfragen. Die Zusammenfassung kommt zuletzt, damit du dir vorher ein eigenes Urteil bildest.

## B.14 Kapitel 14: Schweregrade

1. Die Schwere ist eine Eigenschaft der Lücke. Das Risiko ist eine Eigenschaft der Lage aus Lücke, System, Angreifern und Geschäft. Die Dringlichkeit ist eine Entscheidung über Reihenfolge und Frist.
2. Ein Angreifer kann über das Netz, aber nur unter Umständen, die er nicht in der Hand hat, ohne eigene Rechte und mit Mitwirkung eines anderen Menschen einen Teil der vertraulichen Daten lesen. Er verändert und stört nichts, der Schaden bleibt in der Komponente. Der Wert ist 3,1.
3. Er sagt, ob der Schaden eine Grenze zu einer anderen Komponente mit eigenen Schutzmechanismen überschreitet. Der Scope eines Auftrags sagt, was getestet werden darf.
4. Ausnutzbarkeit: 8,22 × 0,85 × 0,77 × 0,85 × 0,85 = 3,89. Auswirkung: 6,42 × 0,56 = 3,60. Summe 7,49, aufgerundet 7,5.
5. Weil die Auswirkung dann höchstens 3,6 erreicht und die Ausnutzbarkeit höchstens 3,9. Die Summe ist 7,5 und liegt unter der Grenze von 9,0.
6. Das nötige Konto kann sich jeder anlegen, die Menge der betroffenen Daten geht in den Wert nicht ein, und der hohe Schutzbedarf der Daten auch nicht.
7. Sie gewichtet den Verlust der Vertraulichkeit höher, weil diese im betroffenen System besonders wichtig ist. Machen kann sie nur der Betreiber.
8. Drei aus: Der Scope ist durch Auswirkungen auf das verwundbare und auf nachgelagerte Systeme ersetzt, es gibt die neue Metrik Attack Requirements, die Mitwirkung hat drei Stufen, die zeitliche Gruppe ist zur Bedrohungsgruppe mit einer Metrik geworden, und der Name des Werts sagt, welche Gruppen eingeflossen sind.
9. Weil es keinen Angriff gibt, den der Vektor beschreiben würde. Der Befund ist eine fehlende Schutzmaßnahme, durch die andere Angriffe unbemerkt bleiben.
10. Zur Voraussetzung passt PR:H, nicht PR:N. Der Wert sinkt von 9,8 auf 7,2.

## B.15 Kapitel 15: Triage

1. Prüfen und Übertragen betrachten den einzelnen Befund. Bündeln und Verketten betrachten den Zusammenhang. Ordnen fasst zusammen.
2. Die ungeprüfte Werkzeugmeldung, die nicht erreichbare Stelle, die falsch gedeutete Beobachtung und das gewollte Verhalten. Das letzte ist kein falsch positiver Befund, sondern eine Entscheidung, die sich belegen lassen muss.
3. Bei dem, der schließen will. Er muss zeigen, warum der Nachweis nicht trägt.

4. Einen Beleg, etwa die Konfiguration der Schutzschicht. Und du hältst fest, dass der Schutz an dieser einen Stelle hängt und der Service selbst keinen hat.
5. Duplikat: dieselbe Schwäche an derselben Stelle. Klasse: dieselbe Fehlerart an verschiedenen Stellen. Ursache: verschiedene Fehlerarten aus derselben Entscheidung oder demselben Versäumnis.
6. Die gemeinsame Maßnahme, die alle Mitglieder behebt oder verhindert. Die CWE ordnet nach der Art des Fehlers im Code, und zwei Befunde mit derselben Ursache können dort in verschiedenen Ästen stehen.
7. Für jeden Befund festhalten, was ein Angreifer braucht und was er danach hat. Wo das Ergebnis des einen die Voraussetzung des anderen ist, entsteht eine Kette.
8. Die Behebung von F-07 schließt einen von vier Wegen. Das Ziel bleibt erreichbar. Zu beheben ist mindestens ein Befund aus jedem Ast.
9. Die Kette bekommt die Voraussetzungen ihres ersten und die Auswirkung ihres letzten Glieds. Jedes Glied erbt ihre Dringlichkeit, solange sie besteht.
10. Weil der Test eine Stichprobe ist und andere Stellen unter denselben Bedingungen entstanden sind. Das Team kann mit Quellcode und eigenen Regeln gründlicher suchen als die Tester.

## B.16 Kapitel 16: Ursachenanalyse

1. Symptom: Kunde sieht fremde Rechnung. Fehler: Besitzprüfung fehlt im Endpunkt. Ursache: Keine Stelle erzwingt die Prüfung.
2. Code: Was genau ist an dieser Stelle falsch? Architektur: Warum konnte es dort falsch sein, und warum reicht der Schaden so weit? Lifecycle: Warum ist es entstanden, und warum hat es niemand bemerkt?
3. Weil Menschen Dinge vergessen und ein System, das sich darauf verlässt, dass niemand etwas vergisst, falsch entworfen ist. Die Aussage ist der Anfang der nächsten Frage.
4. Warum ist der Fehler entstanden? Und warum wurde er nicht bemerkt?
5. Als Leiter von der Stelle zur Klasse. Von CWE-639 führt der Weg über die fehlerhafte Autorisierung zur unzureichenden Zugriffskontrolle. Der unterste Eintrag nennt die Korrektur, die oberen das Thema für die Architektur.
6. Sie verteilt die Verantwortung auf viele Stellen, sie macht den unsicheren Weg zum bequemen, und sie bestimmt den Wirkungsradius.
7. Das Prinzip der geringsten Berechtigung. Es vergrößert die SQL-Injection F-03 zum Zugriff auf die Daten aller Services.
8. Weil ein Token ohne Zustand nach seinem Entwurf bis zum Ablauf gilt. Ein Widerruf ist eine Fähigkeit, die dem System fehlt, und verlangt eine Entwurfsentscheidung. Die Laufzeit zu kürzen mildert nur.
9. Weil der Fehler an einer Stelle sitzt: Prüfen und Entwerten sind zwei Schritte. Solange das so bleibt, hilft kein Umbau. Die Architektur kann nur den Schaden begrenzen.
10. Die Ursache jeder Schwachstelle ermitteln, über die Zeit nach Mustern suchen, die Software auf ähnliche Schwachstellen durchsehen und den Prozess ändern, damit sie nicht wieder entstehen.
11. Bündel und Befunde, Fehler im Code, begünstigende Struktur, durchlassender Prozess, jeweils mit Beleg, weitere Stellen und offene Fragen. Das Feld „Weitere Stellen“ zeigt, dass der Befund die Stichprobe war und nicht das ganze Problem.

## B.17 Kapitel 17: Beheben, abmildern, nachtesten

1. Beheben (Abfrage parametrisieren), abmildern (Admin-Oberfläche nur aus dem Firmennetz), vermeiden (Abruf über eine Adresse abschaffen), akzeptieren (fehlende Header in einer Oberfläche, die abgelöst wird).
2. Stell dir vor, die Maßnahme fällt weg oder wird umgangen. Ist die Schwäche dann wieder da, war es eine Abmilderung.
3. Es ist eine Sperrliste. Die Struktur der Schwäche, dass Daten als Anweisung ausgewertet werden, besteht fort, und der Interpreter versteht mehr, als die Liste kennt.
4. Gegen Fehler der Autorisierung und der Fachlogik, weil sie Anfragen sieht und keine Bedeutung versteht. Und gegen alles, was nicht durch sie hindurchgeht, etwa den Verkehr zwischen Services.

5. Die Sofortmaßnahme wendet die Gefahr in Tagen ab. Die Korrektur der Klasse behebt in Wochen alle Stellen derselben Art. Die Behebung der Ursache ändert in Monaten Architektur und Prozess.
6. Er muss zuerst fehlschlagen. Er prüft die Eigenschaft und nicht den Nachweis. Er wird auf die Klasse verallgemeinert.
7. Stelle: Ist die gemeldete Stelle behoben? Nachweis ist der grüne Regressionstest. Klasse: Sind alle Stellen derselben Art behoben? Nachweis ist die Liste mit Tests. Ursache: Kann die Art von Fehler noch unbemerkt entstehen? Nachweis ist eine zentrale Stelle oder eine Prüfung in der Pipeline.
8. Nein. Die neuen Zugangsdaten stehen wieder im Klartext im Abbild. Der Tausch war nötig und ist eine Abmilderung. Behoben ist der Befund, wenn die Geheimnisse zur Laufzeit aus einem Tresor kommen.
9. Bestehende Nutzer verlassen sich auf das Fehlverhalten, alte Clients senden, was nicht mehr erlaubt ist, und die Maßnahme kostet andere Qualitäten wie Verfügbarkeit, Geschwindigkeit oder Bedienbarkeit.
10. Ob bei der Behebung neue Schwächen entstanden sind, und alles, was nicht Teil des Auftrags war, etwa weitere Stellen derselben Klasse und die Ursachen.
11. Die Korrektur gilt in der Web-Oberfläche, aber nicht in der Schnittstelle der App. Oder sie wurde an einem von mehreren Endpunkten eingebaut.

## B.18 Kapitel 18: Entscheiden unter Konflikt

1. Er trägt die Folgen und verfügt über die Mittel und die Befugnis. Die Entwicklung schätzt Aufwand und Wirkung, die Informationssicherheit berät und warnt. Beide entscheiden nicht für das Geschäft.
2. Siehe Tab. 65. Sicherheit: kennt die Bedrohungslage, unterschätzt Aufwand. Produkt: ohne Produkt kein Geschäft, hält Sicherheit für nachrüstbar. Betrieb: jede Änderung gefährdet die Verfügbarkeit, zählt den Angriff nicht mit. Compliance: Pflichten gegenüber Dritten, hält dokumentiert für behoben.
3. Tatsache: der Änderungsstopp. Bewertung: Das interne Netz sei sicher, zu prüfen am Bericht und dort durch F-05 widerlegt. Forderung: M3 kommt nicht in Frage.
4. Wirkung auf die Ziele, Aufwand, Nebenwirkungen und Zeitfenster, Abhängigkeiten, Pflichten.
5. Weil die Tests zeigen, welche Endpunkte M1 umstellen muss, danach belegen, dass es geschehen ist, und neue Endpunkte ohne Prüfung abfangen.
6. Weil viele angefangene Maßnahmen entstehen, von denen keine eine Kette bricht oder ein Bündel schließt. Eine halb eingeführte zentrale Kontrolle ist schlechter als keine, weil sich alle darauf verlassen.
7. Dokumentiert, vom Risikoeigner, für ein bestimmtes Risiko, für einen bestimmten Zeitraum, unter bestimmten Bedingungen.
8. Risiken, die Dritte tragen, ohne zugestimmt zu haben, etwa Kunden mit ihren Gesundheitsdaten, und alles, was eine gesetzliche Pflicht verletzt.
9. Lage, Optionen, Empfehlung, und was zu entscheiden ist. Auch die vollständige Option hat einen Preis, hier Ausfälle in der Saison und eine Vertragsstrafe. Eine Vorlage, die das verschweigt, wird als Werbung gelesen.
10. Es erklärt, warum eine Regel gilt, sodass sie nicht als Schikane umgangen wird. Der Pentest-Bericht kann das nicht, weil er vertraulich ist und vergessen wird.

## B.19 Kapitel 19: Die Spezialisierung unterrichten

1. Software Security geht vom Entwurf zum möglichen Fehler. Pentesting geht von außen in das laufende System. Die Spezialisierung geht vom Befund zurück zur Ursache und von dort zu einem Plan.
2. Weil die Befunde dann durch Ursachen verbunden sind, weil sich die Schwierigkeit gezielt einstellen lässt und weil der Korrekturschlüssel vor dem Bericht vorliegt.
3. Sechs aus Tab. 69, etwa Bündel (Kern der Aufgabe), Ketten mit einem niedrigen Glied (gegen das Sortieren nach Schweregrad), verschiedene CWE bei gleicher Ursache (gegen das Bündeln nach Textähnlichkeit), falsch positive Befunde (prüfen statt glauben), nur im Code behebbare Befunde (gegen den Reflex zum Umbau), Befunde ohne tiefere Ursache (Rauschen).

4. Architekturbeschreibung, Ausschnitte aus dem Code, Beschreibung der Pipeline, Entwicklungsrichtlinie, Testfälle, Betriebsunterlagen, geschäftlicher Rahmen. Die Belege für die Ursachen stehen nur dort, und ein Assistent, der nur den Bericht kennt, kann sie nicht nennen.
5. Er zeigt, was eine bequeme Abgabe mit einem Assistenten erreicht. Erreicht sie mehr als ein Genügend, ist die Case Study zu leicht.
6. Er nennt die Ursache im Code zuverlässig und liefert zu Architektur und Lifecycle Gemeinplätze. Verlangt wird deshalb zu jeder Aussage ein Beleg aus dem Dossier.
7. Beurteilung des Berichts, Triage-Tabelle, Bündel und Angriffsbaum, Ursachenanalysen, Maßnahmenplan, zwei Korrekturen im Code, Entscheidungsvorlage, Risikoakzeptanzen, Erklärung zum Einsatz von KI.
8. Sind die falsch positiven Befunde erkannt? Hat jedes Bündel eine Ursache mit Beleg? Sind die Abmilderungen benannt?
9. Erklären („Lesen Sie diesen Vektor als Satz vor“), Begründen („Welche eine Maßnahme behebt beide Befunde?“), Übertragen („Was ändert eine weitere Schutzschicht an Ihrer Bewertung?“), Angreifen („Ich bin der Betrieb. Überzeugen Sie mich.“).
10. Methodik und Namen der Phasen, Form des Berichts, Bewertungsverfahren mit Fassung, Namen der Klassen von Schwächen, und wie weit die Lehrveranstaltung nach dem ersten Zugriff und bei KI-Systemen geht.

## B.20 Kapitel 20: Ein Fall von Anfang bis Ende

1. Die Anbindung an den Zahlungsdienstleister wurde in der Testumgebung durch eine Attrappe ersetzt. Die Einschränkung stand im Bericht und wurde notiert, aber nicht zu einer Aufgabe gemacht.
2. Mit den Protokollen des Gateways der letzten dreißig Tage: Kein Konto hatte mehr als die eigenen Rechnungen abgerufen. Über die Zeit davor ließ sich nichts sagen.
3. Der Entwurf zu M3 beginnt sofort, und vor der Gutscheinaktion werden zwei zusätzliche Testtage für die Fachlogik beauftragt. Grundlage waren die Abhängigkeit zwischen M4 und M3 und die Abdeckungsmatrix.
4. Das Team hatte die Eingabe gefiltert statt die Abfrage zu parametrisieren, und sein Test wiederholte nur die Anfrage aus dem Bericht. Seither prüfen die Tests die Eigenschaft, und eine blockierende Regel der statischen Analyse findet zusammengesetzte Abfragen.
5. Die zentrale Zugriffskontrolle unterband einen Bericht der Buchhaltung, der sich mit einem Kundenkonto alle Rechnungen holte. Die Trennung der Datenbankkonten deckte auf, dass ein Service direkt in Tabellen eines anderen las.
6. Befunde zur Zugriffskontrolle auf Objektebene, zur Injection und zum Vertrauen zwischen den Services. Dass diese Klassen fehlen, zeigt, dass Ursachen behoben wurden und nicht nur Stellen.
7. Die Regel für neue Funktionen stand am Ende, weil sie nur auf Neues wirkt. In einem Jahr mit viel Neuem hätte sie an den Anfang gehört, und der Assistent wäre mit einem Bedrohungsmodell entworfen worden.

# C Glossar

Das Glossar erklärt die Begriffe so, wie dieses Buch sie verwendet. Die Zahl in Klammern nennt das Kapitel, in dem der Begriff eingeführt wird. Kürzel wie CVE, CWE oder CVSS stehen mit Langform und Erklärung im Spickzettel in Anhang A.

**Abdeckungsmatrix**: Gegenüberstellung von geprüften Bereichen und Testkategorien mit dem Vermerk, was geprüft wurde (13).

**Abmilderung**: Maßnahme, die eine Schwäche bestehen lässt und ihre Ausnutzung erschwert oder ihren Schaden begrenzt (17).

**Angriffsbaum**: Darstellung der Wege zu einem Ziel des Angreifers, mit dem Ziel als Wurzel (6, 15).

**Angriffsfläche**: alle Stellen, an denen jemand von außerhalb einer Vertrauensgrenze Daten hineingeben, herausbekommen oder eine Funktion auslösen kann (6).

**Angriffspfad**: Folge von Schritten, die Tester von der ersten Schwäche bis zum weitesten erreichten Punkt gegangen sind (11, 13).

**Assume Breach**: Entwurfshaltung, die davon ausgeht, dass ein Teil des Systems übernommen wird (11).

**Aufklärung**: Phase eines Tests, in der festgestellt wird, was es gibt und was erreichbar ist (7).

**Authentifizierung**: Feststellung, wer jemand ist (9).

**Autorisierung**: Feststellung, was jemand darf, auf den Ebenen Funktion, Objekt und Eigenschaft (9).

**Bedrohungsmodellierung**: strukturierte Analyse eines Systemmodells auf mögliche Bedrohungen (6).

**Befund**: einzelnes Ergebnis eines Tests mit Ort, Nachweis, Auswirkung, Einordnung und Empfehlung (13).

**Behebung**: Maßnahme, die den Fehlerzustand hinter einem Befund beseitigt (17).

**Black Box**: Test ohne Vorwissen über das System (1).

**Bug Bounty**: offene Einladung, Schwächen gegen Prämie zu melden (1).

**Bündel**: Gruppe von Befunden, die durch eine gemeinsame Maßnahme behoben oder verhindert werden können (15).

**Credential Stuffing**: Durchprobieren von Zugangsdaten, die anderswo abgeflossen sind (9).

**Datenflussdiagramm**: Modell aus externen Entitäten, Prozessen, Datenspeichern und Datenflüssen, ergänzt um Vertrauensgrenzen (6).

**Deny by Default**: Was nicht ausdrücklich erlaubt ist, ist verboten (9).

**Deserialisierung**: Wiederherstellen von Objekten aus Daten, gefährlich bei Daten aus nicht vertrauenswürdiger Quelle (8).

**Dossier**: Unterlagen über das System, die die Studierenden zusätzlich zum Bericht bekommen (19).

**Dreispaltentest**: Prüfung einer Behebung auf Stelle, Klasse und Ursache (17).

**Dringlichkeit**: Entscheidung, in welcher Reihenfolge und bis wann behoben wird (14).

**Duplikat**: dieselbe Schwäche an derselben Stelle, mehrfach gemeldet (15).

**Ebenen der Ursache**: Code, Architektur und Lifecycle (16).

**Excessive Agency**: übermäßige Handlungsmacht eines Systems mit Sprachmodell (12).

**Fachlogikfehler**: Verletzung einer Regel des Geschäfts, oft ein verbotener Zustandsübergang (10).

**Falsch positiv**: Befund, der eine Schwachstelle meldet, die es nicht gibt (7, 15).

**Fehlkonfiguration**: unsichere Einstellung eines an sich fehlerfreien Produkts (10).

**Grey Box**: Test mit begrenztem Vorwissen, meist Konten und Schnittstellenbeschreibung (1).

**Härtung**: Einstellen eines Produkts nach einer verbindlichen Vorgabe auf einen sicheren Zustand (10).

**Implizites Vertrauen**: Annahme eines Service, dass jede Anfrage aus dem Inneren schon geprüft wurde (6).

**Injection**: Daten aus einer nicht vertrauenswürdigen Quelle werden von einem Interpreter als Anweisung ausgewertet (8).

**Kette**: Folge von Befunden, bei der jeder die Voraussetzung für den nächsten schafft (15).

**Klasse**: dieselbe Fehlerart an verschiedenen Stellen (15).

**Kodierung**: Umformung von Daten, sodass sie in einem bestimmten Ausgabekontext als Daten gelesen werden (8).

**Lieferkette**: Weg, auf dem fremde Komponenten und neue Versionen in ein System kommen (10).

**Mass Assignment**: automatische Übernahme gesendeter Felder in ein Objekt, auch solcher, die der Benutzer nicht setzen soll (9).

**Nachtest**: erneute Prüfung der Befunde nach der Behebung (17).

**Nachweis**: Angaben im Befund, mit denen ein Dritter die Beobachtung wiederholen kann (13).

**Parametrisierung**: getrennte Übergabe von Anweisung und Daten an einen Interpreter (8).

**Path Traversal**: Zugriff auf Dateien außerhalb des vorgesehenen Verzeichnisses über manipulierte Pfade (8).

**Penetrationstest**: beauftragte, begrenzte, auf Ausnutzung gerichtete Sicherheitsprüfung mit Nachweis (1).

**Prompt Injection**: Text gelangt in den Kontext eines Sprachmodells und wird als Anweisung befolgt, obwohl seine Quelle keine geben dürfte (12).

**Quelle**: Stelle, an der nicht vertrauenswürdige Daten in ein Programm gelangen (8).

**Rechteausweitung**: Schritt von vorhandenen zu höheren Rechten (11).

**Red Teaming**: Einsatz mit einem bestimmten Ziel und allen Mitteln, der auch Erkennung und Reaktion prüft (1).

**Regressionstest zu einem Befund**: automatisierter Test, der die Handlung aus dem Nachweis ausführt und eine Ablehnung erwartet (17).

**Risiko**: Lage aus Lücke, System, Angreifern und Geschäft (14).

**Risikoakzeptanz**: dokumentierte Entscheidung des Risikoeigners, ein bestimmtes Risiko für eine bestimmte Zeit unter bestimmten Bedingungen zu tragen (18).

**Risikoeigner**: Person, die die Folgen eines Risikos trägt und über seine Behandlung entscheiden darf (18).

**Rollenmatrix**: Tabelle, welche Rolle welche Funktion aufrufen darf (7).

**Rules of Engagement**: vereinbarte Spielregeln eines Tests (2).

**Schutzbedarf**: wie wichtig Vertraulichkeit, Integrität und Verfügbarkeit in einem System sind (14).

**Schwäche**: Art von Fehler, die unter Umständen ausnutzbar wird (4).

**Schwachstelle**: konkretes, ausnutzbares Auftreten einer Schwäche in einem Produkt (4).

**Schwachstellenscan**: automatisierter Abgleich eines Systems mit bekannten Lücken und Fehlkonfigurationen (1, 7).

**Schwere**: technische Eigenschaft einer Lücke, wie leicht sie auszunutzen ist und wie viel dabei verloren geht (14).

**Scope (Auftrag)**: Festlegung, was getestet wird (2).

**Scope (CVSS)**: Metrik, die sagt, ob der Schaden eine andere Komponente trifft (14).

**Seitwärtsbewegung**: Übergang von einem übernommenen System zu einem weiteren im selben Netz (11).

**Senke**: Stelle, an der Daten eine gefährliche Wirkung entfalten können, etwa eine Abfrage (8).

**Shift Left**: Sicherheitstätigkeiten früher im Lebenszyklus ansetzen (5).

**Sofortmaßnahme**: Maßnahme, die eine unmittelbare Gefahr in Tagen abwendet (17).

**Symptom**: von außen beobachtetes Fehlverhalten (16).

**Systemanweisung**: Vorgaben des Betreibers an ein Sprachmodell (12).

**Testumgebung**: nachgebaute Umgebung, in der getestet wird, um die Produktion zu schonen (2).

**Token**: signierte Angabe über einen Benutzer, die ein Service prüft, statt eine Sitzung nachzuschlagen (9).

**Triage**: Sichtung der Befunde in fünf Schritten: prüfen, übertragen, bündeln, verketten, ordnen (15).

**Ursache**: warum ein Fehlerzustand entstand und bestehen blieb (16).

**Validierung**: Prüfung an der Quelle, ob eine Eingabe der Erwartung entspricht (8).

**Vektor**: Zeichenkette, die die Ausprägung jeder CVSS-Metrik festhält (4, 14).

**Verwirrter Stellvertreter**: Komponente mit weiten Rechten, die im Auftrag eines Aufrufers handelt und dessen Rechte nicht prüft (9).

**Vertrauensgrenze**: Linie im System, an der sich ändert, wie weit Daten und Absender vertraut werden darf (6).

**Virtuelles Patchen**: Abschirmen einer Lücke durch eine Regel in einer Schutzschicht, bis die Behebung fertig ist (17).

**Vollständige Vermittlung**: Entwurfsprinzip, nach dem jeder Zugriff auf jedes Objekt geprüft wird (16).

**Warum-Kette**: wiederholtes Fragen nach dem Warum, bis die Antwort ein änderbarer Zustand ist (16).

**Wettlaufsituation**: Fehler, bei dem das Ergebnis davon abhängt, in welcher Reihenfolge gleichzeitige Vorgänge ablaufen (10).

**White Box**: Test mit vollem Vorwissen einschließlich Quelltext (1).

**Wirkungsradius**: wie weit ein Einbruch oder Fehler in einem System reicht (11).

**Zustand eines Befunds**: Ergebnis der Prüfung in der Triage, etwa bestätigt oder falsch positiv (15).

# Literaturverzeichnis

Die Nummern verweisen auf die Quellenangaben im Text. Welche Angaben am 12. November 2026 online gegen die Originalquelle geprüft wurden, steht in `QUELLENPRUEFUNG.md` im Buchordner. Das Verzeichnis enthält nur Quellen, die im Text zitiert werden.


[1] K. Scarfone, M. Souppaya, A. Cody, und A. Orebaugh, „Technical Guide to Information Security Testing and Assessment“, technical report NIST SP 800-115, 2008. doi: 10.6028/NIST.SP.800-115.

[2] Europäisches Parlament und Rat, „Verordnung (EU) 2022/2554 über die digitale operationale Resilienz im Finanzsektor (DORA)“. [Online]. Verfügbar unter: https://eur-lex.europa.eu/eli/reg/2022/2554/oj

[3] European Central Bank, „TIBER-EU Framework: How to Implement the European Framework for Threat Intelligence-Based Ethical Red Teaming“. [Online]. Verfügbar unter: https://www.ecb.europa.eu/paym/cyber-resilience/tiber-eu/html/index.en.html

[4] ISO/IEC, „ISO/IEC 29147:2018 Information technology - Security techniques - Vulnerability disclosure“. 2018.

[5] ISO/IEC, „ISO/IEC 30111:2019 Information technology - Security techniques - Vulnerability handling processes“. 2019.

[6] E. Foudil und Y. Shafranovich, „RFC 9116: A File Format to Aid in Security Vulnerability Disclosure“. 2022. doi: 10.17487/RFC9116.

[7] A. Spillner und T. Linz, *Software Testing Foundations*, 5. Aufl. Heidelberg: dpunkt.verlag, 2021.

[8] Europäisches Parlament und Rat, „Richtlinie (EU) 2022/2555 über Maßnahmen für ein hohes gemeinsames Cybersicherheitsniveau in der Union (NIS-2-Richtlinie)“. [Online]. Verfügbar unter: https://eur-lex.europa.eu/eli/dir/2022/2555/oj

[9] Wirtschaftskammer Österreich, „Cybersicherheitsgesetz NISG 2026“. [Online]. Verfügbar unter: https://www.wko.at/oe/oesterreich/cybersicherheitsgesetz-nisg-2026

[10] Europäisches Parlament und Rat, „Verordnung (EU) 2024/2847 über horizontale Cybersicherheitsanforderungen für Produkte mit digitalen Elementen (Cyber Resilience Act)“. [Online]. Verfügbar unter: https://eur-lex.europa.eu/eli/reg/2024/2847/oj

[11] Republik Österreich, „Strafgesetzbuch (StGB), §§ 118a, 119, 119a, 126a, 126b, 126c“. [Online]. Verfügbar unter: https://www.ris.bka.gv.at/GeltendeFassung.wxe?Abfrage=Bundesnormen&Gesetzesnummer=10002296

[12] Europäisches Parlament und Rat, „Verordnung (EU) 2016/679 (Datenschutz-Grundverordnung)“. [Online]. Verfügbar unter: https://eur-lex.europa.eu/eli/reg/2016/679/oj

[13] PTES Team, „The Penetration Testing Execution Standard“. [Online]. Verfügbar unter: http://www.pentest-standard.org/

[14] OWASP Foundation, „OWASP Web Security Testing Guide, Version 4.2“. [Online]. Verfügbar unter: https://owasp.org/www-project-web-security-testing-guide/

[15] OWASP Foundation, „OWASP Application Security Verification Standard, Version 5.0.0“. [Online]. Verfügbar unter: https://owasp.org/www-project-application-security-verification-standard/

[16] P. Herzog, „OSSTMM 3: The Open Source Security Testing Methodology Manual“. [Online]. Verfügbar unter: https://www.isecom.org/OSSTMM.3.pdf

[17] Bundesamt für Sicherheit in der Informationstechnik, „Studie: Durchführungskonzept für Penetrationstests“. [Online]. Verfügbar unter: https://www.bsi.bund.de/

[18] CVE Program, „CVE: Common Vulnerabilities and Exposures“. [Online]. Verfügbar unter: https://www.cve.org/

[19] National Vulnerability Database, „CVE-2014-0160 (Heartbleed)“. [Online]. Verfügbar unter: https://nvd.nist.gov/vuln/detail/CVE-2014-0160

[20] National Vulnerability Database, „CVE-2021-44228 (Log4Shell)“. [Online]. Verfügbar unter: https://nvd.nist.gov/vuln/detail/CVE-2021-44228

[21] National Vulnerability Database, „CVE-2024-3094: Malicious backdoor in xz/liblzma“. [Online]. Verfügbar unter: https://nvd.nist.gov/vuln/detail/CVE-2024-3094

[22] National Institute of Standards and Technology, „National Vulnerability Database“. [Online]. Verfügbar unter: https://nvd.nist.gov/

[23] ENISA, „European Vulnerability Database (EUVD)“. [Online]. Verfügbar unter: https://euvd.enisa.europa.eu/

[24] MITRE Corporation, „CWE: Common Weakness Enumeration“. [Online]. Verfügbar unter: https://cwe.mitre.org/

[25] MITRE Corporation, „2025 CWE Top 25 Most Dangerous Software Weaknesses“. [Online]. Verfügbar unter: https://cwe.mitre.org/top25/archive/2025/2025_cwe_top25.html

[26] OWASP Foundation, „OWASP Top 10:2025“. [Online]. Verfügbar unter: https://owasp.org/Top10/2025/

[27] FIRST, „Common Vulnerability Scoring System v3.1: Specification Document“. [Online]. Verfügbar unter: https://www.first.org/cvss/v3-1/specification-document

[28] FIRST, „Common Vulnerability Scoring System Version 4.0: Specification Document“. [Online]. Verfügbar unter: https://www.first.org/cvss/v4-0/specification-document

[29] Cybersecurity and Infrastructure Security Agency, „Known Exploited Vulnerabilities Catalog“. [Online]. Verfügbar unter: https://www.cisa.gov/known-exploited-vulnerabilities-catalog

[30] Cybersecurity and Infrastructure Security Agency, „Binding Operational Directive 22-01: Reducing the Significant Risk of Known Exploited Vulnerabilities“. [Online]. Verfügbar unter: https://www.cisa.gov/news-events/directives/bod-22-01-reducing-significant-risk-known-exploited-vulnerabilities

[31] FIRST, „Exploit Prediction Scoring System (EPSS)“. [Online]. Verfügbar unter: https://www.first.org/epss/

[32] J. Jacobs, S. Romanosky, B. Edwards, I. Adjerid, und M. Roytman, „Exploit Prediction Scoring System (EPSS)“, *Digital Threats: Research and Practice*, Bd. 2, Nr. 3, 2021, doi: 10.1145/3436242.

[33] M. Roytman, „EPSS V5 Is Here“. [Online]. Verfügbar unter: https://research.empiricalsecurity.com/research/epss-v5-is-here

[34] MITRE Corporation, „CAPEC: Common Attack Pattern Enumeration and Classification“. [Online]. Verfügbar unter: https://capec.mitre.org/

[35] MITRE Corporation, „MITRE ATT&CK“. [Online]. Verfügbar unter: https://attack.mitre.org/

[36] B. E. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington, und C. B. Thomas, „MITRE ATT&CK: Design and Philosophy“, technical report, 2018. [Online]. Verfügbar unter: https://attack.mitre.org/docs/ATTACK_Design_and_Philosophy_March_2020.pdf

[37] MITRE Corporation, „ATT&CK Updates: October 2026 (v20)“. [Online]. Verfügbar unter: https://attack.mitre.org/resources/updates/

[38] MITRE Corporation, „MITRE ATLAS: Adversarial Threat Landscape for Artificial-Intelligence Systems“. [Online]. Verfügbar unter: https://atlas.mitre.org/

[39] MITRE Corporation, „MITRE D3FEND: A Knowledge Graph of Cybersecurity Countermeasures“. [Online]. Verfügbar unter: https://d3fend.mitre.org/

[40] E. M. Hutchins, M. J. Cloppert, und R. M. Amin, „Intelligence-Driven Computer Network Defense Informed by Analysis of Adversary Campaigns and Intrusion Kill Chains“, in *Proceedings of the 6th International Conference on Information Warfare and Security*, 2011.

[41] J. H. Saltzer und M. D. Schroeder, „The Protection of Information in Computer Systems“, *Proceedings of the IEEE*, Bd. 63, Nr. 9, S. 1278–1308, 1975, doi: 10.1109/PROC.1975.9939.

[42] M. Howard und S. Lipner, *The Security Development Lifecycle*. Microsoft Press, 2006.

[43] G. McGraw, *Software Security: Building Security In*. Addison-Wesley, 2006.

[44] M. Souppaya, K. Scarfone, und D. Dodson, „Secure Software Development Framework (SSDF) Version 1.1“, technical report NIST SP 800-218, 2022. doi: 10.6028/NIST.SP.800-218.

[45] H. Booth, M. Souppaya, A. Vassilev, M. Ogata, M. Stanley, und K. Scarfone, „Secure Software Development Practices for Generative AI and Dual-Use Foundation Models“, technical report NIST SP 800-218A, 2024. doi: 10.6028/NIST.SP.800-218A.

[46] OWASP Foundation, „OWASP Software Assurance Maturity Model (SAMM), Version 2“. [Online]. Verfügbar unter: https://owaspsamm.org/model/

[47] Black Duck Software, „Building Security In Maturity Model (BSIMM)“. [Online]. Verfügbar unter: https://www.blackduck.com/services/security-program/bsimm-maturity-model.html

[48] OWASP Foundation, „CycloneDX Bill of Materials Standard“. [Online]. Verfügbar unter: https://cyclonedx.org/

[49] Open Source Security Foundation, „SLSA: Supply-chain Levels for Software Artifacts“. [Online]. Verfügbar unter: https://slsa.dev/

[50] P. Haindl, P. Kochberger, und M. Sveggen, „A Systematic Literature Review of Inter-Service Security Threats and Mitigation Strategies in Microservice Architectures“, *IEEE Access*, Bd. 12, S. 90252–90286, 2024, doi: 10.1109/ACCESS.2024.3406500.

[51] A. Shostack, *Threat Modeling: Designing for Security*. Indianapolis: Wiley, 2014.

[52] Z. Braiterman, A. Shostack, J. Marcil, und others, „Threat Modeling Manifesto“. [Online]. Verfügbar unter: https://www.threatmodelingmanifesto.org/

[53] L. Kohnfelder und P. Garg, „The Threats to Our Products“. [Online]. Verfügbar unter: https://shostack.org/files/microsoft/The-Threats-To-Our-Products.docx

[54] J. Kettle, „Smashing the State Machine: The True Potential of Web Race Conditions“. [Online]. Verfügbar unter: https://portswigger.net/research/smashing-the-state-machine

[55] L. Bass, P. Clements, und R. Kazman, *Software Architecture in Practice*, 4. Aufl. Boston: Addison-Wesley Professional, 2021.

[56] B. Schneier, „Attack Trees“, *Dr. Dobb's Journal*, Bd. 24, Nr. 12, S. 21–29, 1999, [Online]. Verfügbar unter: https://www.schneier.com/academic/archives/1999/12/attack_trees.html

[57] OWASP Foundation, „OWASP Risk Rating Methodology“. [Online]. Verfügbar unter: https://owasp.org/www-community/OWASP_Risk_Rating_Methodology

[58] OWASP Foundation, „OWASP API Security Top 10 2023“. [Online]. Verfügbar unter: https://owasp.org/API-Security/editions/2023/en/0x11-t10/

[59] M. Zalewski, *The Tangled Web: A Guide to Securing Modern Web Applications*. No Starch Press, 2011.

[60] S. Khan, I. Kabanov, Y. Hua, und S. Madnick, „A Systematic Analysis of the Capital One Data Breach: Critical Lessons Learned“, *ACM Transactions on Privacy and Security*, Bd. 26, Nr. 1, 2022, doi: 10.1145/3546068.

[61] P. A. Grassi und others, „Digital Identity Guidelines: Authentication and Lifecycle Management“. 2017. doi: 10.6028/NIST.SP.800-63b.

[62] Center for Internet Security, „CIS Benchmarks“. [Online]. Verfügbar unter: https://www.cisecurity.org/cis-benchmarks

[63] J. Spracklen, R. Wijewickrama, A. H. M. N. Sakib, A. Maiti, B. Viswanath, und M. Jadliwala, „We Have a Package for You! A Comprehensive Analysis of Package Hallucinations by Code Generating LLMs“, in *34th USENIX Security Symposium*, 2025. [Online]. Verfügbar unter: https://arxiv.org/abs/2406.10279

[64] N. Perry, M. Srivastava, D. Kumar, und D. Boneh, „Do Users Write More Insecure Code with AI Assistants?“, in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2023, S. 2785–2799. doi: 10.1145/3576915.3623157.

[65] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, und R. Karri, „Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions“, in *2022 IEEE Symposium on Security and Privacy (SP)*, 2022, S. 754–768. doi: 10.1109/SP46214.2022.9833571.

[66] OWASP GenAI Security Project, „OWASP Top 10 for LLM Applications 2025“. [Online]. Verfügbar unter: https://genai.owasp.org/llm-top-10/

[67] OWASP GenAI Security Project, „OWASP Top 10 for LLM Applications 2026“. [Online]. Verfügbar unter: https://genai.owasp.org/llm-top-10/

[68] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, und M. Fritz, „Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection“, in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023, S. 79–90. doi: 10.1145/3605764.3623985.

[69] J. Wunder, A. Kurtz, C. Eichenmüller, F. Gassmann, und Z. Benenson, „Shedding Light on CVSS Scoring Inconsistencies: A User-Centric Study on Evaluating Widespread Security Vulnerabilities“, in *2024 IEEE Symposium on Security and Privacy (SP)*, 2024. [Online]. Verfügbar unter: https://arxiv.org/abs/2308.15259

[70] J. M. Spring *u. a.*, „Prioritizing Vulnerability Response: A Stakeholder-Specific Vulnerability Categorization (Version 2.0)“, technical report, 2021. [Online]. Verfügbar unter: https://www.sei.cmu.edu/library/prioritizing-vulnerability-response-a-stakeholder-specific-vulnerability-categorization-version-20/

[71] Cybersecurity and Infrastructure Security Agency, „Stakeholder-Specific Vulnerability Categorization (SSVC)“. [Online]. Verfügbar unter: https://www.cisa.gov/stakeholder-specific-vulnerability-categorization-ssvc

[72] ISO/IEC, „ISO/IEC 27005:2022 Information security, cybersecurity and privacy protection - Guidance on managing information security risks“. 2022.

[73] J. Biggs und C. Tang, *Teaching for Quality Learning at University*, 4. Aufl. Maidenhead: Open University Press, 2011.